

Auxiliary File Format for Simulator 24

Last Updated: September 1, 2026



PowerWorld Corporation

2001 South First St

Champaign, IL 61820

(217) 384-6330

<http://www.powerworld.com>

info@powerworld.com

Table of Contents

Introduction.....	15
SCRIPT Section	16
Using Filters in Script Commands.....	16
Specifying Special Keywords in Script Command Parameters.....	17
Specifying File Names in Script Commands	17
Specifying Field Variable Names in Script Commands	18
Specifying Field Values in Script Commands.....	18
AUX Actions.....	19
General Program Actions.....	19
CopyFile("oldfilename", "newfilename");	19
DeleteFile("filename");.....	19
ExitProgram;	19
LogAdd("text");.....	19
LogAddDateTime("label", includedate, includetime, includemilliseconds);.....	19
LogClear;.....	19
LogSave("filename", AppendFile);	20
LogShow(DoShow);	20
SetCurrentDirectory("filedirectory", CreatelfNotFound);.....	20
StopAuxFile;.....	20
WriteTextToFile("filename", "text");.....	20
Data Interaction.....	21
CustomFieldDescriptionAppend(ObjectType, CustomType, FieldString, HeaderString, IncludeInDiff);.....	21
CustomFieldDescriptionModify(ObjectType, CustomType, Location, "FieldString", "HeaderString", IncludeInDiff);....	21
CreateData(objecttype, [fieldlist], [valuelist]);	21
Delete(objecttype, filter);.....	22
DeleteDevice([ObjectIDString]);	22
DeleteIncludingContents(objecttype, filter);	22
ExportAreaSupplyCurves("filename", "User Defined String", NumPoints);	22
ImportData("filename", FileType, HeaderLine, CreatelfNotFound);	23
LoadAux("filename", CreatelfNotFound);	23
LoadAuxDirectory("filedirectory", "filterstring", CreatelfNotFound);	23
LoadCSV("filename", CreatelfNotFound);.....	24
LoadData("filename", DataName, CreatelfNotFound);.....	24
LoadScript("filename", ScriptName, CreatelfNotFound);	24
SaveData("filename", filetype, objecttype, [fieldlist], [subdatalist], filter, [SortFieldList], Transpose, Append);.....	25
SaveDataEPC("filename", objecttype, filter, GEFileType, SaveBuses, Append);	26
SaveDataUsingBuiltInAUXFormat("filename", filetype, [List_of_built_ins], ModelToUse);.....	27
SaveDataUsingExportFormat("filename", filetype, "FormatName", ModelToUse);.....	27
SaveDataWithExtra("filename", filetype, objecttype, [fieldlist], [subdatalist], filter, [SortFieldList], [Header_List], [Header_Value_List], Transpose, Append);.....	28
SaveObjectFields("filename", objecttype, [fieldlist]);.....	29
SelectAll(objecttype, filter);.....	29
SendtoExcel(objecttype, [fieldlist], filter, UseColumnHeaders, "workbookname", "worksheetname", [SortFieldList], [Header_List], [Header_Value_List], ClearExisting, RowShift, ColShift);.....	29
SetData(objecttype, [fieldlist], [valuelist], filter);.....	31

SetElseCreateData(objecttype, [fieldlist], [SetValueList], [DefaultValueList]);	31
UnSelectAll(objecttype, filter);	32
WriteLimitMonitoringSettings("filename");	32
Case Actions	33
AppendCase("filename", OpenFileType, [StarBus, EstimateVoltages]);	33
AppendCase("filename", OpenFileType, [MSLine, VarLimDead, PostCTGAGC, EstimateVoltages]);	33
CaseDescriptionClear;	33
CaseDescriptionSet("text", Append);	34
DeleteExternalSystem;	34
EnterMode(mode);	34
Equivalence;	34
LoadEMS("filename", filetype);	34
NewCase;	34
OpenCase("filename", OpenFileType,[LoadTransactions,StarBus,HowToKeepDuplicates]);	34
OpenCase("filename", OpenFileType,[MSLine,VarLimDead,PostCTGAGC,MSLineDummyBus]);	34
Renumber3WXFormerStarBuses("filename", Delimiter);	35
RenumberAreas(NumCI);	36
RenumberBuses(NumCI);	36
RenumberMSLineDummyBuses("filename", Delimiter);	36
RenumberSubs(NumCI);	36
RenumberZones(NumCI);	37
RenumberCase;	37
SaveCase("filename", SaveFileType, [PostCTGAGC, UseAreaZone]);	37
SaveCase("filename", SaveFileType, [AddCommentForObjectLabels, IncludeSubstations]);	37
SaveExternalSystem("Filename", SaveFileType, WithTies);	38
SaveMergedFixedNumBusCase ("filename", SaveFileType);	38
Scale(scaletype, basedon, [parameters], scalemarker);	39
Modify Case Objects	41
AutoInsertTieLineTransactions;	41
BranchMVALimitReorder(Filter, SetA, SetB, SetC, SetD, SetE, SetF, SetG, SetH, SetI, SetJ, SetK, SetL, SetM, SetN, SetO);	41
CalculateRXBGFromLengthConfigCondType(filter);	41
ChangeSystemMVABase(NewBase);	41
ClearSmallIslands;	42
CreateLineDeriveExisting(FromBus, ToBus, Circuit, NewLength, BranchID, ExistingLength, ZeroG);	42
DirectionsAutoInsert(Source, Sink, DeleteExisting, UseAreaZoneFilters);	42
DirectionsAutoInsertReference(SourceType, ReferenceObject, DeleteExisting, SourceFilterName, OppositeDirection);	43
InitializeGenMvarLimits;	43
InjectionGroupsAutoInsert;	43
InjectionGroupCreate("Name", objecttype, InitialValue, filter, Append);	43
InjectionGroupRemoveDuplicates(PreferenceFilter);	44
InterfaceAddElementsFromContingency(InterfaceName, ContingencyName);	44
InterfacesAutoInsert(Type, DeleteExisting, UseFilters, "Prefix", Limits);	45
InterfaceCreate("Name", DeleteExisting, ObjectType, Filter);	45
InterfaceFlatten("InterfaceName");	45
InterfaceFlattenFilter(Filter);	45
InterfaceModifyIsolatedElements(filter);	46
InterfaceRemoveDuplicates(PreferenceFilter);	46
MergeBuses([element], Filter);	47
MergeLineTerminals(Filter);	47
MergeMSLineSections(Filter);	47
Move([elementA], [destination parameters], HowMuch, AbortOnError);	47

ReassignIDs(objecttype, field, filter, UseRight);.....	48
Remove3WXformerContainer(filter);	49
RenameInjectionGroup("OldName", "NewName");.....	49
RotateBusAnglesInIsland([BUS KeyField], Value);.....	49
SetGenPMaxFromReactiveCapabilityCurve(filter);	50
SetParticipationFactors(Method, ConstantValue, Object);	50
SetScheduledVoltageForABus([bus identifier], voltage);.....	50
SetInterfaceLimitToMonitoredElementLimitSum(filter);	51
SplitBus([element], NewBusNumber, InsertBusTieLine, LineOpen, BranchDeviceType);	51
SuperAreaAddAreas("Name", Filter);.....	51
SuperAreaRemoveAreas("Name", Filter);.....	52
TapTransmissionLine([element], PosAlongLine, NewBusNumber, ShuntModel, TreatAsMSLine, UpdateOnelines, NewBusName);	52
Power Flow	54
ClearPowerFlowSolutionAidValues;	54
ConditionVoltagePockets(VoltageThreshold, AngleThreshold, filter);.....	54
DiffCaseClearBase;	54
DiffCaseKeyType(KeyType);.....	54
DiffCaseMode(diffmode);.....	55
DiffCaseSetAsBase;	55
DiffCaseShowPresentAndBase(How);.....	55
DiffCaseRefresh;.....	55
DiffCaseWriteCompleteModel ("filename", AppendFile, SaveAdded, SaveRemoved, SaveBoth, KeyFields, "ExportFormat", UseAreaZone, UseDataMaintainer, AssumeBaseMeet, IncludeClearPowerFlowSolutionAidValues, DeleteBranchesThatFlipBusOrder);.....	55
DiffCaseWriteBothEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, "ExportFormat", UseDataMaintainer);.....	57
DiffCaseWriteNewEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, UseDataMaintainer);.....	57
DiffCaseWriteRemovedEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, UseDataMaintainer);.....	57
DoCTGAction([contingency action]);.....	58
DoCTGAction("contingency action");	58
EstimateVoltages(filter);.....	58
GenForceLDC_RCC(filter);.....	59
InterfacesCalculatePostCTGMWFlows;	59
ResetToFlatStart (FlatVoltagesAngles, ShuntsToMax, LTCsToMiddle, PSAnglesToMiddle);	59
SaveGenLimitStatusAction("filename");.....	59
SaveJacobian("JacFileName", "JIDFileName", FileType, JacForm);.....	60
SaveYbusInMatlabFormat("filename", IncludeVoltages);	60
SolvePowerFlow (SolMethod, "filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);.....	60
UpdateIslandsAndBusStatus;.....	61
ZeroOutMismatches(ObjectType);	61
DeleteState(WhichState, StateName);	62
RestoreState(WhichState, StateName);	62
StoreState(StateName);	63
VoltageConditioning;.....	63
User Interface	64
Animate(DoAnimate);.....	64
MessageBox("text");	64
ObjectFieldsInputDialog("ObjectIDString", [fieldlist], "DialogCaption", "DialogExplain", [LabelCaptions], [TabBreaks], [TabCaptions], [RowBreaks], [RowCaptions], [ColBreaks], [ColCaptions]);.....	64
OpenDataView("ObjectIDString", "DataGridIDString");	67

Online Actions	68
CloseOnline("OnlineName");.....	68
EditMultipleOnlineAction("Path", LinkType, SaveFileType);.....	68
EnumerateDDLOnline("InputDSET", "OutputList");	68
ExportBusView("filename", "bus key", ImageType, Width, Height, [ExportOptions]);.....	68
ExportOnline("filename", "OnlineName", ImageType, "view", FullScreen, ShowFull, [ExportOptions]);	69
ExportOnlineAsShapeFile("filename", "OnlineName", "ShapeFileExportDescriptionName", UseLonLat, PointLocation);.....	70
ImportDDLAsTranslation("filename");.....	71
LoadAXD("filename", "OnlineName", CreatelfNotFound)	71
OpenOnline("filename", "view", FullScreen, ShowFull, LinkMethod, Left, Top, Width, Height);.....	71
RelinkAllOpenOnelines;	72
SaveOnline("filename", "OnlineName", SaveFileType);	72
OpenBusView("Bus key", ForceNewWindow);	72
OpenSubView("Substation key", ForceNewWindow);	72
Connections Tools.....	73
CreateNewAreasFromIslands;.....	73
DetermineBranchesThatCreatIslands(Filter, StoreBuses, "filename", SetSelectedOnLines, FileType);	73
DeterminePathDistance([start], BranchDistMeas, BranchFilter, BusField);.....	73
DetermineShortestPath([start], [end], BranchDistanceMeasure, BranchFilter, Filename);.....	74
DoFacilityAnalysis ("Filename", SetSelected);.....	75
FindRadialBusPaths(IgnoreStatus, TreatParallelAsNotRadial, BusOrSuperBus);	75
SetBusFieldFromClosest(variablename, BusFilterSetTo, BusFilterFromThese, BranchFilterTraverse, BranchDistMeas);	76
SetSelectedFromNetworkCut(SetHow, [BusOnCutSide], BranchFilter, InterfaceFilter, DCLineFilter, Energized, NumTiers, InitializeSelected, [ObjectsToSelect], UseAreaZone, UsekV, MinkV, MaxkV, LowerMinkV, LowerMaxkV);	77
Sensitivity Calculations	79
CalculateFlowSense([flow element], FlowType);	79
CalculateLODF([BRANCH nearbusnum farbusnum ckt], LinearMethod, PostClosureLCDF);	79
CalculateLODFAdvanced(IncludePhaseShifters, FileType, MaxColumns, MinLODF, NumberFormat, DecimalPoints, OnlyIncludingLinesIncreasing, "FileName", IncludeIslandingCTG);	79
CalculateLODFMatrix(WhichOnes, filterProcess, filterMonitor, MonitorOnlyClosed, LinearMethod, filterMonitorInterface, PostClosureLCDF);	80
CalculateLODFScreening(filterProcess, filterMonitor, IncludePhaseShifters, IncludeOpenLines, UseLODFThreshold, LODFThreshold, UseOverloadThreshold, OverloadLow, OverloadHigh, DoSaveFile, FileLocation, CustomFieldHighLODF, CustomFieldHighLODFLine, CustomFieldHighOverload, CustomFieldHighOverloadLine, DoUseCTGName, CustomFieldOrigCTGName);	81
CalculateLossSense(FunctionType,AreaSALossReference,IslandLossReference);	83
CalculatePTDF([transactor seller], [transactor buyer], LinearMethod);.....	84
CalculatePTDFMultipleDirections(StoreForBranches, StoreForInterfaces, LinearMethod);.....	84
CalculateShiftFactors([flow element], direction, [transactor], LinearMethod, SetOutOfServiceBuses, filter, AbortOnError, BranchDistMeas);	84
CalculateShiftFactorsMultipleElement(TypeElement,WhichElement,direction,[transactor],LinearMethod);.....	86
CalculateTapSense(filter);.....	86
CalculateVoltSelfSense(filter);.....	87
CalculateVoltSense([BUS num]);.....	87
CalculateVoltToTransferSense([transactor seller], [transactor buyer], TransferType, TurnOffAVR);.....	87
LineLoadingReplicatorCalculate([Flow Element], [Injection Group], AGOnly, DesiredFlow, Implement, LinearMethod, UseLoadMinMax, MaxMultiplier, MinMultiplier);.....	88
LineLoadingReplicatorImplement;	89
SetSensitivitiesAtOutOfServiceToClosest(filter, BranchDistMeas);	89
Contingency Analysis	90

CTGApply("ContingencyName");	90
CTGAutoInsert;	90
CTGCalculateOTDF([transactor seller], [transactor buyer], LinearMethod);	90
CTGClearAllResults;	90
CTGCloneMany(filter, "Prefix", "Suffix", SetSelected);	90
CTGCloneOne("ctgname", "newctgname", "Prefix", "Suffix", SetSelected);	91
CTGComboDeleteAllResults;	91
CTGComboSolveAll(DoDistributed, ClearAllResults);	91
CTGCompareTwoListsofContingencyResults (PRESENT or "ControllingFilename",PRESENT or "ComparisonFilename");	92
CTGConvertAllToDeviceCTG(KeepOriginalIfEmpty);	92
CTGConvertToPrimaryCTG(filter, KeepOriginal, "Prefix", "Suffix");	93
CTGCreateContingentInterfaces(filter, maxOption);	93
CTGCreateExpandedBreakerCTGs;	93
CTGCreateStuckBreakerCTGs(filter, AllowDuplicates, "PrefixName", IncludeCTGLabel, BranchFieldName, "SuffixName", "PrefixComment", BranchFieldComment, "SuffixComment");	94
CTGDeleteWithIdenticalActions;	95
CTGJoinActiveCTGs(InsertSolvePowerFlow, DeleteExisting, JoinWithSelf, "filename");	95
CTGPrimaryAutoInsert;	95
CTGProcessRemedialActionsAndDependencies(DoDelete, filter);	95
CTGProduceReport("filename");	95
CTGReadFilePSLF("filename");	96
CTGReadFilePTI("filename");	96
CTGRelinkUnlinkedElements;	96
CTGRestoreReference;	96
CTGSaveViolationMatrices("filename", filetype, UsePercentage, [ObjectTypesToReport], SaveContingency, SaveObjects, FieldListObjectType, [FieldList], IncludeUnsolvableCTGs);	96
CTGSetAsReference;	97
CTGSkipWithIdenticalActions;	97
CTGSolve("ContingencyName");	97
CTGSolveAll(DoDistributed, ClearAllResults);	97
CTGSort([SortFieldList]);	98
CTGVerifyIteratedLinearActions("filename");	98
CTGWriteAllOptions("filename", KeyField, UseSelectedDataMaintainer, SaveDependencies, UseAreaZoneFilters);	98
CTGWriteAuxUsingOptions("filename", Append);	99
CTGWriteFilePTI("filename", BusFormat, TruncateCTGLabels, "filename", Append);	99
CTGWriteResultsAndOptions("filename", [opt1, opt2, opt3, ..., opt22], KeyField, UseDATASection, UseConcise, UseObjectIDs, UseSelectedDataMaintainers, SaveDependencies, UseAreaZoneFilters);	100
Fault Analysis	102
Fault([Bus num], faulttype, R, X);	102
Fault([BRANCH nearbusnum farbusnum ckt], faultlocation, faulttype, R, X);	102
FaultAutoInsert;	102
FaultClear;	102
FaultMultiple(UseDummyBus);	102
LoadPTISEQData("filename", version);	102
ATC (Available Transfer Capability)	103
ATCCreateContingentInterfaces(filter);	103
ATCDeleteAllResults;	103
ATCDeleteScenarioChangeIndexRange(ScenarioChangeType, [IndexRange]);	103
ATCDetermine([transactor seller], [transactor buyer], DoDistributed, DoMultipleScenarios);	103
ATCDetermineMultipleDirections(DoDistributed, DoMultipleScenarios);	104
ATCDetermineATCFor(RL, G, I, ApplyTransfer);	104
ATCDetermineMultipleDirectionsATCFor(RL, G, I);	104

ATCIncreaseTransferBy(amount);	104
ATCRestoreInitialState;	104
ATCSetAsReference;	104
ATCTakeMeToScenario(RL, G, I);	104
ATCWriteAllOptions("filename", AppendFile, KeyField);	104
ATCDataWriteOptionsAndResults("filename", AppendFile, KeyField);	104
ATCWriteResultsAndOptions("filename", AppendFile);	105
ATCWriteScenarioLog("filename", AppendFile, filter);	105
ATCWriteScenarioMinMax("filename", filetype, AppendFile, [fieldlist], Operation, OperationField, GroupScenario, [RLFilter], [GFilter], [IFilter], DirectionFilter, DoGroupDirection, filter, PreferIterativelyFound);	106
ATCWriteToExcel("worksheetname", [fieldlist]);	108
ATCWriteToText("filename", filetype, [fieldlist]);	108
GIC (Geomagnetically Induced Current)	110
GICCalculate(MaxField, Direction, SolvePF);	110
GICClear;	110
GICLoad3DEfield(FileType, "FileName", SetupOnLoad);	110
GICReadFilePSLF("FileName");	110
GICReadFilePTI("FileName");	110
GICSaveGMatrix("GMatrixFileName", "GMatixIDFileName");	110
GICClear;	111
GICSensitivitiesCalculate(SensitivityType,FieldDirectionOption,FieldDirectionDeg,[BranchesToInclude]);	111
GICSetupTimeVaryingSeries(Start, End, Delta);	111
GICShiftOrStretchInputPoints(LatShift, LonShift, MagScalar, StretchScalar, UpdateTimeVaryingSeries);	111
GICTimeVaryingCalculate(TheTime,SolvePF);	112
GICTimeVaryingAddTime(NewTime);	112
GICTimeVaryingDeleteAllTimes;	112
GICTimeVaryingEFieldCalculate(TheTime,SolvePF);	112
GICTimeVaryingElectricFieldsDeleteAllTimes;	112
GICWriteFilePSLF("FileName", UseFilters);	112
GICWriteFilePTI("FileName", UseFilters, Version);	112
GICWriteOptions("FileName", KeyField);	113
ITP (Integrated Topology Processing)	114
CloseWithBreakers(objecttype, filter or [object identifier], OnlyEnergizeSpecifiedObjects, [SwitchingDeviceTypes], CloseNormallyClosedDisconnects);	114
ExpandAllBusTopology;	115
ExpandBusTopology(BusIdentifier, TopologyType);	115
OpenWithBreakers(objecttype, filter or [object identifier], [SwitchingDeviceTypes], OpenNormallyOpenDisconnects);	116
SaveConsolidatedCase("filename", filetype, [BusFormat, TruncateCtgLabels, AddCommentsForObjectLabels]);	117
OPF (Optimal Power Flow) and SCOPF	118
SolvePrimalLP("filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);	118
InitializePrimalLP("filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);	118
SolveSinglePrimalLPOuterLoop("filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);	119
SolveFullSCOPF (BCMMethod, "filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);	119
OPFWriteResultsAndOptions("filename");	120
PV Analysis	121
PVClear;	121
PVDDataWriteOptionsAndResults("filename", AppendFile, KeyField);	121
PVDestroy;	121
PVQVTrackSingleBusPerSuperBus;	121
PVRun([elementSource], [elementSink]);	121
PVSetSourceAndSink([elementSource], [elementSink]);	122
PVStartOver;	122

PVWriteInadequateVoltages("filename", AppendFile, InadequateType);	122
PVWriteResultsAndOptions("filename", AppendFile);	122
RefineModel(objecttype, filter, Action, Tolerance);	123
QV Analysis	124
QVDataWriteOptionsAndResults("filename", AppendFile, KeyField);	124
QVDeleteAllResults;	124
QVRun("filename", InErrorMakeBaseSolvable, DoDistributed);	124
QVSelectSingleBusPerSuperBus;	124
QVWriteCurves("filename", IncludeQuantitiesToTrack, filter, Append);	126
QVWriteResultsAndOptions("filename", AppendFile);	126
Regions	127
RegionLoadShapefile("FileName", "Class Name", [AttributeNames], AddToOpenOnelines, "DisplayStyleName", DeleteExistingShapes);	127
RegionRename("OldName", "NewName", UpdateOnelines);	127
RegionRenameClass("OldClassName", "NewClassName", UpdateOnelines, filter);	127
RegionRenameProper1("OldProper1Name", "NewProper1Name", UpdateOnelines, filter);	127
RegionRenameProper2("OldProper2Name", "NewProper2Name", UpdateOnelines, filter);	128
RegionRenameProper3("OldProper3Name", "NewProper3Name", UpdateOnelines, filter);	128
RegionRenameProper12Flip(UpdateOnelines, filter);	128
RegionUpdateBuses;	128
TS (Transient Stability)	129
TSAutoCorrect;	129
TSAutoInsertDistRelay(Reach, AddRelayAtFromBus, AddRelayAtToBus, DoTransferTrip, Shape, filter);	129
TSAutoInsertZPOTT(Reach, filter);	129
TSAutoSavePlots ([PlotNames], [ContingencyNames], ImageFileType, ImageWidth, ImageHeight, ImageFontScalar, IncludeCaseName, IncludeCategory);	129
TSCalculateCriticalClearTime([branch] or filter);	130
TSCalculateSMIBEigenValues;	130
TSClearAllModels;	130
TSClearModelsforObjects(ObjectType, Filter);	130
TSClearResultsFromRAM(ALL/SELECTED/"ContingencyName", ClearSummary, ClearEvents, ClearStatistics, ClearTimeValues, ClearSolutionDetails);	130
TSDisableMachineModelNonZeroDerivative(DerivativeThreshold);	131
TSGetVCurveData("FileName", filter);	131
TSGetResults("FileName", SINGLE/SEPARATE/JSIS, [Contingencies], [Plots, ObjectFields], StartTime, EndTime));	131
TSInitialize(CheckInitialized);	132
TSJoinActiveCTGs(TimeDelay, DeleteExisting, JoinWithSelf, "fileName", FirstCtg);	132
TSLoadBPA("FileName");	133
TSLoadGE("FileName", GENCCYN, EnableOutOfOrderModels);	133
TSLoadPTI("FileName", "MCREfilename", "MTRLOfilename", "GNETfilename", "BASEGENfilename", "MODREMOVEfilename");	133
TSLoadRDB("filename", ModelType, filter);	133
TSLoadRelayCSV("filename", ModelType, filter);	134
TSPlotSeriesAdd("PlotName", SubPlotNum, AxisGroupNum, ObjectType, FieldType, "Filter", "Attributes");	134
TSResultStorageSetAll(objecttype, YES/NO);	135
TSRunResultAnalyzer("ContingencyName");	135
TSRunUntilSpecifiedTime("ContingencyName", [StopTime, StepSize, StepsInCycles, ResetStartTime, NumberOfTimeStepsToDo]);	135
TSSaveBPA("FileName", DiffCaseModifiedOnly);	135
TSSaveGE("FileName", DiffCaseModifiedOnly);	136
TSSavePTI("FileName", DiffCaseModifiedOnly);	136
TSSaveTwoBusEquivalent ("FileName", [BUS]);	136
TSSolve("ContingencyName", [StartTime, StopTime, StepSize, StepsInCycles]);	136

TSSolveAll(DoDistributed);	137
TSSolveContinue;	137
TSTransferStateToPowerFlow(CalculateMismatch);.....	137
TSValidate;	137
TSWriteModels("FileName", DiffCaseModifiedOnly);	137
TSWriteOptions("FileName",[SaveDynamicModel, SaveStabilityOptions, SaveStabilityEvents, SaveResultsEvents, SavePlotDefinitions, SaveTransientLimitMonitors, SaveResultAnalyzerTimeWindow], KeyField);	137
TSSetSelectedForTransientReferences(SetWhat, SetHow, [ObjectType List],[ModelType List]);	138
TSSaveDynamicModels("FileName", FileType, ObjectType, Filter, Append);.....	138
Scheduled Actions	139
ApplyScheduledActionsAt(StartTime, EndTime, Filter, Revert);	139
IdentifyBreakersForScheduledActions(IdentifyFromNormalStatus);.....	139
RevertScheduledActionsAt(StartTime, EndTime, Filter);	139
ScheduledActionsSetReference;.....	140
SetScheduleView(ViewTime, ApplyActions, UseNormalStatus, ApplyWindow);.....	140
SetScheduleWindow(StartTime, EndTime, Resolution, ResolutionUnits);.....	140
Time Step Simulation	141
TimeStepAppendPWW("FileName","SolutionTypeString")	141
TimeStepAppendPWWRange("FileName", ISO8601StartDateTime, ISO8601EndDateTime, "SolutionTypeString")..	141
TimeStepAppendPWWRangeLatLon("FileName", ISO8601StartDateTime, ISO8601EndDateTime, minLatitude,maxLatitude,minLongitude,maxLongitude,"SolutionTypeString")	142
TimeStepClearResults(ISO8601StartDateTime, ISO8601EndDateTime);	142
TimeStepDeleteAll;	142
TimeStepDoRun(ISO8601StartDateTime,ISO8601EndDateTime)	143
TimeStepDoSinglePoint(ISO8601Dateime)	143
TimeStepLoadB3D("FileName","SolutionTypeString")	143
TimeStepLoadPWW("FileName","SolutionTypeString")	143
TimeStepLoadPWWRange("FileName",ISO8601StartDateTime,ISO8601EndDateTime, "SolutionTypeString").....	144
TimeStepLoadPWWRangeLatLon("FileName",ISO8601StartDateTime,ISO8601EndDateTime, minLatitude,maxLatitude,minLongitude,maxLongitude, "SolutionTypeString")	144
TimeStepLoadTSB("FileName").....	145
TimeStepResetRun.....	145
TimeStepSaveFieldsClear([objecttype])	145
TimeStepSaveFieldsSet(objecttype, [fieldlist], filter).....	145
TimeStepSaveFieldsSetByObject(objecttype, [fieldlist], [objectIDList]).....	146
TIMESTEPSaveSelectedModifyStart;	146
TIMESTEPSaveSelectedModifyFinish;	146
TIMESTEPSaveInputCSV("Filename", [input field list], ", ISO8601StartDateTime, ISO8601EndDateTime).....	147
TimeStepSaveResultsByTypeCSV(ObjectType, "FileCSVName", ISO8601StartDateTime, ISO8601EndDateTime)	147
TimeStepSaveTSB(FileName).....	148
TimeStepSavePWW("FileName")	148
TimeStepSavePWWRange("FileName",ISO8601StartDateTime,ISO8601EndDateTime).....	148
Weather	149
TemperatureLimitsBranchUpdate(RatingSetPrecedence, NormalRatingSet, CTGRatingSet);	149
WeatherLimitsGenUpdate(UpdateMax, UpdateMin);.....	149
WeatherPFWModelsSetInputs;	150
WeatherPFWModelsSetInputsAndApply (SolvePowerFlow).....	150
WeatherPWWFileAllMeasValid ("PWWFileName", [field list], ISO8601StartDateTime, ISO8601EndDateTime).....	150
WeatherPFWModelsRestoreDesignValues;	151
WeatherPWWFileCombine2 ("SourceFileName1", "SourceFileName2", "DestFileName")	151
WeatherPWWFileGeoReduce("SourceFileName", "DestFileName", minLatitude, maxLatitude, minLongitude, maxLongitude);.....	151
WeatherPWWSetDirectory("PWWFileDirectory", IncludeSubDirectories);	152

WeatherPWWLoadForDateTimeUTC(ISO8601DateTime);	152
Distributed Computing.....	153
EnterDistMasterPassword(Password);	153
VerifyDistributedComputersAvailable;.....	153
Trainer	154
ResetStatusChangeCount;	154
SuppressCommandDialog;.....	154
Customer-Specific Actions.....	155
ISONEInterfaceLimitCalculation("filename", variablename);	155
DATA Section	157
Concise Auxiliary File Header	157
Legacy Auxiliary File Header	157
ObjectType	157
File_Type_Specifier.....	158
Required Fields and Create_if_not_found	158
List_of_Fields.....	158
Concise Field Variable Names	159
Legacy Field Variable Naming	159
Special Naming	159
Key Fields.....	159
Data List.....	160
Special Data List Entries	160
Special Identifiers for Model Fields in Data.....	160
Using Labels for Identification.....	162
Saving Auxiliary Files Using Labels	162
Loading Auxiliary Files SUBDATA Sections Using Labels.....	163
Special Use of Labels in SUBDATA.....	163
ATC Scenarios.....	163
ATC Extra Monitors.....	163
Model Conditions	164
Model Expressions.....	164
Bus Load Throw Over Records.....	164
Injection Group Participation Points	164
SubData Sections	165
ATC_Options	166
RLScenarioName	166
GScenarioName	166
IScenarioName	166
ATCMemo	166
ATCExtraMonitor.....	166
ATCFlowValue.....	166
ATCScenario.....	167
TransferLimiter	167
ATCExtraMonitor.....	167
AUXFileExportFormatData.....	168

DataBlockDescription	168
AUXFileExportFormatDisplay	168
DataBlockDescription	168
BGCalculatedField	168
Condition	168
Bus	169
MWMarginalCostValues	169
MvarMarginalCostValues	169
LPOPfMarginalControls	169
BusViewFormOptions	169
BusViewBusField	169
BusViewFarBusField	169
BusViewGenField	169
BusViewLineField	169
BusViewLoadField	169
BusViewShuntField	169
ColorMap	170
ColorPoint	170
Contingency	170
CTGElementAppend	170
CTGElement	170
LimitViol	182
Sim_Solution_Options	183
WhatOccurredDuringContingency	183
ContingencyMonitoringException	183
CTG_Options	183
Sim_Solution_Options	183
CTGElementBlock	184
CTGElement	184
CTGElementAppend	184
CustomColors	184
CustomColors	184
CustomCaseInfo	184
ColumnInfo	184
DataGrid	184
ColumnInfo	184
ColumnContourInfo	185
DynamicFormatting	186
DynamicFormattingContextObject	186
LineThicknessLookupMap	187
LineColorLookupMap	187
FillColorLookupMap	187
FontColorLookupMap	187
FontSizeLookupMap	187
BlinkColorLookupMap	187
XoutColorLookupMap	187
FlowColorLookupMap	187
SecondaryFlowColorLookupMap	187
Filter	188
Condition	188
Gen	189
BidCurve	189
ReactiveCapability	189

GeoDataViewStyle	190
TotalAreaValueMap.....	190
RotationRateValueMap.....	190
RotationAngleValueMap	190
LineThicknessValueMap	191
GlobalContingencyActions.....	191
CTGElementAppend.....	191
CTGElement.....	191
HintDefValues.....	191
HintObject.....	191
InjectionGroup	192
PartPoint	192
Interface.....	193
InterfaceElement.....	193
KMLExportFormat.....	194
DataBlockDescription	194
LimitSet	194
LimitCost.....	194
Load.....	194
BidCurve.....	194
LPVariable	195
LPVariableCostSegment	195
ModelCondition	195
Condition.....	195
ModelExpression.....	195
LookupTable.....	195
ModelFilter	196
ModelCondition.....	196
MTDCRecord.....	197
MTDCBus.....	197
MTDCConverter	197
MTDCTransmissionLine	198
MultiSectionLine.....	199
Bus.....	199
BusRenumber	200
Nomogram	200
InterfaceElementA.....	200
InterfaceElementB.....	200
NomogramBreakPoint	201
NomogramInterface	201
InterfaceElement.....	201
Owner	201
Bus.....	201
Load.....	201
Gen.....	201
Branch	202
PostPowerFlowActions.....	202
CTGElementAppend.....	202
CTGElement.....	202
PWCaseInformation	202
PWCaseHeader.....	202
PWFormOptions.....	202
PieSizeColorOptions	202

PWLPOPCTGViol	203
OPFControlSense	203
OPFBusSenseP	203
OPFBusSenseQ	203
PWLPTabRow	203
LPBasisMatrix	203
PWPVResultListContainer	204
PWPVResultObject	204
LimitViol	204
PVBusInadequateVoltages	204
PWQVResultListContainer	205
PWPVResultObject	205
QVCurve	205
QVPoints	205
QVCurve_Options	206
Sim_Solution_Options	206
RemedialAction	206
CTGElementAppend	206
CTGElement	206
SelectByCriteriaSet	206
SelectByCriteriaSetType	206
Area	207
Zone	207
ScreenLayer	207
ShapefileExportDescription	207
StudyMWTransactions	207
ImportExportBidCurve	207
SuperArea	208
SuperAreaArea	208
TSSchedule	208
SchedPoint	208
UserDefinedDataGrid	209
ColumnInfo	209

SCRIPT Section for Display Auxiliary File..... 210

AXD Actions.....	210
AutoInsertBorders;	210
AutoInsertBuses(LocationSource, MapProjection, AutoInsertBranches, InsertIfNotAlreadyShown, "filename", InsertSelected);	210
AutoInsertGens(MinkV, InsertTextFields);	210
AutoInsertInterfaces(InsertPieCharts, PieChartSize);	210
AutoInsertLineFlowObjects(MinkV, InsertOnlyIfNotAlreadyShown, LineLocation, Size, FieldDigits, FieldDecimals, TextPosition, ShowMW, ShowMvar, ShowMVA, ShowUnits, ShowComplex);	211
AutoInsertLineFlowPieCharts(MinkV, InsertOnlyIfNotAlreadyShown, InsertMSLines, Size);	211
AutoInsertLines(MinkV, InsertTextFields, InsertEquivObjects, InsertZBRPieCharts, InsertMSLines, ZBRImpedance, NoStubsZBRs, SingleCBZR);	211
AutoInsertLoads(MinkV, InsertTextFields);	212
AutoInsertSwitchedShunts(MinkV, InsertTextFields);	212
AutoInsertSubStations(LocationSource, MapProjection, AutoInsertBranches, InsertIfNotAlreadyShown, "filename", InsertSelected);	212
AutoInsertAreas(MapProjection, InsertIfNotAlreadyShown, InsertSelected);	213
AutoInsertInjectionGroups(MapProjection, InsertIfNotAlreadyShown, InsertSelected);	213
AutoInsertOwners(MapProjection, InsertIfNotAlreadyShown, InsertSelected);	213

AutoInsertZones(MapProjection, InsertIfNotAlreadyShown, InsertSelected);	214
FixFlowArrowLineEnds("OnlineName", "LayerName");	214
FixFlowArrowPosition("OnlineName", "LayerName");	214
InsertConnectedBuses("BusIdentifier");	215
LoadAXDFromAXD("filename", CreateIfNotFound)	215
PanAndZoomToObject("ObjectID", "DisplayObjectType", "DoZoom");	215
ResetStubLocations(ZBRImpedance, NoStubsZBRs);	216
General Script Commands	216
DATA Section for Display Auxiliary File	217
Key Fields	217
Special Data Sections	217
GeographyDisplayOptions	217
Picture	217
PWFormOptions	217
View	218
SubData Sections	218
ColorMap	218
CustomColors	218
DisplayDCTramissionLine	219
DisplayInterface	219
DisplayMultiSectionLine	219
DisplaySeriesCapacitor	219
DisplayTransformer	219
DisplayTransmissionLine	219
Line	219
Line	219
DynamicFormatting	219
Filter	219
GeoDataViewStyle	219
PieChartGaugeStyle	220
ColorMap	220
PWFormOptions	220
SelectByCriteriaSet	220
UserDefinedDataGrid	220
View	220
ScreenLayer	220

Introduction

PowerWorld has incorporated the ability to import data to/from data sources other than power flow models into PowerWorld Simulator. The text file interface for exchanging data, as well as for executing a batch script command, is represented by the auxiliary files. The script language and auxiliary data formats are incorporated together. This format is described in this document.

Script/Data files are called data auxiliary files in Simulator and typically have the file extension .AUX. These files mostly contain information about power system elements and options for running the various tools within Simulator. They do not contain any information about the individual display objects contained on a one-line diagram. There are separate files called display auxiliary files that are available for importing display data to/from Simulator in a text format. These files are distinguished from the data auxiliary files by using the extension .AXD. The format for these two types of files is similar, but different object types are supported by each and require that the files be read separately.

Both file types will be generically referred to as auxiliary files. An auxiliary file may be comprised of one or more data or script sections. A data section provides specific data for a specific type of object. A script section provides a list of script actions for Simulator to perform. These sections have the following format:

```
SCRIPT ScriptName1
{
script_statement_1;
.
script_statement_n;
}

object_type DataName1(list_of_fields)
{
data_list_1
.
data_list_n
}

object_type DataName2(list_of_fields)
{
data_list_1
.
data_list_n
}

SCRIPT ScriptName2
{
script_statement_1;
.
script_statement_n;
}
```

Data sections start with the string representing the object_type, but older auxiliary files start with the syntax DATA DataName(object_type, [list_of_fields])

See the description of the data section later in this document for more information.

Note that the keywords script, data, or a valid object_type must occur at the start of a text file line. Auxiliary files may contain more than one section. These sections always begin with the keyword script, data, or a valid object_type. Data sections are followed by an argument list enclosed in (). The actual data or script commands are then contained within curly braces { }. Strings are enclosed in straight quotes – note that smart quotes will not work (this might be encountered when copy/pasting script commands from another program). The Script commands available are described in the next main section. The data sections are then described after this. There are separate sections for describing the data sections for the data auxiliary files and the display auxiliary file.

SCRIPT Section

```
SCRIPT ScriptName
{
script_statement_1;
.
script_statement_n;
}
```

Scripts may optionally contain a ScriptName. This enables you to call a particular SCRIPT by using the LoadScript action (see General Actions). After the optional name, the SCRIPT section begins with a left curly brace and ends with a right curly brace. Inside of this, script statements can be given. In general, a script statement has the following format

```
Keyword(arg1, arg2, ...);
```

- Statement starts with a keyword.
- The keyword is followed by an argument list which is encompassed in parentheses ().
- The arguments are separated by commas.
- If a single argument is a list of things, this list is encompassed by braces [].
- Statements end with a semicolon.
- Statements may take up several lines of the text file.
- You may put more than one statement on a single text line.

Those familiar with using Simulator will know that there is a RUN and EDIT mode in Simulator. Some features in Simulator are only available in one mode or the other. This functionality will be preserved in the script language. In earlier versions of the software, certain functionality was organized by the "submode" feature. While existing scripts designed to work with submodes will still function as before, moving between submodes is no longer necessary.

Various script commands require that you be in RUN or EDIT mode. If a script requires this, then the script will automatically change modes.

Using Filters in Script Commands

Many script commands allow the specification of a filename. Only those objects meeting this filter will be selected for the specified action. Unless otherwise specified, a blank filter will select all objects. This filename can be the name of an advanced filter. The filename should be enclosed in double quotes. Advanced filters belonging to a different objecttype can also be used depending on the objecttype in use. For example, if filtering generator objects a bus filter can also be used. When using an advanced filter that belongs to a different objecttype, the format of the filter is "<Objecttype>filename" instead of just specifying the filename itself.

The filename can also be the name of a device filter. A device filter allows you to specify a particular object for filtering instead of a class of object. For example, you might want to return all buses that belong to a particular substation. You can specify the device filter for the particular substation and then apply this to the bus objects. The format of a device filter is "<DEVICE>objecttype 'key1' 'key2' 'key3'".

In addition to a filename, special keywords can be used to indicate the type of filter desired. These include the following:

AREAZONE : Only objects that meet the area/zone/owner filters will be selected for the specified action.

SELECTED : Only objects whose Selected field is YES will be selected for the specified action.

(Added in October 5, 2023 patch of Simulator 23) FilterName may also be a special string with the syntax such as "MW >= 50" representing a single-condition filter. The syntax is `VariableName Comparison Value1 Value2`. These special single-condition filters are convenient to use because there is no need to create a new filter object, and instead one is created inside the software for temporary use. Examples for a Bus would be the following:
"NomkV between 220 550"
"MW >= 50"

Specifying Special Keywords in Script Command Parameters

There are special keywords that may be used as part of the input for script command parameters. Generally, these are allowed as part of the file name input in commands that save or modify files, as well as some other commands that take text as input. These special keywords will be replaced with their actual values when the script command is processed.

The special keywords that are allowed and their associated replacement strings are shown in the following table:

@BUILDDATE	Simulator patch build date
@CASEFILENAME	File name of the presently open case. This is only the file name without the path.
@CASEFILEPATH	Directory path of the presently open case
@CASENAME	Name of the presently open case including the path and file name
@DATE	Present date
@DATETIME	Actual date and time in the format yyyyymmdd_hhnnss-hhmm with the UTC offset included on the end of the time
@TIME	Present time
@VERSION	Simulator version number

The special keyword @MODELFIELD can be used in combination with an object type and variable name so that any field of any object can be included in the text. The syntax for inserting the value of a model field in the text is the following:

@MODELFIELD<objecttype 'key1' 'key2' variablename:digits:rod>.

Specifying File Names in Script Commands

In place of the "filename" parameter in any script command, specially formatted text can be used to indicate that the user should be prompted to choose the file. Depending on whether or not a file is being opened or saved, an Open or Save dialog will be presented for the user to choose the file. This will not work when using the SimAuto Add-on.

The special syntax of the filename parameter is generally:

"<PROMPT 'Caption' 'FileTypes' 'InitialDirectory' 'CancelAction'>"

The entire string must start with <PROMPT and end with >. The parameters after PROMPT are all optional, must be space delimited, must be enclosed in single quotes, and can be specified as follows:

Caption

Caption to be placed at the top of the file dialog that appears. If omitted, either 'Save' or 'Open' is assumed based on how the prompt is used to access a file.

FileTypes

List of file types and extensions. The list itself is composed of a pipe-delimited string (|) with the first string representing the first file type, the second string representing the first file extension, the third string representing the second file type, the fourth string representing the second file extension and so on. If no file types are specified, 'All Files (*.*)|*.*' is assumed.

InitialDirectory

The initial directory in which the dialog should open.

CancelAction

The CancelAction can either be 'Abort' or 'Continue', with 'Abort' being the default. If the Cancel button is clicked on the resulting dialog, the CancelAction specifies how to proceed. Setting this to 'Abort' will skip the command and abort any remaining auxiliary file commands. Setting this to 'Continue' will skip the command but continue processing the remaining auxiliary file commands.

Here is an example prompt using all options:

"<PROMPT 'Choose an AUX file' 'Auxiliary Files (*.aux)|*.aux|All Files (*.*)|*.*' 'c:\StudyDirectory\AUX Files' 'Abort'>"

The special keywords described in the [Specifying Special Keywords in Script Command Parameters](#) section can also be used as part of a filename.

Specifying Field Variable Names in Script Commands

See the [Field Variable Naming \(Legacy\)](#) topic in the DATA Section for general information about naming fields.

Within select script commands the keyword ALL can be used instead of using the location number of a field when specifying variable names as part of a field list. This will return all fields with the same variable name. This is intended to allow easier access to fields when the exact number of fields is not known, such as with multiple TLR (MultBusTLRSens:ALL) or PTDF (LinePTDFMult:ALL) results. This can be used with SaveData, SaveDataWithExtra, SaveObjectFields, and SendToExcel script actions.

Within select script commands the keyword ALL can be used instead of a list of fields. This will return all fields for a particular objecttype. This can be used with SaveData, SaveDataWithExtra, SaveObjectFields, and SendToExcel script actions.

Within these same Save related script commands for any transient stability data model objecttype (Machines, Exciters, etc...), the special strings may be used to export all the model input parameters for the stability model and the various primary, secondary or label key fields. These special strings are as follows.

AllModelParams	exports all the model input parameters and references to objects necessary to define model
AllModelParamsPriKey	exports the primary keys for the objecttype and the same fields as AllModelParams
AllModelParamsSecKey	exports the secondary keys for the objecttype and the same fields as AllModelParams
AllModelParamsLabel	exports the label key for the objecttype and the same fields as AllModelParams

Specifying Field Values in Script Commands

Several script commands require that a valuelist be specified to assign values to a corresponding fieldlist. Instead of specifying the values explicitly, special formatting is available to assign values from other fields. See the [Special Data List Entries](#) topic in the DATA Section for more information.

AUX Actions

General Program Actions

CopyFile("oldfilename", "newfilename");

Use this action to copy a file from within a script.

- "oldfilename" : The present file name. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- "newfilename" : The new file name desired. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

This command duplicates the file named "B7FLAT.pwb" and saves the copy as "B7FLAT_BACKUP.pwb" in the same directory.
--

CopyFile("B7FLAT.pwb", "B7FLAT_BACKUP.pwb");
--

DeleteFile("filename");

Use this action to delete a file from within a script.

- "filename" : The file name to delete. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

This call deletes the file named "B7FLAT_BACKUP.pwb".

DeleteFile("B7FLAT_BACKUP.pwb");

ExitProgram;

Immediately exits the program with no prompts. Note that this command should not be used in scripts loaded through SimAuto, as the SimAuto instance should be managed from the script which created it.

LogAdd("text");

Use this action to add a personal message to the Message Log.

- "text" : The text that will appear as a message in the log.

This command adds the greeting "hello" in the message log.
--

LogAdd("hello");

LogAddDateTime("label", includedate, includetime, includemilliseconds);

Use this action to add the date and time to the message log

- "label" : A string which will appear at the start of the line containing the date/time.
- includedate : YES – Include the data or NO to not include.
- includetime : YES – Include the time or NO to not include.
- includemilliseconds : YES – Include the milliseconds or NO to not include.

This command adds a log entry labeled "DateTime" to the message log, including the current date, time, and milliseconds.
--

LogAddDateTime("DateTime", YES, YES, YES);
--

LogClear;

Use this action to clear the Message Log.

LogSave("filename", AppendFile);

This action saves the contents of the Message Log to "filename".

- "filename" : The file name to save the information to.
- AppendFile : Set to YES or NO. YES means that the contents of the log will be appended to "filename". NO means that "filename" will be overwritten.

This command saves the current contents of the Message Log to a file named "LogFile1", overwriting any existing data in that file. The NO parameter ensures that the file is replaced rather than appended to.

```
LogSave("LogFile1", NO);
```

LogShow(DoShow);

This action will show or hide the Message Log.

- DoShow : Set to YES to show the Message Log. Set to NO to hide the Message Log.

RenameFile("oldfilename", "newfilename");

Use this action to rename a file from within a script.

- "oldfilename" : The present file name. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- "newfilename" : The new file name desired. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

This command renames the file "results_old.csv" to "results_final.csv".

```
RenameFile("results_old.csv", "results_final.csv");
```

SetCurrentDirectory("filedirectory", CreatelfNotFound);

Use this action to set the current work directory.

- "filedirectory" : The path of the working directory. See the Specifying Special Keywords in Script Command Parameters and Special Identifiers for Model Fields section for information on special keywords that can be used as part of the directory name.
- CreatelfNotFound : Set to YES or NO. YES means that if the directory path cannot be found, the directory will be created. If this parameter is not specified, NO is assumed.

StopAuxFile;

Use this action to treat the remainder of the file after the command as a big comment. This includes any script commands inside the present SCRIPT block, as well as all remaining SCRIPT or DATA blocks.

WriteTextToFile("filename", "text");

Use this action to write text to a file. If the specified file already exists, the text will be appended to the file. Otherwise, it creates the file and writes the text to the file.

- "filename" : The file path and name to save.
- "text" : The text to be written to the file. Special keywords can be entered that will be replaced with their actual values. These include @BUILDDATE, @DATETIME, @DATE, @TIME, @VERSION, and @CASENAME.

Data Interaction

CustomFieldDescriptionAppend(ObjectType, CustomType, FieldString, HeaderString, IncludeInDiff);

(This command was added in the December 4, 2023 patch of Simulator 23)

This command behaves the same as calling CustomFieldDescriptionModify but the Location is assumed to be a negative number. This creates a new CustomFieldDescription by incrementing the CustomMaxOfType.

CustomFieldDescriptionModify(ObjectType, CustomType, Location, "FieldString", "HeaderString", IncludeInDiff);

(This command was added in the December 4, 2023 patch of Simulator 23)

Use this script command to edit the CustomFieldDescription entries for a particular ObjectType/CustomType/Location.

ObjectType	: The object type to which the Custom Field Description is applied
CustomType	: Either Integer, String or Floating Point
Location	: Integer value for the location number of the custom field. Entering a negative value for Location will increment the present CustomMaxOfType by 1 for the present ObjectType/CustomType and this newly added entry will be edited by the command. Remember that "Custom Integer 4" corresponds to CustomInteger:3 and for that example you would use a Location of 3. If a location is chosen that is too large, then the CustomMaxOfType will be increased to accommodate the location.
FieldString	: String shown when field is displayed. Remember that a \ results subfolder being created under the Custom folder. Enter a value of "_same_" if you do not want to change the string.
HeaderString	: String shown in column headers in case information displays. Enter a value of "_same_" if you do not want to change the string.
IncludeInDiff	: This specifies if the field is included in Difference Case comparisons. Set to YES, NO, or "", where a blank indicates that the value is not changed.

This command modifies the description of a custom string field at location 3 for bus objects. The location numbers are zero-indexed meaning that location 3 refers to the fourth custom string field. This sets the internal field name to "Custom_Region_v2", the header displayed in case info to "Updated_Region_Label", and enables inclusion in difference case comparisons (YES).

```
CustomFieldDescriptionModify(BUS, String, 3, "Custom_Region_v2",  
"Updated_Region_Label", YES);
```

CreateData(objecttype, [fieldlist], [valuelist]);

Use this action to create particular objects.

objecttype	: The objecttype being created.
[fieldlist]	: A list of fields to set with the object. The key fields and required fields must be specified.
[valuelist]	: A list of values corresponding to the respective fields.

This command creates a new Bus object in the case with the specified attributes. The bus is named "Eight", assigned the number 8, placed in Area 1 and Zone 1, and given a nominal voltage of 138.00 kV.

```
CreateData(BUS, [Name, Number, AreaNumber, ZoneNumber, NomkV,  
NameNomkV], [Eight, 8, 1, 1, 138.0, Eight_138.0]);
```

Delete(objecttype, filter);

Use this delete objects of a particular type. A filter may optionally be specified to only delete objects that meet a filter.

- objecttype : The objecttype being selected.
- filter : Optional parameter – default is to delete all objects of specified type
See [Using Filters in Script Commands](#) section for more information on specifying the filter.

This command deletes all Buses where the Selected field is YES.

Delete(BUS, SELECTED);

DeleteDevice([ObjectIDString]);

Use this action to delete a specific object.

- [ObjectIDString] : The specific object to delete. The format is the object type followed by the key fields used to identify the object. Examples: DeleteDevice([Bus 234891]), DeleteDevice([Branch 1239 1234 "AB"]), and DeleteDevice([Interface "my interface name"]).

This command deletes Bus 1.

DeleteDevice([Bus 1]);

DeleteIncludingContents(objecttype, filter);

Use this to delete objects of a particular type and other objects that these contain. Currently, only multi-section lines (objecttype = MultiSectionLine) can be used with this command. The branches and dummy buses that belong to multi-section lines will also be deleted along with the multi-section lines. A filter may optionally be specified to only delete objects that meet a filter. The syntax is identical to the Delete(objecttype, filter); action above.

This command deletes all multi section lines along with the branches and dummy buses associated with them.
--

DeleteIncludingContents(MultiSectionLine,);
--

ExportAreaSupplyCurves("filename", "User Defined String", NumPoints);

Use this action to export Area Supply Curves to a CSV file. The output of the file will have 7 entries for each area for Fixed Gen MW, Fixed Load MW, Fixed Shunt MW, Losses MW, Variable Min MW, Variable Max MW, Variable Present MW, followed by a set of Bid MW/Price entries represents the supply curve for the variable MWs.

- "filename.csv" : The name of the CSV file to which results will be written.
- "User Defined String" : This is an optional parameter for specifying a user defined string written to each entry in the resulting CSV file. If this is omitted, blank will be assumed.
- NumPoints : This is an optional parameter and is related to converting a cubic cost model into a piece-wise linear model. If this is omitted, 5 is the default.

This command exports area supply curve data to the file TestAreaSupply.csv, tagging each entry with "ScenarioA" and using 20 points to linearize the variable MW cost curves. The output includes fixed generation, load, shunt values, losses, and a detailed bid-based supply curve for each area.
--

ExportAreaSupplyCurves("TestAreaSupply.csv", "ScenarioA", 20);
--

ImportData("filename", FileType, HeaderLine, CreatelfNotFound);

Use this action to import data in various file formats that are not native to Simulator.

- "filename" : Name of the file to import
- FileType : Parameter that specifies the format of the data this is being read.
Currently supported are two methods of importing CROW files as created by the Equinox Control Room Operations Window application. This is used with the Scheduled Actions add-on tool.
- CSV : Uses CSV Import Settings as specified in the Scheduled Actions dialog to read in a CROW CSV file.
 - PWCSV : Uses the PowerWorld Outage CSV format.
 - CROW : Uses the hardcoded format Scheduled Actions was originally programmed to import.
- HeaderLine : Optional parameter to specify if the row of headers in the CSV file is on the first line (1) or second line (2). If left blank (or any other value is specified), it will use the setting last configured in the Scheduled Actions dialog.
- CreatelfNotFound : Optional parameter that is NO by default. Set this to YES to create objects defined in the data if they do not already exist.

This command imports data from the file "GenFields.csv" using the CSV format. It assumes the header row is on the first line of the file and will create any new objects from the data if they don't already exist in the case.

```
Importdata("GenFields.csv", CSV, 1, YES);
```

LoadAux("filename", CreatelfNotFound);

Use this action to load another auxiliary file from within a script.

- "filename" : The filename of the auxiliary file being loaded.
- CreatelfNotFound : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename".

This command loads the auxiliary file named "ThreeBuses.aux" into the current simulation case. By setting the CreatelfNotFound parameter to YES, any objects contained in Legacy Auxiliary File Header DATA sections that do not already exist in the case will be created during the loading process.

```
LoadAux("ThreeBuses.aux", YES);
```

LoadAuxDirectory("filedirectory", "filterstring", CreatelfNotFound);

Use this action to load multiple auxiliary files from a specified directory. The auxiliary files will be loaded in alphabetical order by name.

- "filedirectory" : The directory where the auxiliary files are located.
- "filterstring" : Optional – if not specified then all files in the directory are loaded. If specified, only files meeting this filter will be loaded. This filtering supports normal Windows wildcard filtering.
- CreatelfNotFound : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.

This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from the files.

This command loads all .aux files from the "C:\SimCases\AuxFiles" directory in alphabetical order. It sets CreatelfNotFound to YES, meaning that any objects contained in Legacy Auxiliary File Header DATA sections in the files that do not already exist in the case will be created during the loading process.

```
LoadAuxDirectory("C:\SimCases\AuxFiles", "*.aux", YES);
```

LoadCSV("filename", CreatelfNotFound);

Use this action to load a CSV file that is formatted the same as the data sent to Excel in the Send All to Excel option found within a case information display, or by choose Save As CSV.

"filename" : The filename of the CSV file being loaded.
CreatelfNotFound : Set to YES or NO. YES means that objects which cannot be found will be created. If this parameter is not specified, NO is assumed.

This command loads data from the CSV file named "3BusCSV.csv". The YES parameter means that if any objects referenced in the CSV file do not already exist in the case, they will be created during the loading process.

```
LoadCSV("3BusCSV.csv", YES);
```

LoadData("filename", DataName, CreatelfNotFound);

Use this action to load a named Data Section from another auxiliary file. This will open the auxiliary file denoted by "filename", but will only read the data section specified.

"filename" : The filename of the auxiliary file being loaded.
DataName : The specific data section name from the auxiliary file that should be loaded.
CreatelfNotFound : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename".

This command loads only the DATA section named "BusData" from the auxiliary file "ScenarioData.aux". By setting the CreatelfNotFound parameter to YES, any objects contained in Legacy Auxiliary File Header DATA sections that do not already exist in the case will be automatically created during the loading process.

```
LoadData("ScenarioData.aux", "BusData", YES);
```

LoadScript("filename", ScriptName, CreatelfNotFound);

Use this action to load a named Script Section from another auxiliary file. This will open the auxiliary file denoted by "filename", but will only execute the script section specified.

"filename" : The filename of the auxiliary file being loaded.
ScriptName : The specific script section name from the auxiliary file that should be loaded.
CreatelfNotFound : This parameter is ignored and always assumed to be NO. This does not mean that objects loaded from DATA sections will not be created. The default behavior for how objects are created is followed depending on

DATA sections being specified in the Concise Auxiliary File Header or the Legacy Auxiliary File Header format. [See DATA Section for more details.](#)

This command opens the "Operations.aux" file and executes only the script section named "RestoreSettings".

```
LoadScript("Operations.aux", "RestoreSettings");
```

SaveData("filename", filetype, objecttype, [fieldlist], [subdatalist], filter, [SortFieldList], Transpose, Append);

Use this action to save data in a custom defined format.

- "filename" : The file path and name to save.
- filetype : There are several options for the filetype
 - AUXCSV : save as a comma-delimited auxiliary data file.
 - AUX : save as a space-delimited auxiliary data file.
 - CSV : save as a normal CSV file without the AUX file syntax. The first few lines of the text file will represent the object name and field variable names.
 - CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included. This option is useful when appending data of the same object type and field list into a common file.
 - CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing column headers you would see in a case information display
- objecttype : The type of object being saved.
- [fieldlist] : A list of fields that you want to save. For numeric fields, the number of digits and the number of decimal places (digits to right of decimal) can be specified by using the following format for the field, `variablenamelegacy:location:digits:rod` or `concisename:digits:rod`. See the [Specifying Field Variable Names in Script Commands](#) topic for more information on specifying this list.
- [subdatalist] : A list of the subdata objecttypes to save with each object record.
- filter : Optional parameter – default is to save all objects of specified type
 - blank : Save all objects of the specified type
 - AREAZONE : Only objects that meet the area/zone/owner filters will be saved
 - SELECTED : Only objects whose Selected field = YES will be saved
 - "FilterName" : Only objects that meet the specified filter will be checked. See [Using Filters in Script Commands](#) section for more information on specifying the filename.
- [SortFieldList] : Optional parameter – the default is to do no sorting
 This allows the specification of a sort order in which the data will be saved. The format is: `[variablename1:+:0, variablename2:-:1]` where
 variablename : the first parameter is the name of the field by which to sort. There is no limit to how many fields can be specified for sorting. For fields that require a location other than zero, variablename can be in the format `fieldname:location`.

	+ or -	: the second parameter indicates sort ascending for + and sort descending for -. This parameter must be specified.
	0 or 1	: for the third parameter 0 means case insensitive and do not use absolute value, 1 mean case sensitive or use absolute value. This parameter is optional.
Transpose		: Optional parameter – default is NO Set this to YES or NO. Set to YES to transpose the columns and rows of the returned data. When transposed the values for the same field for all selected objects will appear in the same row. Transposing the data is only allowed for CSV filetypes and this option will default to NO for all other filetypes.
Append		: Optional parameter – default is YES Set this to YES or NO. Set to YES to append data to an existing file. Set to NO to overwrite an existing file.

This command saves data for selected buses into the file "BusData.csv" using the CSVCOLHEADER format. It includes the bus Number, Name, and NomkV fields. It does not transpose the data or append it to an existing file—any existing file with the same name will be overwritten.

```
SaveData("BusData.csv", CSVCOLHEADER, BUS, [Number, Name, NomkV], [],  
SELECTED, [], NO, NO);
```

SaveDataEPC("filename", objecttype, filter, GEFileType, SaveBuses, Append);

Use this action to save data in the GE EPC format.

"filename"	: The file path and name to save.
objecttype	: The type of object being saved.
filter	: Optional parameter – default is to save all objects of specified type See Using Filters in Script Commands section for more information on specifying the filter.
GEFileType	: Optional parameter – default is to save with the latest version. Valid options: GE (latest version), GE14-GE21
SaveBuses	: Optional parameter – default is NO Set to YES or NO. Set to YES to save any buses associated with the regulated bus of generators or switched shunts so that the scheduled voltage can also be saved.
Append	: Optional parameter – default is YES Set to YES or NO. Set to YES to append data to an existing file. Set to NO to overwrite an existing file.

This command saves data for selected generators in the GE EPC format version GE21 to the file "GenData.epc". It also saves the associated buses to retain scheduled voltage information. Any existing file with the same name will be overwritten.

```
SaveDataEPC("GenData.epc", GEN, SELECTED, GE21, YES, NO);
```

SaveDataUsingBuiltInAUXFormat("filename", filetype, [List_of_built_ins], ModelToUse);

Use this action to save data using built-in categories. If the file already exists, data will be appended to the file.

"filename"	: The file to save the data to
filetype	: There are several options for the filetype
	AUXCSV : save as a comma-delimited auxiliary data file.
	AUX : save as a space-delimited auxiliary data file.
	CSV : save as a normal CSV file without the AUX file syntax. The first few lines of the text file will represent the object name and field variable names.
	CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included. This option is useful when appending data of the same object type and field list into a common file.
	CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing column headers you would see in a case information display
[List_of_built_ins]	: Comma-separated list of built-in data categories to include in the file. Options include: Custom Info, Network Model, Contingency, Transient Models, Transient, Model Info, Voltage Conditioning, and Weather Dependent Limits.
ModelToUse	: Optional parameter that indicates the model to use.
	FULL : Full-topology model. This is the default if the parameter is omitted.
	CONSOLIDATED : (Not currently supported) Consolidated planning-type model. This option will only work with the Topology Processing add-on.

This command saves the Network Model and Contingency built-in data categories to the file "FullModelData.aux" in standard AUX format, using the full-topology model. If the file already exists, the new data will be appended.

```
SaveDataUsingBuiltInAUXFormat("FullModelData.aux", AUX, [Network Model, Contingency], FULL);
```

SaveDataUsingExportFormat("filename", filetype, "FormatName", ModelToUse);

Use this action to save data in a user-defined format that has previously been defined.

"filename"	: The file to save the data to
filetype	: There are several options for the filetype
	AUXCSV : save as a comma-delimited auxiliary data file.
	AUX : save as a space-delimited auxiliary data file.
	CSV : save as a normal CSV file without the AUX file syntax. The first few lines of the text file will represent the object name and field variable names.
	CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included. This option is useful when appending data of the same object type and field list into a common file.

- CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing column headers you would see in a case information display
- FormatName : The name of the Object Export Format Description to use.
- ModelToUse : Optional parameter that indicates the model to use.
- FULL : Full-topology model. This is the default if the parameter is omitted.
- CONSOLIDATED : Consolidated planning-type model. This option will only work with the Topology Processing add-on.

This command saves data to the file "ExportedData.csv" using the user-defined export format named "CustomExportFormat1", in CSV format, and applies the full-topology model.

SaveDataUsingExportFormat("ExportedData.csv", CSV, "CustomExportFormat1", FULL);

SaveDataWithExtra("filename", filetype, objecttype, [fieldlist], [subdatalist], filter, [SortFieldList], [Header_List], [Header_Value_List], Transpose, Append);

Use this action to save data in a custom defined format. User-specified fields and field values can also be specified in the output. The syntax is identical to the SaveData command with the following exceptions:

- Filetype : There are several options for the filetype
- CSV : save as a normal CSV file without the AUX file syntax. The first few lines of the text file will represent the object name and field variable names.
- CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included. This option is useful when appending data of the same object type and field list into a common file.
- CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing column headers you would see in a case information display
- Data cannot be saved using AUX or AUXCSV filetypes with this command.

[Header_List] : Optional parameter – default is that no extra headers are included
This allows the specification of user-defined fields that will appear in the output. Headers should be specified as a list of comma delimited strings. A string should be enclosed in double quotes if the string contains a comma. Header strings cannot be blank.

[Header_Value_List] : Optional parameter – default is that all values are blank
Allows the specification of the values that should be assigned to the user-defined fields specified by Header_List. If specified, there must be as many values specified as there are headers. If not specified, all values are blank. Each object will use the same specified value for the specified field. To use different values for different objects and save these in the same file, make use of the CSVNOHEADER file format and filtering. Special keywords can be entered that will be replaced with their actual values. These include @BUILDDATE, @DATETIME, @DATE, @TIME, @VERSION, and @CASENAME.

Append : Optional parameter – default is YES

Set this to YES or NO. Set to YES to append data to an existing file. Set to NO to overwrite an existing file.

For the Header_List and Header_Value_List, the input should be formatted in a manner to indicate how it should be written to the CSV. Any strings enclosed in double quotes will be stripped of the enclosers. Any strings containing double double quotes will have them replaced with single double quotes.

This command exports generator data to "UtilityPoints.csv" using the CSVHEADER format, which omits headers. It saves the MW and MVR fields for all generator objects and adds an extra column labeled "P" with a value of "0.0" for each row; the header for this column is not included because CSVNOHEADER is used for the format.

```
SaveDataWithExtra("UtilityPoints.csv", CSVNOHEADER, Gen, [GenMW, GenMVR], [], , [], ["P"], ["0.0"]);
```

SaveObjectFields("filename", objecttype, [fieldlist]);

Use this action to save a list of fields available for the specified objecttype to a CSV file. Format of the file is variablename, field, col header, description.

"filename" : The file path and name to save.
objecttype : The type of object for which fields should be saved.
[fieldlist] : List of fields for which information will be saved. See the [Specifying Field Variable Names in Script Commands](#) topic for more information on specifying this list.

This command saves a CSV file named "GenFields.csv" that contains metadata about the specified fields (GenMW, GenMVR, GenStatus) for generator objects. The CSV will include columns for the internal variable name, the field name, the column header as shown in the GUI, and a brief description of each field.

```
SaveObjectFields("GenFields.csv", Gen, [GenMW, GenMVR, GenStatus]);
```

SelectAll(objecttype, filter);

Use this to set the Selected field of objects of a particular type to YES. A filter may optionally be specified to only set this property for objects that meet a filter.

objecttype : The objecttype being selected.
filter : Optional parameter – default is to set all objects of specified type
AREAZONE : Only objects that meet the area/zone/owner filters will be selected
"FilterName" : Only objects that meet the specified filter will be checked. See [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command sets Selected = YES for all Buses based on the current AREAZONE filter.

```
SelectAll(BUS, AREAZONE);
```

SendtoExcel(objecttype, [fieldlist], filter, UseColumnHeaders, "workbookname", "worksheetname", [SortFieldList], [Header_List], [Header_Value_List], ClearExisting, RowShift, ColShift);

Use this action to mimic the behavior of the Send to Excel option found within a case information display.

objecttype : The type of object for which fields should be saved.
[fieldlist] : List of fields for which information will be saved. See the [Specifying Field Variable Names in Script Commands](#) topic for more information on specifying this list.
filter : Optional parameter – default is to send all objects of specified type
See the [Using Filters in Script Commands](#) section for more information on specifying the filter.

- UseColumnHeaders : Set to YES or NO. YES signifies that the first row shows the Column Header, NO signifies that field variable names are used.
- "workbookname" : Path and name of the workbook to save or modify. If no path is specified, the workbook will be saved or opened from the current directory. If the workbook already exists, it will be modified with a new worksheet, or if the worksheet is specified and already exists, the worksheet will be overwritten. If using Excel 2007 or later *.xslm filetypes can be specified.
- "worksheetname" : Optional parameter to specify the worksheet name to save. If blank, a new worksheet will be created, if a value is specified it will overwrite the data in any existing worksheet of that name.
- [SortFieldList] : Optional parameter – the default is to do no sorting
This allows the specification of a sort order in which the data will be saved. The format is: [variablename1+:0, variablename2:-:1] where
variablename : is the name of the field to sort by. There is no limit to how many fields can be specified for sorting. For fields that require a location other than zero , variablename can be in the format fieldname:location.
+ or - : for the second parameter indicates sort ascending for + and sort descending for -. This parameter must be specified.
0 or 1 : for the third parameter 0 means case insensitive and do not use absolute value, 1 mean case sensitive or use absolute value. This parameter is optional.
- [Header_List] : Optional parameter – default is that no extra headers are included
This allows the specification of user-defined fields that will appear in the output. Headers should be specified as a list of comma delimited strings. A string should be enclosed in double quotes if the string contains a comma. Header strings cannot be blank.
- [Header_Value_List] : Optional parameter – default is that all values are blank
Allows the specification of the values that should be assigned to the user-defined fields specified by Header_List. If specified, there must be as many values specified as there are headers. If not specified, all values are blank. Each object will use the same specified value for the specified field. Special keywords can be entered that will be replaced with their actual values. These include @BUILDDATE, @DATETIME, @DATE, @TIME, @VERSION, and @CASENAME.
- ClearExisting : Optional parameter – default is YES. YES means to clear the existing sheet
- RowShift : Optional parameter – default is 0. Set to a positive integer to shift the paste of data downwards from row 1. Negative values treated as 0.
- ColShift : Optional parameter – default is 0. Set to a positive integer to shift the paste of data to the right from column A. Negative values treated as 0.

This command sends data about selected buses (fields BusNum, Name, AreaNum) to an Excel file named "BusData.xlsx" in a worksheet called "SelectedBuses". The first row of the worksheet will include normal column headers. An extra header "ExportDate" with the current date (replaced from "@DATE") is added to each row. If the worksheet already exists, its content is cleared before writing new data.

```
SendtoExcel(Bus, [BusNum, Name, AreaNum], SELECTED, YES,
"BusData.xlsx", "SelectedBuses", [], ["ExportDate"], ["@DATE"], YES, 0,
0);
```

SetData(objecttype, [fieldlist], [valuelist], filter);

Use this action to set fields for particular objects. If a filter is specified, then it will set the respective fields for all objects which meet this filter. Otherwise, if no filter is specified, then the key fields must be included in the field list so that the object can be found.

objecttype	: The objecttype being set.
[fieldlist]	: A list of fields to update.
[valuelist]	: A list of values to which to set the corresponding fields.
filter	: Optional parameter – if omitted the key fields and corresponding values must be included to identify the one object to update
ALL	: Set data for all objects
AREAZONE	: Only objects that meet the area/zone/owner filters will be set
SELECTED	: Only objects whose Selected field = YES will be set
"FilterName"	: Only objects that meet the specified filter will be set. See Using Filters in Script Commands section for more information on specifying the filtername.

This command updates the AreaNum field to 5 for all bus objects that are currently marked as Selected = YES.
--

SetData(Bus, [AreaNum], [5], SELECTED);

SetElseCreateData(objecttype, [fieldlist], [SetValueList], [DefaultValueList]);

(This command was added in the September 7, 2026 patch of Simulator 24)

Because AUX scripts provide no process control to determine if a power flow case contains a particular object, this command provides a way to do that. If the object exists, it will be updated, but if the object does not exist, a new one will be created with default values for any excluded fields. This action only affects a single object of the specified type. If an object does not exist and a new object is being created, Simulator's operating mode is automatically switched to EDIT.

Required fields must be included for a new object to be created. Non-blank values for required fields should be included in either SetValueList or DefaultValueList. For most object types in Simulator, new objects will not be created if a required field is blank.

objecttype	: The type of object being set or created.
[fieldlist]	: The list of fields to set. This list must include keyfields necessary to properly identify the object.
[SetValueList]	: The list of values used when updating an existing object or creating a new one. The values must correspond to the fieldlist. The keyfields must be specified and not left blank. To use default values, leave an entry blank between the field separating commas. A blank entry enclosed in double-quotes is treated as a valid entry and will not use a default value.
[DefaultValueList]	: Optional parameter – If not specified, Simulator's defaults are used. The default list of values used when creating a new object. The values must correspond to the fieldlist. Keyfields in this list are ignored and must be specified in the SetValueList. If an object does not already exist, any blank entries in the SetValueList are replaced with the corresponding entry in the DefaultValueList, so that default values are used if necessary when creating a new object. If this list is not specified, Simulator's defaults are used instead.

This command updates ZoneNumber to 3 for Bus 1 if the bus exists, while leaving all other fields unchanged. If Bus 1 does not exist, a new bus is created setting ZoneNumber to 3, while using the default values for the other required fields so that the bus can be properly created.

```
SetElseCreateData(Bus, [Number, Name, AreaNumber, ZoneNumber, NomkV],  
[1,,3,], [1, "NewBus", 1, 3, 138]);
```

This command sets the VoltSet field to 1.04 and the AVR field to "YES" for the generator at bus 1 with ID "1" if the generator already exists. All other fields are left unchanged.

The Status field is set to "" in SetValueList, meaning that the Status should be left unchanged if the generator already exists. The Status is "Closed" in the DefaultValueList, but this value will not be used because the value in the SetValueList is not blank; an empty set of double-quotes is not treated the same as having a blank entry.

The second example where the Status field is set to blank in SetValueList will use the default of "Closed" and create the object if it does not exist.

```
SetElseCreateData(Gen, [BusNum, ID, Status, MWMin, MWMax, MWSetPoint,  
MvarMin, MvarMax, MvarSetPoint, VoltSet, AGC, AVR],  
[1,"1","",,,,,,,,,1.04,, "YES"], [,, "Closed",0,300,0,-50, 50, 0, 1.03, "YES",  
"YES"]); // Will result in error if generator does not already exist
```

```
SetElseCreateData(Gen, [BusNum, ID, Status, MWMin, MWMax, MWSetPoint,  
MvarMin, MvarMax, MvarSetPoint, VoltSet, AGC, AVR],  
[1,"1",,,,,,,,,1.04,, "YES"], [,, "Closed",0,300,0,-50, 50, 0, 1.03,  
"YES", "YES"]); // Will create the generator if it does not exist
```

UnSelectAll(objecttype, filter);

Same as SelectAll, but this action sets the Selected field of objects meeting the filter to NO.

This command sets Selected = NO for all buses.

```
UnSelectAll(BUS, );
```

WriteLimitMonitoringSettings("filename");

Use this to save Limit Monitoring Settings to an auxiliary file.

"filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

Case Actions

AppendCase("filename", OpenFileType, [StarBus, EstimateVoltages]);

AppendCase("filename", OpenFileType, [MSLine, VarLimDead, PostCTGAGC, EstimateVoltages]);

Use this action to append a case to the currently open case. The optional parameters depend on the type of file being appended.

Filename	: File name of the case to be appended
OpenFileType	: PWB – case file is a PowerWorld Binary file GE – case file is a GE .epc file. GE _{xx} where _{xx} is the appropriate EPC file version number can also be used. PTI – case file is a PTI .raw file. PTI _{xx} where _{xx} is the appropriate RAW file version number can also be used. CF – case file is an IEEE common data format file
StarBus	: Optional parameter – default is NEAR Only used for PTI RAW format, with the following options: NEAR – star buses are numbered starting after the near bus number MAX – star buses are numbered starting after the maximum bus number Value – star bus numbering will start at specified value
MSLine	: Optional parameter – default is MAINTAIN Only used for the GE EPC format, with the following options: MAINTAIN – maintain multisection lines EQUIVALENCE – equivalence multisection lines
VarLimDead	: Optional parameter – default is 2.0 Only used for the GE EPC format NUMBER – set the GE var limit deadband to this value
PostCTGACG	: Optional parameter – default is NO Only used for the GE EPC format. Set to YES to populate the generator field Post-CTG Prevent Response based on the EPC file's generator base load flag.
EstimateVoltages	: Optional parameter – default is YES Used with either GE EPC or PTI RAW format with the following options: YES – voltages and angles are estimated for new buses that are created when appending data to a case. Angle smoothing is done across new lines that are created when appending data to a case. These operations might be necessary if appending data that contains voltages that are not consistent to the case into which it is being appended or contains no voltages at all. This is the default. NO – no voltage and angle estimates are done and no angle smoothing is done. This might be necessary when appending large sections of a case, i.e. such as a new island, or providing voltages that are already good estimates in the appended data.

This command appends the PTI RAW file "B7FlatExtension.raw" (version 33) to the currently open case. Star buses are numbered starting after the maximum existing bus number, and PowerWorld will estimate voltages and angles for all new buses and branches, ensuring smooth integration with the existing system.

```
AppendCase("B7FlatExtension.raw", PTI33, [MAX, YES]);
```

CaseDescriptionClear;

Use this action clear the case description of the presently open case.

CaseDescriptionSet("text", Append);

Use this action to set or append text to the case description.

"text"	: Specify the text to set/append to the case description.
Append	: YES – will append the text specified to the existing case description. NO – will replace the case description.

DeleteExternalSystem;

This action will delete part of the power system. It will delete those buses whose property Equiv is set true.

EnterMode(mode);

This action will change the mode in which Simulator is operating. This is especially necessary when creating new case objects for which you are required to be in EDIT mode. Simulator will automatically change the mode to the correct mode for script actions that are mode-specific.

Mode	: The mode to enter, either RUN or EDIT.
------	--

Equivalence;

This action will equivalence a power system. All options regarding equivalencing are handled by the Equiv_Options objecttype. Use the SetData action, or a DATA section to set these options prior to using the Equivalence action. Also, remember that the property Equiv must be set true for each bus that you want to equivalence.

LoadEMS("filename", filetype);

Use this to open any EMS file. This can be a full case, a contingency file, or remedial action scheme file. Simulator will determine the type of information being loaded and how to handle it based on the records in the file.

"filename"	: Name of the file to open. See the Specifying File Names in Script Commands section for special keywords that can be used when specifying the file name.
filetype	: Type of file to be loaded. Currently the only option is AREVAHDB.

NewCase;

This action clears out the existing case and open a new case from scratch.

OpenCase("filename", OpenFileType,[LoadTransactions,StarBus,HowToKeepDuplicates]);**OpenCase("filename", OpenFileType,[MSLine,VarLimDead,PostCTGAGC,MSLineDummyBus]);**

This action will open a case stored in "filename" of the type OpenFileType. Different sets of optional parameters apply for the PTI and GE file formats. The LoadTransactions and Star bus parameters are available for writing to RAW files. MSLine, VarLimDead, and PostCTGAGC are for writing EPC files.

"filename"	: The file to be opened.
OpenFileType	: An optional parameter indicating the format of the file being opened. If none is specified, PWB will be assumed. It may be one of the following strings: PWB, PTI (latest version), PTI23-PTI35 GE (latest version), GE14-GE23, CF AUX, UCTE, AREVAHDB, OPENNETEMS
LoadTransactions	: valid for PTI RAW format only YES : load transactions when opening case. NO : do not load transactions when opening case. DEFAULT : follow default behavior.
StarBus	: valid for PTI RAW format only NEAR : star buses are numbered starting after the near bus number MAX : star buses are numbered starting with the maximum bus number

VALUE : star bus numbering will start at value
 HowToKeepDuplicates : valid for PTI RAW format only
 LAST : keep only the LAST duplicate of any device in each section of a PTI RAW file
 ALL : keep ALL duplicates of any device in each section of a PTI RAW file, and make them each unique by dynamically changing their IDs
 MSLine : valid for GE EPC format only
 MAINTAIN : maintain multi-section lines
 EQUIVALENCE : equivalence multi-section lines
 VarLimDead : valid for GE EPC format only
 Number : set the GE var limit deadband
 PostCTGACG : valid for GE EPC format only
 set to YES to populate the generator field Post-CTG Prevent Response based on the EPC file's generator base load flag.

 MSLineDummyBus : Optional parameter – default is specified with Simulator Options
 valid for GE EPC format only
 Specifies how the dummy bus numbers for multi-section lines are determined.
 FROM : starting at the from bus number
 MAX : starting with the maximum bus number
 Value or range – starting with the specified value or will be numbered within the specified range. If an unused bus number within the specified range cannot be found, the numbering will start at the highest number specified in the range.

This command opens the PTI RAW file "B7FlatImport.raw" using the PTI33 format. It loads transaction records, assigns star buses starting after the maximum existing bus number, and keeps only the last instance of any duplicate devices found in the file.

```
OpenCase("B7FlatImport.raw", PTI33, [YES, MAX, LAST]);
```

Renumber3WXFormerStarBuses("filename", Delimiter);

Use this action to renumber star buses based on user-specified values.

"filename" : The name of the file containing the renumbering
 Delimiter : Optional parameter – default is BOTH
 Set to COMMA, SPACE, or BOTH to indicate the delimiter to use to separate data in the file.

The file may be comma or space delimited. The contents of the file should be formatted using the format below.

```
primary bus, secondary bus, tertiary bus, circuit, new starbus number, new starbus name
```

or

```
pribusname_nomkV, secbusname_nomkV, terbusname_nomkV, circuit ID, newstarbusnum, newstarbusname
```

Either the bus number or the name_nominalkV identifier may be used to identify the buses. Each bus may be identified using either method even for the same transformer. Lines starting with two slashes (//) will be ignored. The next two lines are sample file contents using different methods to identify buses.

```
11037, 11038, 11199, "1", 11202, "Ki star"  
"WESTWING_500.00" "WESTWNGW_230.00" "WESTWG 4_34.50" "2" 99823 "KI STAR 3"
```

RenumberAreas(NumCI);

Renumber Areas using the new number for the Area located in the Custom Integer field of the area.

NumCI : Custom Integer field containing the new numbers. Variable name location numbers are zero-indexed while the standard column headers are indexed starting with 1. This value corresponds to the standard column header value for the custom integer field.

This will change each area's number to the value stored in its CustomInteger:0 field.

```
RenumberAreas(1);
```

RenumberBuses(NumCI);

Renumber Buses using the new number for the bus located in the Custom Integer field of the bus.

NumCI : Custom Integer field containing the new numbers. Variable name location numbers are zero-indexed while the standard column headers are indexed starting with 1. This value corresponds to the standard column header value for the custom integer field.

This will change each bus' number to the value stored in its CustomInteger:1 field.

```
RenumberBuses(2);
```

RenumberMSLineDummyBuses("filename", Delimiter);

Use this action to renumber dummy buses or a multisection line based on user-specified values.

"filename" : The name of the file containing the renumbering
Delimiter : Optional parameter – default is BOTH
Set to COMMA, SPACE, or BOTH to indicate the delimiter to use to separate data in the file.

The file may be comma or space delimited. Buses may be identified using bus numbers or using the BusName_NominalKV combination. The file format is below:

```
from bus, to bus, circuit //identify multi-section line  
dummybusnumber1, dummybusname1  
dummybusnumber2, dummybusname2
```

where the dummy bus numbers and names give the numbers and names that will be assigned for the dummy buses of a multi-section line. An example of the file contents is below:

```
40039 , 40141, 1 // ALFALFA 230 N BONNVL 230 #1  
49997, "ALFN B11"  
40062 , 40699, 2 // ASHE R1 500 MARION 500 #2  
49990, ASHMAR21  
49989, ASHMAR22  
49988, ASHMAR23
```

RenumberSubs(NumCI);

Renumber Substations using the new number for the substation located in the Custom Integer field of the substation.

NumCI : Custom Integer field containing the new numbers. Variable name location numbers are zero-indexed while the standard column headers are indexed starting with 1. This value corresponds to the standard column header value for the custom integer field.

This will change each substation's number to the value stored in its CustomInteger:2 field.

RenumberSubs(3);

RenumberZones(NumCI);

Renumber Zones using the new number for the Zone located in the Custom Integer field of the zone.

NumCI : Custom Integer field containing the new numbers. Variable name location numbers are zero-indexed while the standard column headers are indexed starting with 1. This value corresponds to the standard column header value for the custom integer field.

This will change each zone's number to the value stored in its CustomInteger:3 field.

RenumberZones(4);

RenumberCase;

(This command was added in the December 5, 2023 patch of Simulator 23)

RenumberCase rennumbers the object in the case according to the swap list in memory.

SaveCase("filename", SaveFileType, [PostCTGAGC, UseAreaZone]);

SaveCase("filename", SaveFileType, [AddCommentForObjectLabels, IncludeSubstations]);

This action will save the case to "filename" in the format SaveFileType.

"filename" : The file name in which to save the information.

SaveFileType : An optional parameter indicating the format of the file to be saved. If none is specified, then PWB will be assumed. It may be one of the following strings:

PWB (latest version), PWB16-PWB23

PTI23-PTI35

GE14-GE23,

CF, UCTE

AUXNETWORK, AUX, AUXSECOND, AUXLABEL

When saving an auxiliary file, AUX, AUXSECOND, and AUXLABEL are supported for compatibility with existing processes that users might have in place, but the recommended option is AUXNETWORK. This option saves the data required for defining the network model of a power system.

PostCTGAGC : An optional parameter, only valid for GE EPC format. If the Governor Response Limits field for a generator is set to Down Only or Fixed, the base load flag will be written as 1 or 2, respectively, and this option is ignored. This option is only used when a generator's Governor Response Limits field is set to Normal. If this option is set to YES and not ignored, the base load flag in the EPC file is based on the Post-Contingency Prevent AGC Response setting. If preventing post-contingency AGC, the base load flag is set to 2. If not preventing post-contingency AGC, the base load flag is set to 0.

UseAreaZone : An optional parameter, only valid for GE EPC format. YES limits the entries in the EPC file based on the area/zone/owner filter (NO by default)

AddCommentsForObjectLabels

: Optional parameter – default is NO

Only valid for PTI RAW format. YES adds object Labels to the end of data records when saving a RAW file.

Label comments will appear as /* [Label] */

IncludeSubstations : (Added in January 30, 2024 patch of Simulator 23)
 Optional parameter – default is NO
 Only valid for PTI RAW format. YES includes substations, which are used for full topology node breaker modeling. NO excludes substations.

This command saves the current case to a PTI RAW file named "B7FlatExport.raw" using version 33 format. The AddCommentsForObjectLabels option is set to YES, so any objects that have Labels defined will include an end-of-line comment. IncludeSubstations is also set to YES, meaning full topology substations will be included in the RAW export for accurate node-breaker representation.

```
SaveCase("B7FlatExport.raw", PTI33, [YES, YES]);
```

SaveExternalSystem("Filename", SaveFileType, WithTies);

This action will save part of the power system to a "filename". It will save only those buses whose Equiv field is set to External.

filename : The file name to save the information to.
 SaveFileType : An optional parameter saying the format of the file to be saved. If none is specified, then PWB will be assumed. May be one of the following strings
 PWB, PWB16-PWB23
 PTI23-PTI35
 GE14-GE23, CF, AUX
 WithTies : An optional parameter. The user must specify the file type explicitly in order to use the WithTies parameter. Allows saving a transmission line that ties a bus marked with Equiv as External and one marked Study. This must be a string which starts with the letter Y, otherwise NO will be assumed.

This command saves a portion of the power system to a PTI RAW file named "B7Flat_External.raw", using the PTI version 33 format. Only buses that have their Equiv field set to External will be included in the saved file. The WithTies parameter is set to "YES", meaning that transmission lines (branches) connecting marked (Equiv = External) and unmarked (Equiv = Study) buses will also be saved, ensuring a complete boundary representation for the external system.

```
SaveExternalSystem("B7Flat_External.raw", PTI33, YES);
```

SaveMergedFixedNumBusCase ("filename", SaveFileType);

This online help topic will explain FixedNumBus in more detail:

https://www.powerworld.com/WebHelp/#MainDocumentation_HTML/FixedNumBus_Features.htm

This action will save the Merged FixedNumBus case to "filename" in the format SaveFileTypes. A FixedNumBus is a group of connection points that is defined by the user. Typically other buses that are assigned to a FixedNumBus represent nodes in a full topology system, and the FixedNumBus is the common electrical point that is represented if this system is merged.

"filename" : The file name in which to save the information.
 SaveFileType : Same options as for [SaveCase](#) command

This command saves a merged version of the currently open power system case—where buses grouped under each FixedNumBus are treated as single electrical points—into a PTI RAW format file named "B7Flat_Merged.raw". This merged representation collapses all buses assigned to the same FixedNumBus into a single bus.

```
SaveMergedFixedNumBusCase("B7Flat_Merged.raw", PTI33);
```

Scale(scaletype, basedon, [parameters], scalemarker);

Use this action to scale the load and generation in the system. This script command should be used in conjunction with the SCALE_OPTIONS object that specifies additional options necessary for the scaling that are not set through the script command.

scaletype	: The objecttype begin scaled. Must be either LOAD, GEN, INJECTIONGROUP, or BUSSHUNT.
basedon	: One of: MW : parameters are given in MW, MVAR units. FACTOR : parameters a factor to multiple the present values by.
[parameters]	: These parameters have different meanings depending on ScaleType. LOAD : [MW, MVAR] or [MW]. If you want to scale load using constant power factor, then do not specifying a MVAR value. GEN : [MW] INJECTIONGROUP: [MW, MVAR] or [MW] . If you want to scale load using constant power factor, then do not specifying a MVAR value. BUSSHUNT : [GMW, BCAPMVAR, BREAMVAR]. The first values scales G shunt values, the second value scales positive (capacitive) B shunt values, and the third value scales negative (reactive) B shunt values
The Scale script command also allows using the [parameters] input to specify the new value or scale factor through a field with the object type to scale. To use this option, the [parameters] input should contain field variable names instead of numeric values. When using a field rather than value, the scaling will be done by individual object rather than the aggregation of all objects selected for scaling.	
scalemarker	: Optional parameter – default is BUS This value specifies whether to look at an element's bus, area or zone to determine whether it should be scaled. This parameter can be omitted and is ignored when scaletype is INJECTIONGROUP. BUS : Means that elements will be scaled according to the Scale property of the element's terminal bus. AREA : Means that elements will be scaled according to the Scale property of the element's Area. Note that it is possible for the area of a load, generator, or switched shunt to be different than the terminal bus's area. ZONE : Means that elements will be scaled according to the Scale property of the element's Zone. Note that it is possible for the zone of a load, generator, or switched shunt to be different than the terminal bus's zone. OWNER : Means that elements will be scaled according to the Scale property of the element's Owner. Note that it is possible for the Owner of a load, generator, or switched shunt to be different than the terminal bus's Owner.

Here are two different ways to scale three areas to a particular load value.

```
Script
{
  SetData(Area, [Scale], ["NO"], All);          // Sets the scale field of all areas to NO
  SetData(Area, [Number, Scale], [111, "YES"]); // For area, set Scale=YES
  Scale(LOAD, MW, [1111.1], AREA);              // Do the scaling for particular area
  SetData(Area, [Number, Scale], [111, "NO"]);  // reset Scale back to NO
  SetData(Area, [Number, Scale], [333, "YES"]); // For area, set Scale=YES
  Scale(LOAD, MW, [3333.3], AREA);              // Do the scaling for particular area
  SetData(Area, [Number, Scale], [333, "NO"]);  // reset Scale back to NO
  SetData(Area, [Number, Scale], [444, "YES"]); // For area, set Scale=YES
  Scale(LOAD, MW, [4444.4], AREA);              // Do the scaling for particular area
  SetData(Area, [Number, Scale], [444, "NO"]);  // reset Scale back to NO
}
Script
{
  SetData(Area, [Scale], ["NO"], All);
  SetData(Area, [Number, Scale, CustomFloat:5], [111, "YES", 1111.1]);
  SetData(Area, [Number, Scale, CustomFloat:5], [333, "YES", 3333.3]);
  SetData(Area, [Number, Scale, CustomFloat:5], [444, "YES", 4444.4]);
  Scale(LOAD, MW, ["CustomFloat:5"], AREA);
}
```

Modify Case Objects

AutoInsertTieLineTransactions;

Use this action to delete all existing MW transactions and set the unspecified MW interchange for each area to zero. It then automatically creates a MW transaction between each pair of connected areas with a MW transaction exactly equal to the sum of the tie-line flows.

BranchMVALimitReorder(Filter, SetA, SetB, SetC, SetD, SetE, SetF, SetG, SetH, SetI, SetJ, SetK, SetL, SetM, SetN, SetO);

This action will modify the MVA limits for a branch. 15 different limits labeled A through O can be specified for a branch. A specified limit set can be updated to the values contained in another set or a specific value.

- Filter : Optional parameter – default is to change all branch limits
See [Using Filters in Script Commands](#) section for more information on specifying the filter.
- SetA, SetB, ... , SetO : Optional parameters – default is to not change the value of a limit set
For each limit set to be changed, the letter identifying another limit set can be specified so that those values populate the current limit set.
Instead of a limit set letter a value for new limits can also be specified. To keep the value of a particular limit set the same leave that entry blank.

The following sets limits A and B to their own values, sets limits C-J to values of other limits, limits K-N at set to 9999, and limit O is left unchanged. This is done for any branch with at least one terminal in Area 3.

```
BranchMVALimitReorder("Area 3", A, B, D, E, G, H, J, K, M, N, 9999, 9999, 9999, 9999);
```

The following sets limits A through O to values of other limits for all branches that meet filter "My Filter Name".

```
BranchMVALimitReorder("My Filter Name", M, N, O, A, B, C, D, E, F, G, H, I, J, K, L);
```

CalculateRXBGFromLengthConfigCondType(filter);

Use this action to go through branches in the power system and recalculate the per unit R, X, G, and B values using the TransLineCalc tool. The branches Conductor Type, Tower Configuration, and Line Length will be passed to the TransLineCalc tool and new R, X, G and B values will be calculated. This is only available if you have installed the TransLineCalc tool.

- filter : Optional parameter – default is to check all branches
Check only branches that meet the specified filter. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command recalculates the per unit resistance (R), reactance (X), conductance (G), and susceptance (B) for all branches where Selected = YES in the case using the TransLineCalc tool. It uses each branch's conductor type, tower configuration, and line length to perform these calculations.

```
CalculateRXBGFromLengthConfigCondType(SELECTED);
```

ChangeSystemMVABase(NewBase);

Use this action to change the system MVA base to the specified value and update all internal data structures to store values on the new base.

- NewBase : New power system base in MVA.

ClearSmallIslands;

Use this action to identify the largest island and de-energize all other islands. The largest island is the island with the most buses. Small islands are de-energized by setting the status of all generators in those island to open.

Combine([elementA], [elementB]);

Use this action to combine two generators.

- [elementA] : The object that should be moved. See the format for [elementA] in the Move script command for information on the formatting of this string.
- [elementB] : The object that element A should be combined with. Same format as for elementA.

Suppose that there were two generators on Bus 2. To combine them into a single equivalent unit by merging their MW/MVar output, operating limits, and other characteristics, one would use this command.

```
Combine([GEN 2 2], [GEN 2 1]);
```

CreateLineDeriveExisting(FromBus, ToBus, Circuit, NewLength, BranchID, ExistingLength, ZeroG);**(Added in December 27, 2024 patch of Simulator 23)**

This command is used to scale the impedance values of a branch onto a new branch. The idea being to create a new branch that is the same but with impedance values scaled to represent a different length.

- FromBus : From bus number of new branch to be added to the case
- ToBus : To bus number of new branch
- Circuit : Circuit ID for new branch
- NewLength : Length of new branch
- BranchID : Object ID for existing branch in the model using the BRANCH FROMBUS TOBUS CIRCUIT format.
- ExistingLength : Optional parameter – default is to use length with existing branch
This parameter is used to specify the existing branch length. If the parameter is omitted, then the existing value from the branch length field will be used. If the branch length field is not populated, then the effect of this command will be to create a new branch with the same impedance and limit values.
- ZeroG : Optional parameter – default is NO
If YES the shunt G value will be set to zero. The mathematics of scaling the impedance values can result in negative g (shunt conductance) values. This value allow you to set the g values to 0 after the scaling calculations.

Create a derived line with a length of 220, based on the existing line from bus 1 to 3 circuit 1. The new line will go from bus 1 to 3 with a circuit ID of 5.

```
CreateLineDeriveExisting(1, 3, 5, 220, BRANCH 1 3 1);
```

DirectionsAutoInsert(Source, Sink, DeleteExisting, UseAreaZoneFilters);

Use this action to auto-insert directions to the case.

- Source : AREA, ZONE, or INJECTIONGROUP – specifies what to use as source
- Sink : AREA, ZONE, INJECTIONGROUP, or SLACK – specifies what to use as sink.
- DeleteExisting : YES – to delete existing direction; NO to not do that.
- UseAreaZoneFilters : YES – to filter Area/Zones by filter.

The command deletes all existing directions and creates new ones from each Area to the Slack bus. No filtering is used to exclude areas.

```
DirectionsAutoInsert(Area, Slack, YES, NO);
```

DirectionsAutoInsertReference(SourceType, ReferenceObject, DeleteExisting, SourceFilterName, OppositeDirection);

Use this to auto-insert directions from multiple source objects to the same ReferenceObject. By default, directions will be created that go from the SourceType objects with the ReferenceObject as a Sink. Specify the parameter OppositeDirection as YES build directions in the opposite direction.

- SourceType : The type of object used as source object. May be either Area, Zone, InjectionGroup, or Bus
- ReferenceObject : The specific object used as the reference. Use the identifying string which starts with the ObjectName and is followed by keyfields or label identifiers. The ObjectNames allowed are Area, Zone, InjectionGroup, or Bus. The string "Slack" can also be specified to indicate the slack bus.
- DeleteExisting : Optional parameter – default is YES. Set to YES to delete existing directions and NO to leave existing directions defined.
- SourceFilterName : Optional parameter – default is a blank string which means to do all objects of the SourceType. If specified, then will only use those that meet this filter. See the [Using Filters in Script Commands](#) in Script Commands section for more information on specifying the filename. May also specify the string ALL to indicate all SourceType objects, though this is equivalent to a blank.
- OppositeDirection : Optional parameter – default is NO. Set to YES to indicate the directions should use the ReferenceObject as the Source for each direction and the SourceType will be the Sink instead.

A direction is added for each bus inside Area 2 with the Sink equal to the InjectionGroup named "MyGroup". Existing directions are not deleted.

DirectionsAutoInsertReference(Bus,"InjectionGroup MyGroup",NO,"<Device>Area 2",NO);

InitializeGenMvarLimits;

Use this action to initialize all generators in the case so that they are appropriately marked as being at Mvar limits or not. This could be useful if manually setting the Mvar output of generators or changing their limits.

InjectionGroupsAutoInsert;

Use this action to insert injection groups according to the options specified in the "IG_AutoInsert_Options" object. The settings available with this object represent what is seen on the Auto Insert Injection Groups Dialog.

InjectionGroupCreate("Name", objecttype, InitialValue, filter, Append);

This action will create or modify an injection group with participation points of a single object type that meet a filter. Repeated calls to this script command can be used to define an injection group with different object types.

- "Name" : Name of the injection group to create or modify.
- objecttype : Type of object to be included in the injection group. Valid options are GEN, LOAD, SHUNT, or BUS.
- InitialValue : Set this to a floating point value to indicate the value of the participation factor to use with each point. Special keywords can also be used to indicate dynamically determined values. (The participation point AutoCalc field is set to YES). The following options are available:
 - Generators:
 - PRESENT, MAX GEN INC, MAX GEN DEC, and MAX GEN MW
 - Loads:
 - LOAD MW
 - Switched Shunts:

MAX SHUNT INC, MAX SHUNT DEC, and MAX SHUNT MVAR

All object types:

<FIELD>variablename can be used to reference a field associated with the object in the participation point.

<EXPRESSION>modelexpressionname can be used to reference a Model Expression.

- filter : Specify a filter to select the objects to add to the injection group. See the [Using Filters in Script Commands](#) section for more information on specifying the filter.
- Append : Optional parameter – default is YES.
Set to YES or NO. Set to YES to add new participation points based on the current settings to an injection group that exists with the same Name. Set to NO to delete all existing points before adding new points to an injection group that already exists with the same Name.

This command creates a new Injection Group called "TopAreaGens" that includes all generators in Area 1 and whose participation factor is set the same as the participation factor specified with the generator. The filter is specified as a single-condition filter that does not require the creation of an advanced filter.

```
InjectionGroupCreate("TopAreaGens", GEN, PRESENT, "AreaNumber = 1", NO);
```

InjectionGroupRemoveDuplicates([PreferenceFilter](#));

([PreferenceFilter](#) added in August 17, 2023 patch of Simulator 22 and 23)

This action will search through all injection groups in the case looking for injection groups with the same elements. For an injection group being the same means that the injection group would have the same behavior as a duplicate. This means that the elements must be the same. The elements must also have the same initial value, participation factor, and auto calculation settings.

- [PreferenceFilter](#) : (optional) Specify a filter to use when determining which object to keep when there are duplicates. If one object meets the filter and the other object does not, then the object which meets the filter will be maintained and the other removed. Otherwise, the one with a name that would occur first in an alphabetic sort is maintained. See the [Using Filters in Script Commands](#) section for more information on specifying the filter. Special filter keywords of SELECTED and AREAZONE cannot be used with this script command.

Suppose that there are multiple injection groups: "NorthAreaGenGroup_1", "NorthAreaGenGroup_A", and "NorthGen_Combined". They all include the exact same set of generators, with the same participation factors and settings — they are functionally identical. To clean up the case and avoid confusion, run this command. This keeps the group named "NorthGen_Combined", while deleting the rest.

```
InjectionGroupRemoveDuplicates("Name contains 'Combined'");
```

InterfaceAddElementsFromContingency([InterfaceName](#), [ContingencyName](#));

(Added in February 1, 2024 patch of Simulator 23)

This action will create or modify an interface by adding the elements from a contingency to an interface as contingent interface elements. Only elements that are can be represented as a contingent interface element will be added to the interface.

- [InterfaceName](#) : Name of interface to add contingent elements to. If the interface does not exist, it will be created.
- [ContingencyName](#) : Name of contingency to get elements from.

This command creates or modifies an interface named "Left-Right" using the elements defined in a contingency called "L_000001One-00000TwoC1".

```
InterfaceAddElementsFromContingency("Left-Right", "L_000001One-00000TwoC1");
```

InterfacesAutoInsert(Type, DeleteExisting, UseFilters, "Prefix", Limits);

Use this action to auto-insert interfaces.

Type	: AREA – insert area-to-area tieline interfaces. ZONE – insert zone-to-zone tieline interfaces.
DeleteExisting	: YES – to delete existing interfaces; NO – to leave existing interfaces alone.
UseFilters	: YES – to use Area/Zone Filters; NO – to insert for entire case.
"Prefix"	: Enter a string which will be a prefix on the interface names.
Limits	: ZEROS – to make all limits zero. AUTO – limits will be set to the sum of the branch limits. [lima, limb, limc, limd, ...] – Enter 8 limits enclosed in brackets, separated by commas. This will set the limits as specified.

This command creates interfaces between all defined areas in the case that are comprised of the tie-lines between pairs of areas. Existing interfaces are deleted. Each interface is named with the prefix "AreaIF_", resulting in names like "AreaIF_Top_Left". The interface limits are set based on the sum of the limits of the branches that connect each pair of areas.

```
InterfacesAutoInsert(AREA, YES, NO, "AreaIF_", AUTO);
```

InterfaceCreate("Name", DeleteExisting, ObjectType, Filter);

This action will create or modify an interface with elements of a single object type that meet a filter.

Repeated calls to this script command can be used to define an interface with different object types.

"Name"	: Name of the interface to create or modify.
DeleteExisting	: Set to YES to delete an interface with the same Name. Set to NO to append elements to an interface with the same Name.
ObjectType	: Type of object to be included in the interface. Valid options are BRANCH or INTERFACE.
Filter	: Specify a filter to select the objects of the specified ObjectType to add to the interface. See the Using Filters in Script Commands section for more information on specifying the filter. A filter must be specified, and this field cannot be blank.

This command creates an interface named "North138Lines", and if an interface with the same name already exists, it will be deleted and recreated. Only branch objects (i.e., transmission lines, transformers) are considered. Using the device filter for Bus 1 means that only branches that are connected to Bus 1 at either terminal are included.

```
InterfaceCreate("North138Lines", YES, BRANCH, "<DEVICE>Bus 1");
```

InterfaceFlatten("InterfaceName");

Interfaces can contain elements that are other interfaces. The "flattening" process will permanently modify an interface to contain the elements that the other interface contains and remove the element that is the other interface.

"InterfaceName"	: Name of the interface to modify.
-----------------	------------------------------------

InterfaceFlattenFilter(Filter);

(command added in February 9, 2024 patch of Simulator 23)

This functions the same as the InterfaceFlatten script command, except you apply a filter to the case to perform the "flattening" process on all Interface objects that meet the filter.

Filter : Specify a filter to select Interfaces to flatten. Enter the string ALL to indicate that all interfaces should be flattened. See the [Using Filters in Script Commands](#) section for more information on specifying the filter.

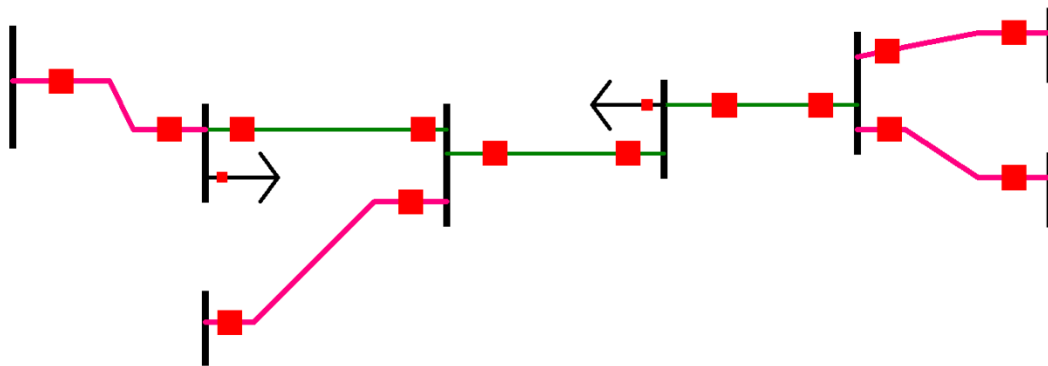
This command flattens all interface objects that have a total MW flow greater than 100 MW, as indicated by the single-condition filter.

```
InterfaceFlattenFilter("MW > 100");
```

InterfaceModifyIsolatedElements(filter);

In interfaces with many BRANCH OPEN actions it is possible for some of the open actions to disconnect portions of the system that contain other open actions and other devices. This can be seen in the illustration below. **All** lines on the diagram represent interface BRANCH OPEN elements. However, the three green lines in the center are completely disconnected from the system by the other BRANCH OPEN interface elements (the ones in pink). The BRANCH OPEN elements also disconnect load. This situation is very hard to deal with numerically when calculating sensitivities. We address this by removing the BRANCH OPEN elements that are contained within the disconnected portion of the system. We also add OPEN actions

filter : Optional parameter – default is to apply command to all interfaces
Filter that specifies which interfaces to modify. See [Using Filters in Script Commands](#) section for more information on specifying the filter.



InterfaceRemoveDuplicates(PreferenceFilter);

(PreferenceFilter added in August 17, 2023 patch of Simulator 22 and 23)

This action will search through all interfaces in the case looking for interfaces with the same elements. For an interface being the same means that the reported flow is the same. In addition to having all the same elements, the interface elements must also have the same monitoring direction and weighting.

PreferenceFilter : (optional) Specify a filter to use when determine which object to keep when there are duplicates. If one object meets the filter and the other object does not, then the object which meets the filter will be maintained and the other removed. Otherwise, the one with a name that would occur first in an alphabetic sort is maintained. See the [Using Filters in Script Commands](#) section for more information on specifying the filter. Special filter keywords of SELECTED and AREAZONE cannot be used with this script command.

This script command searches all interfaces in a case that contains the exact same elements, along with monitoring direction and weighting, and removes them. If duplicates are found, the preference filter "Name contains 'AreaIF'" tells Simulator to retain the interface whose name contains 'AreaIF'.

```
InterfaceRemoveDuplicates("Name contains 'AreaIF'");
```

MergeBuses([element], Filter);

Use this action to merge buses

- Element : The bus object that will be created when the buses meeting the Filter are merged.
- Filter : Optional parameter – default is to merge all buses into a single bus
 - AREAZONE : Only buses that meet the area/zone/owner filters will be merged
 - SELECTED : Merge buses with Selected field = YES
 - "FilterName" : Only buses that meet the specified filter will be merged. See [Using Filters in Script Commands](#) section for more information on specifying the filtername.

This will merge all the buses that meet the area/zone/owner filters into bus 6.

MergeBuses([Bus 6], AREAZONE);

MergeLineTerminals(Filter);

Use this action to merge line terminals. This action can be used to remove a line by merging the terminal buses of that line into a single bus. The only parameter of the script command is a filter parameter, which must be populated with either the name of an advanced filter (with the name in quotation marks) or the text SELECTED (with no quotation marks). If an advanced filter is given, then Simulator will find all branches that meet the advanced filter definition and will individually merge the line terminals of each line one at a time.

- Filter : Any multi-section lines meeting this filter will be merged.
 - "filtername" : See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.
 - SELECTED : Merge objects with Selected field = YES

This command finds all branches in the case that have the Selected field set to YES. For each of these selected lines, it merges the terminal buses into a single bus, thereby removing the line from the network.
--

MergeLineTerminals(SELECTED);

MergeMSLineSections(Filter);

Use this action to eliminate multi-section line records. If possible, the individual sections will be merged into a single line record between the from and to bus and the multi-section line record will be removed. If a multi-section line contains series capacitors or transformers, the multi-section line record will be retained.

- Filter : Any multi-section lines meeting this filter will be merged.
 - "filtername" : See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.
 - SELECTED : Merge objects with Selected field = YES

This attempts to merge the line sections of each multi-section line that has the Selected field set to YES.

MergeMSLineSections(SELECTED);

Move([elementA], [destination parameters], HowMuch, [AbortOnError](#));

Use this action to move a generator, load, transmission line, or switched shunt.

- [elementA] : The object that should be moved. Must be one of the following formats:
 - [GEN busnum id], [GEN "name_nomkv" id],
 - [GEN "label"]
 - [LOAD busnum id] , [LOAD "name_nomkv" id],

```
[LOAD "label"],
[BRANCH busnum1 busnum2 ckt],
[BRANCH "name_kv1" "name_kv2" ckt],
[BRANCH "label"]
[SHUNT busnum id], [SHUNT "name_nomkv" id],
[SHUNT "label"],
[MULTISECTIONLINE busnum1 busnum2 ckt],
[MULTISECTIONLINE "name_kv1" "name_kv2" ckt],
[MULTISECTIONLINE "label"],
[3WXFORMER busnum1 busnum2 busnum3 ckt],
[3WXFORMER "name_kv1" "name_kv2" "name_kv3" ckt]
[3WXFORMER "label"]
```

[destination parameters]

: These parameters have different meanings depending on object type of the element. Must use bus numbers here:

```
GEN          : [busnum id]
LOAD         : [busnum id]
BRANCH       : [busnum1 busnum2 ckt]
SHUNT        : [busnum id]
MULTISECTIONLINE : [busnum1 busnum2 ckt]
3WXFORMER    : [busnum1 busnum2 busnum3 ckt]
```

HowMuch : The amount of the element to move. A value of 100 indicates that 100% should be moved. This parameter is only valid for generators and loads. It is ignored for lines and switched shunts.

AbortOnError : Optional flag that allows users to control if loading an AUX file will continue after an error is encountered.

Added in the October 4, 2024 patch of Simulator 23.

```
YES          : Abort processing script commands after
               encountering error
NO           : Continue processing script commands after error
```

This command moves 100% of the generator located at Bus 1 with ID 1 to Bus 2 with ID 2.

Move([Gen 1 1], [2 2], 100);

ReassignIDs(objecttype, field, filter, UseRight);

Use this action to set the IDs of specified objects to the first two characters of a specified field.

Objecttype : The type of object for which IDs are assigned. BRANCH, GEN, LOAD, and SHUNT are allowed.

field : The field that contains the IDs that will be assigned. Only the first two characters of the field will be assigned. Field is specified in format `variablenamelegacy:location` or `concisename`

Filter : (optional) Any objects meeting this filter will have their IDs reassigned. Blank is the default value:

```
Blank      : means all objects will be modified
ALL        : means all objects will be modified
SELECTED   : means only branches whose Selected field = YES will
              be modified
AREAZONE   : means only branches that meet the area/zone/owner
              filters will be modified
"FilterName" : means only objects that meet the specified filter will be
              modified. See the Using Filters in Script Commands
```

section for more information on specifying the filename.

UseRight : Optional parameter – default is NO
Set to YES or NO. If set to YES, the last two characters of the specified field will be assigned.

This command updates each LOAD ID using the first two letters of the name of the bus to which it is connected.

```
ReassignIDs(LOAD, "BusName", ALL, NO);
```

Remove3WXformerContainer(filter);

Use this action to delete the three-winding transformers matching the specified filter while leaving the internal two-winding transformers intact.

Filter : (optional) Any three-winding transformers meeting this filter will be deleted. Default is blank:

- Blank : means all three-winding transformers will be deleted
- ALL : means all three-winding transformers will be deleted
- SELECTED : means only three-winding transformers whose Selected field = YES will be deleted
- AREAZONE : means only three-winding transformers that meet the area/zone/owner filters will be deleted
- "FilterName" : means only three-winding transformers that meet the specified filter will be deleted. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command will remove all three-winding transformers in the case. However, it will leave the individual two-winding transformers that internally represent the 3-winding transformers intact.

```
Remove3WXformerContainer(ALL);
```

RenameInjectionGroup("OldName", "NewName");

This action will change the name of an existing injection group.

"OldName" : Name of the existing injection group.
"NewName" : New name of the existing injection group.

This command renames the injection group currently named "TopAreaGens" to "IG_Left_New". All references in the case to the old injection group name will now refer to the new name.

```
RenameInjectionGroup("TopAreaGens", "IG_Left_New");
```

RotateBusAnglesInIsland([BUS KeyField], Value);

All angles in the island to which the specified bus belongs will be rotated by the same shift such that the specified bus ends up with a bus angle specified by the **Value** parameter.

[BUS KeyField] : Objecttype identifier BUS followed by the keyfield identifier for the bus whose island identifies the buses that should be shifted and whose angle should be set to the specified **Value**.

Value : Angle value to which the specified bus should be set

This command identifies the electrical island that contains Bus 2 and shifts all bus voltage angles so that Bus 2 is set to 0.0 degrees. The voltage angles of all other buses in this island are adjusted relative to this new reference.

```
RotateBusAnglesInIsland([Bus 2], 0.0);
```

SetGenPMaxFromReactiveCapabilityCurve(filter);

Use this action to change the Maximum MW output of generators that use a capability curve, equal to the second-to-last MW point in the capability curve if the last Max Mvar point on the capability curve is smaller than 0.001 Mvar. If the present MW output is higher than this new Max MW value, then Max MW is set to the present MW output.

- filter : optional parameter that is used to specify which generators are processed. If blank, all generators are processed.
- Selected : means only generators whose Selected field = YES will be processed
- AREAZONE : means process those generators that meet the area/zone/owner filters.
- "FilterName" : See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

This command modifies MWMax based on reactive capability curves for all generators that meet the area/zone/owner filters.

SetGenPMaxFromReactiveCapabilityCurve(AREAZONE);
--

SetParticipationFactors(Method, ConstantValue, Object);

Use this action to modify the generator participation factors in the case

- Method : The formula used to calculate the participation factors for each generator. It may be one of the following strings:
- MAXMWRAT – base factors on the maximum MW ratings.
 - RESERVE – base factors on the (Max MW rating – Present MW).
 - CONSTANT – set factors to a constant value.
- ConstantValue : The value used if CONSTANT method is specified. If CONSTANT method is not specified, enter 0 (zero).
- Object : Specify which generators to set the participation factor for.
- [Area Num], [Area "name"], [Area "label"]
 - [Zone Num], [Zone "name"], [Zone "label"]
 - SYSTEM
 - AREAZONE or DISPLAYFILTERS

This sets the participation factors for all generators in the "Top" area to a constant value of 1.0.
--

SetParticipationFactors(CONSTANT, 1.0, [Area "Top"]);

SetScheduledVoltageForABus([bus identifier], voltage);

Use this action to set the stored scheduled voltage, vsched, for a bus according to how this is defined in the EPC format. This value is not used by Simulator but is stored for purposes of writing out to an EPC file. The setpoint voltages for generators and switched shunts regulating the specified bus are also set to the new voltage. The regulation range for switched shunts is modified for the new setpoint voltage according to how this is defined in the EPC format: $vband = (VHigh - VLow)/2$ with newVHigh = voltage+vband and new VLow = voltage-vband.

- [bus identifier] : This is the key field identifier for the bus being changed. This can contain the objecttype identifier of BUS followed by the key field or this identifier can be omitted. The square brackets are also optional. If using a string identifier such as the secondary key field or a label for the bus and that includes a comma, the identifier must be enclosed in double quotes or the square brackets must be used.
- voltage : the new voltage

This command sets the scheduled voltage value (Vsched) of Bus 2 to 1.02 pu.

```
SetScheduledVoltageForABus([BUS 2], 1.02);
```

SetInterfaceLimitToMonitoredElementLimitSum(filter);

This sets the limits of the interface to the sum of the limits of all branches within the interface. This only includes branches that are monitored and excludes any contingency branches. All limits A through H will be set.

Filter : This parameter is used to specify which interfaces have their limits set.

- ALL : all interfaces will be set
- SELECTED : only interfaces whose Selected field = YES will be set
- AREAZONE : only interfaces that meet the area/zone/owner filters will be set
- "FilterName" : only interfaces that meet the specified filter will be set.

See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command goes through all interfaces and sets each interface's limits (Limit A through Limit H) to be equal to the sum of the limits of all the monitored branches within that interface. It excludes any branches that are only used as contingency elements.

```
SetInterfaceLimitToMonitoredElementLimitSum(ALL);
```

SplitBus([element], NewBusNumber, InsertBusTieLine, LineOpen, BranchDeviceType);

Use this action to split buses

Element : Enter the description of which bus to split by enclosing in brackets the word bus and an identifier. The format looks as follows:

- [BUS num]
- [BUS "name_nomkv"]
- [BUS "buslabel"]

NewBusNumber : This is the number of the new bus to create

InsertBusTieLine : Optional parameter – default is YES
Set to YES or NO. YES will insert a low impedance tie line between the buses; NO will not.

LineOpen : Optional parameter – default is NO
Set to YES or NO. YES set the status of the inserted bus tie to OPEN. NO will set the status of the inserted bus tie to CLOSED.

BranchDeviceType : Optional parameter – default is "Line"
Specify the Branch Device Type of the branch inserted for the bus tie.
Options are: "Line", "Transformer", "Breaker", "Disconnect", "ZBR", "Fuse", "Load Break Disconnect", and "Ground Disconnect".

This command will split Bus 5 into two buses, assign bus number 999 to the new bus, and insert a closed Breaker between the original and new bus.

```
SplitBus([BUS 5], 999, YES, NO, "Breaker");
```

SuperAreaAddAreas("Name", Filter);

(Added in the July 23, 2024 patch of Simulator 23)

Use this action to add areas to an existing Super Area.

"Name" : Enter name of the existing Super Area

Filter : This parameter is used to specify which areas are added.

- ALL : all areas will be added
- SELECTED : only areas whose Selected field = YES will be set

AREAZONE : only areas that meet the area/zone/owner filters will be set

"FilterName" : only areas that meet the specified filter will be set. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command will add all areas where Selected = YES to an existing Super Area called "SuperArea1".

SuperAreaAddAreas("SuperArea1", ALL);

SuperAreaRemoveAreas("Name", Filter);

(Added in the July 23, 2024 patch of Simulator 23)

Use this action to remove areas from a Super Area.

"Name" : Enter name of the Super Area

Filter : This parameter is used to specify which areas are removed.

ALL : all areas will be removed

SELECTED : only areas whose Selected field = YES will be set

AREAZONE : only areas that meet the area/zone/owner filters will be set

"FilterName" : only areas that meet the specified filter will be set. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command will remove all Areas from SuperArea1 that match the active area/zone/owner filters.

SuperAreaRemoveAreas("SuperArea1", AREAZONE);

TapTransmissionLine([element], PosAlongLine, NewBusNumber, ShuntModel, TreatAsMSLine, UpdateOnelines, NewBusName);

Use this action to tap a transmission line by adding in a new bus and splitting the line in two.

Element : A description of the branch being tapped. The first bus listed will be treated as the nearbus that is used as the reference for the PosAlongLine. If the branch is identified by label, the from bus will be used as the reference for the PosAlongLine. Enclose description in brackets:
 [BRANCH busnum1 busnum2 ckt]
 [BRANCH "name_kv1" "name_kv2" ckt]
 [BRANCH "buslabel1" "buslabel2" ckt]
 [BRANCH "label"]

PosAlongLine : The percent distance along the branch at which the line will be tapped.

NewBusNumber : The number of the new bus created at the tap point.

ShuntModel : Optional parameter – default is CAPACITANCE
 How should the shunt charging capacitance values be handled for the split lines:
 LINESHUNTS : Line shunts will be created (keeps exact power flow model).
 CAPACITANCE : Convert shunt values capacitance in the PI model.

TreatAsMSLine : Optional parameter – default is NO
 Set to YES or NO. If set to YES, the two newly created lines will be made part of a mulit-section line.

UpdateOnelines : Optional parameter – default is NO

Set to YES or NO. If set to YES, the original display transmission line object will be replaced with the tapped display transmission line and display bus objects on all open oneline diagrams.

(Added in the February 20, 2024 patch of Simulator 23)

NewBusName : Optional parameter – default is blank
Specify the name of the new bus created at the tap point. If this is left blank, the new name will be the same as the new bus number.

This command taps the transmission line from Bus 1 to Bus 2 circuit 1 at 50% distance from Bus 1, creating a new bus with number 1001 and name "TapBus1_2". The line is split into two segments connected through the new bus. ShuntModel is set to LINESHUNTS to explicitly model line shunts. With TreatAsMSLine = YES, the segments are treated as a multi-section line, and UpdateOnelines = YES updates any oneline diagrams to reflect the changes.

```
TapTransmissionLine([BRANCH 1 2 1], 50, 1001, LINESHUNTS, YES, YES,  
"TapBus1_2");
```

Power Flow

ClearPowerFlowSolutionAidValues;

PowerWorld Simulator maintains several internal flags that keep track of which branches are closed or opened, as well as information to help estimate the generation change needed in a system after making changes to load or generation. PowerWorld uses this to help with various pre-processing steps in the power flow solution. This information is related to angle smoothing and generator MW estimation features of the power flow solution. Typically, this information is a great aid in getting successful power flow solutions, however in some circumstances you may be using an AUX file to edit information you know is good and would not want PowerWorld to modify the initial bus voltage and angle nor the generator MW outputs before a solution is attempted. To clear all this internally stored information so that PowerWorld does not do any of this, call the ClearPowerFlowSolutionAidValues script command.

ConditionVoltagePockets(VoltageThreshold, AngleThreshold, filter);

The goal of this script command is to find pockets of buses that may have bad initial voltage estimates and to get a better voltage estimate of these buses based on assuming that the voltages on buses outside these pockets are good. It will identify pockets of buses bounded by branches that meet the condition that the absolute value of the voltage difference across the branch is greater than VoltageThreshold or the absolute value of the angle difference across the branch is greater than AngleThreshold and the branch meets the specified filter.

- | | | |
|------------------|---|--|
| VoltageThreshold | : | Per-unit voltage difference (absolute value) that determines if a branch can be considered when determining groups of radial buses. |
| AngleThreshold | : | Angle difference in degrees (absolute value) that determines if a branch can be considered when determining groups of radial buses. |
| filter | : | This is an optional parameter that is used to specify which branches are checked. If omitted all branches are considered. |
| | | ALL : All branches will be checked |
| | | SELECTED : Only branches whose Selected field = YES will be checked |
| | | AREAZONE : Only branches that meet the area/zone/owner filters will be checked |
| | | "FilterName" : Only branches that meet the specified filter will be checked. See Using Filters in Script Commands section for more information on specifying the filtername. |

This command checks all branches in the system. If the absolute voltage difference across a branch exceeds 0.1 per unit, or the angle difference exceeds 10 degrees, that branch helps define a "pocket" of buses. The tool will then determine the buses in each pocket and estimate voltages better using known good values outside the pocket.

```
ConditionVoltagePockets(0.1, 10, ALL);
```

DiffCaseClearBase;

Call this action to clear the base case for the difference flows abilities of Simulator.

In Version 21 and earlier this script command was called **DiffFlowClearBase**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseClearBase** or **DiffFlowClearBase**.

DiffCaseKeyType(KeyType);

Use this action to change the key type that should be used when comparing fields when using the difference flows abilities of Simulator.

- | | | |
|---------|---|--|
| KeyType | : | String that starts with 'P' changes key field type to PRIMARY. |
| | | String that starts with 'S' changes key field type to SECONDARY. |
| | | String that starts with 'L' changes key field type to LABEL. |

In Version 21 and earlier this script command was called **DiffFlowKeyType**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseKeyType** or **DiffFlowKeyType**.

This command sets the key type used in PowerWorld Simulator's Difference Case comparison to PRIMARY.
--

<code>DiffCaseKeyType(PRIMARY);</code>
--

DiffCaseMode(diffmode);

Call this action to change the mode for the difference flows abilities of Simulator.

diffmode : String that starts with 'P' changes it to PRESENT.
String that starts with 'B' changes it to BASE.
String that starts with 'D' changes it to DIFFERENCE.
String that starts with 'C' changes it to CHANGE.

In Version 21 and earlier this script command was called **DiffFlowMode**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseMode** or **DiffFlowMode**.

This command sets PowerWorld Simulator's Difference Case mode to DIFFERENCE.
--

<code>DiffCaseMode(DIFFERENCE);</code>
--

DiffCaseSetAsBase;

Call this action to set the present case as the base case for the difference flows abilities of Simulator.

In Version 21 and earlier this script command was called **DiffFlowSetAsBase**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseSetAsBase** or **DiffFlowSetAsBase**.

DiffCaseShowPresentAndBase(How);

Call this action with the parameter of either YES or NO to toggle the difference flows options "Show Present|Base in Difference and Change Mode".

In Version 21 and earlier this script command was called **DiffFlowShowPresentAndBase**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseShowPresentAndBase** or **DiffFlowShowPresentAndBase**.

This command enables the "Show Present Base in Difference and Change Mode" option in PowerWorld Simulator.
--

<code>DiffCaseShowPresentAndBase(YES);</code>

DiffCaseRefresh;

Call this action to refresh the linking between the base case and the present case. This should be used before saving data that identifies objects as being added or removed, especially if any topological differences have been made that affect the comparison.

In Version 21 and earlier this script command was called **DiffFlowRefresh**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseRefresh** or **DiffFlowRefresh**.

DiffCaseWriteCompleteModel ("filename", AppendFile, SaveAdded, SaveRemoved, SaveBoth, KeyFields, "ExportFormat", UseAreaZone, UseDataMaintainer, AssumeBaseMeet, IncludeClearPowerFlowSolutionAidValues, DeleteBranchesThatFlipBusOrder);

In Version 21 and earlier this script command was called **DiffFlowWriteCompleteModel**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseWriteCompleteModel** or **DiffFlowWriteCompleteModel**.

Use this action to create an auxiliary file that contains information about objects that have been added or removed when comparing the present case to the base case when using the difference case comparison. Fields that have changed for objects that exist in both the present and base case can also be written to this auxiliary file. This auxiliary file can then be used to modify cases with these same changes.

"filename" : Name of the auxiliary file to create.

AppendFile	: Set to YES or NO. YES means to append the saved information to "filename". NO means that "filename" will be overwritten.
SaveAdded	: Set to YES or NO. YES means to save the added objects to the file. NO means to exclude the added objects.
SaveRemoved	: Set to YES or NO. YES means to save the removed objects to the file. NO means to exclude the removed objects.
SaveBoth	: Set to YES or NO. YES means to save the changed fields for the objects that exist in both the present and base case. NO means to exclude objects that occur in both cases.
KeyFields	: Optional parameter – default is Primary Set to Primary or Secondary to specify the key field identifiers to use for objects in the resulting file.
"ExportFormat"	: Optional parameter – default is blank This is the name of the Auxiliary File Export Format Description to use for defining the object types and fields that should be included in the auxiliary file.
UseAreaZone	: Optional parameter – default is NO Set to YES or NO. YES means to use the Area/Zone/Owner filter for including objects in the file. NO means to ignore this filter.
UseDataMaintainer	: Optional parameter – default is NO Set to YES or NO. YES means to use the Data Maintainer filter for including objects in the file. NO means to ignore the Data Maintainer filter.
AssumeBaseMeet	: Optional parameter – default is YES Set to YES or NO. YES means that areas/zones/owners and data maintainers that are in the base case and not in the present case meet the Area/Zone/Owner and Data Maintainer filters.
IncludeClearPowerFlowSolutionAidValues	: Optional parameter – default is YES Set to YES or NO. YES means that the ClearPowerFlowSolutionAidValues script command is included in the auxiliary file.
DeleteBranchesThatFlipBusOrder	: Optional parameter – default is NO Set to YES or NO. YES means that branches that have the order of their from and to terminal buses flipped will be included as both removed and added objects. The branch with the old order will be removed and the branch with the new order will be added. This is always done for transformers, but it is optional for non-transformer branches.

This command generates an auxiliary file named "B7FlatChanges.aux" that captures the differences between the Present and Base cases. It overwrites any existing file (rather than appending) and includes objects that were added, removed, or changed between the cases. Primary key fields are used to identify objects, and no special export format or area/zone/owner or data maintainer filters are applied. The base case is assumed to meet any filtering requirements even if certain objects are missing in the present case. The file also includes a command to clear solution aid values before applying changes, but it does not treat non-transformer branches with flipped terminal buses as removed and re-added.

```
DiffCaseWriteCompleteModel("B7FlatChanges.aux", NO, YES, YES, YES,
PRIMARY, "", NO, NO, YES, YES, NO);
```

DiffCaseWriteBothEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, "ExportFormat", UseDataMaintainer);

Call this action to save any elements in both the base and present cases as determined using the difference flows functionality. The elements are saved in the GE EPC format. See the DiffFlowWriteRemovedEPC script command for the descriptions of the parameters that are common among all script commands used to save difference case change files in the EPC format. Parameters that are specific to DiffCaseWriteBothEPC are described here:

"ExportFormat" : Optional parameter – default is blank.
This is the name of the Auxiliary File Export Format Description to use for defining the object types that should be included in the file. Only object types that are allowed in an EPC can be included and others will be skipped. Fields that are specified with the object types in the format will be used to determine if an object has changed based on those fields changing between the present and base case. If any field has changed for an object, the entire object will be written including all fields that are required in the EPC format.

In Version 21 and earlier this script command was called **DiffFlowWriteBothEPC**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseWriteBothEPC** or **DiffFlowWriteBothEPC**.

This command saves all elements that exist in both the base and present cases (and have differences) to a GE EPC file named "B7Flat_Changes.epc". It does not apply the area/zone/owner filters (UseAreaZone = NO), but it assumes the base case meets the filter (BaseAreaZoneMeetFilter = YES). The file is not appended (Append = NO), and no custom export format is specified, so all valid EPC object types and their standard fields will be considered.

```
DiffCaseWriteBothEPC("B7Flat_Changes.epc", GE23, NO, YES, NO, "");
```

DiffCaseWriteNewEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, UseDataMaintainer);

Call this action to save any new elements determined using the difference flows functionality. The elements are saved in the GE EPC format. See the DiffFlowWriteRemovedEPC script command for the descriptions of the parameters.

In Version 21 and earlier this script command was called **DiffFlowWriteNewEPC**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseWriteNewEPC** or **DiffFlowWriteNewEPC**.

This command saves new elements identified by difference case to the file "B7Flat_NewElements.epc". It includes all new elements regardless of area/zone/owner filters, treats areas/zones not in the base case as meeting the filter (YES), and overwrites the file if it already exists.

```
DiffCaseWriteNewEPC("B7Flat_NewElements.epc", GE23, NO, YES, NO);
```

DiffCaseWriteRemovedEPC ("filename", GEFileType, UseAreaZone, BaseAreaZoneMeetFilter, Append, UseDataMaintainer);

In Version 21 and earlier this script command was called **DiffFlowWriteNewEPC**. Simulator 21 patches after January 20, 2021 will handle reading either the **DiffCaseWriteNewEPC** or **DiffFlowWriteNewEPC**. Call this action to save any removed elements determined using the difference flows functionality. The elements are saved in the GE EPC format.

"filename" : The path and name of the file to save.
GEFileType : Optional parameter – default is to save with the latest version. Valid options:
GE14-GE23 (must specify with the version number, GE##)

- UseAreaZone : Optional parameter – default is NO.
Set to YES or NO. YES means to save only those objects that meet the Area/Zone/Owner filter.
- BaseAreaZoneMeetFilter : Optional parameter – default is NO.
Set to YES or NO. YES means that areas or zones that are not in the difference flows base case will be treated as meeting the Area/Zone filter if that is used.
- Append : Optional parameter – default is YES.
Set to YES or NO. YES means to append data to an existing file. NO means to overwrite an existing file.
- (Added in the October 30, 2025 patch of Simulator 24)**
- UseDataMaintainer : Optional parameter – default is NO
Set to YES or NO. YES means to use the Data Maintainer filter for including objects in the file. NO means to ignore the Data Maintainer filter.

The command creates a file named "B7Flat_RemovedElements.epc" to store removed elements in GE EPC format, using version GE23. It includes only elements that pass the current area/zone/owner filters (YES), excludes areas/zones not present in the base case (NO), and overwrites any existing file with the same name.

```
DiffCaseWriteRemovedEPC("B7Flat_RemovedElements.epc", GE23, YES, NO, NO, NO);
```

DoCTGAction([contingency action]);

DoCTGAction("contingency action");

Call this action to use the formats seen in the CTGElement subdata record for Contingency Data. Note that all actions are supported, except COMPENSATION sections are not allowed. The action must be enclosed in either brackets or double quotes.

This command takes the branch from Bus 1 to Bus 2 Circuit 1 out of service. The branch is identified by using the secondary key fields of the buses.

```
DoCTGAction([BRANCH One_138.0 Two_138.0 1 OPEN]);
```

EstimateVoltages(filter);

This will estimate voltages and angles at the buses that meet the filter based on voltages and angles at surrounding buses (these are buses that do not meet the filter). It is assumed that the voltages and angles are correct at the surrounding buses, and in order for this command to work, there must be some buses that do not meet the filter.

- filter : See the [Using Filters in Script Commands](#) section for more information on specifying the filter. A valid filter must be specified. An error will result if a blank filter is specified.

This script estimates voltages and angles at buses in the Area named "Top" based on neighboring buses outside this area. The filter is specified as a single-condition filter.

```
EstimateVoltages("AreaName = Top");
```

GenForceLDC_RCC(filter);

Use this action to force generators in the case onto line drop / reactive current compensation. The present voltage at the point at which the generator is controlling based on the line drop/reactive current compensation impedance is calculated, and the setpoint of the generator is set to this value. If the absolute value of the line drop/reactive current compensation impedance is less than or equal to $2 \times 10^{-6} \times \text{MVA Base}$, the generator will regulate its terminal bus and the setpoint voltage is set to the present value of the terminal bus voltage. For a typical case with an MVA Base of 100 MVA, this value is 0.0002.

filter : Optional parameter – default is to set all generators
See the [Using Filters in Script Commands](#) section for more information on specifying the filter.

InterfacesCalculatePostCTGMWFlows;

Call this action to update the "Interface MW Flow", "Contingent MW Flow" fields on all Contingent Interfaces defined in the case. The fields will be updated according to the option chosen in [Simulator Options > Power Flow Solution > Monitor/Enforce Contingent Interface Elements](#). If "Never" has been chosen, this script command will have no impact, i.e., the "Contingent MW Flow" field will be zero, and the "Interface MW Flow" field will equal "Base MW Flow".

ResetToFlatStart (FlatVoltagesAngles, ShuntsToMax, LTCsToMiddle, PSAnglesToMiddle);

Use this action to initialize the Power Flow Solution to a "flat start." The parameters are all optional and specify a conditional response depending on whether the solution is successfully found. If parameters are not passed then default values will be used.

FlatVoltagesAngles : Set to YES or NO. YES means setting all the voltage magnitudes and generator setpoint voltages to 1.0 per unit and all the voltage angles to zero. Default Value = YES.
ShuntsToMax : Set to YES or NO. YES means to increase Switched Shunts Mvar half way to maximum. Default Value = NO.
LTCsToMiddle : Set to YES or NO. YES means setting the LTC Transformer Taps to middle of range. Default Value = NO.
PSAnglesToMiddle : Set to YES or NO. YES means setting Phase Shifter angles to middle of range. Default Value = NO.

This command resets the power flow case to a flat start. It sets all voltage magnitudes and generator setpoints to 1.0 per unit and all voltage angles to zero (FlatVoltagesAngles = YES). Additionally, it adjusts all switched shunts to halfway between their current and maximum Mvar values (ShuntsToMax = YES). LTC transformer taps and phase shifter angles are left unchanged (LTCsToMiddle = NO, PSAnglesToMiddle = NO).

ResetToFlatStart(YES, YES, NO, NO);

SaveGenLimitStatusAction("filename");

Use this action to save Mvar information about generators in a text file. The information saved includes the generator bus number, generator ID, Mvar, Max Mvar, Min Mvar, AVRable flag (user specified), and internal AVRable flag (set by Simulator). This information is useful for debugging.

"filename" : Name of the text file in which to save the generator information.

This command saves information about the reactive power (Mvar) limits and AVR (automatic voltage regulation) status for all generators in the case to a file named "GenLimitReport.txt".

SaveGenLimitStatusAction("GenLimitReport.txt");

SaveJacobian("JacFileName", "JIDFileName", FileType, JacForm);

Use this action to save the Jacobian Matrix to a text file or a file formatted for use with Matlab

"JacFileName"	: File in which to save the Jacobian.
"JIDFileName"	: File to save a description of what each row and column of the Jacobian represents.
FileType	: One of: M – Matlab form TXT – Text file EXPM – Save Jacobian in Exponential form (ex. 1.2E-2) in Matlab form
JacForm	: One of: R – AC Jacobian in Rectangular coordinates P – AC Jacobian in Polar coordinates DC – B' matrix of DC power flow

This command saves the Jacobian matrix of the current power system case to a file named "B7FlatJacobian.mat" in MATLAB format (M), using polar coordinates (P). The file "B7FlatJacobianIDs.txt" will contain descriptions of what each row and column in the Jacobian matrix represents—such as voltage angles, magnitudes, or power injections at each bus.

SaveJacobian("B7FlatJacobian.mat", "B7FlatJacobianIDs.txt", M, P);

SaveYbusInMatlabFormat("filename", IncludeVoltages);

Use this action to save the YBus to a file formatted for use with Matlab

"filename"	: File in which to save the YBus.
IncludeVoltages	: YES includes the per unit bus voltages in the file; NO does not include.

This command saves the Y-bus matrix of the current system case to a file named "B7FlatYbus.mat" in a format that is compatible with MATLAB. By setting the second parameter to YES, the file will also include the per-unit voltage magnitudes and angles for each bus in the system.

SaveYbusInMatlabFormat("B7FlatYbus.mat", YES);

SolvePowerFlow (SolMethod, "filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);

Call this action to perform a single power flow solution. The parameters are all optional and specify a conditional response depending on whether the solution is successfully found. If parameters are not provided, default values will be used.

SolMethod	: Optional parameter – default is RECTNEWT if the case is in AC power flow mode or DC if the case is in DC power flow mode. The solution method to be used for the Power Flow calculation. If the DC method is selected, the case is switched to DC power flow mode. If one of the other AC methods is selected, the case is switched to AC power flow mode. It might be difficult to solve a case with an AC power flow method once the case has been switched to DC power flow mode, so take this into consideration before using the DC method. The options are: RECTNEWT : for Rectangular Newton-Raphson. POLARNEWTON : for Polar Newton-Raphson. GAUSSSEIDEL : for Gauss-Seidel. FASTDEC : for Fast Decoupled. ROBUST : for attempting the robust solution process DC : for DC power flow calculation
-----------	--

"filename1"	: Optional parameter – default is blank
-------------	---

The filename of the auxiliary file to be loaded if there is a successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default = "".

"filename2" : Optional parameter – default is blank

The filename of the auxiliary file to be loaded if there is a NOT successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default = "".

CreaterIfNotFound1 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.

This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename1".

CreaterIfNotFound2 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.

This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename2".

This command attempts to solve the power flow using the Rectangular Newton-Raphson method. If the solution is successful, the auxiliary file "OnSuccess.aux" is loaded with object creation allowing for any missing elements referred to in DATA sections to be created. If the solution fails, "OnFailure.aux" is loaded, but new objects will not be created during the load for DATA sections in the Legacy Auxiliary File Header format.

```
SolvePowerFlow(RECTNEWT, "OnSuccess.aux", "OnFailure.aux", YES, NO);
```

UpdateIslandsAndBusStatus;

Changes to branch and generator status impact islands and whether or not buses are connected. Islands and bus status are always updated at the beginning of a power flow solution if necessary, but this script command makes it convenient to update this information without requiring a power flow solution.

ZeroOutMismatches(ObjectType);

With this script command, bus shunts or loads are changed at each bus that has a mismatch greater than the MVA convergence tolerance so that the mismatch at that bus is forced to zero.

ObjectType : Optional parameter – default is BUSSHUNT

Set to BUSSHUNT or LOAD to indicate how the mismatch should be adjusted. When set to BUSSHUNT the Bus Shunt fields at each bus where there is a mismatch are adjusted to eliminate the mismatches. When set to LOAD, a new load is added to a bus where there is a mismatch to eliminate the mismatches. The new loads are identified by a unique ID of Q1. If a load with this ID already exists, a unique ID is determined by incrementing the ID until a unique ID is found.

This command checks every bus in the system and identifies those where the power flow mismatch exceeds the MVA convergence tolerance. For each of those buses, it adds a new load (with an ID like Q1, Q2, etc.) to cancel out the mismatch.

```
ZeroOutMismatches(LOAD);
```

DeleteState(WhichState, StateName);

Apply this script command to delete a specified system state that is currently in memory. This script command will fail if the specified state has not been set. The following options are available for specifying which system state to delete:

- WhichState : Optional parameter – default is USER. This determines which state to restore.
- USER : Delete the user set system state that is set with the StoreState script command.
- BEFOREFAILED : Before the power flow is solved, either through the GUI or the SolvePowerFlow script command, a system state will be stored in the event that the power flow solution fails. This pre-solution state is deleted with this option. If the power flow is successful in solving at any time after this pre-solution state is stored, this pre-solution state is removed.
- LASTSUCCESSFUL : If the power flow solution is successful when solving the power flow either through the GUI or the SolvePowerFlow script command, a system state will be stored with this successful solution. This post-solution state is deleted with this option.
- StateName : Optional parameter – default is blank
This option is only used when WhichState = USER. This specifies a named system state that was stored using the StoreState script command. If no name is specified, the unnamed system state stored using StoreState will be deleted.

This command deletes a previously stored user-defined system state called "PreContingency".

DeleteState("User", "PreContingency");
--

RestoreState(WhichState, StateName);

Apply this script command to restore a specified system state that is currently in memory. This script command will fail if the specified state has not been set. The following options are available for specifying which system state to restore:

- WhichState : Optional parameter – default is USER. This determines which state to restore.
- USER : Restore the user set system state that is set with the StoreState script command.
- BEFOREFAILED : Before the power flow is solved, either through the GUI or the SolvePowerFlow script command, a system state will be stored in the event that the power flow solution fails. This pre-solution state is restored with this option. If the power flow is successful in solving at any time after this pre-solution state is stored, this pre-solution state is removed and cannot be restored.
- LASTSUCCESSFUL : If the power flow solution is successful when solving the power flow either through the GUI or the SolvePowerFlow script command, a system state will be stored with this successful solution.

This post-solution state is restored with this option.

StateName : Optional parameter – default is blank
This option is only used when WhichState = USER. This specifies a named system state that was stored using the StoreState script command. If no name is specified, the unnamed system state stored using StoreState will be restored.

This command restores a previously stored user-defined system state called "PreContingency".
--

RestoreState("User", "PreContingency");

StoreState(StateName);

Apply this script command to store the current system state to memory. Use the RestoreState script command to restore the state. Multiple states can be stored by providing a unique name.

StateName : Optional parameter
Multiple states can be stored by providing a unique name for each state.
If StateName is not specified, an unnamed state is stored.

This command stores the current system state and saves it under the name "PreContingency".
--

StoreState("PreContingency");

VoltageConditioning;

Perform voltage conditioning based on the Voltage Conditioning tool options and case voltage targets.

User Interface

Animate(DoAnimate);

Use this action to animate all the open oneline diagrams.

DoAnimate : Set to YES or NO. YES means to start the animation of the open oneline diagrams, while NO means that the animation will be paused.

This command starts the animation of flow objects on all open oneline diagrams.

Animate(YES);

MessageBox("text");

Use this action to open a dialog box that will display the entered text. This script command will fail if using the SimAuto add-on.

"text" : Text that will appear in the dialog box.

Opens a message box with the greeting "Hello".
--

MessageBox("Hello");

ObjectFieldsInputDialog("ObjectIDString", [fieldlist], "DialogCaption", "DialogExplain", [LabelCaptions], [TabBreaks], [TabCaptions], [RowBreaks], [RowCaptions], [ColBreaks], [ColCaptions]);

Use this action to open a dialog box displaying the list of specified fields for the specified object. This will allow the fields to be modified in the same manner as they can through case information displays. This script command will fail if using the SimAuto add-on.

"ObjectIDString" : The specific object for which display fields are displayed. The format is the object type followed by the key fields used to identify the object. Examples: "Bus 234891", "Gen 16445 'A'", "Branch 1239 1234 'AB'".

[fieldlist] : A list of fields to to display for the specified object.

"DialogCaption" : Optional with default of blank. This is the caption that will appear on the dialog.

"DialogExplain" : Optional with default of blank. This is an explanation that will appear in a text at the top of the dialog underneath the caption.

[LabelCaptions] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of captions that will appear with the respective fields. The captions must be enclosed in double quotes if there are any commas in the string. If now label captions are specified, then the concise variable names will be used to indicate what each field is

[TabBreaks] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of integers that indicate that a tab break occurs before the field at the particular index. The fields are indexed starting at zero. The dialog that appears will be created with the first "tab" representing a panel at the TOP of the dialog. This top panel will be made a fixed height so that all rows of fields can be seen. Any subsequent tabs will be placed inside a Tabbed control. The tabbed control will take up the remainder of the size of the dialog.

[TabCaptions] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of captions that will appear with the respective tab break. Each tab break will represent a TAB on the tabbed control. These will be the captions. If nothing is specified, the captions will simply numbered.

[RowBreaks] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of integers that indicate that a row break occurs before the field at the particular index. The fields are indexed starting at zero. Each tab of the dialog will be drawn with controls optionally grouped into

- rows and then these rows optionally grouped into columns. A particular "cell" of this table can then have multiple fields inside it.
- [RowCaptions] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of captions that will appear with the respective group box that starts with the field at this index. The group box will contain all fields up until the next Column or Row break. Blank captions are also allowed, in which case a group box is not drawn.
- [ColBreaks] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of integers that indicate that a column break occurs before the field at the particular index. The fields are indexed starting at zero. Each tab of the dialog will be drawn with controls optionally grouped into rows and then these rows optionally grouped into columns. A particular "cell" of this table can then have multiple fields inside it.
- [ColCaptions] : Optional with default of []. Inside brackets, you may enter a comma-delimited list of captions that will appear with the respective group box that starts with the field at this index. The group box will contain all fields up until the next Column or Row break. Blank captions are also allowed, in which case a group box is not drawn.

The following image depicts what the resulting dialog would show for the following script command. Note the field list is abbreviated but for this example there are 21 fields listed in the same manner as other script commands.

```
ObjectFieldsInputDialog("Branch 5 6 1", [Field0, ... Field20],
    "Add Caption Here", "Explain Stuff Here", [],
    [4, 12], [My Cap, Another],
    [5, 11, 12, 17, 19], [EDFG, "Test, Cap", "Heref", ABCD, ""],
    [8, 14, 15, 18, 18], [HIJK, , LMNO, , XYZ]
);
```

A few things of note in this example.

Column caption that goes with Index 14 is blank so no group box is drawn.

Column index 18 is listed twice in the ColBreaks which results in the empty column between Field 17 and 18.

RowBreak index 12 would seem to be unnecessary but provide the mechanism to add the caption "Heref".

Initial is results in pane at the top

Additional tabs are inside tabbed control. Captions are strings in [TabCaptions]

My Cap Another

LabelCaption0 Field 0

LabelCaption1 Field 1

LabelCaption2 Field 2

LabelCaption3 Field 3

LabelCaption4 Field 4

LabelCaption5 Field 5

LabelCaption6 Field 6

LabelCaption7 Field 7

LabelCaption8 Field 8

LabelCaption9 Field 9

LabelCaption10 Field 10

LabelCaption11 Field 11

LabelCaption12 Field 12

LabelCaption13 Field 13

LabelCaption14 Field 14

LabelCaption15 Field 15

LabelCaption16 Field 16

LabelCaption17 Field 17

LabelCaption18 Field 18

LabelCaption19 Field 19

LabelCaption20 Field 20

Close

OpenDataView("ObjectIDString", "DataGridIDString");

Use this action to open the Data View Dialog to a particular object using a particular set of customized string grid options. The string grid options determine the fields to show as well as whether to have any Tab, Row or Column breaks on the dialog in the same manner as is done for the ObjectFieldsInputDialog() script command.

- "ObjectIDString" : The specific object for which fields are displayed. The format is the object type followed by the key fields used to identify the object.
Examples: "Bus 234891", "Gen 16445 'A'", "Branch 1239 1234 'AB'".
- "DataGridIDString" : Optional. If not specified dialog will open with the first customized grid. This is a reference to either a DataGrid or a UserDefinedDataGrid object. DataGrid objects store the customizations used on various case information displays in PowerWorld Simulator. A part of the customization for a DataGrid includes information about the Data View Layout (allows tab, row, and column breakers along with captions for tabs and group boxes). The format for this string is object type string DataGrid or UserDefinedDataGrid the key fields for that object (only a name for the DataGrid, and name followed by object type for UserDefinedDataGrid). Examples:

"DataGrid 'BranchRun'"

"DataGrid 'BranchEdit'"

"UserDefinedDataGrid 'My named grid' Bus"

Note: you may also simply enter a string showing the name of either the DataGrid or UserDefinedDataGrid. If you do this, then Simulator will first look for a DataGrid with that name. If a DataGrid is not found, then we will look for a UserDefinedDataGrid that matches the name specified and assumes the object type matches what is specified for the ObjectIDString.

This command opens the Data View Dialog for the branch connecting Bus 1 to Bus 2 with circuit ID "1", using a customized grid layout named "BranchEdit". This layout controls which fields are shown and how they are organized.
--

OpenDataView("Branch 1 2 '1'", "DataGrid 'BranchEdit'");
--

Online Actions

CloseOnline("OnlineName");

Use this action to close an open online diagram without saving it. If the name is omitted, the last focused online diagram will be closed.

"OnlineName" : The name of the online diagram to close.

This call closes the online diagram named "B7FaultExample.pwd".

CloseOnline("B7FaultExample.pwd");

EditMultipleOnlineAction("Path", LinkType, SaveFileType);

Use this action to convert all files with a PWD extension in a specified directory to a new format. This is useful for converting files from a newer version of Simulator to an older version. The files will be saved with the same name but with an extension appropriate for the SaveFileType.

"Path" : Specify a valid path where the files are located.

LinkType : Specify the key field identifier to use for linking objects in the online diagrams to a power flow case. Options are NUMBER, NAMENOMKV, and LABEL.

SaveFileType : Specify the new format for the online diagrams. Valid options are: PWB, PWB16-PWB23, and AUX.

EnumerateDDLOnelines("InputDSET", "OutputList");

(Added in the February 29, 2024 patch of Simulator 23)

Use this action to analyze an Areva/Alstom/GE DSET DDL file and write out a listing of all diagrams contained within the file.

"InputDSET" : Specify a valid path to the DSET file in DDL text format that is to be analyzed.

"OutputList" : Specify a valid path to a text file that will contain a listing of all the diagram names contained within the DSET DDL file separated by line.

ExportBusView("filename", "bus key", ImageType, Width, Height, [ExportOptions]);

Use this action to export an image of a bus view online diagram to a file.

"filename" : Name of the file in which the exported image will be saved. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

"Bus key" : The specific bus. The format is the object type followed by the key fields

ImageType : The type of image to save. Valid options are: BMP, GIF, JPG, EMF, WMF, and PDF.

Width : Width of the saved image in pixels.

Height : Height of the saved image in pixels.

[ExportOptions] : Optional parameter

This is a comma separated list of options based on the ImageType that is being exported.

When exporting an image of type JPG, the following options case be specified:

ImageQuality : Quality of the image specified from 1 to 100 with 100 being the highest quality image. The larger the image quality the larger the resulting file will be. Default is 80.

ResolutionScalar : The resolution can be changed from the default resolution by adjusting by this scalar. To increase the resolution set the scalar to something greater

than 1. Increasing the resolution will also increase the file size. Default is 1.

When exporting an image of type GIF, the following options can be specified:

- NumFrames : GIF images can be animated by introducing multiple frames. This value specifies the number of frames. Default is 1.
- FrameDelay : Number of seconds to wait between frames. Default is 0.1.
- ResolutionScalar : The resolution can be changed from the default resolution by adjusting by this scalar. To increase the resolution set the scalar to something greater than 1. Increasing the resolution will also increase the file size. Default is 1.

This command exports the one-line diagram view centered on Bus 1 to a JPG image file named "bus_1.jpg" saved in the H:\ directory. The image will be 800×600 pixels, with a high quality (ImageQuality = 90) and a resolution scaling factor of 1.5 for enhanced clarity.

```
ExportBusView("H:\bus_1.jpg", "BUS 1", JPG, 800, 600, [90, 1.5]);
```

ExportOnline("filename", "OnlineName", ImageType, "view", FullScreen, ShowFull, [ExportOptions]);

Use this action to export an image of the open online diagram to a file containing the specified image type.

- "filename" : Name of the file in which the exported image will be saved. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- "OnlineName" : The name of the online diagram to export. The online diagram must be open. Use the OpenOnline script command if necessary to open the appropriate online.
- ImageType : The type of image to save. Valid options are: BMP, GIF, JPG, EMF, WMF.
- "view" : Optional parameter. The view name that should be opened. Pass an empty string to denote no specific view.
- FullScreen : Optional parameter with default of NO. Set to YES or NO. YES means that the online diagram will be open in full screen mode. If this parameter is not specified, then NO is assumed.
- ShowFull : Optional parameter with default of NO. Set to YES to open the online and apply the Show Full option. Set to NO to open the online and leave the online as is.
- [ExportOptions] : Optional parameter
This is a comma separated list of options based on the ImageType that is being exported.

When exporting an image of type JPG, the following options can be specified:

- ImageQuality : Quality of the image specified from 1 to 100 with 100 being the highest quality image. The larger the image quality the larger the resulting file will be. Default is 80.
- ResolutionScalar : The resolution can be changed from the default resolution by adjusting by this scalar. To increase the resolution set the scalar to something greater

than 1. Increasing the resolution will also increase the file size. Default is 1.

When exporting an image of type GIF, the following options can be specified:

- NumFrames : GIF images can be animated by introducing multiple frames. This value specifies the number of frames. Default is 1.
- FrameDelay : Number of seconds to wait between frames. Default is 0.1.
- ResolutionScalar : The resolution can be changed from the default resolution by adjusting by this scalar. To increase the resolution set the scalar to something greater than 1. Increasing the resolution will also increase the file size. Default is 1.

This command exports the currently open online diagram named "B7Flat" to a high-resolution JPG image called "main_view.jpg" in H:\. No specific view is selected, and full-screen mode is not enabled (FullScreen = NO), but the "Show Full" option is applied (YES), so the entire diagram view is captured. The image is saved with high quality (ImageQuality = 90) and a resolution scaling factor of 1.5 for enhanced clarity.

```
ExportOnline("H:\main_view.jpg", "B7Flat", JPG, "", NO, YES, [90, 1.5]);
```

ExportOnlineAsShapeFile("filename", "OnlineName", "ShapeFileExportDescriptionName", UseLonLat, PointLocation);

Use this action to save an open online diagram to a shapefile.

- "filename" : The file name of the shapefile to save.
- "OnlineName" : The name of the online diagram to save to a shapefile. The online diagram must be open. Use the OpenOnline script command if necessary to open the appropriate online.
- "ShapeFileExportDescriptionName" : Name of the ShapeFile Export Description to use when saving the shapefile.
- UseLonLat : Set to YES or NO. YES means that the coordinates of objects on the online diagram will be saved using longitude,latitude. This will only be true if a valid map projection is in use with the online diagram. Otherwise, the coordinates will be saved in x,y. If this parameter is set to NO, the coordinates will be saved in x,y. If this parameter is not specified, YES is assumed.
- PointLocation : Determines where points are specified – object centers, or the upper left corner. Specify "center" to define points as the shape centers, or "ul" to define them as the upper left corner of the shapes. If not specified, upper left is assumed.

This command exports the open online diagram named "B7Flat" to a shapefile named "grid_map.shp", saved in the H:\ directory. The export uses the shapefile export description "GenAndLineShapes", which defines which objects and fields are included. Since UseLonLat = YES, the coordinates will be saved using longitude and latitude, assuming the online uses a valid map projection. The point location is set to center, meaning point-based shapes will use the center of each object (e.g., for generator or bus icons) when written to the shapefile.

```
ExportOnlineAsShapeFile("H:\grid_map.shp", "B7Flat", "GenAndLineShapes", YES, center);
```

ImportDDLAsTranslation("filename");

This loads an ESET DDL and converts some definitions into translations within Simulator. This currently only works for keyset definitions that set links to open onelines.

"filename" : The name of file to load.

LoadAXD("filename", "OnlineName", CreatelfNotFound)

Use this action to apply a display auxiliary file to an open online diagram.

"filename" : The file name of the display auxiliary file to load.

"OnlineName" : The name of the online diagram to which to apply the display auxiliary file. If the online is not already open, the OpenOnline script command can be used to open the appropriate online. If the specified online is not open, a new one will be created with the given name.

CreatelfNotFound : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.

This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename".

OpenOnline("filename", "view", FullScreen, ShowFull, LinkMethod, Left, Top, Width, Height);

Use this action to open a online diagram. When using SimAuto, this action cannot be used to actually view a online. This script can be used in SimAuto to associate onelines with a PWB file. Any online that is opened using the script command and while the case is saved will opened in the GUI once the case is reopened.

"filename" : The file name of the online diagram to open. Wildcards are allowed when opening a DDL file type. This is useful for loading DDL files via browsing patch searches.

"view" : The view name that should be opened. Pass an empty string to denote no specific view.

FullScreen : Set to YES, NO, or MAX. YES means that the online diagram will be open in full screen mode. If this parameter is not specified, then NO is assumed. [If MAX is specified, then FullScreen is NO, but the online will be maximized when it is opened \(Added in July 25, 2024 patch for Simulator Version 23\)](#).

ShowFull : Optional parameter. Set to YES to open the online and apply the Show Full option. Set to NO to open the online and leave the online as is. Default is NO if not specified.

LinkMethod : Optional Parameter that controls online linking. LABELS, NAMENOMKV, and NUMBER will link using the respective key fields.

Left : Optional with default of 0. Value between 0 and 100 that indicates the location of the left edge of the online as a percentage of the Simulator/Retriever window width.

Top : Optional with default of 0. Value between 0 and 100 that indicates the top edge of the online as a percentage of the Simulator/Retriever window height.

Width : Optional with default of 0. Value between 0 and 100 that indicates the width of the online as a percentage of the Simulator/Retriever window width.

Height : Optional with default of 0. Value between 0 and 100 that indicates the height of the online as a percentage of the Simulator/Retriever window height.

This command opens the oneline diagram file "B7Flat.pwd" from the H:\ directory. No specific view is selected, and the oneline is opened in maximized mode (FullScreen = MAX) with the Show Full option applied (YES), meaning the entire diagram will be zoomed to fit. Objects on the oneline will be linked to power system elements using the Name and Nominal kV (NAMENOMKV) method. The oneline window will be positioned to start at 10% from the left and 10% from the top of the screen, and occupy 80% of the screen width and height.

```
OpenOnline("H:\B7Flat.pwd", "", MAX, YES, NAMENOMKV, 10, 10, 80, 80);
```

RelinkAllOpenOnelines;

Making modifications to the power flow case could cause objects on a oneline from becoming unlinked. This action will attempt to relink all objects on all open onelines.

SaveOnline("filename", "OnlineName", SaveFileType);

Use this action to save an open oneline diagram to file

"filename"	: The path and file name of the file to save. If a full path is not specified, then the file is saved to the current directory.
"OnlineName"	: Name of the open oneline to save.
SaveFileType	: Type of file to save. Valid options are AXD, PWB, and PWB16-PWB23. If omitted, PWB, which is the most recent version, will be assumed. Note the use of "PWB" instead of "PWD" is not a typo. The version of the PWD file corresponding to the PWB version will be used.

OpenBusView("Bus key", ForceNewWindow);

Opens the Bus View to a particular bus specified in the first parameter.

"Bus key"	: The specific bus. The format is the object type followed by the key fields
ForceNewWindow	: Optional with default of NO. Set to YES to force a new bus view to be opened regardless. If NO, then if a bus view is already open the command will update that bus view instead of opening a new one.

OpenSubView("Substation key", ForceNewWindow);

Opens the Substation View to a particular substation specified in the first parameter.

"Substation key"	: The specific substation. The format is the object type followed by the key fields
ForceNewWindow	: Optional with default of NO. Set to YES to force a new substation view to be opened regardless. If NO, then if a substation view is already open the command will update that substation view instead of opening a new one.

Connections Tools

CreateNewAreasFromIslands;

Use this action to create permanent areas that match the area Simulator creates temporarily while solving the power flow. New areas are created if and area is on AGC, spans multiple viable islands, and only one of those islands has more than one area in it.

DetermineBranchesThatCreateIslands(Filter, StoreBuses, "filename", SetSelectedOnLines, FileType);

Use this action to determine the branches whose outage results in island formation. Note that setting the Selected field will overwrite the Selected fields.

Filter	: This parameter is used to specify which branches are checked. ALL : means all branches will be checked SELECTED : means only branches whose Selected field = YES will be checked AREAZONE : means only branches that meet the area/zone/owner filters will be checked "FilterName" : means only branches that meet the specified filter will be checked. See the Using Filters in Script Commands section for more information on specifying the filename.
StoreBuses	: YES to store the buses in the island to the output file
"filename"	: file to which the results will be written. The format of the file is based on the auxiliary file format. Each branch that was checked will be followed by the list of buses that are islanded. The branch and bus information will be written in appropriate auxiliary file DATA format. If this is left blank, SetSelectedOnLines will be assumed to be YES.
SetSelectedOnLines	: YES to set the SELECTED field to YES for branches that create islands
FileType	: Optional parameter used to specify the format of the file. This is AUX by default. AUX : The saved file is based on an auxiliary file data format. Each branch that causes an island appears in the file in the auxiliary file data format followed by a auxiliary file bus data section containing all of the buses that are islanded by the preceding branch. CSV : The saved file is a comma-delimited text file. Each unique bus/branch pair appears on a single line. A unique bus/branch pair is determined by a bus that is islanded and a particular branch that causes it to be islanded. A header appears in the file specifying the fields used to identify the branch and bus in each record.

Evaluates each branch to see if its removal causes part of the system to become electrically isolated. If so, it sets the branch's Selected field to YES and logs each affected bus in a separate row of a CSV file.

DetermineBranchesThatCreateIslands(ALL, YES, "H:\branches.csv", YES, CSV);

DeterminePathDistance([start], BranchDistMeas, BranchFilter, BusField);

Use this action to calculate a distance measure at each bus in the entire model. The distance measure will represent how far each bus is from the starting group specified. The distance measure can be related to impedance, geographical distance, or simply the number of nodes.

[start]	: The starting location. The starting location may be either a Bus, Area, Zone, SuperArea, Substation, or Injection Group. Format of string is
---------	--

	[Bus Num], [Bus "Name_Nominal kV"], or [Bus "label"]
	[Area Num], [Area "Name"], or [Area "label"]
	[Zone Num], [Zone "Name"], or [Zone "label"]
	[SuperArea "Name"] or [SuperArea "label"]
	[Substation Num] or [Substation "label"]
	[InjectionGroup "Name"] or [InjectionGroup "label"]
BranchDistMeas	: is either X, Z, Length, Nodes, or a field variable name for a branch. X : means use the series reactance, Z : means use $\sqrt{X^2 + R^2}$, Length : means us the Length field, and Nodes : means treat each branch as a length of one FixedNumBus : means treat each branch between different FixedNumBuses has length 1 and each branch between the same FixedNumBuses has length 0 SuperBus : means treat each branch between different SuperBuses has length 1 and each branch between the same SuperBuses has length 0 "VariableName" : Otherwise use any Branch object field variable name.
BranchFilter	: is either All, Selected, Closed or the name of a branch Advanced Filter. This parameter is used to specify which branch can be traversed at all. All : means all branches can be traversed Selected : means only branches whose Selected field = YES can be traversed Closed : means only branches that are CLOSED can be traversed. "FilterName" : See the Using Filters in Script Commands section for more information on specifying the filtername.
BusField	: is the variable name of a Bus field. This field is populated with the minimum distance from the Start Place to that bus. All buses in the start group will have a distance measure of zero. Buses which cannot be reached from the start group will have a distance measure of -1.

Calculates the shortest distance from Bus 1 to all other buses using the series reactance (X) of closed branches only. Open branches are ignored. Results are stored in each bus's CustomFloat field: Bus 1 is set to 0, reachable buses receive a positive value based on the shortest path's total reactance, and unreachable buses are assigned -1.

DeterminePathDistance([Bus 1], X, CLOSED, CustomFloat);

DetermineShortestPath([start], [end], BranchDistanceMeasure, BranchFilter, Filename);

Use this action to calculate the shortest path between a starting group and an ending group. The results will be written to a textfile specified by filename. In the text file, the first bus listed will be in the end grouping and the last bus listed will be the start grouping. The result text file will have a line for each bus passed. Each line will contain three entries delimited by a space: "Number DistanceMeasure Name".

[start]	: same as the starting place for the DeterminePathDistance script command
[end]	: same as the starting place for the DeterminePathDistance script command
BranchDistanceMeasure	: same as for DeterminePathDistance script command
BranchFilter	: same as for DeterminePathDistance script command
Filename	: is a filename (may need to be enclosed in quotes) to which the results will be written.

This command computes the lowest-impedance path between Bus 1 and Bus 7, using impedance magnitude $Z = \sqrt{R^2 + X^2}$ as the distance metric. It evaluates all branches and saves the path details—bus number, cumulative impedance from Bus 1, and bus name—to the file "SP_1_to_7.txt".

```
DetermineShortestPath([Bus 1], [Bus 7], Z, ALL, "SP_1_to_7.txt");
```

DoFacilityAnalysis ("Filename", SetSelected);

Do Facility Analysis (Minimum Cut) is used to determine the branches that would isolate the Facility from the External region as specified in the Select Bus Dialog in the Simulator Tool dialog. It is assumed that the user will set the options before using the script command. The script will be used to identify the minimum number of branches that need to be opened or removed from the system in order to isolate the Facility (power system device) from an External region.

- "Filename" : The auxiliary file to which the results will be written. The results will show the buses of the different paths in a data section consisting of the buses that form the respective path. It will also show the branches of the minimum cut.
- SetSelected : (Added in the October 19, 2023 patch for Simulator version 23)
Optional parameter – default is NO
Set to YES or NO. Set to YES to set the Selected field to YES for the branches contained in the minimum cut.

Identifies the minimal set of branches needed to isolate Facility buses from External buses and saves the results to "cut_results.aux". The file includes bus paths for each isolating route and a list of branches in the minimum cut. With the second parameter set to YES, all cut branches are flagged with their Selected field set to YES, highlighting the lines to open for isolation.

```
DoFacilityAnalysis("H:\cut_results.aux", YES);
```

FindRadialBusPaths(IgnoreStatus, TreatParallelAsNotRadial, BusOrSuperBus);

This online help topic will explain radial bus paths in more detail:

https://www.powerworld.com/WebHelp/#MainDocumentation_HTML/Find_Radial_Bus_Paths.htm

Use this action to calculate series paths of buses or superbuses that are radial. The following fields for buses and branches will be populated with the results and indicate unique radial paths: Radial Path End Number, Radial Path Index, and Radial Path Length.

- IgnoreStatus : Optional parameter – default is NO
Set to YES or NO. Set to YES to ignore the status when traversing branches.
- TreatParallelAsNotRadial : Optional parameter – default is NO
Set to YES or NO. Set to YES to treat parallel branches as not radial when traversing branches.
- BusOrSuperBus : Optional parameter – default is BUS
Set to BUS or SUPERBUS. This determines groupings to traverse. When using SUPERBUS, any branch that has both terminal buses in the same superbus will have blank results because it is not part of the path.

This command scans the network for any series of buses that end in a dead-end (radial path), and it assigns RadialEndBusNum, RadialEndIndex, and RadialEndLength values to all involved buses and branches.

```
FindRadialBusPaths(YES, NO, BUS);
```

SetBusFieldFromClosest(variablename, BusFilterSetTo, BusFilterFromThese, BranchFilterTraverse, BranchDistMeas);

(This command was added to the January 15, 2025 Patch of Version 23)

Set buses field values equal to the closest bus's value. The parameters for this command are as follows.

BusField	:	variable name of the Bus object to set (and also the one copied from the closest bus)
BusFilterSetTo	:	specifies which Bus objects the should have their variablename overwritten. See the Using Filters in Script Commands section for more information on specifying the filter.
BusFilterFromThese	:	specifies which Bus objects that can have their variablename used to overwrite another bus. See the Using Filters in Script Commands section for more information on specifying the filter.
BranchFilterTraverse	:	specifies which AC Branch objects can be traversed when searching for the closest bus. Value is either All, Selected, Closed or the name of a branch Advanced Filter. This parameter is used to specify which branch can be traversed at all. All : means all branches can be traversed Selected : means only branches whose Selected field = YES can be traversed Closed : means only branches that are CLOSED can be traversed. "FilterName" : See the Using Filters in Script Commands section for more information on specifying the filtername.
BranchDistMeas	:	is either X, Z, Length, Nodes, or a field variable name for a branch. X : means use the series reactance, Z : means use $\sqrt{X^2 + R^2}$, Length : means us the Length field, and Nodes : means treat each branch as a length of one FixedNumBus : means treat each branch between different FixedNumBuses has length 1 and each branch between the same FixedNumBuses has length 0 SuperBus : means treat each branch between different SuperBuses has length 1 and each branch between the same SuperBuses has length 0 "Variablename" : Otherwise use any Branch object field variable name.

This command assigns buses that do not belong to a substation equal to the substation closest to the bus based on impedance magnitude (Z).

```
SetBusFieldFromClosest(SubNumber, "SubNumber IsBlank", "SubNumber  
NotIsBlank", All, Z);
```

SetSelectedFromNetworkCut(SetHow, [BusOnCutSide], BranchFilter, InterfaceFilter, DCLineFilter, Energized, NumTiers, InitializeSelected, [ObjectsToSelect], UseAreaZone, UsekV, MinkV, MaxkV, LowerMinkV, LowerMaxkV);

Use this action to set the Selected field of specified object types if they are on the specified side of a network cut created by specified branches, interfaces, and/or dc lines.

SetHow : Set to YES or NO. This is the value to which the Selected field will be set if an object is within the network cut.

[BusOnCutSide] : Specify the bus that is on the desired side of the network cut. Objects that are on the same side as this bus will have their Selected field set.

At least one of the following filters MUST not be blank:

BranchFilter : Specify a filter to select the branches that define the network cut. See the [Using Filters in Script Commands](#) section for more information on specifying the filter. A blank filter means that no branches are selected.

InterfaceFilter : Specify a filter to select the interfaces that define the network cut. See the [Using Filters in Script Commands](#) section for more information on specifying the filter. A blank filter means that no interfaces are selected.

DCLineFilter : Specify a filter to select the dc lines that define the network cut. See the [Using Filters in Script Commands](#) section for more information on specifying the filter. A blank filter means that no dc lines are selected.

Energized : Set to YES or NO. Set to YES to only include branches with a closed status when traversing branches to determine which side of the network cut each bus is on. This option does not apply to the branches that are specified to define the network cut. Those branches will not be traversed regardless of their status.

NumTiers : Once the network cut has been defined by a set of branches and the bus defining which side of the cut is being examined has been chosen, this value indicates that buses will be included within this number of tiers of the network cut boundary, on the opposite side of the cut as the specified bus. If the number of tiers is set to zero, the buses examined will only be those on the same side of the cut as the specified bus.

InitializeSelected : Set to YES or NO. Set to YES to set the Selected field for all objects to the opposite of the value specified in the SetHow parameter.

[ObjectsToSelect] : Comma separated list of object types enclosed in square brackets. These objects will have their Selected fields set if they are within the network cut. Valid options are: BRANCH, BUS, DCTRANSMISSIONLINE, GEN, LOAD, and SHUNT.

UseAreaZone : Optional parameter – default is NO.
Set to YES or NO. Set to YES to only set the Selected field for objects that are within the network cut that meet the area/zone/owner filter.

UsekV : Optional parameter – default is NO.
Set to YES or NO. Set to YES to only set the Selected field for objects that are within the network cut and within the nominal kV range specified.

MinkV : Optional parameter – default is 0.
An object's nominal kV must be greater than or equal to this value to have its Selected field set if it is also in the network cut. Branches can have different nominal voltages at each terminal; the largest nominal voltage must be greater than or equal to this value.

MaxkV : Optional parameter – default is 9999.
An object's nominal kV must be less than or equal to this value to have its Selected field set if it is also in the network cut. Branches can have

	different nominal voltages at each terminal; the largest nominal voltage must be less than or equal to this value.
LowerMinkV	: Optional parameter – default is 0. This value is only used with branches. Branches can have different nominal voltages at each terminal; the smallest nominal voltage must be greater than or equal to this value.
LowerMaxkV	: Optional parameter – default is 9999. This value is only used with branches. Branches can have different nominal voltages at each terminal; the smallest nominal voltage must be less than or equal to this value.

This script sets the Selected field to YES for all BRANCH, GEN, and LOAD objects that are electrically on the same side of the network cut as Bus 2. The network cut is defined by the branches with Selected = YES (BranchFilter = SELECTED). Only energized branches (Energized = YES) are considered when determining connectivity. The InitializeSelected = YES parameter resets the Selected field for all objects to NO before applying the new selection. Voltage filtering is not applied (UsekV = NO), and no additional connectivity tiers are considered (NumTiers = 0).

```
SetSelectedFromNetworkCut(YES, [BUS 2], SELECTED, , , YES, 0, YES,
[BRANCH, GEN, LOAD], NO, NO, 0, 9999, 0, 9999);
```

Sensitivity Calculations

CalculateFlowSense([flow element], FlowType);

This calculates the sensitivity of the MW, MVAR, or MVA flow of a line or interface to a real and reactive power injections at all buses in the system. (Note: this assumes that the power is injected at a given bus and taken out at the slack bus).

[flow element] : This is the flow element we are interested in. Choices are:
[INTERFACE "name"]
[INTERFACE "label"]
[BRANCH busnum1bus num2 ckt]
[BRANCH "name_kv1" "name_kv2" ckt]
[BRANCH "buslabel1" "buslabel2" ckt]
[BRANCH "label"]
FlowType : The type of flow to calculate this for. Either MW, MVAR, or MVA.

This command calculates the sensitivity of the MW flow of the Interface Left-Right to real and reactive power injections at all buses in the system.
--

CalculateFlowSense([INTERFACE Left-Right], MW);

CalculateLODF([BRANCH nearbusnum farbusnum ckt], LinearMethod, PostClosureLCDF);

Use this action to calculate the Line Outage Distribution Factors (or the Line Closure Distribution Factors) for a particular branch. If the branch is presently closed, then the LODF values will be calculated, otherwise the LCDF values will be calculated. You may optionally specify the linear calculation method as well. If no Linear Method is specified, Lossless DC will be used.

[BRANCH nearbusnum farbusnum ckt]
: the branch whose status is being changed. Can also use strings
[BRANCH "nearbusname_kv" "farbusname_kv" ckt]
[BRANCH "nearbuslabel" "farbuslabel" ckt]
[BRANCH "label"]
LinearMethod : The linear method to be used for the LODF calculation. The options are:
DC : for lossless DC.
DCPS : for lossless DC that takes into account phase shifter operation.
Note: AC is NOT an option for the LODF calculation.
PostClosureLCDF : Optional parameter – default is YES
Set to YES to calculate any line closure sensitivities relative to post-closure flow on the line being closed. This is known as the LCDF value.
Set to NO to calculate any line closure sensitivities based on calculating the flow on the line being closed from pre-closure voltages and angles. This is known as the MLCDF value.

This calculates the Line Outage Distribution Factors (LODFs) for the branch that connects bus 1 to bus 2 with circuit ID 1 using the lossless DC method.
--

CalculateLODF([BRANCH 1 2 1], DC,);

CalculateLODFAdvanced(IncludePhaseShifters, FileType, MaxColumns, MinLODF, NumberFormat, DecimalPoints, OnlyIncludingLinesIncreasing, "FileName", IncludesIslandingCTG);

Use this action to to mimic what is done on the Advanced LODF Calculation dialog in the GUI.

IncludePhaseShifters : Set to YES to calculate the LODF/LCDF values assuming that phase shifters are allowed to operate and will see no impact due to an outage or closure. Set to NO to not enforce the flow on phase shifters.
FileType : Either PROMOD or MATRIX. For PROMOD, save only "Monitored Branch, Contingency" pairs for PROMOD; and for MATRIX save Matrix as comma-delimited text file.

MaxColumns : Maximum number of columns per text file.
 MinLODF : Only Save pairs with an LODF whose absolute value is greater than this minimum.
 NumberFormat : LODF Number format, either, EXPONENTIAL or DECIMAL
 DecimalPoints : Fixed Decimals Points.
 OnlyIncludingLinesIncreasing: Only include monitored branches whose MW flow increases.
 "FileName" : The name of the text file to write.
 IncludeIslandingCTG : Optional parameter – default is YES
 LODF values cannot be calculated for contingencies that will cause a new island to be created. When including these contingencies in the results, the LODF will be reported as a very large number to indicate that these values were not actually calculated. Set to NO to completely omit these contingencies from the results.

This command performs an advanced LODF calculation allowing phase shifters and includes only monitored branches with increasing MW flow. Results are saved in a comma-delimited matrix format with up to 10 columns per file, including only LODF values above 0.03. The values are in decimal format with four decimal places. Contingencies causing islanding are excluded from the output.

```
CalculateLODFAdvanced(YES, MATRIX, 10, 0.03, DECIMAL, 4, YES,
"AdvancedLODFResults.txt", NO);
```

CalculateLODFMatrix(WhichOnes, filterProcess, filterMonitor, MonitorOnlyClosed, LinearMethod, filterMonitorInterface, PostClosureLCDF);

Use this action to calculate the Line Outage Distribution Factors (or the Line Closure Distribution Factors) for a particular branch. If the branch is presently closed, then the LODF values will be calculated, otherwise the LCDF values will be calculated. You may optionally specify the linear calculation method as well. If no Linear Method is specified, Lossless DC will be used.

WhichOnes : Specify the type of sensitivities to be calculated.
 OUTAGES : Outage sensitivities will be calculated for those branches meeting the filterProcess.
 CLOSURES : Closure sensitivities will be calculated for those branches meeting the filterProcess.
 filterProcess : Specify a filter for the branches for which the outages or closures will be implemented.
 ALL : All AC transmission lines.
 SELECTED : Only those branches whose Selected field is YES.
 AREAZONE : Only those branches meeting the area/zone filter.
 "filename": See the [Using Filters in Script Commands](#) section for more information on specifying the filename.
 filterMonitor : Specify a filter for the branches for which the impact of the outages or closures will be determined.
 ALL : All AC transmission lines.
 SELECTED : Only those branches whose Selected field is YES.
 AREAZONE : Only those branches meeting the area/zone filter.
 "filename": See the [Using Filters in Script Commands](#) section for more information on specifying the filename.
 SAME : Same as set of branches to process as specified by filterProcess.
 MonitorOnlyClosed : Set to YES to monitor only those branches that are closed. Set to NO to monitor branches regardless of their status.
 LinearMethod : Optional parameter – default is DC
 The linear method to be used for the LODF calculation.

DC : for lossless DC.

DCPS : for lossless DC that takes into account phase shifter operation.

Note: AC is NOT an option for the LODF calculation.

filterMonitorInterface : Optional parameter – default is to not monitor interfaces

Specify a filter for the interfaces for which the impact of the outages or closures will be determined. Using this option will add the individual lines in the interface to the list of lines to monitor.

ALL : All interfaces.

SELECTED : Only those interfaces whose Selected field is YES.

AREAZONE : Only those interfaces meeting the area/zone filter.

"filtername": See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

PostClosureLCDF : Optional parameter – default is YES

Set to YES to calculate any line closure sensitivities relative to post-closure flow on the line being closed. This is known as the LCDF value.

Set to NO to calculate any line closure sensitivities based on calculating the flow on the line being closed from pre-closure voltages and angles. This is known as the MLCDF value.

This command calculates Line Outage Distribution Factors (LODFs) using the lossless DC method for all AC transmission branches in the case and monitors how all other closed branches are affected when each one is taken out of service.

<code>CalculateLODFMatrix(OUTAGES, ALL, ALL, YES, DC, ,);</code>

CalculateLODFScreening(filterProcess, filterMonitor, IncludePhaseShifters, IncludeOpenLines, UseLODFThreshold, LODFThreshold, UseOverloadThreshold, OverloadLow, OverloadHigh, DoSaveFile, FileLocation, CustomFieldHighLODF, CustomFieldHighLODFLine, CustomFieldHighOverload, CustomFieldHighOverloadLine, DoUseCTGName, CustomFieldOrigCTGName);

Use this action to do the LODF Screening calculation. This calculation uses LODF/LCDF factors to determine how significant a branch open/close action will be on monitored lines. The significance of the action can be determined by LODF/LCDF magnitude or line loading on monitored lines. Significant single contingency actions can then be combined to form pairs of contingency actions that will be used to create new contingencies that can be saved to an auxiliary file.

filterProcess : Specify a filter for the branches for which the outage or closure impact will be determined.

ALL : All AC transmission lines.

AREAZONE : Only those branches meeting the area/zone filter.

CTG : Only those branches included in any currently defined contingency.

LIMITMONITOR : Only those branches meeting the Limit Monitoring Settings.

SELECTED : Only those branches whose Selected field is YES.

"filtername" : See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

filterMonitor : Specify a filter for the branches on which the impact of the outages or closures will be determined.

ALL : All AC transmission lines.

AREAZONE : Only those branches meeting the area/zone filter.

LIMITMONITOR : Only those branches meeting the Limit Monitoring Settings.

SAME : Same as branches to process specified by filterProcess.

SELECTED : Only those branches whose Selected field is YES.

- "filtername" : See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.
- IncludePhaseShifters : Set to YES to calculate the LODF/LCDF values assuming that phase shifters are allowed to operate and will see no impact due to an outage or closure. Set to NO to not enforce the flow on phase shifters.
- IncludeOpenLines : Set to NO to monitor only those branches that are closed. Set to YES to monitor branches regardless of their status.
- UseLODFThreshold : Set to YES to screen outages/closures by LODF/LCDF magnitude. Set to NO to not screen by LODF/LCDF magnitudes.
- LODFThreshold : Threshold above which LODF/LCDF magnitudes are considered significant.
- UseOverloadThreshold : Set to YES to screen outages/closures by monitored branch loading. Set to NO to not screen by branch loading.
- OverloadLow : Threshold above which a monitored branch loading is considered significant. This value should be entered as a percent.
- OverloadHigh : Threshold below which a monitored branch loading is considered significant. This value should be entered as a percent.
- DoSaveFile : Set to YES to save an auxiliary file of new contingencies created by joining pairs of significant single outage/closure actions. Set to NO to not save the file.
- FileLocation : Specify a directory path where the auxiliary file containing new contingencies will be saved. The filename will be determined by Simulator.
- CustomFieldHighLODF : Optional parameter – default is 0
Integer indicating which Custom Floating Point field for a processed branch will store the highest magnitude LODF/LCDF determined for any monitored branch.
- CustomFieldHighLODFLine : Optional parameter – default is 0
Integer indicating which Custom String field for a processed branch will store the identifier for the monitored branch that has the highest magnitude LODF/LCDF.
- CustomFieldHighOverload : Optional parameter – default is 0
Integer indicating which Custom Floating Point field for a processed branch will store the highest overload determined for any monitored branch.
- CustomFieldHighOverloadLine : Optional parameter – default is 0
Integer indicating which Custom String field for a processed branch will store the identifier for the monitored branch that has the highest overload.
- DoUseCTGName : Optional parameter – default is NO
Set to YES to use the available active contingency names available in the CTG Tool. If a contingency in the CTG Tool has the same branch as the only contingency element it will use the contingency name in the tool to create the new CTG Label for the new contingency combination of branches. Set to NO to only use the branch info to create the new CTG Label for the combination of branches.
- CustomFieldOrigCTGName : Optional parameter – default is 0

Integer indicating which Custom String field for a processed branch will store the name of the contingency from which the branch originated.

This command performs LODF-based screening to identify significant single-line outages among selected branches and evaluates their impact on all monitored branches. It considers both LODF magnitude (threshold 0.05) and monitored line overloads (between 95% and 120%) as criteria for significance. Phase shifters and open lines are excluded from analysis. When impactful contingencies are found, the command combines them into new two-element contingencies, saves them to an AUX file, and logs detailed results—such as max LODF, max overload, and related identifiers—into designated custom fields. Existing contingency names are reused when applicable for consistency.

```
CalculateLODFScreening(SELECTED, ALL, NO, NO, YES, 0.05, YES, 95, 120, YES, "H:\", 1, 1, 2, 2, YES, 3);
```

CalculateLossSense(FunctionType,AreaSALossReference,IslandLossReference);

This calculates the loss sensitivity at each bus for an injection of power at the bus. The parameter FunctionType determines which losses are referenced.

- FunctionType : This is the losses for which sensitivities are calculated.
- NONE : all loss sensitivities will be set to zero
 - ISLAND : all loss sensitivities are referenced to the total loss in the island
 - AREA : For each bus it calculates how the losses in the bus' area will change (Note: this means that sensitivities at buses in two different areas cannot be directly compared because they are referenced to different losses)
 - AREASA : same as Each Area, but if a Super Area exists it will use this instead (Note: this means that sensitivities at buses in two different areas cannot be directly compared because they are referenced to different losses)
 - SELECTED: Calculates how the losses in the areas selected on the Loss Sensitivity Form will change
- AreaSALossReference : Optional parameter - default is NO
This parameter will only be used if the FunctionType is AREA or AREASA. This option specifies whether or not the Cost of Energy, Losses, and Congestion Reference for each area or super area that is used for OPF calculations will be used in this calculation.
- IslandLossReference : Optional parameter - default is EXISTING
This parameter specifies the loss reference that will be used when the FunctionType is ISLAND.
- EXISTING : Use existing loss sensitivities
 - LOADS : MW value of all loads within an island will be used for weighting in the loss reference calculation
 - "InjGroupName" : Name of injection group where the participation factor of each participant will be used for weighting in the loss reference calculation

This command calculates loss sensitivity values at each bus in the case, showing how much losses in the bus' island will change due to an injection of power at that bus.

```
CalculateLossSense(ISLAND, , EXISTING);
```

CalculatePTDF([transactor seller], [transactor buyer], LinearMethod);

Use this action to calculate the PTDF values between a seller and a buyer. You may optionally specify the linear calculation method. Note that the buyer and seller must not be same thing. If no Linear Method is specified, Lossless DC will be used.

- [transactor seller] : The seller (or source) of power. There are six possible settings:
[AREA num], [AREA "name"], [AREA "label"]
[ZONE num], [ZONE "name"], [ZONE "label"]
[SUPERAREA "name"], [SUPERAREA "label"]
[INJECTIONGROUP "name"], [INJECTIONGROUP "label"]
[BUS num], [BUS "name_nomkv"], [BUS "label"]
[SLACK]
- [transactor buyer] : The buyer (or sink) of power. There are six possible settings which are the same as for the seller.
- LinearMethod : The linear method to be used for the PTDF calculation. The options are:
AC : for calculation including losses
DC : for lossless DC
DCPS : for lossless DC that takes into account phase shifter operation

This command calculates the PTDF values between Area Top (seller), and Bus 7 (buyer) using the lossless DC with phase shifters method.
--

CalculatePTDF([Area Top], [Bus 7], DCPS);

CalculatePTDFMultipleDirections(StoreForBranches, StoreForInterfaces, LinearMethod);

Use this action to calculate the PTDF values between all the directions specified in the case. You may optionally specify the linear calculation method. If no Linear Method is specified, Lossless DC will be used.

- StoreForBranches : Specify YES to store the values calculated for each branch.
- StoreForInterfaces : Specify YES to store the values calculated for each interface.
- LinearMethod : the linear method to be used for the PTDF calculation. The options are:
AC : for calculation including losses.
DC : for lossless DC.
DCPS : for lossless DC that takes into account phase shifter operation.

This command calculates the PTDF values for all directions where Include = YES. PTDFs are calculated for all branches and interfaces using the DC lossless method.
--

CalculatePTDFMultipleDirections(YES, YES, DC);
--

CalculateShiftFactors([flow element], direction, [transactor], LinearMethod, SetOutOfServiceBuses, filter, AbortOnError, BranchDistMeas);

In Version 21 and earlier this script command was called **CalculateTLR**. Simulator 21 patches after January 20, 2021 will handle reading either the **CalculateShiftFactors** or **CalculateTLR**.

Use this action to calculate the Shift Factor Sensitivity values for all buses on a particular flow element (transmission line or interface). There are some additional options that are set with the TLR_Options object rather than through parameters with this command.

- [flow element] : This is the flow element we are interested in. Choices are:
[INTERFACE "name"]
[INTERFACE "label"]
[BRANCH nearbusnum farbusnum ckt]
[BRANCH "nearbusname_kv" "farbusname_kv" ckt]
[BRANCH "nearbuslabel" "farbuslabel" ckt]
[BRANCH "label"]
- direction : The type of the transactor. Either BUYER or SELLER. Shift factors are calculated between each bus in the case and this transactor.

[transactor]	: The transactor of power. Shift factors are calculated between each bus in the case and this transactor. These are the possible settings: [AREA num], [AREA "name"], [AREA "label"] [ZONE num], [ZONE "name"], [ZONE "label"] [SUPERAREA "name"], [SUPERAREA "label"] [INJECTIONGROUP "name"], [INJECTIONGROUP "label"] [BUS num], [BUS "name_nomkv"], [BUS "label"] [SLACK]
LinearMethod	: Optional parameter – default is DC The linear method to be used for the calculation. The options are: AC : for calculation including losses DC : for lossless DC DCPS : for lossless DC that takes into account phase shifter operation
SetOutOfServiceBuses	: Optional parameter – default is NO Set to YES or NO. If YES then set the sensitivities for out-of-service buses equal to the sensitivity to the value at the closest in-service bus. The "distance" to the in-service buses will be measured by the number of nodes. If an out-of-service bus is equally close to a set of buses, then the average of that set of buses will be used.
filter	: Optional parameter – default is to include all buses The filter that will determine the buses for which sensitivities will be set when SetOutOfServiceBuses = YES. In addition to meeting the filter, only out-of-service buses will be included. blank : All buses AREAZONE : Only those buses meeting the area/zone filter SELECTED : Only those buses whose Selected field is YES "filtername" : See the Using Filters in Script Commands section for more information on specifying the filtername.
AbortOnError	: Optional parameter – default is YES Set to YES or NO. If YES, the script command will fail and the auxiliary file containing the script command will terminate without processing the remainder of the file. If NO, an error message is printed to the message log but the command is not treated as failing and the remainder of the auxiliary file containing the command will be processed.
BranchDistMeas	: (Added in September 27, 2023 patch of Simulator Version 23) Optional parameter – default is blank. If omitted or set as blank, then the old behavior of taking the average of the closest nodes measured by the number of Nodes is used. Otherwise set to either X, Z, Length, Nodes, or a field variable name for a branch. X : means use the series reactance, Z : means use $\sqrt{X^2 + R^2}$, Length : means use the Length field, and Nodes : means treat each branch as a length of one FixedNumBus : means treat each branch between different FixedNumBuses has length 1 and each branch between the same FixedNumBuses has length 0 SuperBus : means treat each branch between different SuperBuses has length 1 and each branch between the same SuperBuses has length 0 "Variablename" : Otherwise use any Branch object field variable name.

This command calculates Shift Factors for how a transfer from AREA "Top" (SELLER) to each bus affects the flow on the branch from Bus 1 to Bus 2 circuit "1", using the DC method.

```
CalculateShiftFactors([BRANCH 1 2 "1"], SELLER, [AREA "Top"], DC, NO,
, YES, );
```

CalculateShiftFactorsMultipleElement(TypeElement,WhichElement,direction,[transactor],LinearMethod);

In Version 21 and earlier this script command was called **CalculateTLRMultipleElement**. Simulator 21 patches after January 20, 2021 will handle reading either the **CalculateShiftFactorsMultipleElement** or **CalculateTLRMultipleElement**.

Use this action to calculate Shift Factor Sensitivity values for multiple elements. There are some additional options that are set with the TLR_Options object rather than through parameters with this command.

- TypeElement : May be either INTERFACE, BRANCH, or BOTH
- WhichElement : There are three choices that represent which elements of the TypeElement specified will have shift factor calculations performed.
 - SELECTED : Only branches or interfaces with their Selected Field = YES will be used.
 - OVERLOAD : Only branches that are presently overloaded using their normal ratings will be used
 - CTGOVERLOAD : You must have first run the contingency analysis. A branch or interface is included in the calculation if it has been overloaded during at least one contingency.
- Direction : The type of the transactor. Either BUYER or SELLER. Shift factors are calculated between each bus in the case and this transactor.
- [transactor] : The transactor of power. Shift factors are calculated between each bus in the case and this transactor. These are the possible settings:
 - [AREA num], [AREA "name"], [AREA "label"]
 - [ZONE num], [ZONE "name"], [ZONE "label"]
 - [SUPERAREA "name"], [SUPERAREA "label"]
 - [INJECTIONGROUP "name"], [INJECTIONGROUP "label"]
 - [BUS num], [BUS "name_nomkv"], [BUS "label"]
 - [SLACK]
- LinearMethod : Options parameter – default is DC
 - The linear method to be used for the calculation. The options are:
 - AC : for calculation including losses.
 - DC : for lossless DC.
 - DCPS : for lossless DC that takes into account phase shifter operation.

This command calculates shift factor sensitivities for all branches where Selected = YES, showing how a power injection from AREA "Top" (SELLER) to each bus affects flow on each of those branches, using the lossless DC method.

```
CalculateShiftFactorsMultipleElement(BRANCH, SELECTED, SELLER, [AREA
"Top"], DC);
```

CalculateTapSense(filter);

(Added to June 4, 2025 patch of Simulator 24)

Voltage to tap sensitivities are calculated as needed in the power flow, but the field is not always kept up to date. For example, if you want the voltage to tap sensitivity values without solving the power flow, they would not previously have been calculated. This command forces the voltage to tap sensitivity calculation so that the dV/dtap values are current for the system state.

- Filter : Optional parameter- if omitted the sensitivities for all transformers will be calculated. Branch filter returning the transformers for which sensitivity values are updated.

- ALL : Calculate the sensitivities for all transformers
- "FilterName" : Only sensitivities for transformers that meet the specified filter will be calculated. See [Using Filters in Script Commands](#) section for more information on specifying the filtername.

CalculateVoltSelfSense(filter);

This calculates the sensitivity of a particular bus' voltage to real and reactive power injections at the same bus. (Note: This assumes that the power is injected at a given bus and taken out at the slack bus.)

- filter : Optional parameter – default is to calculate sensitivities for all buses in the system
- "FilterName" : Only buses that meet the specified filter will be included. See [Using Filters in Script Commands](#) section for more information on specifying the filtername.

CalculateVoltSense([BUS num]);

This calculates the sensitivity of a particular buses voltage to real and reactive power injections at all buses in the system. (Note: this assumes that the power is injected at a given bus and taken out at the slack bus).

- [BUS num] : the bus for which sensitivities are calculated.

CalculateVoltToTransferSense([transactor seller], [transactor buyer], TransferType, TurnOffAVR);

This calculates the sensitivity of bus voltage to a real or reactive power transfer between a seller and a buyer. The sensitivity is calculated for all buses in the system.

- [transactor seller] : This is the seller (or source) of power. There are six possible settings:
 [AREA num], [AREA "name"], [AREA "label"]
 [ZONE num], [ZONE "name"], [ZONE "label"]
 [SUPERAREA "name"], [SUPERAREA "label"]
 [INJECTIONGROUP "name"], [INJECTIONGROUP "label"]
 [BUS num], [BUS "name_nomkv"], [BUS "label"]
 [SLACK]
- [transactor buyer] : This is the buyer (or sink) of power. There are six possible settings, which are the same as for the seller.
- TransferType : The type of power transfer. The options are:
 P : real power transfer
 Q : reactive power transfer
 PQ : both real and reactive power transfer. (Note: Real and reactive power transfers are calculated independently, but both are calculated.)
- TurnOffAVR : Set to YES or NO. Set to YES to turn off AVR control for generators participating in the transfer. Set to NO to leave the AVR control unchanged for generators participating in the transfer.

This command calculates how bus voltages across the system will change if you transfer real power (MW) from AREA "Top" (the seller) to AREA "Left" (the buyer), with generator AVR control turned off during the calculation.

CalculateVoltToTransferSense([AREA "Top"], [AREA "Left"], P, YES);

LineLoadingReplicatorCalculate([Flow Element], [Injection Group], AGCOnly, DesiredFlow, Implement, LinearMethod, UseLoadMinMax, MaxMultiplier, MinMultiplier);

This command calculates an injection change list containing the injection changes needed to alter a line or interface flow to a desired value. The user supplies the element to manipulate the flow on, the injection group, and the desired line or interface flow. This command calculates the injection changes that will result in that line or interface flow. This command can optionally implement the changes, or the changes can be applied separately with the LineLoadingReplicatorImplement script command.

[Flow Element]	: This is the flow element we are interested in. Choices are: [INTERFACE "name"] [INTERFACE "label"] [BRANCH nearbusnum farbusnum ckt] [BRANCH "nearbusname_kv" "farbusname_kv" ckt] [BRANCH "nearbuslabel" "farbuslabel" ckt] [BRANCH "label"]
[Injection Group]	: Injection group containing elements that are available to move to implement the desired line or interface flow. Multiple injection groups can be specified by using a comma-delimited list of injection group names: ["IGName1", "IGName2", "IGName3"].
AGCOnly	: YES indicates that only elements on AGC control in the injection group will move to implement the desired line flow. A value of NO indicates that all elements in the injection group can move as needed.
DesiredFlow	: The new desired flow on the flow element.
Implement	: YES indicates that the injection change should be implemented after it is calculated. NO indicates that the injection change will not be implemented. The list is stored in memory and may be implemented later with the LineLoadingReplicatorImplement command. This option lets you implement the change with one command if you have no need to check anything before implementing it.
LinearMethod	: DC : for lossless DC. DCPS : for lossless DC that takes into account phase shifter operation.
UseLoadMinMax	: Optional parameter - default is YES This option indicates that the maximum and minimum values specified with the load records should be used to limit the load movement while calculating the injection changes. A value of NO indicates that the values specified in the MaxMultiplier and MinMultiplier should be applied to set the load change limits. When set to NO the Max and Min multipliers are required.
MaxMultiplier	: Optional parameter - default is 1.0 When UseLoadMinMax is set to NO this parameter is used as a scaling factor on the present load value to set the maximum limit that bounds the load's change.
MinMultiplier	: Optional parameter - default is 1.0 When UseLoadMinMax is set to NO this parameter is used as a scaling factor on the present load value to set the minimum limit that bounds the load's change.

This command calculates and applies changes in generator/load injections (defined in the group "GenShiftGroup") to adjust the MW flow on the branch from Bus 1 to Bus 2 circuit "1" to 100 MW, using the lossless DC linear method, while respecting load min/max limits.

LineLoadingReplicatorCalculate([BRANCH 1 2 "1"], ["GenShiftGroup"], NO, 100, YES, DC, YES);

LineLoadingReplicatorImplement;

This command takes no parameters. It applies the changes in the injection change list calculated by the LineLoadingReplicatorCalculate command.

SetSensitivitiesAtOutOfServiceToClosest(filter, BranchDistMeas);

This will take the P Sensitivity and Q Sensitivity values calculated using the CalculateTLR, CalculateFlowSense, or CalculateVoltSense actions and then populate the respective values at out-of-service buses so that they are equal to the value at the closest in service bus. The "distance" to the in-service buses will be measured by the number of nodes. If an out-of-service bus is equally close to a set of buses, then the average of that set of buses will be used.

- filter : Optional parameter – default is to include all buses
The filter that will determine the buses for which sensitivities will be set.
In addition to meeting the filter, only out-of-service buses will be included.
 - Blank : All buses
 - AREAZONE : Only those buses meeting the area/zone filter
 - SELECTED : Only those buses whose Selected field is YES
 - "filtername": See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.
- BranchDistMeas : (Added in September 27, 2023 patch of Simulator Version 23)
Optional parameter – default is blank. If omitted or set as blank, then the old behavior of taking the average of the closest nodes measured by the number of Nodes is used. Otherwise set to either X, Z, Length, Nodes, or a field variable name for a branch.
 - X : means use the series reactance,
 - Z : means use $\sqrt{X^2 + R^2}$,
 - Length : means us the Length field, and
 - Nodes : means treat each branch as a length of one
 - FixedNumBus : means treat each branch between different FixedNumBuses has length 1 and each branch between the same FixedNumBuses has length 0
 - SuperBus : means treat each branch between different SuperBuses has length 1 and each branch between the same SuperBuses has length 0
 - "Variablename" : Otherwise use any Branch object field variable name.

This command assigns P and Q sensitivity values to all out-of-service buses by copying the values from the closest in-service bus, where "closeness" is based on impedance magnitude (Z) between buses.

SetSensitivitiesAtOutOfServiceToClosest(, Z);

Contingency Analysis

CTGApply("ContingencyName");

Call this action to apply the actions in a contingency without solving the power flow.
"ContingencyName" : This is the name of the contingency to apply.

This command applies the actions in the contingency named "L_000001One-000002TwoC1".
--

CTGApply("L_000001One-000002TwoC1");

CTGAutoInsert;

This action will auto insert contingencies for you case. Prior to calling this action, all options for this action must be specified in the Ctg_AutoInsert_Options object using the SetData script command or DATA sections.

CTGCalculateOTDF([transactor seller], [transactor buyer], LinearMethod);

This action first performs the same action as done by the CalculatePTDF([transactor seller], [transactor buyer], LinearMethod) call. It then goes through all the violations found by the contingency analysis tool and determines the OTDF values for the various contingency/violation pairs.

This command computes OTDFs using the lossless DC with phase shifters linear method. PTDFs are calculated for a transfer from "Area Top" (the source) to "Bus 7" (the sink), and then OTDFs are calculated from these PTDFs for the violations under each contingency.
--

CTGCalculateOTDF([Area Right], [Bus 7], DCPS);
--

CTGClearAllResults;

This action will delete all contingency violations and any contingency comparison results.

CTGCloneMany(filter, "Prefix", "Suffix", SetSelected);

This command creates copies of any contingencies returned by the filter. If neither the prefix nor suffix is supplied, the command will default to appending "- Copy" to the name of the contingency being cloned to define the new contingency name. Integer indices contained in parentheses, " (0)", " (1)", etc., will be added as needed until a unique contingency name is found.

filter	: Optional parameter – by default all contingencies will be cloned All contingencies meeting the filter will be cloned. See the Using Filters in Script Commands section for more information on specifying the filtername.
Prefix	: Optional parameter – default is blank Prefix to prepend to the existing contingency name
Suffix	: Optional parameter – default is blank Suffix to append to the existing contingency name
SetSelected	: Optional parameter – default is NO If this value is YES, then as new contingencies are created for these clones, the Selected field of the new contingencies will be set to YES.

This command duplicates all existing contingencies, naming each new one by prepending "Clone_" and appending "_v2" to the original contingency name. Each newly created contingency will have its Selected field set to YES.
--

CTGCloneMany(, "Clone_", "_v2", YES);

CTGCloneOne("ctgname", "newctgname", "Prefix", "Suffix", SetSelected);

This command creates a copy of a single existing contingency. If newctgname, prefix, and suffix are all blank, "- Copy" will be added to the end of the existing contingency name to specify the name of the new contingency. Regardless of how the new contingency name is specified, integer indices contained in parentheses, " (0)", " (1)", etc., will be added as needed until a unique contingency name is found.

ctgname	: name of contingency to clone
newctgname	: Optional parameter – default is blank Name of new contingency. If the special keyword @CTGName is specified it will be replaced with the name of the existing contingency that is being cloned.
Prefix	: Optional parameter – default is blank Prefix to prepend to the existing contingency name. This will only be used if newctgname is blank.
Suffix	: Optional parameter – default is blank Suffix to append to the existing contingency name. This will only be used if newctgname is blank.
SetSelected	: Optional parameter – default is NO If this value is YES, then as new contingencies are created for these clones, the Selected field of the new contingencies will be set to YES.

This command creates a copy of the contingency named "L_One-TwoC1", naming the new contingency "Backup_L_One-TwoC1_Test". The new contingency will have its Selected field set to YES.

```
CTGCloneOne("L_One-TwoC1", "", "Backup_", "_Test", YES);
```

CTGComboDeleteAllResults;

Deletes all results that are associated with contingency combination analysis. This includes violations, what occurred, combination summary results, and injection sensitivities.

CTGComboSolveAll(DoDistributed, ClearAllResults);

Call this command to run contingency combination analysis for all primary and regular/secondary contingencies that are set to not be skipped.

DoDistributed	: Optional parameter – default is NO Set to YES or NO. If set to YES, distributed methods will be used to solve contingency combination analysis if the Distributed Contingency Analysis add-on is installed. Distributed analysis requires the proper configuration and security settings to work.
ClearAllResults	: Optional parameter – default is YES Set to YES or NO. If set to YES, all existing contingency combination results will be cleared even if a primary contingency is marked to be skipped. If set to NO, only those primary contingencies that are marked to not be skipped will have their results cleared.

The command runs contingency combination analysis using standard (non-distributed) methods and clears all existing contingency combination results before running. All primary contingencies that are not marked to be skipped will be run.

```
CTGComboSolveAll(NO, YES);
```

CTGCompareTwoListsofContingencyResults (PRESENT or "ControllingFilename",PRESENT or "ComparisonFilename");

This command compares two different contingency result lists. The first parameter is to set the Controlling List. The second parameter is to set the Comparison List.

PRESENT or "ControllingFilename"

: PRESENT will set the present contingency analysis results as the Controlling List. If the results are in a file then you can set the path to the list as the "ControllingFilename". See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

PRESENT or "ComparisonFilename"

: PRESENT will set the present contingency analysis results as the Comparison List. If the results are in a file then you can set the path to the list as the "ComparisonFilename". See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

The file types allowed are: Simulator Contingency File (*.aux), Simulator (Ver 5,6,7) Contingency Files (*.ctg), PTI Contingency Files (*.con), PTI Load Throw Over Files (*.thr;*.dat), and GE Contingency Files (*.otg).

This command compares the currently loaded contingency analysis results (as the controlling list) with those stored in the file "B7Flat_ContingencyResults.aux" (as the comparison list).

CTGCompareTwoListsofContingencyResults(PRESENT, "B7Flat_ContingencyResults.aux")

CTGConvertAllToDeviceCTG(KeepOriginalIfEmpty);

This command is intended for use with full topology models, where breakers and disconnects are defined in addition to generators, loads, transmission lines, and so on. This function would have no affect on a traditional planning model representation, which has no breakers or disconnects explicitly defined.

The purpose of the function is to allow the user to take a contingency set that is defined with outages of breakers and disconnects in a full topology model and convert them to outages of the traditional planning model elements, such as generators, loads, transmission lines, etc. This would be used in conjunction with the ability to save a full topology model as a consolidated model. A consolidated model reduces the full model down to a traditional planning model by examining the breaker and disconnect statuses, and reducing the system down by consolidating breakers and disconnects that are in service. The resulting model is a smaller model with the traditional planning elements represented, but breakers and disconnects have been removed and nodes aggregated into bus representations. This function will also take the breaker and disconnect statuses and convert contingencies defined with the breakers and disconnects and convert them into contingencies of the planning model devices affected by opening the original breakers and disconnects. Thus you could create a contingency set that is defined for the consolidated model, and can be run on the consolidated model with the same results as if the original contingency set is run on the full topology model.

The parameter KeepOriginalIfEmpty is a YES or NO option to retain or not the original contingency definitions for any contingencies that do not end up isolating any devices. This is an optional parameter that is NO by default if it is not specified.

Note that the contingency set generated depends on the statuses of the breakers and disconnects, and that the contingencies created will be different for different statuses of breakers and disconnects in the full topology model.

CTGConvertToPrimaryCTG(filter, KeepOriginal, "Prefix", "Suffix");

(Added in the April 19, 2024 patch of Simulator 23)

Converts regular/secondary contingencies to Primary contingencies that are used with CTG Combo Analysis. Not all actions that are supported for regular/secondary contingencies are supported for Primary contingencies. Examine any messages in the log after the conversion to determine if actions were not converted.

filter	: Optional parameter – default is to convert all contingencies Contingencies meeting this filter will be converted to Primary contingencies. See Using Filters in Script Commands section for more information on specifying the filter.
KeepOriginal	: Optional parameter – default is YES Set to YES or NO. YES means to retain the original contingencies. NO means to delete the original contingencies.
"Prefix"	: Optional parameter – default is blank The newly created Primary contingency will be named with the original contingency name including this as the prefix and the specified suffix. If a Primary contingency already exists with that name, an integer will be appended to create a unique name.
"Suffix"	: Optional parameter – default is "-Primary" The newly created Primary contingency will be named with the original contingency name including this as the suffix and the specified prefix. If a Primary contingency already exists with that name, an integer will be appended to create a unique name.

This command converts all regular (secondary) contingencies into Primary contingencies used for contingency combination (combo) analysis. The original contingencies are retained, and each new Primary contingency is named by appending "-Primary" to the original name.
--

CTGConvertToPrimaryCTG(, YES, "", "-Primary");
--

CTGCreateContingentInterfaces(filter, maxOption);

This command creates an interface based on contingency violations. The contingency elements are included as contingent elements in the new interface, and the violated element is included as a monitored element.

filter	: This is the name of an Advanced Filter. Only violation objects of type ViolationCTG that meet the named filter will be used to create new interfaces.
maxOption	: Set to BRANCH, CTG, BRANCHCTG, or leave blank to select all violations. BRANCH – for each branch, this selects the worst violation out of all contingency violations for that branch. CTG – for each contingency, this selects the worst violation out of all branch violations for that contingency. BRANCHCTG – union of violations selected in both BRANCH and CTG.

CTGCreateExpandedBreakerCTGs;

This will convert any "Open with Breakers" or "Close with Breakers" contingency actions into OPEN or CLOSE actions on explicit breakers. This will permanently modify the contingency definitions.

CTGCreateStuckBreakerCTGs(filter, AllowDuplicates, "PrefixName", IncludeCTGLabel, BranchFieldName, "SuffixName", "PrefixComment", BranchFieldComment, "SuffixComment");

This command creates new contingencies from contingencies that have explicit breaker outages defined. New contingencies will be created by treating each breaker as stuck in turn. The new contingencies will be comprised of all existing elements, minus the stuck breaker outage, plus open actions for breakers that are identified to isolate the stuck breakers. Only branches with Branch Device Type of Breaker will be considered in determining the stuck breakers.

All of the following parameters are optional. If not specified, the defaults will be used.

- filter : Only contingencies that meet the specified filter will be set. See the [Using Filters in Script Commands](#) section for more information on specifying the filtername. Default is to process all contingencies.
- AllowDuplicates : Set to YES or NO. YES means that contingencies with the same actions as existing or newly created contingencies will be allowed. Default is NO.

"PrefixName", IncludeCTGLabel, BranchFieldName, and "SuffixName" are used to name the new contingencies in the format: PrefixName_Contingency_Label_BranchFieldName_SuffixName.

- "PrefixName" : string that is used as the prefix of the new contingency name. Default is blank.
- IncludeCTGLabel : Set to YES or NO. YES means that the name of the existing contingency will be used as part of the new contingency. Default is YES.
- BranchFieldName : variablename of the Branch field whose value will be used in the naming of the new contingency in the format
variablenamelegacy:location:digits:rod or
concisename:digits:rod. The Branch used to evaluate the variablename is the stuck breaker. Default is blank.
- "SuffixName" : string that is used as the suffix of the new contingency name. Default is "STK".

"PrefixComment", BranchFieldComment, and "SuffixComment" are used to create a comment for new contingency actions in the format: PrefixComment_BranchFieldComment_SuffixComment.

- "PrefixComment" : string that is used as the prefix of the new contingency action comment. Default is blank.
- BranchFieldComment : variablename of the Branch field whose value will be used in the naming of the new contingency action comment in the format
variablenamelegacy:location:digits:rod or
concisename:digits:rod. The Branch used to evaluate the variablename is the breaker in the new contingency action. Default is blank.
- "SuffixComment" : string that is used as the suffix of the new contingency action comment. Default is blank.

This command creates new stuck breaker contingencies from all existing contingencies that contain explicit breaker outages. It avoids duplicates, includes the original contingency name in the new names, and uses the breaker's label to uniquely identify each new contingency. The resulting contingency names follow the format "SB_OriginalCTGName_BreakerLabel_STK". Each new contingency action comment describes the breaker involved, using the format "StuckBreaker_BreakerLabel_Isolation".

```
CTGCreateStuckBreakerCTGs(, NO, "SB", YES, "Label", "STK",  
"StuckBreaker", "Label", "Isolation");
```

CTGDeleteWithIdenticalActions;

This action deletes contingencies that have identical actions. The first contingency alphabetically is retained while all others with identical actions are deleted. Messages are added to the log detailing which contingencies are deleted.

CTGJoinActiveCTGs(InsertSolvePowerFlow, DeleteExisting, JoinWithSelf, "filename");

This command creates new contingencies that are a join of the current contingency list and a list read in from an auxiliary file or the current list itself. Contingencies with their Skip field set to YES will not be included in the join.

- InsertSolvePowerFlow : Set to YES or NO. YES means to insert the solve power flow solution action between the joined contingency actions.
- DeleteExisting : Set to YES or NO. YES means to delete the existing contingencies and only keep the joined contingencies.
- JoinWithSelf : Set to YES or NO. YES means that the current contingency list will be joined with itself instead of contingencies specified in a file. If set to YES, the "filename" parameter does not have to be specified.
- "filename" : Name of auxiliary file containing contingencies to join with the current contingency list. This does not have to be specified if JoinWithSelf = YES.

This command joins the currently active list of contingencies with those stored in the auxiliary file "AdditionalContingencies.aux" (filename is specified and JoinWithSelf = NO). A SolvePowerFlow action is inserted between the two sets of actions, and the existing contingencies are retained.

```
CTGJoinActiveCTGs(YES, NO, NO, "AdditionalContingencies.aux");
```

CTGPrimaryAutoInsert;

This action will auto insert Primary Contingencies. Prior to calling this action, all options for this action must be specified in the Ctg_AutoInsert_Options object using the SetData script command or DATA sections.

CTGProcessRemedialActionsAndDependencies(DoDelete, filter);

Remedial Actions and any Model Conditions, Model Filters, Model Expressions, and Model Planes being used by the specified Remedial Actions will be deleted or have their Selected field set to YES. Model Conditions, Model Filters, Model Expressions, and Model Planes that are being used by other objects or not being used by a specified Remedial Action will not be deleted or marked.

- DoDelete : Set to YES to delete remedial actions and dependencies. Set to NO to set the Selected field to YES instead of deleting.
- filter : Optional parameter - default is blank
Only Remedial Actions that meet the specified filter will be processed.
See the [Using Filters in Script Commands](#) section for more information on specifying the filtername. AREAZONE is not a valid filter. Default is to process all remedial actions.

This command deletes the Remedial Action that matches the single-condition filter identifying a Remedial Action with a particular name along with associated dependencies—such as Model Conditions, Model Filters, Model Expressions, and Model Planes—provided these dependencies are not used by other objects.

```
CTGProcessRemedialActionsAndDependencies(YES,  
"Name='VoltageViolationRA'");
```

CTGProduceReport("filename");

Produces a text-based contingency analysis report using the settings defined in CTG_Options.

CTGReadFilePSLF("filename");

Use this action to load a file in the PSLF OTG format and create contingencies.

"filename" : Name of the file to read. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

CTGReadFilePTI("filename");

Use this action to load a file in the PTI CON format and create contingencies.

"filename" : Name of the file to read. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

CTGRelinkUnlinkedElements;

This will attempt to relink unlinked elements in the contingency records.

CTGRestoreReference;

Call this action to reset the system state to the reference state for contingency analysis.

CTGSaveViolationMatrices("filename", filetype, UsePercentage, [ObjectTypesToReport], SaveContingency, SaveObjects, FieldListObjectType, [FieldList], IncludeUnsolvableCTGs);

This command will save contingency violations in a matrix format. Multiple files can be created for each type of violation as well as a file showing all contingencies that have violations and the objects that are violations under each contingency.

"filename" : Base name of the files to save. It is possible to save multiple files. This file name will be appended with an underscore and name of the object type that is being saved in the file.

filetype : There following options are available:

CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included.

CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing column headers you would see in a case information display

UsePercentage : Set to YES or NO. Set to YES the values will be reported as a percentage loading. If NO, the actual values will be reported.

[ObjectTypesToReport] : Comma delimited list of object types to include in the results. Options are BRANCH, BUS, INTERFACE, and CUSTOMMONITOR. If saving results by contingency (rows show contingencies that contain violations), the columns will only contain objects of these specified types. If saving results by object (row show objects that are violated under any contingency), files will only be created for these specified types.

SaveContingency : Set to YES or NO. Set to YES to save a file containing all contingencies that have at least one violation. The results will be reported as contingencies in rows and the violated elements in columns. Only violations of the specified [ObjectTypesToReport] will be included. Contingencies that have no violations because they failed to solve can be included by setting the IncludeUnsolvableCTGs to YES.

SaveObjects : Set to YES or NO. Set to YES to save a file for each of the object types specified in [ObjectTypesToReport]. The objects will be in rows and the columns will be the contingencies under which the objects are violated.

FieldListObjectType : Optional parameter – default is blank. Additional fields can be included depending on the object type that is being saved in the rows of each file. This parameter specifies the object type associated with the FieldList. Valid options are BRANCH, BUS, INTERFACE, CUSTOMMONITOR, or CONTINGENCY. As an example,

suppose that this parameter is set to BRANCH and a FieldList is specified. If SaveObjects is also YES, a file will be saved showing branch violations in each row of the file. In addition to columns showing contingencies and the violation value of the branch, columns will be added showing the value of the branch in the present system state for each of the fields in the FieldList.

- FieldList : Optional parameter – default is blank.
Additional fields can be included depending on the object type that is being saved in the rows of each file. This parameter specifies the additional fields to save. See the FieldListObjectType parameter for an explanation of how these extra fields are saved to file.
- IncludeUnsolvableCTGs : Optional parameter – default is NO.
Set to YES or NO. Set to YES to include contingencies that have been processed but did not solve either because the power flow failed or an abort action took place. The Solved field will be added to the file that lists results by contingency. Set to NO to only include contingencies that solved. This option is only relevant if the SaveContingency option is set to YES.

This command generates a detailed report of contingency violations, including one file that lists all contingencies causing branch overloads and another that shows each overloaded branch along with the specific contingencies under which the violations occur. It also includes additional data such as each branch's name and its LimitAmpA value. Contingencies that failed to solve are also included.

```
CTGSaveViolationMatrices("Violations", CSVCOLHEADER, YES, [BRANCH],
YES, YES, BRANCH, [Name, LimitAmpA], YES);
```

CTGSetAsReference;

Call this action to set the present system state as the reference for contingency analysis.

CTGSkipWithIdenticalActions;

(Added in the November 6, 2025 patch of Simulator 24)

Contingencies that have identical actions will have their Skip field set to YES. Only contingencies that originally have their Skip field set to No will be processed. The first contingency alphabetically is retained with Skip = NO while all others with identical actions will have Skip = YES. Messages are added to the log detailing which contingencies are set to skip.

CTGSolve("ContingencyName");

Call this action solve a particular contingency. The contingency is denoted by the "Contingency Name" parameter. The system state remains in the post-contingency state following the application of this command.

CTGSolveAll(DoDistributed, ClearAllResults);

Call this action to solve all of the contingencies that are not marked to be skipped.

- DoDistributed : Optional parameter – default is NO
Set to YES or NO. If set to YES, distributed methods will be used to solve contingency analysis if the Distributed Contingency Analysis add-on is installed. Distributed analysis requires the proper configuration and security settings to work.
- ClearAllResults : Optional parameter – default is YES
Set to YES or NO. If set to YES, all existing contingency results will be cleared even if a contingency is marked to be skipped. If set to NO, only

those contingencies that are marked to not be skipped will have their results cleared.

This command solves all contingencies that are not marked as skipped using local (non-distributed) processing. By setting ClearAllResults to YES, it clears all existing contingency analysis results before running the new analysis.

CTGSolveAll(NO, YES);

CTGSort([SortFieldList]);

This command sorts the contingencies stored in Simulator's internal data structure. This is different than sorting contingencies in case information displays in the GUI or sorting data when it is written to an auxiliary file. Contingencies are processed in the order in which they are stored in the internal data structure, and they are not sorted by default in this structure; contingencies are added to the internal data structure in the order in which they are created. This could be significant for other actions like CTGJoinActiveCTGs if the goal is to join contingencies alphabetically.

[SortFieldList] : Optional parameter – the default is to sort alphabetically by contingency name
This allows the specification of a sort order based on the available fields for the contingency object type. The format is: [variablename1:+:0, variablename2:-:1] where
variablename : the first parameter is the name of the field by which to sort. There is no limit to how many fields can be specified for sorting. For fields that require a location other than zero, variablename can be in the format fieldname:location.
+ or - : the second parameter indicates sort ascending for + and sort descending for -. This parameter must be specified.
0 or 1 : for the third parameter 0 means case insensitive and do not use absolute value, 1 means case sensitive or use absolute value. This parameter is optional.

This command sorts all contingencies in the internal data structure alphabetically by their Name field in ascending order, ignoring case sensitivity. This affects the order in which contingencies are processed or joined using script commands.

CTGSort ([Name:+:0]);

CTGVerifyIteratedLinearActions("filename");

Creates a text file that contains validation information relevant for using the option to **Iterate on Action Status** when using an linear method with contingency analysis or running ATC.

"filename" : Name of the text file into which validation information is written.

CTGWriteAllOptions("filename", KeyField, UseSelectedDataMaintainer, SaveDependencies, UseAreaZoneFilters);

Writes out all information related to contingency analysis as an auxiliary file using concise variable names and headers. Data is written using DATA sections instead of SUBDATA sections.

"filename" : Name of the auxiliary file to save.

KeyField : Optional parameter – default is PRIMARY

Indicates the identifier that should be used for the data. Valid entries are PRIMARY, SECONDARY, or LABEL. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and

nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used.

UseSelectedDataMaintainer

: Optional parameter – default is NO
Set to YES or NO. YES means to save only the information belonging to Data Maintainers where the Selected field is set to YES. NO means to save all information. Default is NO.

SaveDependencies

: Optional parameter – default is NO
Set to YES or NO. YES means that all relevant objects that are required to define the selected objects will also be saved. NO means to only save the selected objects.

UseAreaZoneFilters

: Optional parameter – default is NO
Set to YES or NO. YES means to save only the information for objects that meets the Area, Zone, Owner filters for Contingency Options and Limit Monitoring Settings related to the Area, Zone, Bus, Gen, and Shunt Objects. (Opt2 and Opt3 of CTGWriteResultsAndOptions)

See the CTGWriteResultsAndOptions script command for a list of the option settings that are considered. The equivalent options are set as follows for this script command:

Opt1 = NO , Opt2 = YES, Opt3 = YES, Opt4 = YES ,Opt5 = NO ,Opt6 = NO, Opt7 = NO, Opt8 = YES, Opt9 = YES , Opt10 = NO , Opt11 = NO, Opt12 = YES, Opt13 = YES, Opt14 = YES, Opt15 = YES, Opt16 = YES, Opt17 = NO, Opt18 = YES, Opt19 = YES, Opt20 = NO, Opt21 = NO, Opt22 = NO
--

The UseObjectIDs parameter with the CTGWriteResultsAndOptions script command is set automatically for this script command. The equivalent setting is YES_MS_3W.

CTGWriteAuxUsingOptions("filename", Append);

(Added in the August 18, 2023 patch of Simulator 23)

Writes out information related to contingency analysis as an auxiliary file. The CTGWriteAux_Options object is used to specify what object types are written and other relevant user specified parameters for how data is written.

"filename" : Name of the auxiliary file to save.
Append : Optional parameter – default is YES
Set to YES or NO. YES means to append the saved information to "filename". NO means that "filename" will be overwritten.

CTGWriteFilePTI("filename", BusFormat, TruncateCTGLLabels, "filtername", Append);

Write contingencies to file in the PTI CON format.

"filename" : The name of the text file to write out.
BusFormat : How to identify buses:
 Number : Using numbers
 Name8 : Using BusName_NomkV strings truncated to 8 characters
 Name12 : Using BusName_NomkV strings truncated to 12 characters
TruncateCTGLLabels : Set to YES or NO. YES means that the contingency labels will be truncated after 12 characters.
"filtername" : Optional – default is blank and all contingencies will be saved
This filter will be applied to the Contingency object type to specify which contingencies should be saved to file.
See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.
Append : Optional – default is NO
Set to YES or NO. YES means to append the saved information to "filename". NO means that "filename" will be overwritten.

This command exports contingencies all contingencies to a file named "contingencies.con" in the PTI CON format. Bus names will be represented using Name12 format—i.e., truncated BusName_NomkV strings up to 12 characters. Contingency names will also be truncated after 12 characters (TruncateCTGLLabels = YES). Since Append is set to NO, any existing content in the file will be overwritten. This is commonly used to generate PTI-compatible contingency files for external tools.

```
CTGWriteFilePTI("contingencies.con", Name12, YES, "", NO);
```

CTGWriteResultsAndOptions("filename", [opt1, opt2, opt3, ..., opt22], KeyField, UseDATASection, UseConcise, UseObjectIDs, UseSelectedDataMaintainers, SaveDependencies, UseAreaZoneFilters);

Writes out all information related to contingency analysis as an auxiliary file.

"filename" : Name of the auxiliary file to save.

[opt1, opt2, ..., opt22] : Each entry in the Option Settings parameter is either a YES or NO entry corresponding to the following options. These are all optional parameters, so if they are not specified or blank, the default entry given for each will be used.

- Opt1 : Save Unlinked Contingency Actions, default = NO
- Opt2 : Save Contingency Options, default = YES
- Opt3 : Save Limit Monitoring Settings, default = NO
- Opt4 : Save General Power Flow Solution Options, default = YES
- Opt5 : Save List Display Settings, default = NO
- Opt6 : Save Contingency Results, default = YES
- Opt7 : Save Inactive Violations, default = YES
- Opt8 : Save Interface Definitions, default = NO
- Opt9 : Save Injection Group Definitions, default = NO
- Opt10 : Save Distributed Computing Options, default = YES
- Opt11 : Suppress Gen and Load options when writing out Options, default = NO
- Opt12 : Save Contingency Definitions, default = YES
- Opt13 : Save Remedial Actions and Global Actions, default = YES
- Opt14 : Save Custom Monitor Definitions, default = YES
- Opt15 : Save Model Conditions, Model Filters, and Model Expressions, default = YES
- Opt16 : Save Advanced Filters used as part of Custom Monitors, Model Conditions, Remedial Actions, and Global Actions, default = YES
- Opt17 : Save Limit Cost Functions with Limit Sets, default = YES
- Opt18 : Automatically Convert Contingency Blocks and Global Actions, default = NO
- Opt19 : Save Voltage Control Groups, default = YES
- (Following added in the August 9, 2023 patch of Simulator 23)
- Opt20 : Save Primary Contingency Options for Combo Analysis, default = NO
- Opt21 : Save Primary Contingencies for Combo Analysis, default = NO
- Opt22 : Save CTG Combo Results, default = NO

KeyField : Optional parameter – default is PRIMARY

Indicates the identifier that should be used for the data. Valid entries are PRIMARY, SECONDARY, or LABEL. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used.

- UseDATASection : Optional parameter – default is NO
Set this to YES or NO. If YES, data that by default is specified using SUBDATA sections will instead be specified using DATA sections. For example, the actions that define a contingency by default are specified using a SUBDATA section. If choosing to use the DATA section instead, each action will be specified in a DATA record belonging to the ContingencyElement objecttype.
- UseObjectIDs : Optional parameter – default is NO
Possible settings are YES, NO, YES_MS, YES_3W, and YES_MS_3W.
YES : Any input with YES means to use the ObjectID field when writing objects with contingency settings instead of using multiple key fields to identify an object. The advantage to using ObjectIDs is that you only have one field to be used as an identifier rather than a changing number of fields that depends on the type of object. This will simplify your auxiliary file.
NO : This means to use the specified key fields to identify an object.
MS : Any input with MS means to write out a multi-section line by identifying it by the from bus, to bus, and circuit ID of the multi-section followed by the number of the particular section. This follows the PSLF format. If not writing out in this manner, individual sections will be written based on their from bus, to bus, and circuit ID.
3W : Any input with 3W means to write out a three-winding transformer using the buses at the three terminals of the transformer followed by the circuit ID with the first bus listed being the particular winding that is desired. If not writing out in this manner, a particular winding will be written with its terminal bus, the star bus of the transformer, and the circuit ID of the transformer.
- UseSelectedDataMaintainer : Optional parameter – default is NO
Set to YES or NO. YES means to save only the information belonging to Data Maintainers where the Selected field is set to YES. NO means to save all information.
- SaveDependencies : Optional parameter – default is NO
Set to YES or NO. YES means that all relevant objects that are required to define the selected objects will also be saved. NO means to only save the selected objects.
- UseAreaZoneFilters : Optional parameter – default is NO
Set to YES or NO. YES that to save only the information for objects that meets the Area, Zone, Owner filters for Contingency Options and Limit Monitoring Settings related to the Area, Zone, Bus, Gen, and Shunt Objects. (Opt2 and Opt3)

This command saves a comprehensive snapshot of contingency analysis data to "CTG_Export.aux", capturing components like contingency options, limit monitoring settings, power flow configurations, results, remedial actions, custom monitors, and model logic. It uses primary key fields for identification and stores elements in DATA sections rather than SUBDATA. Dependencies are included, and the saved data is filtered according to area/zone/owner filters.

```
CTGWriteResultsAndOptions("CTG_Export.aux", [NO, YES, YES, YES, NO,
YES, YES, NO, NO, YES, NO, YES, YES, YES, YES, YES, YES, NO, YES, NO,
NO, NO], PRIMARY, YES, NO, YES_MS_3W, NO, YES, YES);
```

Fault Analysis

Fault([Bus num], faulttype, R, X);

Fault([BRANCH nearbusnum farbusnum ckt], faultlocation, faulttype, R, X);

Call this function to calculate the fault currents for a fault. If the fault element is a bus then do not specify the fault location parameter. If the fault element is a branch, then the fault location is required.

- [BUS num] : This specifies the bus at which the fault occurs. You may also specify the bus using secondary keys or labels.
[BUS "name_nomkv"]
[BUS "label"]
- [BRANCH nearbusnum farbusnum ckt] : This specifies the branch on which the fault occurs. You may also specify the branch using secondary keys or labels.
[BRANCH "name_kv1" "name_kv2" ckt]
[BRANCH "buslabel1" "buslabel2" ckt]
[BRANCH "label"]
- Faultlocation : This specifies the percentage distance along the branch where the fault occurs. This percent varies from 0 (meaning at the nearbus) to 100 (meaning at the far bus)
- Faulttype : This specified the type of fault which occurs. There are four options:
SLG : Single Line To Ground fault
LL : Line to Line Fault
3PB : Three Phase Balanced Fault
DLG : Double Line to Group Fault.
- R, X : These parameters are optional and specify the fault impedance. If none are specified, then a fault impedance of zero is assumed.

This places a single-line-to-ground fault at Bus 1 with a fault impedance of $0.001 + j0.01$.
--

<code>Fault([BUS 1], SLG, 0.001, 0.01);</code>
--

FaultAutoInsert;

Multiple fault definitions are inserted using the options in the CTG_AutoInsert_Options object that are relevant for fault analysis. Faults can only be inserted for transmission lines or buses.

FaultClear;

Clears a single fault that has been calculated with the Fault script command.

FaultMultiple(UseDummyBus);

Runs fault analysis on a list of defined faults.

- UseDummyBus : Optional parameter – default is NO
Set to YES or NO. If YES, dummy buses should be created and inserted at the specified percent location for branch faults. Faults will be calculated at the dummy buses. If NO, the fault will be calculated at the branch terminal bus that is closest to the specified location.

LoadPTISEQData("filename", version);

Loads sequence data in the PTI format.

- "filename" : Name of file containing sequence data.
version : Integer representing the PTI version of the SEQ file to open.

This command loads a PTI-format sequence data file named "sequence_data.seq". The file is in PTI version 33 format.

<code>LoadPTISEQData("sequence_data.seq", 33);</code>

ATC (Available Transfer Capability)

ATCCreateContingentInterfaces(filter);

This command creates an interface based Transfer Limiter results from an ATC run. Each Transfer Limiter is comprised of a Limiting Element/Contingency pair. Each interface is then created with contingent elements from the contingency and the Limiting Element included as the monitored element.

filter : Optional – default is blank and all transfer limiters will be used
This is the name of an Advanced Filter. Only objects of type TransferLimiter that meet the named filter will be used to create new interfaces.

ATCDeleteAllResults;

Deletes all ATC results including TransferLimiter, ATCExtraMonitor, and ATCFlowValue object types.

ATCDeleteScenarioChangeIndexRange(ScenarioChangeType, [IndexRange]);

ATC scenarios are defined by RL (line rating and zone load), G (generator), and I (interface rating) changes. This command allows the deletion of entries within one of these change types by specifying the indices of the changes that should be deleted.

ScenarioChangeType : RL, G, or I to indicate the scenario change type to delete.
IndexRange : Comma-delimited list of integer ranges that must be enclosed in square brackets. The indices start at 0.

The following will delete line rating and zone load scenarios for indices 0 to 2, 5, and 7 to 9.
--

ATCDeleteScenarioChangeIndexRange(RL, [0-2, 5, 7-9]);

ATCDetermine([transactor seller], [transactor buyer], DoDistributed, DoMultipleScenarios);

Use this action to calculate the Available Transfer Capability (ATC) between a seller and a buyer. The buyer and seller must not be the same. Other options regarding ATC calculations should be set with the ATC_Options object type. If the distributed ATC add-on is installed, the optional DoDistributed flag may be set to indicate that the ATC should be solved using the distributed methods.

[transactor seller] : The seller (or source) of power. There are six possible settings:
[AREA num], [AREA "name"], [AREA "label"]
[ZONE num], [ZONE "name"], [ZONE "label"]
[SUPERAREA "name"], [SUPERAREA "label"]
[INJECTIONGROUP "name"], [INJECTIONGROUP "label"]
[BUS num], [BUS "name_nomkv"], [BUS "label"]
[SLACK]
[transactor buyer] : The buyer (or sink) of power. There are six possible settings, which are the same as for the seller.
DoDistributed : Optional parameter – default is NO
Set to YES to use the distributed ATC solution method.
DoMultipleScenarios : Optional parameter – default is set to YES if scenarios are defined
Set to YES to process each defined scenario.

This command will calculate how much real power can be transferred from Area "Top" to Area "Left" under current system conditions, ignoring any ATC scenarios or distributed processing.
--

ATCDetermine([AREA "Top"], [AREA "Left"], NO, NO);
--

ATCDetermineMultipleDirections(DoDistributed, DoMultipleScenarios);

Use this action to calculate the Available Transfer Capability (ATC) for all defined directions. Other options regarding ATC calculations should be set with the ATC_Options object type. If the distributed ATC add-on is installed, the optional DoDistributed flag may be set to indicate that the ATC should be solved using the distributed methods.

- DoDistributed : Optional parameter – default is NO
Set to YES to use the distributed ATC solution method.
- DoMultipleScenarios : Optional parameter – default is NO
Set to YES to process each defined scenario for all defined directions.

This command calculates the Available Transfer Capability (ATC) for all defined transfer directions using local (non-distributed) processing.

ATCDetermineMultipleDirections(NO, NO);

ATCDetermineATCFor(RL, G, I, ApplyTransfer);

Call this action to determine the ATC for Scenario RL, G, I.

- ApplyTransfer : Optional parameter – default is NO
Set to YES or NO. Set this value to YES to leave the system state at the transfer level that was determined. When using the Iterated Linear then Full Contingency solution method, the system state will retain the transfer level but the contingency will not be applied.

ATCDetermineMultipleDirectionsATCFor(RL, G, I);

Call this action to determine the ATC for Scenario RL, G, I for all defined directions.

ATCIncreaseTransferBy(amount);

Call this action to increase the transfer between the seller and buyer.

ATCRestoreInitialState;

Call this action to restore the initial state for the ATC tool.

ATCSetAsReference;

Call this action to set the present system state to the reference state for ATC analysis.

ATCTakeMeToScenario(RL, G, I);

Call this action to set the present case according to the scenarios along the RL, G, and I axes. All three parameters must be specified, with no defaults allowed. All entries must be specified as integers indicating the index of the respective scenario. Indices start at 0.

ATCWriteAllOptions("filename", AppendFile, KeyField);

Renamed to ATCDataWriteOptionsAndResults in December 9, 2021 patch of Simulator 22

ATCDataWriteOptionsAndResults("filename", AppendFile, KeyField);

Writes out all information related to ATC analysis to an auxiliary file. Saves the same information as the ATCWriteResultsAndOptions script command. Auxiliary file is formatted using the concise format for DATA section headers and variable names. Data is written using DATA sections instead of SUBDATA sections.

- "filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- AppendFile : Optional parameter - default is YES
YES means to append results to existing "filename." NO means to overwrite "filename" with the results.
- KeyField : Optional parameter – default is PRIMARY
Indicates the identifier that should be used for the data. Valid entries are

PRIMARY, SECONDARY, or LABEL. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used.

This command saves all ATC (Available Transfer Capability) options and results to an AUX file named "ATC_Results.aux". An existing file with this name will be overwritten and primary key fields will be used to identify objects in the file.

ATCDataWriteOptionsAndResults("ATC_Results", NO, PRIMARY);

ATCWriteResultsAndOptions("filename", AppendFile);

Writes out all information related to ATC analysis to an auxiliary file. This includes Contingency Definitions, Remedial Action Definitions, Limit Monitoring Settings, Solution Options, ATC Options, ATC results, as well as any Model Criteria that are used by the Contingency and Remedial Action Definitions.

Contingency and Remedial Action definitions will always be saved along with dependencies and only those object types that are dependencies will be saved. This means that if a Remedial Action definition uses a Model Filter, that Model Filter along with any Model Filter Conditions, i.e. other Model Filters or Model Conditions, will be saved. If a model criteria object is not being used by a Remedial Action or Contingency it will not be saved. Objects that are saved if they are dependencies include: Model Conditions, Model Filters, Model Planes, Model Expressions, Model Result Overrides, Interfaces, Injection Groups, Calculated Fields, and Expressions.

Dependencies for the ATC setup are also included. This includes Injection Groups that are used as the seller or buyer, Interfaces that are used in ATC Extra Monitor definitions, and Interfaces that are used in Multiple ATC Scenario definitions.

"filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

AppendFile : Optional parameter - default is YES
YES means to append results to existing "filename." NO means to overwrite "filename" with the results.

ATCWriteScenarioLog("filename", AppendFile, filter);

Writes out detailed log information for ATC Multiple Scenarios to a text file. If no scenarios have been defined, no file will be created; this is not treated as a fatal error, and an auxiliary file containing this command will continue to process subsequent commands.

"filename" : Name of log file. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

AppendFile : Optional parameter - default is NO
YES means to append log information to existing "filename." NO means to overwrite "filename" with the log information.

filter : Optional parameter – default is blank
Log information will only be written for ATC scenarios that meet the specified filter. See the [Using Filters in Script Commands](#) section for more information on specifying the filename. Default is to write the log for all ATC scenarios.

ATCWriteScenarioMinMax("filename", filetype, AppendFile, [fieldlist], Operation, OperationField, GroupScenario, [RLFilter], [GFilter], [IFilter], DirectionFilter, DoGroupDirection, filter, PreferIterativelyFound);

Writes out TransferLimiter results from multiple scenario ATC calculations. The results are grouped based on the input parameters, and the minimum, maximum, or minimum and maximum limiter from each group is written to file.

"filename"	: Name of file. See the Specifying File Names in Script Commands section for special keywords that can be used when specifying the file name.
filetype	: There are several options for the filetype - AUXCSV : save as a comma-delimited auxiliary data file - AUX : save as a space-delimited auxiliary data file - CSV : save as a normal CSV file without the AUX file syntax. The first few lines of the text file will represent the object name and field variable names. - CSVNOHEADER : save as a normal CSV text file, without the AUX file formatting. The object name and field variable names are NOT included. - CSVCOLHEADER : save as a normal CSV without the AUX syntax and with the first row showing the normal column headers seen in a case information display
AppendFile	: YES means to append to existing "filename." NO means to overwrite "filename."
[fieldlist]	: Comma-delimited list of fields to save. For numeric fields, the number of digits and the number of decimal places (digits to right of decimal) can be specified by using the following format for the field, <code>variablenamelegacy:location:digits:rod</code> or <code>concisename:digits:rod</code> . See the Specifying Field Variable Names in Script Commands topic for more information on specifying this list.
Operation	: This is the operation to perform on each grouping of transfer limiters. The OperationField specifies the value to use when performing the operation. - MIN : Write the minimum limiter for each grouping. If there are limiters for the grouping, at most one limiter per grouping will be written to the file. - MAX : Write the maximum limiter for each grouping. If there are limiters for the grouping, at most one limiter per grouping will be written to the file. - MIN/MAX : Write the minimum and maximum limiter per grouping. If there are limiters for the grouping, at most two limiters per grouping will be written to the file. There will only be one limiter if the minimum and maximum limiters are the same.
OperationField	: Variable name of the field to use for the value in the operation. Valid entries are TransferLimit or any available ATC_ExtraMonitor field.
GroupScenario	: Indicates which type of scenario to use for the grouping. - None : The results are not grouped by scenarios. All scenario results are included as one group. Direction grouping also applies. - RL : The results are grouped by RL scenario. For each RL scenario all results for the G and I scenarios are included. Direction grouping also applies.

	G	: The results are grouped by G scenario. For each G scenario all results for the RL and I scenarios are included. Direction grouping also applies.
	I	: The results are grouped by I scenario. For each I scenario all results for the RL and G scenarios are included. Direction grouping also applies.
[RLFilter]		: Optional parameter – default is blank. Determines which RL scenarios are included in the groupings. If left blank, all RL scenarios are included. Specify the scenarios using an integer range list enclosed in square brackets, which is a comma-separated list of either a single integer or a range of integers such as [1, 3-4, 7]. This parameter is ignored if filter is not blank.
[GFilter]		: Optional parameter – default is blank Determines which G scenarios are included in the groupings. If left blank, all G scenarios are included. Specify the scenarios using an integer range list enclosed in square brackets, which is a comma-separated list of either a single integer or a range of integers such as [1, 3-4, 7]. This parameter is ignored if filter is not blank.
[IFilter]		: Optional parameter – default is blank Determines which I scenarios are included in the groupings. If left blank, all I scenarios are included. Specify the scenarios using an integer range list enclosed in square brackets, which is a comma-separated list of either a single integer or a range of integers such as [1, 3-4, 7]. This parameter is ignored if filter is not blank.
DirectionFilter		: Optional parameter – default is IGNORE A single direction or multiple directions can be run for multiple scenarios. This parameter determines which set of directions to write in the results. IGNORE : Use only the single direction results in the groupings. Scenario grouping also applies. ALL : Use all of the multiple direction results in the groupings. The DoGroupDirection parameter will determine if groupings will be formed for each direction or if all directions will be combined before determining the scenario groupings. "Direction Name" : Only use the specified direction in the groupings. Scenario grouping also applies.
DoGroupDirection		: Optional parameter – default is YES This parameter is used only if DirectionFilter = ALL. Set to YES if groupings should be formed for each of the multiple direction results. Within a direction grouping, scenarios will also be grouped based on specified parameters. Set to NO if all directions are considered together and the only grouping that should be done is based on scenario groupings.
filter		: Optional parameter – default is blank Groupings will be formed only for TransferLimiters that meet this filter. RLFilter , GFilter , and IFilter will be ignored when specifying this filter, but all other options will apply. See the Using Filters in Script Commands section for more information on specifying the filename.
PreferIterativelyFound		: Optional parameter – default is YES Set to YES so that transfer limiters that have been iteratively found will take precedence when determining the limiter that meets the selected operation for each grouping. If there are no limiters that have been

iteratively found or this is set to NO, the first limiter that is found that meets the operation will be reported for each grouping.

This command writes the minimum and maximum Available Transfer Capability (ATC) limiters for RL (Rating & Load) scenarios indexed 0 through 3 into a CSV file named "ATC_Summary.csv". The results are grouped by both scenario and direction and include the specified field list for each limiter.

```
ATCWriteScenarioMinMax("ATC_Summary", CSV, NO, [DirectionName,
LineZoneChange, GeneratorChange, InterfaceChange, TransferLimit,
ObjectDesc, Contingency, Sensitivity, ValuePreTrans, Limit], MIN/MAX,
TransferLimit, RL, [0-3], , , ALL, YES, , YES);
```

ATCWriteToExcel("worksheetname", [fieldlist]);

Sends ATC analysis results to an Excel spreadsheet. This script command is available only for Multiple Scenarios ATC analysis.

"worksheetname" : The name of the Excel sheet where the results will be sent to.
fieldlist : Optional parameter
If specified, the results will be saved including this list of fields. If not specified, the results will be saved with the fields that are specified with the TransferLimiter DataGrid. It is possible that the results could be blank if the TransferLimiter DataGrid has not been initialized by either opening the ATC dialog or loading the DataGrid settings from an auxiliary file. To make sure that results are stored and the desired fields are included, it is suggested that the fieldlist option be used.

This writes ATC (Available Transfer Capability) results from a Multiple Scenarios ATC analysis into an Excel worksheet named "ATCResults".

```
ATCWriteToExcel("ATCResults", [DirectionName, LineZoneChange,
GeneratorChange, InterfaceChange, TransferLimit, ObjectDesc,
Contingency, Sensitivity, ValuePreTrans, Limit]);
```

ATCWriteToText("filename", filetype, [fieldlist]);

This is used with Multiple Scenario ATC analysis. Multiple files are created with "filename" as the primary identifier and the Interface scenario label appended to the end of the filename. Separate files are created for each of the Interface scenarios. Results inside the files are separated into sections based on the number of Rating/Load scenarios.

"filename" : Primary identifier for the name of the file in which to save the results.
"filename" gets appended with the Interface scenario label to complete the filename.
filetype : Either TAB or CSV. This indicates the delimiter to use when writing out the file(s). This is an optional parameter with TAB being the default if omitted.
fieldlist : Optional parameter
If specified, the results will be saved including this list of fields. If not specified, the results will be saved with the fields that are specified with the TransferLimiter DataGrid. It is possible that the results could be blank if the TransferLimiter DataGrid has not been initialized by either opening the ATC dialog or loading the DataGrid settings from an auxiliary file. To make sure that results are stored and the desired fields are included, it is suggested that the fieldlist option be used.

This script writes separate CSV files for each interface scenario in the ATC analysis using the specified field list.

```
ATCWriteToText("ATCOutput", CSV, [DirectionName, LineZoneChange,  
GeneratorChange, InterfaceChange, TransferLimit, ObjectDesc,  
Contingency, Sensitivity, ValuePreTrans, Limit]);
```

GIC (Geomagnetically Induced Current)

GICCalculate(MaxField, Direction, SolvePF);

Calculates the "Single Snapshot" using GICSolution Options

MaxField	: Maximum Electric Field in Volts/km
Direction	: Storm Direction, Degrees from 0 to 360
SolvePF	: Select YES or NO to include GIC in the Power Flow

This command calculates a "Single Snapshot" Geomagnetically Induced Current (GIC) solution using a maximum electric field strength of 5.0 V/km and a storm direction of 90 degrees (eastward). By setting SolvePF to YES, the GIC solution is integrated into the power flow analysis.

```
GICCalculate(5.0, 90, YES);
```

GICClear;

Clear GIC values.

GICLoad3DEfield(FileType, "FileName", SetupOnLoad);

Loads GIC data including time varying fields.

FileType	: Type of file to be loaded. Options are CSV, B3D, JSON, and DAT.
FileName	: Name of the file to be loaded
SetupOnLoad	: Select YES to run procedure to setup time varying series after loading file or NO to skip the setup process.

This command loads GIC data from the file "TimeSeriesGIC.b3d" in B3D format, which contains time-varying electric field values used for GIC simulations. By setting SetupOnLoad to YES, the command initiates the procedure to configure the time series data upon loading.

```
GICLoad3DEfield(B3D, "TimeSeriesGIC.b3d", YES);
```

GICReadFilePSLF("FileName");

Added in the August 27, 2024 patch of Simulator 23.

Reads GIC supplemental data from a GMD text file format.

"FileName"	: Name of the file to be loaded, with extension GMD
------------	---

GICReadFilePTI("FileName");

Added in the August 27, 2024 patch of Simulator 23.

Reads GIC supplemental data from a GIC text file format.

"FileName"	: Name of the file to be loaded, with extension GIC
------------	---

GICSaveGMatrix("GMatrixFileName", "GMatixIDFileName");

Added on November 19, 2024 to Simulator 24

Use this action to save the GMatrix used with the GIC calculations in a file formatted for use with Matlab

"GMatrixFileName"	: File in which to save the G Matrix.
"GMatrixIDFileName"	: File to save a description of what each row and column of the G Matrix represents.

This command saves the G Matrix used in GIC calculations to a MATLAB-compatible file named "GIC_GMatrix.mat", with an accompanying identifier file "GIC_GMatrix_IDs.txt" that describes what each row and column in the matrix represents.

```
GICSaveGMatrix("GIC_GMatrix.mat", "GIC_GMatrix_IDs.txt");
```

GICClear;

Clear GIC Values

GICSensitivitiesCalculate(SensitivityType,FieldDirectionOption,FieldDirectionDeg,[BranchesToInclude]);

Added on March 8, 2026 to Simulator 25.

Calculates the GIC sensitivities associated with either Transformer Ieffective values or Line Amp Input Sensitivities. Hence, this command duplicates the **Recalculate Sensitivities** found on the **GIC Analysis**

Dialog, Sensitivity Analysis Page. The command has the following options.

- Sensitivity Type : Use 0 for Line Amp Input Sensitivities or 1 for Transformer Ieffective GIC Sensitivities; default is 1.
- FieldDirectionOption : For the Assumed Field Direction, select 0 for parallel to each line, or 1 in the angle direction specified in the FieldDirectionDeg field. Default is 0.
- FieldDirectionDeg : The assumed field direction in degrees when the FieldDirectionOption is 1; ignored if this field is 0. Valid values are ≥ 0 degrees and < 360 degrees.
- BranchesToInclude : Specifies the line to use if the Sensitivity Type is set to 0, or the list of transformers to include in the Sensitivity Type is set to 1. Each object is identified in double quotes using its busnum1 busnum2 ckt. Multiple transformers are separated with commas. For example, ["1 2 1", "5 6 2"] calculates the sensitivities for the transformer between Buses 1 and 2 circuit 1 and Buses 5 and 6 circuit 2. For Sensitivity Type 1 the list may be empty. In that case the already selected transformers are used. Otherwise the set of selected transformers is changed to the list entries.

GICSetupTimeVaryingSeries(Start, End, Delta);

Added in the April 19, 2024 patch of Simulator 23.

Creates a set of Branch series DC input voltages in the "Time-Varying Series Voltage Inputs" Calculation Mode from the Active Event(s) in the "Time-Varying Electric Field Inputs" Calculation Mode.

Note: Set all arguments to 0 to create a complete set of Time-Varying Series Voltage Inputs with the source time offset values.

- Start : Start Time Offset (seconds), (optional) default is 0.0
- End : End Time Offset (seconds), (optional) default is 0.0
- Delta : Sampling Rate (seconds), (optional) default is 0.0

This command generates time-varying series DC voltage inputs for GIC analysis by converting active electric field event data into voltage input series. It starts at 0 seconds, ends at 3600 seconds (1 hour), and samples the data every 60 seconds.

GICSetupTimeVaryingSeries(0, 3600, 60);

GICShiftOrStretchInputPoints(LatShift, LonShift, MagScalar, StretchScalar, UpdateTimeVaryingSeries);

Scales, shifts, or stretches the active set of Time Varying Electric Field Inputs.

- LatShift : Latitude Shift in degrees, default is 0.0
- LonShift : Longitude Shift in degrees, (optional) default is 0.0
- MagScalar : E-Field Magnitude scalar, (optional) default is 1.0 (to NOT scale)
- StretchScalar : Geographic Stretch scalar, (optional) default is 1.0 (to NOT stretch)
- UpdateTimeVaryingSeries : Select YES or NO to update the time varying voltage input values, (optional) default is NO.

Added in the April 19, 2024 patch of Simulator 23.

This command modifies the active set of Time Varying Electric Field Inputs by shifting them 1.0° north and 2.0° west, scaling the electric field magnitudes by 1.2 (increasing intensity by 20%), and stretching the geographic area by a factor of 1.1. Setting UpdateTimeVaryingSeries to YES ensures that the corresponding voltage inputs are recalculated based on the adjusted field data.

GICShiftOrStretchInputPoints(1.0, -2.0, 1.2, 1.1, YES);

GICTimeVaryingCalculate(TheTime,SolvePF);

Calculate GIC Values using the "Time-Varying Series Voltage Inputs" Calculation Mode.

TheTime : Current Time Offset from Reference (seconds)
SolvePF : Select YES or NO to include GIC in the Power Flow and Transient Stability

This command calculates GIC values at 1800 seconds (30 minutes) into a time-varying event using the "Time-Varying Series Voltage Inputs" mode. By setting SolvePF to YES, the calculation integrates GIC effects directly into the power flow and transient stability analysis.

GICTimeVaryingCalculate(1800, YES);

GICTimeVaryingAddTime(NewTime);

Adds a new input values at specified time

NewTime : New Time for new input values

GICTimeVaryingDeleteAllTimes;

Delete All Input time varying voltage input values

GICTimeVaryingEFieldCalculate(TheTime,SolvePF);

Calculate GIC Values using the "Time-Varying Electric Field Inputs" Calculation Mode.

TheTime : Current Time Offset from Reference (seconds)
SolvePF : Select YES or NO to include GIC in the Power Flow and Transient Stability

This command calculates GIC values at 900 seconds (15 minutes) into a time-varying event using the "Time-Varying Electric Field Inputs" mode. By setting SolvePF to YES, it ensures that the calculated GIC values are factored into the power flow and transient stability simulations.

GICTimeVaryingEFieldCalculate(900, YES);

GICTimeVaryingElectricFieldsDeleteAllTimes;

Clear all the time varying electric field input values.

GICWriteFilePSLF("FileName", UseFilters);

Added in the August 27, 2024 patch of Simulator 23.

Writes GIC supplemental data from a GMD text file format.

"FileName" : Name of the file to be loaded, with extension GMD
UseFilters : YES – to user Area/Zone Filters; NO – to insert for entire case.

GICWriteFilePTI("FileName", UseFilters, Version);

Added in the August 27, 2024 patch of Simulator 23.

Writes GIC supplemental data from a GIC text file format.

"FileName" : Name of the file to be loaded, with extension GIC
UseFilters : YES – to user Area/Zone Filters; NO – to insert for entire case.
Version : The version number of the GIC file, 1 – 4, (optional) default is 4.

GICWriteOptions("FileName", KeyField);

Calculates the "Single Snapshot" using GICSolution Options

FileName : Name of Aux file name to write out the options
KeyField : KeyField indicates the identifier that should be used for the data. Valid entries are PRIMARY, SECONDARY, or LABEL. The default setting is PRIMARY. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used.

This command writes the current GIC solution options to an auxiliary file named "GIC_Options.aux", using primary key fields (like bus numbers) to identify system elements.

GICWriteOptions("GIC_Options.aux", PRIMARY);
--

ITP (Integrated Topology Processing)

CloseWithBreakers(objecttype, filter or [object identifier], OnlyEnergizeSpecifiedObjects, [SwitchingDeviceTypes], CloseNormallyClosedDisconnects);

This action is used to specify which objects are to be energized by closing breakers and to actually close those breakers. The status of an object will be set to closed if necessary in addition to closing the breakers. If only the status of an object needs to be changed to close an object, that will occur without requiring any breakers to be closed.

objecttype	: Objects that are valid to be energize. Only allowed for Buses, Generators, Loads, Transmission Lines, Switched Shunts, DC Lines, Injection Groups, and Interfaces.
Filter	: The second parameter can either be a filter specification or an object identifier. When specifying a filter, the following options are available: SELECTED : only objects whose Selected field = YES will be energized AREAZONE : only objects that meet the area/zone/owner filters will be energized "FilterName" : only objects that meet the specified filter will be energized. See the Using Filters in Script Commands section for more information on specifying the filename.
[object identifier]	: The second parameter can either be a filter specification or an object identifier. When using an object identifier, the objecttype is applicable and no further specification of the type needs to be included with the object identifier as is done with some other script commands. The following describe the possible objecttypes and identifier options: BUS : [busnum] ["name_nomkv"] ["label"] GEN : [busnum id] ["name_nomkv" id] ["buslabel" id] ["label"] LOAD : [busnum id] ["name_nomkv" id] ["buslabel"] ["label"] BRANCH : [busnum1 busnum2 ckt] ["name_kv1" "name_kv2" ckt] ["buslabel1" "buslabel2" ckt] ["label"] SHUNT : [busnum id] ["name_nomkv" id] ["buslabel" id] ["label"] INJECTIONGROUP: ["name"] INTERFACE : ["name"] DCLINE : [num rectnum invnum] [num "rectnam_nomkv" "invname_nomkv"] [num "rectlabel" "invlabel"] ["label"]

OnlyEnergizeSpecifiedObjects

- : optional parameter, default is NO.
 - YES : No extra objects in addition to those specified in the filter can be energized. Each object will be evaluated individually.
 - NO : Extra objects could be energized in addition to those specified if a group of breakers required to energize a specified object also causes other objects to be energized. All objects will be evaluated collectively for determining which objects can be energized, i.e. breakers that cause one object to be energized might also be needed for another object to be energized.

[SwitchingDeviceTypes]

- : optional parameter, default is "Breaker". This is a comma-separated list naming the Branch Device Types for switching devices that should be included when determining which devices to close to energize objects. Options include "Breaker" and "Load Break Disconnect".

CloseNormallyClosedDisconnects

- : optional parameter, default is NO.
 - YES : When searching for the specified SwitchingDeviceTypes and a Disconnect is encountered that is open but normally closed, it will be closed and the search for open switching devices will continue past the Disconnect. Additionally, Disconnects that are in series with any open devices of the specified SwitchingDeviceTypes will be closed if they are normally open.
 - NO : Only switching devices of the specified SwitchingDeviceTypes will be closed.

This command energizes the generator at bus 1 with ID 1, allowing other connected elements to be energized in the process. It uses both breakers and load-break disconnects as valid switching devices and will also close any disconnects that are normally closed but currently open.

```
CloseWithBreakers(GEN, [1 1], NO, ["Breaker", "Load Break Disconnect"], YES);
```

ExpandAllBusTopology;

This action is used to expand the topology around all buses in the case according to a topology type that is specified with a custom string field (currently Custom String 5) for the bus. New breakers and nodes (buses) will be inserted as necessary.

ExpandBusTopology(BusIdentifier, TopologyType);

This action is used to expand the topology around the specified bus according to the specified topology type. New breakers and nodes (buses) will be inserted as necessary.

- BusIdentifier : A bus can be identified in one of these formats: BUS busnum, BUS name_nomkv, BUS label.
- TopologyType : These types of breaker configurations are allowed:
DOUBLEBUSDOUBLEBREAKER, MAINTRANSFER, RINGBUS, BREAKERANDAHALF, SINGLEBUS, and SECTIONALIZEBUS.

This command modifies the topology around Bus 1, expanding it into a Breaker-and-a-Half configuration. It inserts the necessary breakers and additional nodes (buses) to match the selected topology type.

```
ExpandBusTopology(BUS 1, BREAKERANDAHALF);
```

**OpenWithBreakers(objecttype, filter or [object identifier], [SwitchingDeviceTypes],
OpenNormallyOpenDisconnects);**

This action is used to specify which objects are to be disconnected by opening breakers and to actually open those breakers.

- objecttype : Objects that are valid to be disconnected. Only allowed for Buses, Generators, Loads, Transmission Lines, Switched Shunts, DC Lines, Injection Groups, and Interfaces.
- Filter : The second parameter can either be a filter specification or an object identifier. When specifying a filter, the following options are available:
 - SELECTED : only objects whose Selected field = YES will be energized
 - AREAZONE : only objects that meet the area/zone/owner filters will be energized
 - "FilterName" : only objects that meet the specified filter will be energized. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.
- [object identifier] : The second parameter can either be a filter specification or an object identifier. When using an object identifier, the objecttype is applicable and no further specification of the type needs to be included with the object identifier as is done with some other script commands. The following describe the possible objecttypes and identifier options:
 - BUS : [busnum]
["name_nomkv"]
["label"]
 - GEN : [busnum id]
["name_nomkv" id]
["buslabel" id]
["label"]
 - LOAD : [busnum id]
["name_nomkv" id]
["buslabel"]
["label"]
 - BRANCH : [busnum1 busnum2 ckt]
["name_kv1" "name_kv2" ckt]
["buslabel1" "buslabel2" ckt]
["label"]
 - SHUNT : [busnum id]
["name_nomkv" id]
["buslabel" id]
["label"]
 - INJECTIONGROUP : ["name"]
 - INTERFACE : ["name"]
 - DCLINE : [num rectnum invnum]
[num "rectnam_nomkv" "invname_nomkv"]
[num "rectlabel" "invlabel"]
["label"]
- [SwitchingDeviceTypes] : Optional parameter – default is "Breaker"
This is a comma-separated list naming the Branch Device Types for switching devices that should be included when determining which

devices to open to disconnect objects. Options include "Breaker" and "Load Break Disconnect".

OpenNormallyOpenDisconnects

: optional parameter, default is NO.

YES : When searching for the specified SwitchingDeviceTypes and a Disconnect is encountered that is closed but normally open, it will be opened and the search for closed switching devices will terminate along that path.

NO : Only switching devices of the specified SwitchingDeviceTypes will be opened.

This command disconnects the transmission line from bus 1 to bus 2 ckt 1, by opening associated breakers only. Since OpenNormallyOpenDisconnects is set to NO, the command will not change the status of any disconnects that are normally open.

OpenWithBreakers(BRANCH, [1 2 1], ["Breaker"], NO);

SaveConsolidatedCase("filename", filetype, [BusFormat, TruncateCtgLabels, AddCommentsForObjectLabels]);

This action saves the full topology model into a consolidated case.

"filename" : The name of the consolidated case file to be saved.

Filetype : Optional parameter to specify the type of the file to be saved. If omitted, the latest version of the PWB will be used.

PWB : save a pwb file with the most recent version

PWBX: save a pwb with version X

PTIXX: save the file with PTI version XX, where XX is between 23 and 35

GEXX : save the file with GE PSLF version XX, where XX is between 14 and 23

BusFormat : optional parameter used to specify the bus identifier format in the .CON file used to store contingencies when saving a PTI file

Number : identify buses using number

Name8 : identify buses using the Name_kV identifier truncated to 8 characters

Name12 : identify buses using the Name_kV identifier truncated to 12 characters

TruncateCTGLabels : optional parameter used to specify if the contingency labels should be truncated to 12 characters when saving the contingencies in PTI format

YES : truncate the contingency labels to 12 characters

NO : do not truncate the contingency labels

AddCommentsForObjectLabels

: (optional) YES adds object labels to the end of data records when saving a RAW file. (default NO)

This command saves the consolidated case of a full topology model to a .pwb file named "B7Flat_ConsolidatedCopy.pwb".

SaveConsolidatedCase("B7Flat_ConsolidatedCopy.pwb", PWB);

OPF (Optimal Power Flow) and SCOPF

SolvePrimalLP("filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);

Call this action to perform a primal LP OPF solution. The parameters are all optional and specify a conditional response depending on whether the solution is successfully found. If parameters are not passed then default values will be used.

- "filename1" : The filename of the auxiliary file to be loaded if there is a successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".
- "filename2" : The filename of the auxiliary file to be loaded if there is a NOT successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".
- CreatelfNotFound1 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename1".
- CreatelfNotFound2 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename2".

This command attempts to solve a primal linear programming optimal power flow (LP OPF) problem. If the solution is successful, the auxiliary file "LP_Success.aux" is loaded with object creation allowing for any missing elements referred to in DATA sections to be created. If the solution fails, "LP_Failure.aux" is loaded, but new objects will not be created during the load for DATA sections in the Legacy Auxiliary File Header format.

SolvePrimalLP("LP_Success.aux", "LP_Failure.aux", YES, NO);

InitializePrimalLP("filename1", "filename2", CreatelfNotFound1, CreatelfNotFound2);

This commands clears all the structures and results of previous primal LP OPF solutions. The parameters are all optional and specify a conditional response depending on whether the solution is successfully found. If parameters are not passed then default values will be used.

- "filename1" : The filename of the auxiliary file to be loaded if there is a successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".
- "filename2" : The filename of the auxiliary file to be loaded if there is a NOT successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".
- CreatelfNotFound1 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename1".

CreaterIfNotFound2 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename2".

This command clears all data and results from previous primal LP OPF runs. If the initialization is successful, it loads the auxiliary file "LP_InitSuccess.aux"; if it fails, it loads "LP_InitFail.aux". In both cases any objects contained in DATA sections using the Legacy Auxiliary File Header format will not be created.

InitializePrimalLP("LP_InitSuccess.aux", "LP_InitFail.aux", NO, NO);

SolveSinglePrimalLPOuterLoop("filename1", "filename2", CreaterIfNotFound1, CreaterIfNotFound2);

This action is basically identical to the SolvePrimalLP action, except that this will only perform a single optimization. The SolvePrimalLP will iterate between solving the power flow and an optimization until this iteration converges. This action will only solve the optimization routine once, then resolve the power flow once and then stop.

SolveFullSCOPF (BCMethod, "filename1", "filename2", CreaterIfNotFound1, CreaterIfNotFound2);

Call this action to perform a full Security Constrained OPF solution. The parameters are all optional and specify a conditional response depending on whether the solution is successfully found. If parameters are not passed then default values will be used.

BCMethod : The solution method to be used for solving the base case. The options are:
POWERFLOW : for single power flow algorithm (default)
OPF : for the optimal power flow algorithm.

"filename1" : The filename of the auxiliary file to be loaded if there is a successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".

"filename2" : The filename of the auxiliary file to be loaded if there is a NOT successful solution. You may also specify STOP, which means that all AUX file execution should stop under the condition. Default Value = "".

CreaterIfNotFound1 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename1".

CreaterIfNotFound2 : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename2".

This command performs a full Security Constrained Optimal Power Flow (SCOPF) using the OPF algorithm for the base case solution. If the SCOPF completes successfully, "SCOPF_Success.aux" is loaded with object creation allowing for any missing elements referred to in DATA sections to be created. If it fails, "SCOPF_Failure.aux" is loaded, but new objects will not be created during the load for DATA sections in the Legacy Auxiliary File Header format.

```
SolveFullSCOPF(OPF, "SCOPF_Success.aux", "SCOPF_Failure.aux", YES, NO);
```

OPFWriteResultsAndOptions("filename");

Writes out all information related to OPF analysis as an auxiliary file. This includes Limit Monitoring Settings, options for Areas, Buses, Branches, Interfaces, Generators, SuperAreas, OPF Solution Options.

PV Analysis

Changes were made with Simulator version 14 to eliminate the need for a PV study name. To maintain functionality with any existing processes that users might have in place using older script definitions, scripts from older versions of Simulator will still be supported if the name is specified. However, the name will just be ignored. The script formats given here reflect the changes for versions 14 and later.

The PVCreate script required in previous versions is no longer necessary starting with Simulator version 14. Versions starting with 14 will still recognize this action if is included and will simply set the source and sink for the study. This does the same thing as PVSetSourceAndSink.

It is highly recommended that for any new processes the new script formats specified here be used.

PVClear;

Call the function to clear all the results of the PV study.

PVDataWriteOptionsAndResults("filename", AppendFile, KeyField);

Writes out all information related to PV analysis to an auxiliary file. Saves the same information as the PVWriteResultsAndOptions script command. Auxiliary file is formatted using the concise format for DATA section headers and variable names. Data is written using DATA sections instead of SUBDATA sections.

- | | | |
|------------|---|--|
| "filename" | : | Name of the auxiliary file to save. See the Specifying File Names in Script Commands section for special keywords that can be used when specifying the file name. |
| AppendFile | : | Optional parameter - default is YES
YES means to append results to existing "filename." NO means to overwrite "filename" with the results. |
| KeyField | : | Optional parameter – default is PRIMARY
Indicates the identifier that should be used for the data. Valid entries are PRIMARY, SECONDARY, or LABEL. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used. |

This command saves all information related to PV (Power-Voltage) analysis to an auxiliary file named "PV_Results.aux". It overwrites any existing file (AppendFile = NO) and uses primary key fields (like bus numbers) for identifying elements in the output (KeyField = PRIMARY).

```
PVDataWriteOptionsAndResults("PV_Results.aux", NO, PRIMARY);
```

PVDestroy;

Call the function to destroy the PV study. This will remove all results and prevent any restoration of the initial state that is stored with the PV study.

PVQVTrackSingleBusPerSuperBus;

If the topology processing add-on is installed, then this script command can be used to reduce the number of monitored buses. The script action examines each monitored value for each bus and determines if that bus is part of a super bus and selects monitored buses so that only the pnode is monitored.

PVRun([elementSource], [elementSink]);

Call this function start the PV study and optionally specify the source and sink elements.

- | | | |
|-----------------|---|--|
| [elementSource] | : | Optional parameter – default to using element already set
The source of power for the PV study. Only injection groups can be used:
[INJECTIONGROUP "name"] or [INJECTIONGROUP "label"] |
| [elementSink] | : | Optional parameter – default to using element already set |

The sink of power for the PV study. Only injection groups can be used:
[INJECTIONGROUP "name"] or [INJECTIONGROUP "label"]

This command starts a PV (Power-Voltage) analysis using "SolarGen" as the source injection group and "UrbanLoad" as the sink injection group. These groups define where power will be incrementally injected and withdrawn during the study to analyze system voltage response.

```
PVRun([INJECTIONGROUP "SolarGen"], [INJECTIONGROUP "UrbanLoad"]);
```

PVSetSourceAndSink([elementSource], [elementSink]);

Call the function to specify the source and sink elements to perform the PV study.

[elementSource] : The source of power for the PV study. There is only one possible setting:
[INJECTIONGROUP "name"] or [INJECTIONGROUP "label"]
[elementSink] : The sink of power for the PV study. There is only one possible setting,
which is the same as for the source.

This command sets up the source and sink for a PV study, designating "SolarGen" as the power source and "UrbanLoad" as the sink.

```
PVSetSourceAndSink([INJECTIONGROUP "SolarGen"], [INJECTIONGROUP  
"UrbanLoad"]);
```

PVStartOver;

Call the function to start over the PV study. This includes clear the activity log, clear results, restore the initial state, set the current state as initial state, and initialize the step size.

PVWriteInadequateVoltages("filename", AppendFile, InadequateType);

Call this action to save PV Inadequate Voltages in a CSV file.

"filename" : Name of the CSV file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
AppendFile : Optional parameter – default is YES
Set this to YES or NO. Setting this to YES will cause the data to be appended to an existing file. Setting this to NO will cause any existing file to be overwritten.
InadequateType : Optional parameter – default is LOW
Set this to HIGH or LOW to indicate the type of inadequate voltages to save to file.

This command saves all low inadequate voltages identified during a PV study to the file "PV_LowVoltages.csv". It overwrites any existing file (AppendFile = NO) and specifically filters for LOW voltage violations (InadequateType = LOW).

```
PVWriteInadequateVoltages("PV_LowVoltages.csv", NO, LOW);
```

PVWriteResultsAndOptions("filename", AppendFile);

Writes out all information related to PV analysis to an auxiliary file. This includes Contingency Definitions, Remedial Action Definitions, Solution Options, PV Options, PV results, ATC Extra Monitors, as well as any Model Criteria that are used by the Contingency and Remedial Action Definitions.

Contingency and Remedial Action definitions will always be saved along with dependencies and only those object types that are dependencies will be saved. This means that if a Remedial Action definition uses a Model Filter, that Model Filter along with any Model Filter Conditions, i.e. other Model Filters or Model Conditions, will be saved. If a model criteria object is not being used by a Remedial Action or Contingency it will not be saved. Objects that are saved if they are dependencies include: Model

Conditions, Model Filters, Model Planes, Model Expressions, Model Result Overrides, Interfaces, Injection Groups, Calculated Fields, and Expressions.

Dependencies for the PV setup are also included. This includes Injection Groups that are used as the seller or buyer and Interfaces that are used as part of interface ramping options.

- "filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- AppendFile : Optional parameter – default is YES
Set this to YES or NO. Setting this to YES will cause the data to be appended to an existing file. Setting this to NO will cause any existing file to be overwritten.

RefineModel(objecttype, filter, Action, Tolerance);

Call this function to refine the system model to fix modeling idiosyncrasies that cause premature loss of convergence during PV and QV studies.

- Objecttype : The objecttype being selected:
AREA
ZONE
- Filter : Specify a filter to limit the objects that will be included.
 - Blank : Select all objects of specified type
 - AREAZONE : Only objects that meet the area/zone/owner filters will be selected
 - SELECTED : Only objects whose Selected field = YES will be selected
 - "FilterName" : Only objects that meet the specified filter will be selected. See [Using Filters in Script Commands](#) section for more information on specifying the filename.
- Action : The way the model will be refined. Choices are:
 - TRANSFORMERTAPS : Fix all transformer taps at their present values if their Vmax – Vmin is less than or equal to the user specified tolerance.
 - SHUNTS : Fix all shunts at their present values if their Vmax – Vmin is less than or equal to the user specified tolerance.
 - OFFAVR : Remove units from AVR control, thus locking their MVAR output at its present value if their Qmax – Qmin is less or equal to the user specified tolerance.
- Tolerance : Tolerance value.

This command refines the power system model by fixing transformer tap settings in all areas where the difference between the transformer's voltage maximum and minimum (Vmax - Vmin) is less than or equal to 0.02 p.u.

```
RefineModel(AREA, , TRANSFORMERTAPS, 0.02);
```

QV Analysis

QVDataWriteOptionsAndResults("filename", AppendFile, KeyField);

Writes out all information related to QV analysis to an auxiliary file. Saves the same information as the QVWriteResultsAndOptions script command. Auxiliary file is formatted using the concise format for DATA section headers and variable names. Data is written using DATA sections instead of SUBDATA sections.

- "filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- AppendFile : Optional parameter - default is YES
YES means to append results to existing "filename." NO means to overwrite "filename" with the results.
- KeyField : Optional parameter – default is PRIMARY
Indicates the identifier that should be used for the data. Valid entries are PRIMARY, SECONDARY, or LABEL. PRIMARY will save using bus numbers and other primary key fields. SECONDARY will save using bus name and nominal kV and other secondary fields. LABEL will save using device labels. If no labels are specified then the primary key field will be used.

This command writes all QV (Reactive Power vs. Voltage) analysis data and configuration options to an auxiliary file named "QV_Results.aux". It overwrites any existing content (AppendFile = NO) and uses primary key fields (like bus numbers) for identifying devices in the file.

```
QVDataWriteOptionsAndResults("QV_Results.aux", NO, PRIMARY);
```

QVDeleteAllResults;

Added in October 31, 2023 patch of Simulator version 23

Deletes all QV results including QVCurve and PWQVResultListContainer object types.

QVRun("filename", InErrorMakeBaseSolvable, DoDistributed);

Call the function to start a QV study for the list of buses whose QVSELECTED field is set to YES.

- "filename" : Optional parameter
This specifies the file to which to save a comma-delimited version of the results. If not specified the QV curve results will not be saved to file during the analysis.
- InErrorMakeBaseSolvable : Optional parameter – default is YES
This specifies whether to perform a solvability analysis of the base case if the pre-contingency base case cannot be solved.
- DoDistributed : Optional parameter – default is NO
Set to YES or NO. If set to YES, distributed methods will be used to solve QV if the Distributed QV add-on is installed. Distributed analysis requires the proper configuration and security settings to work.

This command starts a QV analysis for all buses where the QVSELECTED field is set to YES. The results are saved to "QV_CurveResults.csv". If the base case cannot be solved initially, setting InErrorMakeBaseSolvable to YES will trigger a solvability check and fix the issue if possible.

```
QVRun("QV_CurveResults.csv", YES);
```

QVSelectSingleBusPerSuperBus;

If the QV tool is being used on a full topology model, this action can be used to modify the monitored buses. This action examines the monitored buses and sets the monitored status so that only one bus is monitored for each node.

QVWriteCurves("filename", IncludeQuantitiesToTrack, filter, Append);

This will save a comma-separated text file with the QV curve points.

- "filename" : Name of the comma-separated file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- IncludeQuantitiesToTrack : Set this to YES or NO. Set this to YES to also include any Quantities to Track along with the QV curve points.
- filter : Specify a filter that is applied to the QVCurve object type. Specifying a blank will select all curve results. See [Using Filters in Script Commands](#) section for more information on specifying the filename. AREAZONE filtering is ignored with the QVCurve object type and will return all results.
- Append : Set this to YES or NO. Setting this to YES will cause the data to be appended to an existing file. Setting this to NO will cause any existing file to be overwritten.

This command saves QV curve data to the file "QV_CurvePoints.csv", including any Quantities to Track (IncludeQuantitiesToTrack = YES). It does not use a filter, meaning all QV curves will be included, and it overwrites any existing file with the same name (Append = NO).
--

QVWriteCurves("QV_CurvePoints.csv", YES, , NO);

QVWriteResultsAndOptions("filename", AppendFile);

Writes out all information related to QV analysis to an auxiliary file. This includes Contingency Definitions, Remedial Action Definitions, Solution Options, QV Options, QV results, as well as any Model Criteria that are used by the Contingency and Remedial Action Definitions.

Contingency and Remedial Action definitions will always be saved along with dependencies and only those object types that are dependencies will be saved. This means that if a Remedial Action definition uses a Model Filter, that Model Filter along with any Model Filter Conditions, i.e. other Model Filters or Model Conditions, will be saved. If a model criteria object is not being used by a Remedial Action or Contingency it will not be saved. Objects that are saved if they are dependencies include: Model Conditions, Model Filters, Model Planes, Model Expressions, Model Result Overrides, Interfaces, Injection Groups, Calculated Fields, and Expressions.

- "filename" : Name of the auxiliary file to save. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
- AppendFile : Optional parameter – default is YES
Set this to YES or NO. Setting this to YES will cause the data to be appended to an existing file. Setting this to NO will cause any existing file to be overwritten.

Regions

RegionLoadShapefile("FileName", "Class Name", [AttributeNames], AddToOpenOnelines, "DisplayStyleName", DeleteExistingShapes);

(Added in the August 12, 2025 patch of Simulator 24)

This action loads the shapes from a shapefile with options to add the shapes to any open valid onelines and to delete existing shapes. An example, is

RegionLoadShapeFile("c:\tmp\cb_2018_us_county_500k.shp", "US_Counties", [STATEFP,NAME,GEOID], YES);

This example files is available at www.census.gov/geographies/mapping-files/time-series/geo/cartoboundary-file.html

- "FileName" : Name of the shape file (*.shp) to be loaded.
- "Class Name" : Class name field; though not recommended, it may be blank.
- [AttributeNames] : Set of up to three shapefile attribute names, which will be used to set the proper names; at least one must be set to a valid attribute. This command will fail if any of the specified attributes are not found.
- AddToOpenOnelines : If YES then then new regions are automatically added to any open onelines the have a valid geographic projection; default is NO.
- "DisplayStyleName" : Optional parameter to specify the DisplayStyle to use if regions are being added to open onelines; default is blank and the field is only used when adding regions to open onelines.
- DeleteExistingShapes : Optional parameter to first delete any existing shapes; default is NO.

RegionRename("OldName", "NewName", UpdateOnelines);

(Added in the April 21, 2025 patch of Simulator 24)

This action will change the name of an existing region.

- "OldName" : Name of an existing region.
- "NewName" : New name of the region.
- UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.

RegionRenameClass("OldClassName", "NewClassName", UpdateOnelines, filter);

(Added in the April 21, 2025 patch of Simulator 24)

This action will change the class name of all the regions with the OldClassName names.

- "OldClassName" : Class name for some existing region. May be empty to select those with no existing class name.
- "NewClassName" : New class name for the specified regions. May be empty.
- UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.
- filter : Optional parameter with default apply to all that meet the original name criteria. Filter is used to limit the regions by their initial names. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

RegionRenameProper1("OldProper1Name", "NewProper1Name", UpdateOnelines, filter);

(Added in the April 21, 2025 patch of Simulator 24)

This action will change the proper1 name of all the regions with the OldProper1Name names.

- "OldProper1Name" : Old Proper1 name. May be empty.
- "NewProper1Name" : New Proper1 name. May be empty.
- UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.

filter : Optional parameter with default apply to all that meet the original name criteria. Filter is used to limit the regions by their initial names. See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

RegionRenameProper2("OldProper2Name", "NewProper2Name", UpdateOnelines, filter);

(Added in the April 21, 2025 patch of Simulator 24)

This action will change the proper2 name of all the regions with the OldProper2Name name.

"OldProper2Name" : Old proper2 name. May be empty.
"NewProper2Name" : New proper2 name. May be empty.
UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.
filter : Optional parameter with default apply to all that meet the original name criteria. Filter is used to limit the regions by their initial names. See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

RegionRenameProper3("OldProper3Name", "NewProper3Name", UpdateOnelines, filter);

(Added in the May 24, 2025 patch of Simulator 24)

This action will change the proper3 name of all the regions with the OldProper3Name name.

"OldProper3Name" : Old proper3 name. May be empty.
"NewProper3Name" : New proper3 name. May be empty.
UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.
filter : Optional parameter with default apply to all that meet the original name criteria. Filter is used to limit the regions by their initial names. See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

RegionRenameProper12Flip(UpdateOnelines, filter);

(Added in the April 21, 2025 patch of Simulator 24)

This command flips the proper1 and proper2 names for either all regions if the filter is empty, or for the regions that meet the filter.

UpdateOnelines : Optional parameter – default is YES
Set to YES or NO. If set to YES, any regions on all open onelines with the old name will be changed to the new name.
filter : Optional parameter – default is to flip all regions. Filter is used to limit the regions by their initial names. See the [Using Filters in Script Commands](#) section for more information on specifying the filtername.

RegionUpdateBuses;

(Added in the April 18, 2025 patch of Simulator 24)

Updates the buses in all the regions.

TS (Transient Stability)

TSAutoCorrect;

Runs the auto correction of parameters for a transient stability run. If there are still validation errors after running this script that would prevent the stability simulation from running, then the remainder of a script will be aborted.

TSAutoInsertDistRelay(Reach, AddRelayAtFromBus, AddRelayAtToBus, DoTransferTrip, Shape, filter);

Inserts DistRelay models on the lines meeting the specified filter.

Reach : Zone 1 reach
AddRelayAtFromBus : Add relay at the FROM bus: YES/NO.
AddRelayAtToBus : Add relay at the TO bus: YES/NO.
DoTransferTrip : Do transfer trip: YES/NO
Shape : Shape: 0 - Circle; 1 - Rectangle; 2 - Reactance Distance; 3 - Impedance Distance.
filter : Lines meeting this filter will have DistRelay models inserted. See the [Using Filters in Script Commands](#) section for information on specifying the filter.

This command inserts impedance distance relays with 80% Zone 1 reach at both ends of all transmission lines meeting the specified area/zone/owner filters, enables transfer tripping, and uses an impedance-based protection shape.

TSAutoInsertDistRelay(80, YES, YES, YES, 3, AREAZONE);

TSAutoInsertZPOTT(Reach, filter,);

Inserts ZPOTT models on the lines meeting the specified filter.

Reach : Zone 1 reach
filter : Lines meeting this filter will have ZPOTT models inserted. See the [Using Filters in Script Commands](#) section for information on specifying the filter.

This command inserts ZPOTT protection (Zone 1 permissive overreaching transfer trip) on all lines that meet the area/zone/owner filters, with the Zone 1 reach set to 50% of each line's impedance.

TSAutoInsertZPOTT(50, AREAZONE);

TSAutoSavePlots ([PlotNames], [ContingencyNames], ImageFileType, ImageWidth, ImageHeight, ImageFontScalar, IncludeCaseName, IncludeCategory);

Added in November 21, 2024 patch of Simulator 23

Create and save images of the plots in the same manner as the AutoSave an image in the transient stability dialog works. The script will ONLY plot the plots defined in the [PlotNames] and only will plot the result of one contingency as defined in the [ContingencyNames]. It will NOT PLOT Multiple contingencies in the same plot. Files are saved to the directory specified in the Result Storage -> Save to Hard Drive Options. Filename is by default "ContingencyName_PlotName.jpg".

PlotNames : Specifies for which Plot/Plots will be plotted. If left blank it will plot all of the plots in the case.
ContingencyNames : Specifies for which Transient Contingencies the plots will be plotted. If left in blank it will use all the contingencies in the case.
ImageFileType : (optional) ImageFileType choices are: JPG, EMF, BMP, GIF, PNG or PDF. If an invalid string is entered, JPG is used. Default is JPG.
ImageWidth : (optional) Image Pixel Width. Default is 800.
ImageHeight : (optional) Image Pixel Height. Default is 600.
ImageFontScalar : (optional) Image Font Scalar. Default is 1.
IncludeCaseName : (optional) Include Case Name in the file name: YES/NO. Default is NO.

IncludeCategory : (optional) Include Category in the file name: YES/NO. Default is NO.

Here are two examples of saving the plot named "The Plot" to file for the contingency named "My Transient Contingency". The first uses all default settings and the second specifies the settings.

```
TSAutoSavePlots(["The Plot"], ["My Transient Contingency"]);  
TSAutoSavePlots(["The Plot"], ["My Transient Contingency"], JPEG, 800, 600, 1, No, No);
```

TSCalculateCriticalClearTime([branch] or filter,);

Use this action to calculate critical clearing time for faults on the lines that meet the specified filter.

[line] or filter : A single line can be specified in the format [BRANCH keyfield1 keyfield2 ckt] or [BRANCH label]. Multiple lines can be selected by specifying a filter. See the Using Filters in Script Commands section for information on specifying the filter. For the specified lines, this calculation will determine the first time a violation is reached (critical clearing time), where a violation is determined based on all enabled Transient Limit Monitors. For each line, results are saved as a new Transient Contingency (named CritCTG) on a line, with the fault duration equal to the critical clearing time.

This command finds the line that goes from Bus 1 to Bus 2 with circuit ID 1 and runs a transient fault simulation on that line to compute its critical clearing time (CCT).

```
TSCalculateCriticalClearTime([BRANCH 1 2 1]);
```

TSCalculateSMIBEigenValues;

Calculate single machine infinite bus eigenvalues. Initialization to the start time is always done before calculating eigenvalues.

TSClearAllModels;

Clears all transient stability models. Load Model Groups are considered part of the case and will not be deleted.

TSClearModelsforObjects(ObjectType, Filter);

(Added in the October 17, 2025 patch of Simulator 24)

This action deletes all transient stability models associated with the objects that meet the filter.

ObjectType : Specifies which object types to act on. The ObjectType is the object name of supported objects such as GEN, BUS, BRANCH, etc.
Filter : (optional) Only objects that meet the specified filter will be modified. If the filter is not specified, all objects of the specified type will have their transient stability models deleted. See the [Using Filters in Script Commands](#) section for more information on specifying the filter.

This is an example of clearing the transient model of an individual generator using a device filter.

```
TSClearModelsForObjects(Gen, "<Device>Gen 123 1");
```

TSClearResultsFromRAM(ALL/SELECTED/"ContingencyName", ClearSummary, ClearEvents, ClearStatistics, ClearTimeValues, ClearSolutionDetails);

Clears results from RAM of one or more specified Transient Contingencies.

ALL/SELECTED/"ContingencyName"
: Specifies for which Transient Contingencies results are cleared.
ClearSummary : (optional) YES to clear, NO to leave them alone.
ClearEvents : (optional) YES to clear, NO to leave them alone.
ClearStatistics : (optional) YES to clear, NO to leave them alone.

ClearTimeValues : (optional) YES to clear, NO to leave them alone.
ClearSolutionDetails : (optional) YES to clear, NO to leave them alone.

This command clears specific transient stability simulation results from RAM for the contingency named "Contingency1". It clears the summary, statistics, and time value data, while retaining the event log and solution details.

TSClearResultsFromRAM("Contingency1", YES, NO, YES, YES, NO);

TSDisableMachineModelNonZeroDerivative(DerivativeThreshold);

The transient stability solution is initialized and state derivatives are calculated. The state derivatives for all generator models are checked, and for any where the absolute value is greater than the DerivativeThreshold, the machine model for the generator is disabled. Some models may have state derivatives where it is valid for the derivatives to initialize to a non-zero value. These states will be excluded from the derivative check.

DerivativeThreshold : Optional parameter – default is 0.001
Absolute value of a state's derivative must be greater than this value to disable a model.

TSGetVCurveData("FileName", filter);

For a synchronous generator a curve of points for field current and field voltage is created from a fixed terminal voltage and MW (P) power output and varying Mvar (Q) output.

FileName : Name of file to create. If no file extension is specified this defaults to CSV.
filter : Specifies for which generators curves are created.
SELECTED : only generators whose Selected field = YES will be included
AREAZONE : only generators that meet the area/zone/owner filters will be included
"FilterName" : only generators that meet the specified filter will be included. See the [Using Filters in Script Commands](#) section for more information on specifying the filename.

This command generates V-curve data (field current and voltage vs. reactive power output) for all generators with their Selected field set to YES and saves the results to a CSV file named "GenVCurves.csv".

TSGetVCurveData("GenVCurves.csv", SELECTED);

TSGetResults("FileName", SINGLE/SEPARATE/JSIS, [Contingencies], [Plots, ObjectFields], StartTime, EndTime);

Use this to save out results for specific variables from plots, subplots, and object/field pairs after a transient stability simulation has been run. If StartTime and StopTime are not specified, results for the entire simulation time are obtained.

FileName : Name of the CSV result file to write out
SINGLE/SEPARATE/JSIS : Determines whether the results are all saved in one file (SINGLE) with name "filename" or whether results for each transient contingency is saved in a separate file (SEPARATE) with name "filename_ctgname.csv." A separate header file is also saved out, with a name of "filename_Header.csv". If using the JSIS format, a single file with the name "filename" is written in the WECC JSIS format.
Contingencies : A list of contingency names for which results are saved
Plots, ObjectFields : A list of plots and object/field pairs to save out for the specified contingencies. For plots, write PLOT and then the name of the plot. For

ObjectsFields you need the Object name followed by the key fields and a vertical line (|), then the field.

[Added in November 17, 2023 patch of Simulator 23]

May also specify the syntax

"ObjectName FilterName | FieldName".

For this syntax anything after the first space and before the | character is considered a FilterName. The FilterName can be All indicating that all objects of the type ObjectName are used, or it can use the syntax available for defining a FilterName as described in the [Using Filters in Script Commands](#) section.

StartTime : Start of the window of simulation time for retrieved results
EndTime : End of the window of simulation time for retrieved results

This is an example of saving transient stability results to file using a combination of plot definitions, specific fields for specific objects, and a specific field for all objects of a particular type.

```
TSGetResults("D:\Cases\Scripts\Test.CSV",SINGLE,["My Transient  
Contingency"], ["PLOT Gen_Rotor Angle","Bus 4 | frequency","PLOT  
Bus_Frequency","GEN 1 '1'| TSGenMachineState:2", "Bus All |  
TSFrequency"],0,10);
```

TSInitialize(CheckInitialized);

Initialize the transient stability solution. This is useful to calculate the initial state and state derivative values. It is not necessary to call this step manually as it will be done automatically with any other transient analysis, but it is useful if analysis of the initial states is desired without doing any additional calculations. Any validation errors must be corrected before initialization can be done.

CheckInitialized : Optional parameter – default is NO
Set to YES to check if the transient solution has already been initialized and not to initialize again if it has. Set to NO to initialize whether or not the transient solution has already been initialized.

TSJoinActiveCTGs(TimeDelay,DeleteExisting,JoinWithSelf,"fileName",FirstCtg);

Added in March 21, 2024 patch of Simulator 23

This command joins two lists of TSContingency objects with a specified time delay in seconds.

TimeDelay : Time delay in seconds
DeleteExisting : Set to YES or NO. YES means to delete the existing contingencies and only keep the joined contingencies.
JoinWithSelf : Set to YES or NO. YES means that the current contingency list will be joined with itself instead of contingencies specified in a file. If set to YES, the "filename" parameter does not have to be specified.
"filename" : Name of auxiliary file containing contingencies to join with the current contingency list. This does not have to be specified if JoinWithSelf = YES. It must be specified if JoinWithSelf=NO.
FirstCtg : Optional parameter which can be **Active**, **AUX**, or **Both** and defaults to **Both** if not specified.
If FirstCTG=**Active**, then each existing active TSContingency object is joined with the active Contingencies in the AUX file specified by Filename. All TSContingencyElement objects in the AUX file are shifted by the TimeDelay value.
If FirstCTG=**AUX**, TSContingency objects in the AUX file have the same timing and they are joined with the existing Active TSContingency objects with the existing elements time shifted by the time delay.

If FirstCTG=**Both**, then you get both options of list (A + BTimeDelay) and (B + ATimeDelay).

Here are two examples of joining two lists of transient stability contingencies. The first example joins the contingencies in memory with the contingencies in the specified file. The second example joins the contingencies in memory with each other.

```
TSJoinActiveCTGs(10.0, NO, NO, "c:\temp\Myfile.aux", Both);
TSJoinActiveCTGs(10.0, NO, YES);
```

TSLoadBPA("FileName");

Loads transient stability data stored in the BPA format.

FileName : Name of the BPA file to load

TSLoadGE("FileName", GENCCYN, EnableOutOfOrderModels);

Loads transient stability data stored in the GE DYD format.

FileName : Name of the DYD file to load

GENCCYN : YES to split combined cycle units, NO to leave them alone

EnableOutOfOrderModels

: (optional) Default is YES. If set to YES, models that are specified out of order in the file will be enabled. If set to NO, out of order models will be disabled.

This command loads transient stability data from "case.dyd". It splits combined cycle units (GENCCYN = YES) and enables models even if they are specified out of order (EnableOutOfOrderModels = YES).

```
TSLoadGE("case.dyd", YES, YES);
```

TSLoadPTI("FileName", "MCREfilename", "MTRLOfilename", "GNETfilename", "BASEGENfilename", "MODREMOVEfilename");

Loads transient stability data in the PTI format.

FileName : Name of the DYR file to load

MCREfilename : (optional) If not loading a MCRE file, specify ""

MTRLOfilename : (optional) If not loading a MTRLO file, specify ""

GNETfilename : (optional) If not loading a GNET file, specify ""

BASEGENfilename : (optional) If not loading a BASEGEN file, specify ""

MODREMOVEfilename : (optional) if not loading a MODREMOVE file, specify ""

This command loads transient stability dynamic model data from the PTI DYR file "case.dyr". All optional file parameters like MCRE, MTRLO, GNET, BASEGEN, and MODREMOVE are left empty, indicating that only the DYR file is used for loading data.

```
TSLoadPTI("C:\TSData\case.dyr", "", "", "", "", "");
```

TSLoadRDB("filename", ModelType, filter);

Loads a SEL RDB file. The RDB file is a Schweitzer format for describing a relay. This command will load the file and translate the relay settings into a PowerWorld transient stability model. An attempt is made to match up the protected lines in the power flow model with the SID field in the RDB data. A SID in the format "FROM SUB/BREAKERS/TO SUB" is expected. If the SID does not match this format, or a match is not found in the case, the objects can be linked to the transient stability model in the user interface manually.

"filename" : The directory path or filename of the RDB file(s). If pointed to a directory, the script command will load every RDB file in that directory. If pointed to a file, the script command will load the single RDB file. See

the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.

ModelType : One of the following:
DISTRELAY : create a Simulator DistRelay model from the RDB data
ZPOTT : create a Simulator ZPOTT model from the RDB data

filter : Optional parameter
Lines meeting this filter are searched to find ones matching the SID. See the [Using Filters in Script Commands](#) section for information on specifying the filter.

This command loads relay settings from the specified "substation_relay.rdb" file and creates PowerWorld DISTRELAY models for branches with their Selected field set to YES.

```
TSLoadRDB("substation_relay.rdb", DISTRELAY, SELECTED);
```

TSLoadRelayCSV("filename", ModelType, filter);

This is a quicker alternative to using the TSLoadRDB command for loading RDB files. Relevant relay data can be exported to a CSV file so that a single CSV file contains multiple relay models. This is much faster because it does not contain the unused data that an RDB file contains.

"filename" : Name of the CSV file to load

ModelType : One of the following:
DISTRELAY : create a Simulator DistRelay model from the RDB data
ZPOTT : create a Simulator ZPOTT model from the RDB data

filter : Optional parameter
Lines meeting this filter are searched to find ones matching the SID. See the [Using Filters in Script Commands](#) section for information on specifying the filter.

This command loads relay model data from the "relays.csv" file, specifically creating Simulator DISTRELAY models. Only relays for lines meeting the area/zone/owner filters are loaded.

```
TSLoadRelayCSV("relays.csv", DISTRELAY, AREAZONE);
```

TSPlotSeriesAdd("PlotName", SubPlotNum, AxisGroupNum, ObjectType, FieldType, "Filter", "Attributes");

Added in February 8, 2024 patch of Simulator 23

This command adds one or multiple plot series to a new or existing plot definition

"PlotName" : A string to specify a certain plot

SubPlotNum : A positive integer to specify a certain subplot of a plot

AxisGroupNum : A positive integer to specify to a certain subplot of a subplot

ObjectType : Specifies which objects to set. The ObjectType is the object name of supported objects such as GEN, BUS, BRANCH, etc

FieldName : Specified a transient stability field name that can be plotted

"Filter" : (optional) Only objects that meet the specified filter will be included. See the [Using Filters in Script Commands](#) section for more information on specifying the filename

"Attributes" : (optional) A comma-delimited list of other attributes for the PlotSeries.
Blank by default just uses the defaults for a new PlotSeries

Here are two examples of creating new plot series.

```
TSPlotSeriesAdd("Gen 2 1 | Rotor Angle", 1, 1, Gen, TSRotorAngle, "<Device>Gen 2 1");
TSPlotSeriesAdd("Area 2 buses", 1, 1, Bus, TSVpu, "<Device>Area 2", "ValueType=Actual Deviation, LineDashed=Dot");
```

TSSResultStorageSetAll(objecttype, YES/NO);

This command will allow setting which object types are stored in memory during a transient stability run. This will affect all fields and states for the specified objecttype.

- objecttype : Specifies which objects to set. The objecttype is the object name of supported objects such as GEN, BUS, BRANCH, etc. ALL can be used to set all supported object types.
- YES/NO : Using this command will toggle all the "Save All" fields to YES/NO. It will also toggle all the "state" fields (such as exciter, machine, governor, etc.) to YES/NO.

This command configures the transient stability simulation to store all fields and state variables for all generator objects (GEN) during the run.
--

TSSResultStorageSetAll(GEN, YES);

TSSRunResultAnalyzer("ContingencyName");

Run the Transient Result Analyzer for the specified contingency or all contingencies.

- ContingencyName : Optional parameter – default is blank
Name of contingency for which the analysis is run. If the name is not specified or blank, the analysis is run for all contingencies.

TSSRunUntilSpecifiedTime("ContingencyName", [StopTime, StepSize, StepsInCycles, ResetStartTime, NumberOfTimeStepsToDo]);

This command allows manual control of the transient stability run. The simulation can be run until a specified time or number of time steps and then paused for further evaluation.

- ContingencyName : The name of the contingency to solve.
- StopTime : (optional) This is the time to which the simulation will be run. This should be entered in seconds. If NumberOfTimeStepsToDo > 0, this field will be ignored. If not specified, the stop time specified with the contingency will be used.
- StepSize : (optional) Simulation step size in either seconds or cycles. If StepsInCycles = YES this should be specified in cycles. If not specified, the step size specified with the contingency will be used.
- StepsInCycles : (optional) Set to YES to specify StepSize in cycles. If not specified, the units of the step size specified with the contingency will be used.
- ResetStartTime : (optional) Set to YES to reset the simulation start time. Default value is NO.
- NumberOfTimeStepsToDo : (optional) Number of time steps to run. If NumberOfTimeStepsToDo > 0, StopTime is ignored. Default value is 0.

This command runs the transient stability simulation for the contingency named "FaultBus1" until time 5.0 seconds, using a step size of 0.005 seconds. The step size is interpreted in seconds (StepsInCycles = NO), and the simulation start time is reset (ResetStartTime = YES). Since NumberOfTimeStepsToDo is set to 0, the simulation runs until the specified StopTime.
--

TSSRunUntilSpecifiedTime("FaultBus1", [5.0, 0.005, NO, YES, 0]);
--

TSSaveBPA("FileName", DiffCaseModifiedOnly);

Save transient stability data stored in the BPA IPF format.

- FileName : Name and path for the output file. Typically this will be an *.swi file extension.
- DiffCaseModifiedOnly : (optional) Default is NO. When set to YES, it will only save models that are either new or models which have had a parameter modified as compared to the difference case tool base case.

This command saves transient stability data in the BPA IPF format to "export_data.swi". With DiffCaseModifiedOnly set to YES, it includes only those models that are newly added or have had parameters modified from the base case defined in the difference case tool.

TSSaveBPA("export_data.swi", YES);

TSSaveGE("FileName", DiffCaseModifiedOnly);

Save transient stability data stored in the GE DYD format.

FileName : Name and path for the output file. Typically this will be an *.dyd file extension.

DiffCaseModifiedOnly : (optional) Default is NO. When set to YES, it will only save models that are either new or models which have had a parameter modified as compared to the difference case tool base case.

This command exports transient stability models in GE DYD format to the file "output_models.dyd". By setting DiffCaseModifiedOnly to YES, only those models that are new or have been changed relative to the difference case base will be included in the output.

TSSaveGE("output_models.dyd", YES);

TSSavePTI("FileName", DiffCaseModifiedOnly);

Save transient stability data stored in the PTI DYP format.

FileName : Name and path for the output file. Typically this will be an *.dyp file extension.

DiffCaseModifiedOnly : (optional) Default is NO. When set to YES, it will only save models that are either new or models which have had a parameter modified as compared to the difference case tool base case.

This command saves transient stability data in PTI DYP format to the file "output_models.dyp". By setting DiffCaseModifiedOnly to YES, it restricts the saved content to only include models that are newly added or modified compared to the difference case base model.

TSSavePTI("output_models.dyp", YES);

TSSaveTwoBusEquivalent ("FileName", [BUS]);

Save the two bus equivalent model of a specified bus to a PWB file. Initialization to the start time is always done before saving the two bus equivalent.

FileName : Name and path for the output PWB file

BUS : Bus can be specified in three ways:
 Number : [BUS busnum]
 Name/NomkV : [BUS "busname_nominalKV"]
 Label : [BUS "buslabel"]

This command saves a two-bus equivalent model of Bus 1 to a PWB file named "TwoBusEq_Bus1.pwb".

TSSaveTwoBusEquivalent("TwoBusEq_Bus1.pwb", [BUS 1]);

TSSolve("ContingencyName", [StartTime, StopTime, StepSize, StepsInCycles]);

Solves only the specified contingency.

ContingencyName : The name of the contingency to solve

Times : (optional) Parameter is a comma delimited list of strings enclosed in square brackets. This string then consists of 4 optional parameters which override the corresponding property of the contingency.

 StartTime : (optional) Start time in seconds

StopTime : (optional) Stop time in seconds
 StepSize : (optional) Step size (in seconds unless StepInCycles = YES)
 StepInCycles : (optional) Set to YES to mean that the StepSize is given in cycles

This command solves the transient stability simulation for the contingency named "GEN_TRIP_BUS1". It runs the simulation from 0 to 10 seconds using a time step of 0.01 seconds, interpreting the step size in seconds because StepInCycles is set to NO.

```
TSSolve("GEN_TRIP_BUS1", [0, 10, 0.01, NO]);
```

TSSolveAll(DoDistributed);

Solves all defined transient contingencies that are not set to skip.

DoDistributed : (optional) Set to YES to use Distributed Computing with the transient analysis. Default is NO.

TSSolveContinue;

(This command was added on December 22, 2025 to Simulator 25)

Continues the solution of the currently active contingency that has either been saved as a Snapshot or set using a Restore Time Point. Returns false if the contingency is not actually paused.

TSTransferStateToPowerFlow(CalculateMismatch);

Transfer the transient stability state to the power flow.

CalculateMismatch : (optional) Default is NO
 Set to YES to calculate power mismatch when transferring transient state to the power flow case. Any input other than YES will set the option to NO, which means no calculation of power mismatch is done.

TSValidate;

Validate transient stability models and input values. This command is useful for examining model errors and warnings when preparing a case for analysis. Validation will be done automatically when running transient analysis, so this command does not need to be run manually prior to analysis.

TSSaveModels("FileName", DiffCaseModifiedOnly);

Save transient stability dynamic model records only the auxiliary file format.

FileName : Name and path for the output file. Typically this will be an *.aux file extension.

DiffCaseModifiedOnly : (optional) Default is NO. When set to YES, it will only save models that are either new or models which have had a parameter modified as compared to the difference case tool base case.

This command saves only the modified transient stability dynamic model records to the "Modified_Models.aux" file. It includes only those models that are new or have parameter changes compared to the difference case base model.

```
TSSaveModels("Modified_Models.aux", YES);
```

TSSaveOptions("FileName",[SaveDynamicModel, SaveStabilityOptions, SaveStabilityEvents, SaveResultsEvents, SavePlotDefinitions, SaveTransientLimitMonitors, SaveResultAnalyzerTimeWindow], KeyField);

Save the transient stability option settings to an auxiliary file.

FileName : Name and path of the file to save
 SaveDynamicModel : (optional) NO doesn't save dynamic model (default YES)
 SaveStabilityOptions : (optional) NO doesn't save stability options (default YES)

SaveStabilityEvents : (optional) NO doesn't save stability events (default YES)
 SaveResultsSettings : (optional) NO doesn't save results settings (default YES)
 SavePlotDefinitions : (optional) NO doesn't save plot definitions (default YES)
 SaveTransientLimitMonitors: (optional) NO doesn't save transient limit monitors (default YES)
 SaveResultAnalyzerTimeWindows: (optional) NO doesn't save result analyzer time windows (default YES)
 KeyField : (optional) Specifies key: can be Primary, Secondary, or Label (default Primary)

This command saves transient stability settings to the file "TS_Options.aux". It includes dynamic models, stability options, stability events, plot definitions, and result analyzer time windows. It excludes result settings and transient limit monitors. The data is saved using primary key fields.

```
TSSaveOptions("TS_Options.aux", [YES, YES, YES, NO, YES, NO, YES], PRIMARY);
```

TSSetSelectedForTransientReferences(SetWhat, SetHow, [ObjectType List],[ModelType List]);

Set the Custom Integer field to a corresponding integer or Selected field to Yes/No for objects referenced in a transient stability model with extra objects.

SetWhat : Either Selected or Custom Integer number (1, 2, 3...). Default is number 1 for the Custom Integer
 SetHow : when SetWhat = Selected, will be either YES or NO. Otherwise it will be an integer to be used with Custom Integer. Default is 1.
 [ObjectType List] : A comma-delimited list of objecttypes. These are the objects on which the custom field will be set.
 [ModelType List] : A comma-delimited list of transient stability model types on which references will be queried.

This command sets the Selected field to YES for all generator objects that are referenced in the transient stability model, REPC_A.

```
TSSetSelectedForTransientReferences(SELECTED, YES, [GEN], [REPC_A]);
```

TSSaveDynamicModels("FileName", FileType, ObjectType, Filter, Append);

Save dynamics models for specified object types to file.

FileName : Name and path of the file to save
 FileType : AUX or DYD.
 ObjectType : The object type to save - Gen, Load, SwitchedShunt, DCLine, VSCDCLine, Branch, among others available in Transient Stability..
 Filter : See [Using Filters in Script Commands](#) section for more information on specifying the filter.
 Append : YES or NO to whether replace an existing file or append to it.

This command saves dynamic models for all generators with their Selected field set to YES in the file "gen_models.aux" in AUX format. Since Append is set to NO, any existing file with the same name will be overwritten rather than appended to.

```
TSSaveDynamicModels("gen_models.aux", AUX, Gen, SELECTED, NO);
```

Scheduled Actions

ApplyScheduledActionsAt(StartTime, EndTime, Filter, Revert);

Applies any scheduled actions that meet the specified filter and are active during the specified window of time.

StartTime	: The beginning of the window in which to apply actions. Must be specified in the current locale's date/time format.
EndTime	: Optional parameter – default is the same as StartTime The end of the window in which to apply actions. Must be specified in the current locale's date/time format.
Filter	: Optional parameter – default is to apply all actions See Using Filters in Script Commands section for more information on specifying the filter.
Revert	: Optional parameter – default is NO Set to YES if actions should be reverted rather than applied. Use this command with Revert = YES is the same as calling the RevertScheduledActionsAt command.

This command applies all scheduled actions that are active between 8:00 AM and 12:00 PM on June 1, 2025. No filter is used, so all eligible scheduled actions during this time frame are applied. The Revert parameter is set to NO, meaning the actions will be implemented, not undone.

```
ApplyScheduledActionsAt("01/06/2025 08:00", "01/06/2025 12:00", , NO);
```

IdentifyBreakersForScheduledActions(IdentifyFromNormalStatus);

For each Scheduled Outage, identifies breakers that are necessary to implement OpenBreakers and CloseBreakers actions. New Scheduled Actions are added to the Scheduled Outage if new breakers are identified that do not already exist as actions.

IdentifyFromNormalStatus	: Set to YES to return all branches to their normal status before searching for breakers. If using this option, all branches will be returned to their current status at the end of the process. Set to NO to leave all branches at their current status before searching for breakers.
--------------------------	---

This command identifies which breakers are required to implement each Scheduled Action defined using OpenBreakers or CloseBreakers and adds new breaker actions as necessary. By setting IdentifyFromNormalStatus to YES, all branches are first reset to their normal status prior to breaker identification and restored to their current status afterward.

```
IdentifyBreakersForScheduledActions(YES);
```

RevertScheduledActionsAt(StartTime, EndTime, Filter);

Applies the opposite of any relevant actions (e.g. Closes breakers if an Action is to Open them).

StartTime	: The beginning of the window in which to revert actions. Must be specified in the current locale's date/time format.
EndTime	: Optional parameter – default is the same as StartTime The end of the window in which to revert actions. Must be specified in the current locale's date/time format.
Filter	: Optional parameter – default is to revert all actions See Using Filters in Script Commands section for more information on specifying the filter.

This command reverts any scheduled actions that were active between 8:00 AM and 12:00 PM on June 1, 2025. Since no filter is specified, it applies to all scheduled actions within the time window.

```
RevertScheduledActionsAt("01/06/2025 08:00", "01/06/2025 12:00", );
```

ScheduledActionsSetReference;

Set the reference state that is restored prior to applying a time stamp.

SetScheduleView(ViewTime, ApplyActions, UseNormalStatus, ApplyWindow);

Sets the View Time for Scheduled Actions.

- ViewTime : The desired view time. Must be between the currently configured Start and End times, and fall on a valid view point given current resolution settings.
- ApplyActions : (optional) Set to YES or NO to override current "Apply Actions" settings to either apply actions at this view time or suppress the application of actions at this time. If left blank, current "Apply Actions" settings will be used.
- UseNormalStatus : (optional) Set to YES or NO to override current "Use Normal Status" settings to either restore devices to their Normal Status after they are restored, or to use whatever status they had before the action was applied. If left blank, current "Use Normal Status" settings will be used.
- ApplyWindow : (optional) Set to YES or NO to override current "Apply Window" settings to either apply any actions active in the window starting at this View Time and extending forward one Resolution step forward in time, or only apply actions active at the specific View Time specified. If left blank, current "Apply Window" settings will be used.

This command sets the schedule view time to 10:00 AM on June 1, 2025, and applies scheduled actions at that time (ApplyActions = YES). It uses the current device status instead of restoring them to their normal status (UseNormalStatus = NO) and applies actions that are active within the window starting at this time and extending one resolution step forward (ApplyWindow = YES).

```
SetScheduleView("01/06/2025 10:00", YES, NO, YES);
```

SetScheduleWindow(StartTime, EndTime, Resolution, ResolutionUnits);

Defines the window of interest for Scheduled Actions.

- StartTime : Defines the start of the window.
- EndTime : Defines the end of the window. Must fall on a valid time relative to the Start Time as defined by the Resolution (e.g. if the Start Time is 5/20/17 8:00 and the resolution is 1 DAY, the End Time must be at 8:00 as well on a later date.)
- Resolution : (optional) Decimal value defining the time step between the Start Time and any valid View Time.
- ResolutionUnits : (optional but must be specified if Resolution is specified) May be MINUTES, HOURS, or DAYS

This command sets the scheduled actions window from midnight on June 1, 2025, to midnight on June 7, 2025, with a resolution of 1 day. This means valid view times for scheduled action evaluations will occur every day at midnight during this week.

```
SetScheduleWindow("01/06/2025 00:00", "07/06/2025 00:00", 1, DAYS);
```

Time Step Simulation

TimeStepAppendPWW("FileName", "SolutionTypeString")

(This command was added in the December 19, 2023 patch of Simulator 23)

Loads the specified PWW file into the Time Step Simulation, appending it to any timepoints that already exist.

- "FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".
- "SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "Single Solution" is used.

This command appends the "SummerScenario.pww" file to the current Time Step Simulation, adding its time points to any that already exist. It sets the solution type for these time points to SCOPF (Security Constrained Optimal Power Flow).

<code>TimeStepAppendPWW("SummerScenario.pww", "SCOPF");</code>
--

TimeStepAppendPWWRange("FileName", ISO8601StartDateTime, ISO8601EndDateTime, "SolutionTypeString")

This command is similar to TimeStepAppendPWW in that it loads timepoints from the file and appends them to any existing timepoints. However it only loads timepoints for the date and time values between the start and end datetime values (expressed in ISO8601 format).

- "FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".
- ISO8601StartDateTime: The desired start date and time specified in ISO8601 format. If string is empty ("") or before the start datetime then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.
- ISO8601EndDateTime: The desired end date and time specified in ISO8601 format. If string is empty ("") or after the end datetime then go until the end. The datetime should either be a UTC value, or local time specified with the time zone offset.
- "SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "Single Solution" is used.

This command appends only the time points from "SummerScenario.pww" that fall between June 1, 2025, 00:00 and June 3, 2025, 23:59, using OPF (Optimal Power Flow) as the solution type. The date and time values are provided in ISO8601 format with a time zone offset of -05:00 (e.g., Central Daylight Time).
--

<code>TimeStepAppendPWWRange("SummerScenario.pww", 2025-06-01T00:00:00-05:00, 2025-06-03T23:59:59-05:00, "OPF");</code>

TimeStepAppendPWWRangeLatLon("FileName", ISO8601StartDateTime, ISO8601EndDateTime, minLatitude,maxLatitude,minLongitude,maxLongitude,"SolutionTypeString")

(This command was added in the November 3, 2025 patch of Simulator 24)

This command is similar to TimeStepAppendPWWRange in that it loads a range of timepoints from the file and appends them to any existing timepoints. However, it can also optionally only append weather data that is within a specified geographic rectangle.

- "FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".
- ISO8601StartDateTime: The desired start date and time specified in ISO8601 format. If string is empty ("") or before the start datetime then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.
- ISO8601EndDateTime: The desired end date and time specified in ISO8601 format. If string is empty ("") or after the end datetime then go until the end. The datetime should either be a UTC value, or local time specified with the time zone offset.
- minLatitude : Optional minimum latitude for the bounding rectangle. Must be greater than or equal to -90, and less than the maxLatitude; default is -90.
- maxLatitude : Optional maximum latitude for the bounding rectangle. Must be less than or equal to 90 and greater than the minLatitude; default is 90.
- minLongitude : Optional minimum latitude for the bounding rectangle. Must be great than or equal to -180 and less than the maxLatitude. Hence rectangles spanning the international date line are not allowed. Default is -180.
- maxLongitude : Optional maximum latitude for the bounding rectangle. Must be less than or equal to 180 and greater than the minLatitude. Hence rectangles spanning the international date line are not allowed. Default is 180.
- "SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "Single Solution" is used.

TimeStepClearResults(ISO8601StartDateTime, ISO8601EndDateTime);

(This command was added in the November 1, 2024 patch of Simulator 23)

Clears all the Time Step Simulation results, but does not delete the time points.

- ISO8601StartDateTime: The optional desired start date and time specified in ISO8601 format for the clear. If not specified, then start the clear at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.
- ISO8601EndDateTime: The optional desired end date and time specified in ISO8601 format for the clear. If not specified, then finish the clear at the end. This datetime should either be a UTC value, or local time specified with the time zone offset.

This command clears all Time Step Simulation results between January 1, 2025, 00:00 and January 3, 2025, 23:59 (Central Daylight Time), but retains the time points themselves.

TimeStepClearResults(2024-01-01T00:00:00-05:00, 2024-01-03T23:59:59-05:00);

TimeStepDeleteAll;

(This command was added in the December 19, 2023 patch of Simulator 23)

Deletes all of the time points in the Time Step Simulation.

TimeStepDoRun(ISO8601StartDateTime,ISO8601EndDateTime)

Solves the Time Step Simulation either for all time points (when there are no parameters) or for the timepoints between ISO8601StartDateTime and ISO8601EndDateTime.

ISO8601StartDateTime: The optional desired start date and time specified in ISO8601 format. If not specified then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.

ISO8601EndDateTime: The optional desired end date and time specified in ISO8601 format. If not specified then finish at the end. This datetime should either be a UTC value, or local time specified with the time zone offset.

This command runs the Time Step Simulation for all time points scheduled between June 1, 2025, 00:00 and June 3, 2025, 23:59 (Central Daylight Time).

<code>TimeStepDoRun(2025-06-01T00:00:00-05:00, 2025-06-03T23:59:59-05:00);</code>

TimeStepDoSinglePoint(ISO8601DateTime)

Solves the Time Step Simulation (TSS) for the specified date and time that should be specified in the ISO8601 format. The command returns with an error if the datetime is not within TSS start/end datetime values or there are no datetime values.

ISO8601DateTime : The desired date and time specified in ISO8601 format. This datetime should either be a UTC value, or local time specified with the time zone offset.

This command runs the Time Step Simulation for a single time point: June 2, 2025, at 12:00 PM Central Daylight Time.
--

<code>TimeStepDoSinglePoint(2025-06-02T12:00:00-05:00);</code>
--

TimeStepLoadB3D("FileName","SolutionTypeString")

(This command was added in the September 2, 2024 patch of Simulator 23)

Loads the specified B3D file into the Time Step Simulation, deleting any timepoints that already exist.

"FileName" : Name of the B3D file; it should either have a ".b3d" extension or no extension; if no extension then it is set to ".b3d".

"SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "GIC Only (No Power Flow)" is used.

This command loads the B3D file named "GeomagneticStormData.b3d" into the Time Step Simulation, replacing any existing timepoints. All loaded timepoints are configured to use the "GIC Only (No Power Flow)" solution type, which focuses on geomagnetically induced currents without running full power flow calculations.
--

<code>TimeStepLoadB3D("GeomagneticStormData.b3d", "GIC Only (No Power Flow)");</code>

TimeStepLoadPWW("FileName","SolutionTypeString")

(This command was added in the December 19, 2023 patch of Simulator 23)

Loads the specified PWW file into the Time Step Simulation, deleting any timepoints that already exist.

"FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

"SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)",

"Weather Only"; if not specified the default value of "Single Solution" is used.

This command loads the PWW file named "HourlySimulationData.pww" into the Time Step Simulation, replacing any existing timepoints. All the loaded timepoints are configured to use the "OPF" (Optimal Power Flow) solution type.

```
TimeStepLoadPWW("HourlySimulationData.pww", "OPF");
```

TimeStepLoadPWWRange("FileName",ISO8601StartDateTime,ISO8601EndDateTime,"SolutionTypeString")

This command is similar to TimeStepLoadPWW in that it loads a PWW file, but only for the date and time values between the start and end datetime values (expressed in ISO8601 format) are loaded. Also, like TimeStepLoadPWW it deletes any existing time points. An example is

TimeStepLoadPWWRange("c:\tmp\test.pww", 2024-03-06T00Z,2024-03-06T23Z), which loads data for March 6, 2024 (UTC).

"FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

ISO8601StartDateTime: The desired start date and time specified in ISO8601 format. If string is empty ("") or before the start datetime then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.

ISO8601EndDateTime: The desired end date and time specified in ISO8601 format. If string is empty ("") or after the end datetime then go until the end. The datetime should either be a UTC value, or local time specified with the time zone offset.

"SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "Single Solution" is used.

This command loads only the timepoints for May 1, 2025, from the "dailyLoad.pww" file into the Time Step Simulation. Any existing timepoints are deleted first. The solution type for all these timepoints is set to "SCOPF" (Security Constrained Optimal Power Flow).

```
TimeStepLoadPWWRange("dailyLoad.pww", 2025-05-01T00:00:00Z, 2025-05-01T23:59:59Z, "SCOPF");
```

TimeStepLoadPWWRangeLatLon("FileName",ISO8601StartDateTime,ISO8601EndDateTime,minLatitude,maxLatitude,minLongitude,maxLongitude, "SolutionTypeString")

(This command was added in the November 3, 2025 patch of Simulator 24)

This command is similar to TimeStepLoadPWWRange in that it loads a PWW file for the date and time values between the start and end datetime values (expressed in ISO8601 format). However, it can also optionally only load weather data that is within a specified geographic rectangle. Also, like

TimeStepLoadPWW it deletes any existing time points. An example is

TimeStepLoadPWWRange("c:\tmp\test.pww", 2024-03-06T00Z,2024-03-06T23Z,30,31,-88,-87), which loads data for March 6, 2024 (UTC) within the geographic region between latitude 30 and 31 and longitude -88 and -87.

"FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

ISO8601StartDateTime: The desired start date and time specified in ISO8601 format. If string is empty ("") or before the start datetime then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.

- ISO8601EndDateTime: The desired end date and time specified in ISO8601 format. If string is empty ("") or after the end datetime then go until the end. The datetime should either be a UTC value, or local time specified with the time zone offset.
- minLatitude : Optional minimum latitude for the bounding rectangle. Must be greater than or equal to -90, and less than the maxLatitude; default is -90.
- maxLatitude : Optional maximum latitude for the bounding rectangle. Must be less than or equal to 90 and greater than the minLatitude; default is 90.
- minLongitude : Optional minimum latitude for the bounding rectangle. Must be great than or equal to -180 and less than the maxLatitude. Hence rectangles spanning the international date line are not allowed. Default is -180.
- maxLongitude : Optional maximum latitude for the bounding rectangle. Must be less than or equal to 180 and greater than the minLatitude. Hence rectangles spanning the international date line are not allowed. Default is 180.
- "SolutionTypeString" : Optional solution type string; when the file is loaded, all of the time points are set to use this solution type. Valid entries are "Single Solution", "Unconstrained OPF", "OPF", "SCOPF", "GIC Only (No Power Flow)", "Weather Only"; if not specified the default value of "Single Solution" is used.

TimeStepLoadTSB("FileName")

Loads the specified TSB file into the Time Step simulation, deleting any timepoints that already exist.

"FileName" : Name of the TSB file; it should have a ".tsb" extension.

TimeStepResetRun

Resets the run to the beginning.

TimeStepSaveFieldsClear([objecttype])

(This command was added in the September 2, 2024 patch of Simulator 23)

Clears all the fields associated with the saving of the result values during the time step simulation.

- [objecttype] : Optional parameter that is a comma-delimited list of the objecttypes whose save fields should be cleared. If omitted all objecttypes are cleared.

This command clears all configured save fields specifically for the BUS and GEN object types used in Time Step Simulation.
--

TimeStepSaveFieldsClear ([BUS, GEN]);

TimeStepSaveFieldsSet(objecttype, [fieldlist], filter)

(This command was added in the September 2, 2024 patch of Simulator 23)

Sets the fields associated with determining which result values are saved during the time step simulation, first clearing any previously selected fields and objects for this objecttype. As an example, TimeStepSaveFieldsSet(Gen,[MW]) sets saving all the generator MW fields, while TimeStepSaveFieldsSet(Transformer,[GICNeutralCurrent],AREAZONE) sets saving the GIC transformer neutral current for the transformers with their AreaZone filter set.

- objecttype : The objecttype being set.
- [fieldlist] : A comma-delimited list of the A list of fields to update.
- filter : Optional parameter – if omitted the default is ALL
- ALL : Set data for all objects
- SELECTED : Only objects whose Selected field = YES will be set. In the time step simulation the selected field can be set before calling this command by loading a TSB file.

"FilterName" : Only objects that meet the specified filter will be set. See [Using Filters in Script Commands](#) section for more information on the specifying the filename and other filter options.

This command configures the Time Step Simulation to save the MW and Mvar output for selected generators only. It first clears any previously saved fields for the GEN object type, then applies the new configuration.

```
TimeStepSaveFieldsSet(GEN, [MW, Mvar], SELECTED);
```

TimeStepSaveFieldsSetByObject(objecttype, [fieldlist], [objectIDList])

(This command was added in the October 21, 2024 patch of Simulator 23)

Sets the fields associated with determining which result values are saved during the time step simulation by individual objects. This command is similar to TimeStepSaveFieldsSet, but rather than using a filter to set the objects to save, a list of the objects is provided. Also, in contrast to TimeStepSaveFieldsSet previously selected objects and fields are not cleared. Note, in the time step simulation all the saved fields for a particular objecttype must be the same. Hence executing this command multiple times on the same object type results in the union of the fieldlists being stored. An example is TimeStepSaveFieldsSetByObject(Transformer,[GICLeff],[1 2 1, 1 3 1]) saves the GICLeff for the two transformers. A second example is TimeStepSaveFieldsSetByObject(Bus,[Vpu],[1,2]) saves the per unit voltage at buses 1 and 2. If this command is followed by TimeStepSaveFieldsSetByObject(Bus,[Vangle],[3]) both the per unit voltage magnitude and angle are saved at buses 1, 2 and 3.

objecttype : The objecttype being set.
[fieldlist] : A comma-delimited list of the A list of fields to update.
[objectIDList] : A comma-delimited list of the key fields for specific objects the type given by objecttype.

This command sets the Time Step Simulation to save the per-unit voltage (Vpu) for Bus 1 and Bus 2.

```
TimeStepSaveFieldsSetByObject(BUS, [Vpu], [1, 2]);
```

TIMESTEPSaveSelectedModifyStart;

(This command was added in Simulator 25)

This command is associated with setting the power system objects whose fields should be saved during the time step simulation. The objects themselves are specified by calling SetData with the field TimeDomainSelected (e.g. SetData(Gen,["BusNum","GenID","TimeDomainSelected"],[1,"1","YES"])). However, before using the SetData command for this field you must call this function. Then, when the changes are finished, you must call TIMESTEPSaveSelectedModifyFinish.

TIMESTEPSaveSelectedModifyFinish;

(This command was added in Simulator 25)

This command is associated with setting the power system objects whose fields should be saved during the time step simulation. The objects themselves are specified by calling SetData with the field TimeDomainSelected (e.g. SetData(Gen,["BusNum","GenID","TimeDomainSelected"],[1,"1","YES"])). However, before using the SetData command for this field you must call TIMESTEPSaveSelectedModifyStart. Then, when the changes are finished, you must call this function. Changing the selected objects will delete any saved results for that object type. Note, the selected objects are saved in the *.tsb file, not the pwb.

TIMESTEPSaveInputCSV("Filename", [input field list], ", ISO8601StartDateTime, ISO8601EndDateTime)

This command saves the input fields listed below. These include the regular input tab fields and the weather input data.

Filename : The output filename.

[input field list] : The list of input fields to save. See list of valid fields below.

ISO8601StartDateTime: The optional desired start date and time specified in ISO8601 format for the save. If not specified, then start the save at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.

ISO8601EndDateTime: The optional desired end date and time specified in ISO8601 format for the save. If not specified, then finish the save at the end. This datetime should either be a UTC value, or local time specified with the time zone offset.

Valid fields that can be specified in the input field list:

LOAD_MW	WEATHERSTATION_WINDSPEEDMSEC
LOAD_MVAR	WEATHERSTATION_WINDSPEEDKNOTS
GEN_MW	WEATHERSTATION_WINDSPEEDKMPH
GEN_MWMAX	WEATHERSTATION_WINDDIRECTION
BRANCH_STATUS	WEATHERSTATION_CLOUDCOVERPERC
AREA_LOADMW	WEATHERSTATION_WINDSPEED100MPH
ZONE_LOADMW	WEATHERSTATION_WINDSPEED100MS
INJECTIONGROUP_MW	WEATHERSTATION_WINDSPEED100KNOTS
WEATHERSTATION_TEMPF	WEATHERSTATION_WINDSPEED100KMPH
WEATHERSTATION_TEMPC	WEATHERSTATION_GLOBALHORZIRRADWM2
WEATHERSTATION_DEWPOINTF	WEATHERSTATION_DIRECTHORZIRRADWM2
WEATHERSTATION_DEWPOINTC	WEATHERSTATION_WINDTERRFRICTCOEFF
WEATHERSTATION_HUMIDITY	WEATHERSTATION_WINDGUSTMPH
WEATHERSTATION_HEATINDEXF	WEATHERSTATION_WINDGUSTMS
WEATHERSTATION_HEATINDEXC	WEATHERSTATION_WINDGUSTKNOTS
WEATHERSTATION_WINDCHILLF	WEATHERSTATION_SMOKEVERTINTMGMW
WEATHERSTATION_WINDCHILLC	WEATHERSTATION_PRECIPRATEMMHR
WEATHERSTATION_TRANSMITTANCE	WEATHERSTATION_PRECIPPERCFROZEN
WEATHERSTATION_INSOLATIONPERC	WEATHERSTATION_DIRECTNORMIRRADWM2
WEATHERSTATION_WINDSPEEDMPH	WEATHERSTATION_DIFFUSEHORZIRRADWM2

This command saves a CSV file named inputs.csv containing the LOAD_MW and GEN_MW input data from January 1, 2024, 00:00 UTC to January 2, 2024, 00:00 UTC.

```
TIMESTEPSaveInputCSV("inputs.csv", [LOAD_MW, GEN_MW], 2024-01-01T00:00Z, 2024-01-02T00:00Z);
```

TimeStepSaveResultsByTypeCSV(ObjectType, "FileCSVName", ISO8601StartDateTime, ISO8601EndDateTime)

(This command was added in the January 8, 2024 patch of Simulator 23)

Saves the Time Step Simulation results for the specified object type in a CSV file.

ObjectType : The name of the object type being saved with only the first three letters significant, case insensitive.

"FileCSVName" : Name of the CSV file for saving the results; it should have a ".csv" extension.

ISO8601StartDateTime: The optional desired start date and time specified in ISO8601 format for the save. If not specified, then start the save at the beginning. This

datetime should either be a UTC value, or local time specified with the time zone offset.

ISO8601EndTime: The optional desired end date and time specified in ISO8601 format for the save. If not specified, then finish the save at the end. This datetime should either be a UTC value, or local time specified with the time zone offset.

This command saves the Time Step Simulation results for generators into a file named "gen_results.csv", including data from January 1, 2025, 00:00 UTC to January 2, 2025, 00:00 UTC.
<code>TimeStepSaveResultsByTypeCSV(GEN, "gen_results.csv", 2025-01-01T00:00:00Z, 2025-01-02T00:00:00Z);</code>

TimeStepSaveTSB(FileName)

Saves the Time Step Simulation results using the specified TSB file in the latest TSB version.

"FileName" : Name of the TSB file; it should have a ".tsb" extension.

TimeStepSavePWW("FileName")

(This command was added in the December 19, 2023 patch of Simulator 23)

Saves the existing weather data in the Time Step Simulation using the specified PWW file.

"FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

TimeStepSavePWWRange("FileName",ISO8601StartDateTime,ISO8601EndTime)

This command is similar to TimeStepSavePWW in that it saves a PWW file, but only for the date and time values between the start and end datetime values (expressed in ISO8601 format). An example is

TimeStepSavePWWRange("c:\tmp\test.pww, 2024-03-01T0Z,2024-03-01T23Z), which would save data for March 1, 2024.

"FileName" : Name of the PWW file; it should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

ISO8601StartDateTime: The desired start date and time specified in ISO8601 format. If string is empty ("") or before the start datetime then start at the beginning. This datetime should either be a UTC value, or local time specified with the time zone offset.

ISO8601EndTime: The desired end date and time specified in ISO8601 format. If string is empty ("") or after the end datetime then go until the end. The datetime should either be a UTC value, or local time specified with the time zone offset.

This command saves a PWW file named "march1.pww" containing Time Step Simulation data specifically for the time window from March 1, 2024, 00:00 UTC to 23:00 UTC.
<code>TimeStepSavePWWRange("march1.pww", 2024-03-01T00:00Z, 2024-03-01T23:00Z);</code>

Weather

TemperatureLimitsBranchUpdate(RatingSetPrecedence, NormalRatingSet, CTGRatingSet);

Updates branch limits based on a temperature limit curve and weather station temperature if valid temperature limit curves and weather station data are available for a branch.

- RatingSetPrecedence : Optional parameter – default is NORMAL
Valid entries are NORMAL, CTG, or blank. A blank entry is the same as NORMAL. This is used when the same rating set is specified to be overwritten by both the normal and CTG temperature limit curves.
- NormalRatingSet : Optional parameter – default is DEFAULT
Use this parameter to specify which limit for a given branch should be updated with the normal temperature dependent limit. Valid entries are the following:
DEFAULT : When updating limits for a given branch, update the Normal Rating Set that is specified with the Limit Monitoring Settings with the normal temperature-dependent limit.
NO : Do not use the normal temperature-dependent limit to update any limit.
A through O : When updating limits, update this letter-specified limit with the normal temperature-dependent limit.
- CTGRatingSet : Optional parameter – default is DEFAULT
Use this parameter to specify which limit for a given branch should be updated with the CTG temperature dependent limit. Valid entries are the following:
DEFAULT : When updating limits for a given branch, update the CTG Rating Set that is specified with the Limit Monitoring Settings with the normal temperature-dependent limit.
NO : Do not use the CTG temperature-dependent limit to update any limit.
A through O : When updating limits, update this letter-specified limit with the CTG temperature-dependent limit.

This command updates branch limits based on temperatures using predefined temperature limit curves and associated weather station data. With RatingSetPrecedence set to NORMAL, the update process prioritizes normal ratings. The NormalRatingSet parameter set to "A" ensures that the temperature-adjusted normal limit is applied to limit set A, while the CTGRatingSet set to "B" applies the temperature-adjusted contingency limit to limit set B for each branch.

TemperatureLimitsBranchUpdate(NORMAL, A, B);

WeatherLimitsGenUpdate(UpdateMax, UpdateMin);

Updates generator MW limits based on a weather limit curve and weather station temperature if valid weather limit curves and weather station data are available for a generator.

- UpdateMax : Optional parameter – default is YES
Set this to YES or blank to update the Max MW limit based on the calculated weather-dependent MWMax limit. Set to NO to not change the Max MW limit.
- UpdateMin : Optional parameter – default is YES
Set this to YES or blank to update the Min MW limit based on the calculated weather-dependent MWMin limit. Set to NO to not change the Min MW limit.

This command updates generator MW limits using temperature-dependent curves and weather station data.

<code>WeatherLimitsGenUpdate(YES, NO);</code>

WeatherPFWModelsSetInputs;

(This command was added in the January 8, 2024 patch of Simulator 23)

Sets the inputs for all the case PFWModels, but does not apply them to the power flow case. Usually these inputs require the availability of weather measurements.

WeatherPFWModelsSetInputsAndApply (SolvePowerFlow)

(This command was added in the January 8, 2024 patch of Simulator 23; the required SolvePowerFlow parameter was added in the December 31, 2024 patch of Simulator 23)

Sets the inputs for all the case PFWModels and also applies them to the power flow case. Usually these inputs require the availability of weather measurements, which can be loaded using the WeatherPWWLoadForDateTimeUTC command. When the PFWModels are applied to the case, some existing case values can be changed, such as the generator MaxMW fields. Use WeatherPFWModelsRestoreDesignValues to restore these values to the design values specified by the PFWModels.

SolvePowerFlow : If YES then it also solves the power flow using the default power flow solution method. If you do not wish to solve the power flow, or wish to solve it using either a different method or another solution type (e.g., the OPF) set this parameter to NO. Then optionally call the another solution command (e.g., SolvePowerFlow, or SolvePrimalLP).

This command applies all PFWModel-based weather adjustments—like generator MW limits—and solves the power flow using current weather data.
--

<code>WeatherPFWModelsSetInputsAndApply(YES);</code>
--

WeatherPWWFileAllMeasValid ("PWWFileName", [field list], ISO8601StartDateTime, ISO8601EndDateTime)

Returns true if the specified PWW file 1) has the all the specified fields, and 2) all the measurements for those fields are valid. This command only works with a version 2 or greater pww file.

"PWWFileName" : The pww file to check.
[field list] : The list of fields to check. At least one field must be provided. Valid fields are given below. Note, in contrast to commands that are providing numerical weather values, like TimeStepSaveInputCSV, here the units are not included in the field names.

ISO8601StartDateTime: Optional start datetime specified in the ISO8601 format. If provided, then the command only returns true if the starting datetime in the pww file is at or before this datetime.

ISO8601EndDateTime: Optional end datetime specified in the ISO8601 format. If provided, then the command only returns true if the ending datetime in the pww file is at or after this datetime.

Valid fields that can be specified in the field list:

TEMP	DIRECTHORZIRRAD
DEWPOINT	WINDGUST
WINDSPEED	SMOKEVERTINT
WINDSPEED100	PRECIPRATE
GLOBALHORZIRRAD	PRECIPPERCFROZEN

This command checks if the "weather_data.pww" file includes valid temperature and wind speed data throughout March 1, 2024. Returns true only if both fields are present and all their values are valid for the entire specified time range.

```
WeatherPWWFileAllMeasValid("weather_data.pww", [TEMP, WINDSPEED], 2024-03-01T00:00Z, 2024-03-01T23:59Z);
```

WeatherPFWModelsRestoreDesignValues;

(This command was added in the January 8, 2024 patch of Simulator 23)

Restores the case values changed by WeatherPFWModelsSetInputsAndApply to the design values specified with each PFWModel.

WeatherPWWFileCombine2 ("SourceFileName1", "SourceFileName2", "DestFileName")

(This command was added in the March 4, 2024 patch of Simulator 23)

Combines two PWW files provided the PWW files have the same weather stations. The combined file is stored in DestFileName. The files names should either have a ".pww" extension or no extension; if no extension then it is set to ".pww". The datetime ranges for the files cannot overlap.

"SourceFileName2" : Second source file name; it must exist. Should be the second file chronologically.

"DestFileName" : Destination file name; it does not need to exist and can be either of the source file names.

This command merges two PWW weather files—"weather_march.pww" and "weather_april.pww"—into a new file called "weather_combined.pww", provided both source files reference the same weather stations and have non-overlapping date ranges.

```
WeatherPWWFileCombine2("weather_march.pww", "weather_april.pww", "weather_combined.pww");
```

WeatherPWWFileGeoReduce("SourceFileName", "DestFileName", minLatitude, maxLatitude, minLongitude, maxLongitude);

(This command was added in the March 4, 2024 patch of Simulator 23)

Reduces the geographic scope of a PWW file. The minLatitude, maxLatitude, minLongitude, and maxLongitude contain the bounding coordinates for data in the destination file. The files names should either have a ".pww" extension or no extension; if no extension then it is set to ".pww".

"SourceFileName" : Source file name; it must exist.

"DestFileName" : Destination file name; it does not need to exist and can be the source file.

minLatitude : Minimum latitude for the bounding rectangle. Must be greater than or equal to -90, and less than the maxLatitude.

maxLatitude : Maximum latitude for the bounding rectangle. Must be less than or equal to 90 and greater than the minLatitude.

minLongitude : Minimum longitude for the bounding rectangle. Must be greater than or equal to -180 and less than the maxLongitude. Hence rectangles spanning the international date line are not allowed.

maxLongitude : Maximum longitude for the bounding rectangle. Must be less than or equal to 180 and greater than the minLongitude. Hence rectangles spanning the international date line are not allowed.

This command extracts weather data only for the geographic region bounded by latitudes 30.0 to 40.0 and longitudes -100.0 to -90.0 from "weather_full.pww", and saves it in "weather_region.pww".

```
WeatherPWWFileGeoReduce("weather_full.pww", "weather_region.pww", 30.0, 40.0, -100.0, -90.0);
```

WeatherPWWSetDirectory("PWWFileDirectory", IncludeSubDirectories);

(This command was added in the March 10, 2024 patch of Simulator 23)

When getting weather information from PWW files, this command specifies the directory to search, and optionally its subdirectories.

"PWWFileDirectory" : Directory that contains the PWW files.

IncludeSubDirectories : Optional field indicating whether to include subdirectories in the search path. Either YES or NO, with YES the default.

This command sets "C:\WeatherData\PWW" as the main directory to search for PWW files and includes all its subdirectories in the search path.
--

WeatherPWWSetDirectory("C:\WeatherData\PWW", YES);
--

WeatherPWWLoadForDateTimeUTC(ISO8601DateTime);

(This command was added in the March 10, 2024 patch of Simulator 23)

This command loads the weather for the specified date and time. It does this by searching the directory, and optionally the subdirectories set with the WeatherPWWSetDirectory command.

ISO8601DateTime : The desired date and time specified in ISO8601 format. This datetime should either be a UTC value, or local time specified with the time zone offset. For example, for weather on March 6, 2024 at 18:00 UTC the value could be in UTC "2024-03-06T18:00Z" or in local time (with an offset of -6 hours from UTC) "2024-03-06T12:00-06".

This command loads weather data for March 6, 2024, at 18:00 UTC. It searches the directory specified earlier using the WeatherPWWSetDirectory script command, including any subdirectories if configured, to find and load the appropriate weather conditions for that exact time.
--

WeatherPWWLoadForDateTimeUTC("2024-03-06T18:00Z");
--

Distributed Computing

EnterDistMasterPassword(Password);

Use this action to enter the master password used to unlock distributed machine login credentials.

 Password : Password that must be specified to unlock the credentials.

VerifyDistributedComputersAvailable;

Allows distributed computer status to be verified.

Trainer

ResetStatusChangeCount;

There is a field with each branch called Status Change Count that tracks the number of times a branch changes status. This script action sets this count to 0 for all branches. This field is only used with Trainer.

SuppressCommandDialog;

Tells trainer to only send status change commands instead of opening a dialog, which in the case of generators and other objects has additional options.

Customer-Specific Actions

ISONEInterfaceLimitCalculation("filename", variablename);

This script command has been developed specifically for ISO-NE to determine the limit of a specified interface based on a set of logical conditions being met on the given state of the power system.

"filename" : Name of the file containing all input data. This is a text-based format with the description of the contents given below.

variablename : Interface floating point field that will contain the calculated limit for each studied interface.

The file containing the input data contains specific sections to properly calculate an interface limit. Each section is identified by a keyword. Any sections with keywords that are not recognized will be ignored. The data for each section is enclosed in curly braces, { }. Curly braces must be on their own lines. Comments can be added using the double slash, //, notation at the start of a line. Only entire lines can be commented. The contents of an example file are shown below:

```
// Put comments here
Interface
{
MyInterface
}
ModelExpressions
{
Branch1_Online
GeneratorA_Online
GeneratorB_Offline
LoadGroup_Online
}
MatrixM
{
-0.5,0.5,  0.5,1.5, -9,9, -9,9
-0.5,0.5, -0.5,0.5, -9,9, -9,9
}
VectorL
{
123
456
}
MatrixA
{
0,0,350,100
0,0,120,80
}
VectorB
{
700
400
}
```

The section keywords and data contents are described below :

Interface : Name of the interface for which the limit is being calculated. Only a single interface limit can be calculated per input file.

ModelExpressions	: List of Model Expressions that correspond to the columns of MatrixM and MatrixA. These Model Expressions must be specified by name as the key field for identifying them within the power flow case. There must be as many Model Expressions defined as there are columns of MatrixM and MatrixA (m).
MatrixM	: This is an nxm matrix of floating point values. Each entry in the matrix contains two values that represent the minimum and maximum range of a power system value. It is possible that there will be no limit in either the minimum or maximum direction and -INF or INF can be used to indicate this instead of a floating point value.
VectorL	: This is an nx1 vector of floating point values. This vector must be the same size as the number of rows in MatrixM.
MatrixA	: This is an nxm matrix of floating point values. This matrix must be the same size as MatrixM. This matrix is optional and does not have to exist in the input file. If not specified, it will be assumed that all entries are 0 and it is the same size as MatrixM. Previous versions of the script command named this MatrixL1. To maintain older functionality, Simulator will look for either MatrixA or MatrixL1.
VectorB	: This is an nx1 vector of floating point values. Each entry is the upper bound limit for the corresponding rows of MatrixM. To maintain older functionality, it is not required that this vector be included in the input data. If it is not included, the calculations will be done without imposing these limits.

Implementation of the limit calculation is determined by the following:

Each column of Matrix M corresponds to a Model Expression. All defined Model Expressions will be evaluated in the present power system state. Each row, n , of M is evaluated for the logical conditions defined for each cell entry. The logical condition for each cell entry m is: $\min_m \leq \text{Model Expression}_m \leq \max_m$. There is only one row, call this p , of M where all logical conditions are met.

The interface limit is then calculated based on row p using the following equation:

$$\text{Limit}_{\text{prelim}} = \text{VectorL}_p + \sum_{j=1}^m \text{MatrixA}_{p,j} \cdot V_j$$

where V_j is the value of $\text{Model Expression}_j$.

If VectorB has been specified,

$$\text{Limit} = \text{Min}\{\text{Limit}_{\text{prelim}}, \text{VectorB}_p\}$$

otherwise,

$$\text{Limit} = \text{Limit}_{\text{prelim}}$$

If there is no row p where all logical conditions are met, the *Limit* will be set to 99999 if the input is specified using MatrixA. If the older format that uses MatrixL1 is specified, the *Limit* will be set to 0. To access the calculated *Limit* value, the field that is specified by the input variable name parameter is set to the *Limit*.

During the calculation, the *Selected* field will be set to YES for all Model Expressions that are used in the calculation and to NO for all Model Expressions that are not used as part of the calculation. The *Selected* field for the interface for which the limit is determined will be set to YES, and the *Selected* field will be set to NO for all other interfaces.

DATA Section

Concise Auxiliary File Header

PowerWorld defaults to using a concise header that starts with the `object_type` string followed by the `list_of_fields` as the argument between the parentheses.

```
object_type DataName(list_of_fields)
{
  data_list_1
  .
  .
  data_list_n
}
```

Simulator Version 19 and later support reading both this syntax and the older Legacy header. Older versions of Simulator only support the Legacy Auxiliary File Header below. All versions of Simulator support reading the legacy header format. When writing out an auxiliary file there is an option starting in Simulator Version 19 called Use Concise Variable Names and Auxiliary File Headers that determines whether to write out the DATA keyword and other arguments or to write out the concise auxiliary file header. See the online help documentation for more details:

http://www.powerworld.com/WebHelp/#MainDocumentation_HTML/Auxiliary_Files.htm

When using the concise format, the data between the curly braces is always written using space delimiter and Create_if_not_found is always assumed to be YES.

Legacy Auxiliary File Header

```
DATA DataName(object_type, [list_of_fields], file_type_specifier, create_if_not_found)
{
  data_list_1
  .
  .
  data_list_n
}
```

Immediately following the DATA keyword, you may optionally include a DataName. By including the DataName, you can make use of the script command LoadData("filename", DataName) to call this particular data section from another auxiliary file. Following the optional DataName is an argument list. The argument list is contained inside left and right parentheses "()". There are 4 arguments in this list which will be described shortly: `object_type`, `[list_of_fields]`, `file_type_specifier`, and `create_if_not_found`.

ObjectType

The `object_type` parameter identifies the type of object or data element the information section describes or models. For example, if `object_type` equals BUS, then the data describes BUS objects.

There are some special object types that start with the keyword REMOVED. If these are loaded into Simulator while in Edit mode, the corresponding objects will be deleted. For example REMOVEDBUS will delete BUS objects, REMOVEDBRANCH will delete BRANCH objects, etc. Not all object types have a corresponding REMOVED object type, and simply prepending this keyword to the front of an `object_type` will not allow this functionality. The objects that exist of this with this functionality are the ones that allow comparison of topological changes through the Difference Flows tool.

The list of object types Simulator's auxiliary file parser can recognize will grow as new applications are developed. Within Simulator, you will always be able to obtain a list of the available object types by going to the main menu and choosing Window, Export Case Object Fields, and then exporting the objects to Excel or a text file.

File_Type_Specifier

When using the Legacy Auxiliary File Header, the `file_type_specifier` parameter distinguishes the information section as containing custom auxiliary data (as opposed to Simulator's native auxiliary formats), and indicates the format of the data. The parser recognizes two values for `file_type_specifier`:

(blank) or AUXDEF or DEF	Data fields are space delimited
AUXCSV or CSV or CSVAUX	Data fields are comma delimited

Required Fields and Create_if_not_found

Each ObjectType in PowerWorld has a set of Required Fields that must be included in the List_of_Fields when using an Auxiliary File to *create a new object*. These Required Fields are highlighted in green in the user interface. If these Required Fields are not included in the List_of_Fields, then new objects will never be created, and the Auxiliary file will only edit existing objects. When using the Concise Auxiliary File Header, if the required fields are given, then new objects will be created. There are also some restrictions for objects that define the network topology and these objects may only be created while in Edit Mode: Bus, Gen, Load, Shunt, Branch, LineShunt, DCTransmissionLine, VSCDCLine, MSLine, 3WXFormer, MTDCRecord, MTDCBus, MTDCLine, and MTDCConverter.

In addition to specifying these fields, when using the Legacy Auxiliary File Header, an optional field `Create_if_not_found` must be specified. In the Legacy header, objects will only be created if all Required Fields are given and the `Create_if_not_found=YES`. If the value is NO, objects will not be created. If `Create_if_not_found` is omitted in the Legacy header, YES is assumed. If loading an auxiliary file using the LoadAux script command, the `Create_if_not_found` field for the data section will override the `Create_if_not_found` field with the script. Only if the `Create_if_not_found` field for the data section is set to PROMPT will the field for the script command be used. To repeat, regardless of the choice of CreateIfNotFound, a new object will only be created if a particular list of Required Fields is in the List_of_Fields.

List_of_Fields

The `list_of_fields` parameter lists the types of values the ensuing records in the data section contain. The order in which the fields are listed in `list_of_fields` dictates the order in which the fields will be read from the file. Simulator currently recognizes many different field types, each identified by a specific field variable name. Because the available fields for an object may grow as new applications are developed for the convenience of our customers, you will always be able to obtain a list of the available object types and fields by going to the main menu and choosing Window, Export Object Fields, and then choosing to export to Excel or a text file. Certainly, only a subset of these fields would be found in a typical custom auxiliary file. In crafting applications to export custom auxiliary files, developers need concern themselves only with the fields they need to communicate between their applications and Simulator. A few points of interest regarding the `list_of_fields` are:

- The `list_of_fields` may take up several lines of the text file.
- When using the older heading starting with the keyword DATA, the `list_of_fields` should be enclosed by square brackets []. When using the concise heading these square brackets are not used.
- When encountering the PowerWorld comment string `'/'` in one of these lines of the text file, all text to the right is ignored.
- Blank lines, or lines whose first characters are `'/'` will be ignored as comments.
- Field variable names must be separated by commas.

Example: the following are equivalent representations

<code>DATA (BUS, [NomKV, Number, // comment here VAngleA, VAngleB, VpuA, VpuB, // comments allowed here too // note that blank rows are ignored AreaNum, VAngle, ActB, Equiv, ActG, kV, MargCostMW, LoadMVA, // more comment LoadMW, LongName])</code>	<code>BUS (NomKV, Number, // comment here VAngleA, VAngleB, VpuA, VpuB, // comments allowed here too // note that blank rows are ignored AreaNum, VAngle, ActB, Equiv, ActG, kV, MargCostMW, LoadMVA, // more comment LoadMW, LongName)</code>
---	---

Concise Field Variable Names

Variable names within Simulator were overhauled starting with Version 19. Most no longer utilize the special location integer and instead spell out such information in the field variable name. In general, the variable names have been made more concise or at least more understandable. Therefore what was once called for a BRANCH object LineMW:1 is now called MWTo. Similarly LineMW:2 is now called MWFromCalc (representing the MW flow at the from bus of branch calculated from the terminal voltages). The only fields that continue to use the location integer are those that represent fields for which a dynamic number of fields are available. Examples of this include the CustomInteger, CustomString, and CustomFloat fields which use the location integer to specify which value is used. Other examples include the multiple direction PTDF results fields PTDFMult:0, PTDFMult:1, and so on.

When writing out an auxiliary file there is an option within Simulator Version 19 called Use Concise Variable Names and Auxiliary File Headers that determines whether to write out the DATA keyword and other arguments or to write out the concise auxiliary file header. See the online help documentation for more details:

http://www.powerworld.com/WebHelp/#MainDocumentation_HTML/Auxiliary_Files.htm

Legacy Field Variable Naming

When listing fields, some field variable names may be augmented with a field location. These are in the format `variablename:location`. One example of this is the field LineMW. For a branch, there are two MW flows associated with the line: one MW flow at the from bus, and one MW flow at the to bus. So that the number of fields does not become huge, the same field variable name is used for both of these values. For the from bus flow, we write LineMW:0, and for the to bus flow, we write LineMW:1. Note that field variable names using a location of 0, such as LineMW:0, may simply leave off the :0.

These field variable names have been updated with Concise Field Variable Names starting in Simulator version 19. These are described in the next section. The Legacy field variable names can still be used to support existing auxiliary files or files needed for loading into earlier versions of Simulator.

Special Naming

There are several fields that can be referred to by the user-defined name for the field rather than using the location number. These are fields that might have their location numbers change when different auxiliary files are merged in the same case. Referring to these by name can eliminate this possible confusion. These fields can be defined in the format `variablename:location_by_name`. They can also be referred to by location number as well.

Fields that allow referring to the location by name are:

- Expressions – "CustomExpression:my expression name"
- String Expressions – "CustomExpressionStr:my string expression name"
- Custom fields (Floating Point, Integer, and String) – "CustomSingle:my custom single name". Using this format for custom fields requires that Custom Field Descriptions be created for the fields to be used.
- Calculated Fields – "BGCalcField:my calculated field variable name"

Key Fields

Simulator uses certain fields to identify the specific object being described. These fields are called key fields. For example, the key field for BUS objects is BusNum, because a bus can be identified uniquely by its number. The key fields for GEN objects are BusNum and GenID. To properly identify each object, the object's key fields must be present. They can appear in any order in the `list_of_fields` (i.e. they need not be the first fields listed in `list_of_fields`). As long as the key fields are present, Simulator can identify the specific object. By going to the main menu and choosing Window and then Export Case Object Fields you will obtain a list of fields available for each object type in either Excel or text format. In this output, the key fields will appear with asterisks *.

Data List

After the data argument list is completed, the Data list is given. The data section lists the values of the fields for each object in the order specified in `list_of_fields`. The data section begins with a left curly brace and ends with the a right curly brace. A few points of interest regarding the `value_list`:

- The `value_list` may take up several lines of the text file.
- Each new data object must start on its own line of text.
- When encountering the PowerWorld comment string `'/'` in one of these lines of the text file, all text to the right of this is ignored.
- Blank lines, or lines whose first characters are `'/'` will be ignored as comments.
- Remember that the right curly brace must appear on its own line at the end of the `data_list`.
- If the `file_type_specifier` is CSV, the values should be separated by commas. Otherwise, separate the field variable names using spaces.
- Strings can be enclosed in double quotes, but this is not required. You should however always inclose strings that contain spaces (or commas) in quotes. Otherwise, strings containing commas would cause errors for comma-delimited files, and spaces would cause errors for space-delimited formatted files.

Special Data List Entries

When specifying values in case information displays, AUX files, and script commands, there are some special formats that can be used. These formats will allow the value of a model or object field to be used instead of specifying an explicit value. Formats that are allowed include those described in the [Special Identifiers for Model Fields in Data](#) section as well as others mentioned below in this section.

A string of the format `"@Variablenamelegacy:location:digits:decimals"`

Or `"@concisename:location:digits:decimals"`

will be treated as though the value of the named variable is entered in the field, with `digits` total and `decimals` digits to the right of the decimal point. This format may be used in case information displays, AUX files, and script commands that set values. String fields that can be converted to a valid numeric value can be used to populate either floating point or integer fields. Floating point values that are used to populate integer fields are truncated before populating the integer field.

When parsing an AUX file while reading, the treatment of concise and legacy variable names is automatically handled by the parser.

There are specific fields for specific object types that allow referencing a model field definition as the value of the field. The model field definition is specified in this syntax: `@MODELFIELD<objecttype 'key1' 'key2' variablename:digits:rod>`. The model field definition and not the value of the model field itself is used as the value of the field until the field is used for its intended purpose. The fields that can be specified in this manner include those that specify directory paths and file locations. When an action is carried out that requires using the directory path or file location, the model field definition is converted to the value that it represents and that gets used as the location of the file or directory path.

The following object types and fields (given by variable name) can use this special model field syntax: `Transient_Options` (`RSHD_Directory`), `PVCurve_Options` (`PVCOutFile`, `PVCStoreStatesWhere`), `QVCurve_Options` (`QVOutputDir`, `QVOutputFileName`), `CTG_Options` (`PostCTGSolAuxFile`, `PostCTGAuxFile`, `HardDriveFileName`), `Sim_Environment_Options` (`SEOSpecifiedAuxFile:0`, `SEOSpecifiedAuxFile:1`, `SEOSpecifiedAuxFile:2`) and `MessLog_Options` (`LogAutoFileName`).

Special Identifiers for Model Fields in Data

The following special formats can be used in case information displays, AUX files, and script commands that set values. They are also used in some script commands as part of parameters that input text. These formats will allow the value of a model or object field to be used instead of specifying an explicit value.

A string in the format "&ModelExpressionName:digits:decimals" will be treated as though the value of the named model expression is entered in the field, with `digits` total and `decimals` digits to the right of the decimal point. If no `digits` or `decimals` are specified, 7 decimal places will be used. Trailing zeros will be removed if no `decimals` are specified.

A string of the format "&Objecttype 'key fields' variablenamelegacy:location:digits:decimals"
Or "&Objecttype 'key fields' concisename:digits:decimals"

will be treated as though the value of the named object and object field is entered in the field, with `digits` total and `decimals` digits to the right of the decimal point. If no `digits` or `decimals` are specified, 7 decimal places will be used. Trailing zeros will be removed if no `decimals` are specified.

This syntax can be used when specifying the values of key fields used for identifying objects when using `SetData` and `CreateData` script commands.

Example:

Use a `SetData` command to open a branch that has the highest loading in the case. Create a calculated field that returns the maximum loading for a branch object and then apply this to the `PWCaseInformationObject`. Use the `ObjectID` for the branch identification and the `CalcFieldExtra` field to return the string identifier for the object that meets the calculated field conditions.

```
SetData(Branch, [ObjectID, Status], ["&PWCaseInformation  
'CalcFieldExtra:HighestLoading'", "Open"]);
```

A string of the format "&Objecttype '@variablenamelegacy:location'
variablenamelegacy2:location:digits:decimals"

Or "&Objecttype '@concisename' concisename2:digits:decimals"

will use a specific field for one object, `variablenamelegacy` or `concisename`, to determine the key field value for the `objecttype` object. This will be treated as though the value of the named object and object field, `variablename2` or `concisename2`, is entered in the field, with `digits` total and `decimals` digits to the right of the decimal point. If no `digits` or `decimals` are specified, 7 decimal places will be used. Trailing zeros will be removed if no `decimals` are specified.

Example:

Set the `CustomFloat` field for every load to the MW value of that load's zone.

```
SetData(Load, [CustomFloat], ["&Zone '@ZoneNumber' LoadMW"], All);
```

(Added in the August 1, 2023 patch for Simulator 23)

A string of the format "&@variablenamelegacy variablenamelegacy2:location:digits:decimals"

Or "&@concisename concisename2:digits:decimals"

will use a specific field for one object, `variablenamelegacy` or `concisename`, to determine the `objecttype` and key field for identifying another object. This will be treated as though the value of the named object and object field, `variablename2` or `concisename2`, is entered in the field, with `digits` total and `decimals` digits to the right of the decimal point. If no `digits` or `decimals` are specified, 7 decimal places will be used. Trailing zeros will be removed if no `decimals` are specified.

Example:

Set the `CustomFloat` field for every load to the MW value of the zone identified by the `objecttype` and key field identifiers contained in the `CustomString` field for each load.

```
SetData(Load, [CustomFloat], ["&@CustomString LoadMW"], All);
```

Using Labels for Identification

Most data objects (such as buses, generators, loads, switched shunts, transmission lines, areas, zones, and interfaces) may have an alternative names assigned to them. These alternative names are called labels. Labels allow you to refer to equipment in the model in a way that may be unique to your organization. Labels may thus help clarify which elements are described by a particular set of data, especially when the short names employed by the power system model prove cryptic. Furthermore, since labels are likely to change less frequently than bus numbers, and since a label must, by definition, identify only one power system component, they may function as an immutable key for importing data from auxiliary files into different cases, even when bus numbering schemes change between the cases. Labels must be unique for devices of the same type, but the same label can be used for a device of a different type.

Information dialogs corresponding to buses, generators, loads, switched shunts, transmission lines, areas, zones, and interfaces feature a button called Labels. If you press this button, the device's Label Manager Dialog will appear. The Label Manager Dialog lists the labels associated with the device. You can delete a label from the list by selecting it and pressing the delete key on the keyboard or clicking the Delete button. You may add a label to the device by typing its name in the textbox and pressing the Add New button. You will not be allowed to add a Label that already exists for the same type of device. A single power system device may have multiple labels, but each label may be associated with only one device of a given type. For example, a bus could have the label Bus North while a generator could also have the same label, but there could not be another bus or generator with this same label.

You also may designate a particular label to be the primary label for the device by checking the box Primary before adding the label. Alternatively, you can select the device from the list and click the Make Primary button. A device's primary label is the one that is listed first in the Labels (All) field (`variablename = LabelsAll`) in a Case Information Display. This field lists all labels assigned to a device as a comma-delimited string. Any label can be used to import data from auxiliary data files.

Labels can be used to map data from an auxiliary data file to a power system device. Recall that auxiliary data files require you to include a device's key fields in each data record so that data may be mapped to the device. Labels provide an alternative key. Instead of supplying the bus number to identify a bus, for example, you can supply one of the bus's labels. The label will enable Simulator to associate the data with the device associated with that label. This mechanism performs most efficiently when the primary label is used, but other labels will also provide the mapping mechanism. The Label (for use in input from AUX or Paste) field (`variablename = Label`) is used for importing data using labels and is blank when viewing in a case information display. Keep in mind that all devices read via an auxiliary file using the label field should have a non-blank label. Otherwise, information for that device will not be read. Even if the primary or secondary key fields are provided with the device, as long as the label field is present, that is the only field that will be used to identify the device. New devices cannot be created by simply identifying them by label. Either the primary or secondary key fields must be present to create a new device and the label field should not be present.

Again, it is important to remember this: a single power system device may have multiple labels, but each label may be associated with only one device of a particular type. This is the key to enabling data to be imported from an auxiliary file using labels.

Saving Auxiliary Files Using Labels

All devices that can be identified by labels will have the Labels (All) and Label (for use in input from AUX or Paste) fields available in their case information displays. In order to save auxiliary files that identify devices by label, the two label fields should be added to the case information display prior to saving the data in an auxiliary file. Because the Label (for use in input from AUX or Paste) field will be blank when saved in the auxiliary file, this field must be populated with one of the labels in the Labels (All) field before loading the auxiliary file back in. Keep in mind that devices with blank labels cannot be identified when loading in an auxiliary file, so avoid saving auxiliary files by label if all devices do not have labels. Note that when saving out an entire case as an auxiliary file, the field "AllLabels" is included for each object type that allows labels and has some labels defined.

Many devices require SUBDATA sections. These sections have custom formats specific to the type of information that they contain. When saving auxiliary files with devices that require SUBDATA sections, the user can choose to use primary or

secondary key fields or labels to identify devices in the SUBDATA sections. The user will either be prompted when saving the devices, or there is an option to change the key field to use when saving subdata sections on the PowerWorld Simulator Options dialog under the Case Information Displays category. When choosing to use labels, if a device has a label, it will be used. If it is a device that can be identified by buses and bus labels exist, bus labels will be used. Finally, if the device does not have a label and the buses do not have labels, the primary key for the device will be used for identification.

Devices that have SUBDATA sections that contain other devices that can be identified by labels include: contingencies, interfaces, injection groups, post power flow solution actions, and owners.

The setting to choose which identifier to use for the SUBDATA sections does not just apply to SUBDATA sections. Often when saving groups of options, this setting will apply to everything being saved with those options and not just the SUBDATA sections. This includes contingency options, ATC options, limit monitoring settings, and PVQV options. In these cases, there will be a prompt asking the user to decide which identifier to use in the auxiliary file.

Loading Auxiliary Files SUBDATA Sections Using Labels

The various SUBDATA sections that represent references to other objects can also be read using labels. Examples include contingencies, interfaces, injection groups, post power flow solution actions, and owners. When reading a SUBDATA section such as this, PowerWorld makes no assumption ahead of time about what identification was used to write this SUBDATA section. Instead, an order of precedence for the identification is as follows

	Identification	Explanation	Example
1	Key Fields	assumes that the strings represent Key Fields	BRANCH 8 9 1
2	Secondary Key Fields	assumes that the strings represent Secondary Key Fields	BRANCH Eight_138 Nine_230 1
3	Labels for component objects	the key/secondary key fields for some objects consist of references to other objects. An example of this is the BRANCH object that is described by the From Bus, To Bus, and Circuit ID. This assumes that labels of the component objects are used.	BRANCH Label8 Label9 1
4	Labels	Assumes that the string represents one of the Labels of the object	BRANCH LabelForBranch

Special Use of Labels in SUBDATA

There are a few special cases where objects have fields that identify other devices. These devices can be identified by label but not in the conventional means because the label field applies to the object that contains the device and a SUBDATA section is not necessary. These special cases include: (Note all fields given below are by variable name because the use of labels is most relevant with auxiliary files.)

ATC Scenarios

ATC Scenario change records usually contain primary key fields to identify the devices that should be adjusted during the scenario. If using labels, these primary key fields will be replaced with a single Label field. The use of this field is different because the Label field refers to the device in the change record and not to the change record itself. When labels are used with ATC scenarios, device labels only can be used. Bus labels cannot be used to identify devices for which no label exists but a bus label does.

ATC Extra Monitors

ATC Extra Monitors identify either branches or interfaces to monitor during the ATC analysis. These devices are identified in the WhoAml field of ATC Extra Monitor records. Usually, the WhoAml field is a special format that contains key field tags. Optionally, this field can use the label of the device for the

extra monitor. If the device label is not available, the standard format will be used. There is no option to use bus labels if they exist and the device labels do not.

Model Conditions

Devices in Model Conditions are usually identified by the WhoAml field which is in a special format that contains key field tags. Optionally, this field can use the label of the device. If the device label does not exist, the standard format will be used. There is no option to use bus labels if they exist and the device labels do not.

Model Expressions

Model Expressions contain Model Fields. Model Fields are identified by the WhoAml fields in the Model Expressions. Usually, the WhoAml fields are in a special format that contains key field tags. Optionally, these fields can use the label of the device associated with the Model Field. If the device does not exist, the standard format will be used. There is no option to use bus labels if they exist and the device labels do not.

Bus Load Throw Over Records

Bus Load Throw Over Records are used with contingency analysis. These records have an option to identify the bus to which the load will be transferred by either number or name_kV combination. If choosing to identify objects by label, the BusName_NomVolt:1 field will contain the label of the bus instead of the name_kV combination. Bus Load Throw Over Records will be saved in an auxiliary file if choosing to Save settings on the Contingency Analysis dialog.

Injection Group Participation Points

All participation points and the injection groups to which they belong can be listed on the Injection Group Display. Load, generator, bus, and shunt devices that can be assigned to a participation point must be identified by bus and ID. The bus can be identified by either the number or name. When identifying by name, the BusName_NomVolt field is used to provide the name_kV combination for the bus. If choosing to identify devices by label, this field instead will contain the label of the device. If the device does not have a label but the bus does, the bus label will be used instead in conjunction with the ID of the device. Even if the device does contain a label, the ID field must be included in any auxiliary file that is going to be loaded because it is a key field. Injection groups can be included in other injection groups. Injection groups can be identified by label, even though this is not a normal thing to do. If any injection groups have labels and these injection groups are included in other injection groups, their labels will also appear in the BusName_NomVolt field. If they do not have labels, they will be identified by the injection group name that appears in the PPntID field.

SubData Sections

The format described thus far works well for most kinds of data in Simulator. It does not work as well however for data that stores a list of objects. For example, a contingency stores some information about itself (such as its name), and then a list of contingency elements, and possibly a list of limit violations as well. For data such as this, Simulator allows `<SubData>`, `</SubData>` tags that store lists of information about a particular object. This formatting looks like the following

```
object_type (list_of_fields)
{
  value_list_1
    <SUBDATA subobject_type1>
      precise format describing an object_type1
      precise format describing an object_type1
      .
      .
      .
    </SUBDATA>
    <SUBDATA subobject_type2>
      precise format describing an object_type2
      precise format describing an object_type2
      .
      .
      .
    </SUBDATA>
  value_list_2
    .
    .
    .
  value_list_n
}
```

Note that the information contained inside the `<SubData>`, `</SubData>` tags may not be flexibly defined. It must be written in a precisely defined order that will be documented for each SubData type. The description of each of these SubData formats follows.

ATC Options

RLScenarioName

GScenarioName

IScenarioName

These three sections contain the pretty names of the RL Scenarios, G Scenarios, and I Scenarios. Each line consists of two values: Scenario Number and a name string enclosed in quotes.

Scenario Number : The scenarios are number 0 through the number of scenarios minus 1.
Scenario Name : These represent the names of the various scenarios.

Example:

```
<SUBDATA RLScenarioName>
//Index Name
  0 "Scenario Name 0"
  1 "Scenario Name 1"
</SUBDATA>
```

ATCMemo

This section contains the memo text for the ATC analysis.

Example:

```
<SUBDATA ATCMemo>
//Memo
"Comments for the ATC analysis"
</SUBDATA>
```

ATCExtraMonitor

ATCFlowValue

This subdata section contains a list of a flow values for specified transfer levels. Each line consists of two values: Flow Value (flow on the monitored element) and a Transfer Level (in MW).

Flow Value : Contains a string describing which monitor this belongs to.
Transfer Level : Contains the value for this extra monitor at the last linear iteration.

Example:

```
<SUBDATA ATCFlowValue>
//MWFlow TransferLevel
  94.05 55.30
  105.18 80.58
  109.02 107.76
</SUBDATA>
```

ATCScenario

TransferLimiter

This subdata section contains a list of the TransferLimiters for this scenario. Each line contains fields relating one of the Transferlimiters. The fields are written out in the following order:

- Limiting Element : Contains a description of the limiting element. The possible values are:
"PowerFlow Divergence"
"AREA num"
"SUPERAREA name"
"ZONE num"
"BRANCH num1 num2 ckt"
"INJECTIONGROUP name"
"INTERFACE name"
- Limiting Contingency : The name of the limiting contingency. If blank, then this means it's a limitation in the base case.
- MaxFlow : The transfer limitation in MW in per unit.
- PTDF : The PTDF on the limiting element in the base case (not in percent).
- OTDF : The OTDF on the limiting element under the limiting contingency.
- LimitUsed : The limit which was used to determine the MaxFlow in per unit.
- PreTransEst : The estimated flow on the line after the contingency but before the transfer in per unit.
- MaxFlowAtLastIteration : The total transfer at the last iteration in per unit.
- IterativelyFound : Either YES or NO depending on whether it was iteratively determined.

Example:

```
<SUBDATA TransferLimiter >
  "BRANCH 40767 42103 1" "contin" 2.84 -0.0771 -0.3883 -4.35 -4.35 -0.01 "-55.88"
YES
  "BRANCH 42100 42321 1" "Contin" 4.42 0.1078 0.5466 6.50 5.64 1.57 " 22.59" NO
  "BRANCH 42168 42174 1" "Contin" 7.45 -0.0131 -0.0651 -1.39 -1.09 4.60 "-33.31" NO
  "BRANCH 42168 42170 1" "Contin" 8.54 0.0131 0.0651 1.39 1.02 5.69 " 26.10" NO
  "BRANCH 41004 49963 1" "Contin" 9.17 -0.0500 -0.1940 -4.39 -3.16 6.32 " 68.73" NO
  "BRANCH 46403 49963 1" "Contin" 9.53 0.0500 0.1940 4.46 3.16 6.68 "-68.68" NO
  "BRANCH 42163 42170 1" "Contin" 10.14 -0.0131 -0.0651 -1.39 -0.92 7.29 "-15.58" NO
</SUBDATA>
```

ATCExtraMonitor

This subdata section contains a list of the ATCExtraMonitors for this scenario. Each line contains three fields relating one of the ATCExtraMonitors. The first field describes the ATCExtraMonitor which this subdata corresponds to. The second and third variables are the initial value and sensitivity for this extra monitor for the scenario. An optional fourth field may be included if we are using one of the iterated ATC solution options. This field must be the String "ATCFlowValue".

- Monitor Description : Contains a string describing which monitor this belongs to.
- InitialValue : Contains the value for this extra monitor at the last linear iteration.
- Sensitivity : Contains the sensitivity of this monitor.
- ATCFlowValue : A string which signifies that a block will follow which stores a list of flow values for specified transfer levels. Each line of the block consists of two values: Flow Value (flow on the monitored element) and a Transfer Level (in MW). The block is terminated when a line of text that starts with 'END' is encountered.

Example:

```
<SUBDATA ATCEXtraMonitor>
  "Interface<KEY1>Left-Right</KEY1>"          40.0735 0.633295
  "Branch<KEY1>2</KEY1><KEY2>5</KEY2><KEY3>1</KEY3>" 78.7410 0.266589
</SUBDATA>
```

AUXFileExportFormatData

DataBlockDescription

This subdata section is used to define the objects that should be included in an auxiliary file along with their fields, subdata sections, and any filter used to specify which objects should be included. Each line contains the following:

- | | |
|---------------|---|
| ObjectType | : Name of the object to include in the auxiliary file. |
| [FieldList] | : List of fields to include. Must be enclosed in brackets. This list can either be space-delimited or comma-delimited. |
| [SubdataList] | : List of subdata sections to include. This list must be enclosed in brackets and can be either space-delimited or comma-delimited. Include empty brackets to not include subdata or for objects that do not have any subdata sections. |
| "Filter" | : Description of the filter to use for determining which objects to include. This must be enclosed in double quotes. If no filter is to be used, empty double quotes should be included. Valid entries are: "", "filename", "AREAZONE", and "SELECTED". See the Using Filters in Script Commands section for more information on specifying the filename. |

Example:

```
<SUBDATA DataBlockDescription>
  // ObjectType [FieldList] [SubdataList] "Filter"
  Area [AreaName, AreaNum] [] "SELECTED"
  Gen [BusNum, BusName, GenID] [BidCurve, ReactiveCapability] ""
</SUBDATA>
```

AUXFileExportFormatDisplay

DataBlockDescription

Same format as for the AUXFileExportFormatData subdata section.

Example:

```
<SUBDATA DataBlockDescription>
  // ObjectType [FieldList] [SubdataList] "Filter"
  DisplayArea [AreaName, AreaNum, SOAuxiliaryID] [] ""
  DisplayTransmissionLine [BusNum, BusNum:1, LineCircuit, SOAuxiliaryID]
    [Line] "Nominal Voltage > 138 kV"
</SUBDATA>
```

BGCalculatedField

Condition

Calculated Fields allow you to define a calculation over most network and aggregation objects along with a few other types of objects. The calculation can then be used to show an aggregation calculation on objects that link to these calculation objects in some manner. Part of the definition is a filter which specifies which objects to operate over. This subdata section is identical to the Condition subdata section of the Filter object type.

Bus

MWMarginalCostValues

MvarMarginalCostValues

LPOPFMarginalControls

These three sections contain specific values computed for an OPF solution. In MWMarginalCostValues or MvarMarginalCostValues these specific values are the MW or Mvar marginal prices for each constraint. In LPOPFMarginalControls the values are the sensitivities of the controls with respect to the cost of each bus.

Example:

```
<SUBDATA MWMarginalCostValues>
//Value
16.53
0.00
21.80
</SUBDATA>
```

BusViewFormOptions

BusViewBusField

BusViewFarBusField

BusViewGenField

BusViewLineField

BusViewLoadField

BusViewShuntField

The values represent specific fields on the custom defined bus view onelines. Each line contains two values:

- | | | | | | | | | | | | | | | | | | |
|------------------|---|---|---------------|---|---|-------------|---|---|----------------|---|---|-------|---|---|-----------------|---|--|
| Location | : | The various locations on the customized bus view contain slots for fields. This is the slot number. | | | | | | | | | | | | | | | |
| FieldDescription | : | This is a string enclosed in double quotes. The string itself is delimited by the @ character. The string contains five values: <table border="0"><tr><td>Name of Field</td><td>:</td><td>The name of the field. Special fields that appear on dialog by default have special names. Otherwise these are the same as the fieldnames of the AUX file format (for the "other fields" feature on the dialogs).</td></tr><tr><td>Total Digit</td><td>:</td><td>Number of total digits for a numeric field.</td></tr><tr><td>Decimal Points</td><td>:</td><td>Number of decimal points for a numeric field.</td></tr><tr><td>Color</td><td>:</td><td>This is the color of the field. It is not presently used.</td></tr><tr><td>Increment Value</td><td>:</td><td>This is the "delta per mouse" click for the field.</td></tr></table> | Name of Field | : | The name of the field. Special fields that appear on dialog by default have special names. Otherwise these are the same as the fieldnames of the AUX file format (for the "other fields" feature on the dialogs). | Total Digit | : | Number of total digits for a numeric field. | Decimal Points | : | Number of decimal points for a numeric field. | Color | : | This is the color of the field. It is not presently used. | Increment Value | : | This is the "delta per mouse" click for the field. |
| Name of Field | : | The name of the field. Special fields that appear on dialog by default have special names. Otherwise these are the same as the fieldnames of the AUX file format (for the "other fields" feature on the dialogs). | | | | | | | | | | | | | | | |
| Total Digit | : | Number of total digits for a numeric field. | | | | | | | | | | | | | | | |
| Decimal Points | : | Number of decimal points for a numeric field. | | | | | | | | | | | | | | | |
| Color | : | This is the color of the field. It is not presently used. | | | | | | | | | | | | | | | |
| Increment Value | : | This is the "delta per mouse" click for the field. | | | | | | | | | | | | | | | |

Example:

```
<SUBDATA BusViewLineField>
0 "MW Flow@6@1@0@0"
1 "MVar Flow@6@1@0@0"
2 "MVA Flow@6@1@0@0"
3 "BusAngle:1@6@2@0@0"
</SUBDATA>
```

ColorMap

ColorPoint

A colorpoint is simply described by a real number (between 0 and 100) indicating the percentage breakpoint, an integer describing the color, and a field indicating if the color should be used or the contour should be transparent. These three values are written on a single line of text. Each line contains two values:

cmvalue	: Real number between 0 and 100 (minimum to maximum value).
cmcolor	: Integer between 0 and 16,777,216. Value is determined by taking the red, green and blue components of the color and assigning them a value between 0 and 255. The color is then equal to red + 256*green + 256*256*blue.
cmalpha	: Integer between 0 and 255, where only 0 and 255 are valid values. A value of 0 indicates that the color point is transparent, while a value of 255 indicates that the color point is opaque. If the alpha channel is omitted, a default value of 255 (opaque) will be assigned.

Example:

```
<SUBDATA ColorPoint>
// Value Color Alpha
100.0000 127 255
62.5000 65535 255
50.0000 8388479 0
12.5000 16711680 0
0.0000 8323072 255
</SUBDATA>
```

Contingency

CTGElementAppend

Normally when reading in contingency definitions, the CTGElement SubData section is used to define the list of elements. When reading a CTGElement SubData section, all existing elements of the contingency are deleted and replaced with the ones read from the file. Using the CTGElementAppend as the SubData section will modify this behavior so that the elements are appended to the existing ones instead of deleted.

CTGElement

A contingency element is described by up to the following entries. All entries must be on a single line of text:

Action	: String describing the action associated with this element. See below for actions available.
ModelCriteria	: This is the name of a ModelFilter or ModelCondition under which this action should be performed. This entry is optional. If it is not specified, then a blank (or no criteria) is assumed. If you want to enter a Status, then use must specify "" as the ModelCriteria.

Status	: The following options are available: CHECK : perform action if ModelCriteria is true ALWAYS : perform action regardless of ModelCriteria NEVER : do not perform action TOPOLOGYCHECK: perform action if ModelCriteria is true following implementation of other actions and before solving the power flow POSTCHECK : perform action if ModelCriteria is true following implementation of other actions and solving the power flow SOLUTIONFAIL : perform the action if ModelCriteria is true or not defined following the failure of the power flow solution.
InclusionFilter	This entry is optional. If it is not specified, then CHECK is assumed. : This entry is optional and will only exist for elements of RemedialAction or GlobalContingencyActions objects. This is the name of an advanced filter or device filter that gets applied to each contingency. If the contingency meets the filter, that contingency will include this element. Otherwise, the element will be ignored.
TimeDelay	: This entry is optional. If not specified, 0 is assumed. This entry will only exist for elements of Contingency, RemedialAction, or GlobalContingencyActions objects. This is the time delay in seconds to wait before the action takes place.
Persistent	: This entry is optional. If not specified, NO is assumed. Normally after a contingency action has been implemented it will not be applied again. Setting this option to YES to mark an action as persistent will change this behavior. Any action marked as persistent that also has a Status of TOPOLOGYCHECK, POSTCHECK, or SOLUTIONFAIL will be applied in the appropriate section of the overall contingency process any time that its ModelCriteria is met. An exception is that a SOLUTIONFAIL element will only remain persistent until a solution is successfully achieved.
ArmingCriteria	: This entry is optional and will only exist for elements of RemedialAction objects. This is the name of a ModelFilter or ModelCondition under which this action should be armed. If it is not specified, then a blank (or no criteria) is assumed. If you want to enter a ArmingStatus, then use must specify "" as the ArmingCriteria.
ArmingStatus	: This entry is optional and will only exist for elements of RemedialAction objects. If not specified, CHECK is assumed. The following options are available: CHECK : action is armed if ArmingCriteria is true ALWAYS : action is considered armed regardless of ArmingCriteria NEVER : action is not armed
Comment	: All text to the right of the comment symbol (/) will be saved with the CTGElement as a comment.

Possible Actions:

Many actions have a value field that can be specified. This value can be expressed in three ways:

1. A numerical value that will be used directly.
2. The variable name of a field for the object in the action preceded by the tag <Field>. This field will be evaluated and that value will be used. Including the keyword REF in the appropriate place in the action string will cause the field to be evaluated in the contingency reference case. Otherwise, the field will be evaluated at the moment the action is implemented.

- The name of a Model Expression preceded by the tag <Expression>. Single quotes should enclose the entirety of the tag and the name if the name contains spaces. The model expression will be evaluated and the result will be used as the value. Including the keyword REF in the appropriate place in the action string will cause the model expression to be evaluated in the contingency reference case. Otherwise, the model expression will be evaluated at the moment the action is implemented.

Transmission Line or Transformer outage or insertion

BRANCH	bus1# bus2# ckt	OPEN				
		CLOSE				
		OPENCBS				
		CLOSECBS				
		SET_TO	value	LimitMVA	REF	

Takes branch out of service, or puts it in service. The contingency rating of the branch can also be set for the duration of the contingency using the SET_TO action. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the branch.

Generator, Load, or Switched Shunt outage or insertion

GEN	bus# id	OPEN, CLOSE, OPENCBS, or CLOSECBS
LOAD	bus# id	OPEN, CLOSE, OPENCBS, or CLOSECBS
SHUNT	bus# id	OPEN, CLOSE, OPENCBS, or CLOSECBS
INJECTIONGROUP	name	OPEN, CLOSE, OPENCBS, or CLOSECBS

Takes a generator, load, or shunt out of service, or puts it in service. If specifying an injection group, the status of all devices in the injection group will be changed. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the sequence [bus1# ckt] or [name] may be replaced by the label of the device.

Generator, Load or Switched Shunt movement to another bus

For the following set of actions, all of the object types can use the same action keywords which are associated with the value keyword following the actual value. The move is based on specifying a bus:

GEN	bus1#	MOVE_P_TO	bus2#	value	MW	REF
LOAD		MOVE_Q_TO			MVR	
SHUNT						

For the following set of actions, all of the object types can use the same action keywords which are associated with the value keyword following the actual value. The move is based on specifying a particular device:

GEN	bus1# id	MOVE_P_TO	bus2#	value	MW	REF
LOAD		MOVE_Q_TO			MVR	
SHUNT						

Generator actions that move a generator by a percentage only apply to the generator MW:

GEN	bus1#	MOVE_P_TO	bus2#	value	PERCENT	REF
GEN	bus1# id	MOVE_P_TO	bus2#	value	PERCENT	REF

The following set of actions are used for specifying a load move by maintaining a constant power factor:

LOAD	bus1#	MOVE_PQ_TO	bus2#	value	MW	REF
LOAD	bus1# id	MOVE_PQ_TO	bus2#		MW	

The following set of actions apply to loads and shunts and are used to move a percentage of the entire MW and Mvar output. The move is based on specifying a bus:

LOAD	bus1#	MOVE_PQ_TO	bus2#	value	PERCENT	REF
SHUNT						

The following set of actions apply to loads and shunts and are used to move a percentage of the entire MW and Mvar output. The move is based on specifying a particular device:

LOAD	bus1# id	MOVE_PQ_TO	bus2#	value	PERCENT	REF
SHUNT						

Use to move generation, load or shunt at a bus1 over to bus2. This can be used on a bus or specific device basis in specifying what to move. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus. When identifying specific devices, the device label can replace the bus number and device id.

Generator, Load or Switched Shunt set or change a specific value

For the following set of actions, all of the object types can use the same action keywords which are associated with the value keyword following the actual value. These changes are based on specifying a bus:

GEN	bus#	SET_P_TO	value	MW	REF
LOAD		SET_Q_TO		MVR	
SHUNT		CHANGE_P_BY		MW	
		CHANGE_Q_BY		MVR	

For the following set of actions, all of the object types can use the same action keywords which are associated with the value keyword following the actual value. These changes are based on specifying a bus:

GEN	bus# id	SET_P_TO	value	MW	REF
LOAD		SET_Q_TO		MVR	
SHUNT		CHANGE_P_BY		MW	
		CHANGE_Q_BY		MVR	

The following set of actions are used to set or change the MW output of generation at a bus by a percentage:

GEN	bus#	SET_P_TO	value	PERCENT	REF
GEN		CHANGE_P_BY			

The following set of actions are used to set or change the MW output of a particular generator by a percentage:

GEN	bus# id	SET_P_TO	value	PERCENT	REF
GEN		CHANGE_P_BY			

The following set of actions apply to loads and shunts and are used to set or change a percentage of the entire MW and Mvar output. This based on specifying a bus:

LOAD	bus#	SET_PQ_TO	value	PERCENT	REF
SHUNT		CHANGE_PQ_BY			

The following set of actions apply to loads and shunts and are used to set or change a percentage of the entire MW and Mvar output. This based on specifying a specific device:

LOAD	bus# id	SET_PQ_TO	value	PERCENT	REF
SHUNT		CHANGE_PQ_BY			

The following set of actions are used to specify a load set or change by maintaining a constant power factor. This is based on specifying a bus:

LOAD	bus#	SET_PQ_TO	value	MW	REF
		CHANGE_PQ_BY		MW	

The following set of actions are used to specify a load set or change by maintaining a constant power factor. This is based on specifying a specific load:

LOAD	bus# id	SET_PQ_TO	value	MW	REF
		CHANGE_PQ_BY		MW	

The following set of actions apply to generators and shunts and are used to set or change the setpoint voltage of the devices at the specified bus:

GEN	bus#	SET_VOLT_TO	value	PU	REF
SHUNT		CHANGE_VOLT_BY			

The following set of actions apply to generators and shunts and are used to set or change the setpoint voltage of the specified device:

GEN	bus# id	SET_VOLT_TO	value	PU	REF
SHUNT		CHANGE_VOLT_BY			

Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus. When identifying specific devices, the device label can replace the bus number and device id.

Bus outage causes all lines connected to the bus to be outage

BUS	bus#	OPEN
		OPENCBS

Takes all branches connected to the bus out of service. Also outages all generation, load, or shunts attached to the bus. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus.

Interface outage or insertion

INTERFACE	name	OPEN
		CLOSE
		OPENCBS
		CLOSECBS

Takes all monitored branches in the interface out of service, or puts them all in service. Open actions will also open all generators and loads contained in the interface including generators and loads inside any injection groups or other interfaces. Note: the [name] may be replaced by the label of the interface.

Interface change specific value

INTERFACE	name	CHANGE_P_BY	value	Option	REF	PPREF
		SET_P_TO				

The following Option settings are allowed to set or change the MW flow of an interface by or to a particular value:

MWMERITORDEROPEN

- : Value will be interpreted as the amount of MW flow change or new MW flow. The element in the interface with the highest participation factor will be opened, followed by the second generator and so on. This will continue until the amount of MW flow opened is as close to the desired amount as possible without exceeding the amount of MW flow open. If an element will cause a change in flow that is not in the desired direction, that element is not opened and the next element is examined.

PERCENTMERITORDEROPEN or %MERITORDEROPEN

- : Same as MWMERITORDEROPEN except that the value will be interpreted as percentage of the contingency reference state MW flow.

MWMERITORDEROPENEXCEED

- : Same as MWMERITORDEROPEN except that the amount of MW opened is allowed to exceed the desired amount of change. Interface elements will be opened in merit order until the desired amount is met or exceeded.

PERCENTMERITORDEROPENEXCEED or %MERITORDEROPENEXCEED

- : Same as MWMERITORDEROPENEXCEED except that the value will be interpreted as percentage of the contingency reference state MW injection.
- MWEFFECTOPEN**
- : Value that is specified with the action is the desired MW Effect that the action should have. The participation factors defined with the interface elements will be interpreted as effectiveness factors akin to transfer distribution factors. These factors are supplied as input by the user when defining the interface. The effectiveness factors are multiplied by the present MW flow of elements in the interface to determine how much effect they will have if dropped. The flow of an element is determined by its MW flow multiplied by the Weighting factor specified with the element. If the effect of a particular element is in the opposite direction of the desired effect, that element is skipped. The action will find the smallest number of elements to drop that results in a total MW Effect that is within 5% of the desired MW Effect, but does not exceed the desired MW Effect.
- This option is not valid with SET_P_TO actions.
- MWEFFECTOPENEXCEED**
- : Same as MWEFFECTOPEN but it will ensure that the total MW Effect is within 5% of the total desired MW Effect and also meets or exceeds the desired MW Effect.
- This option is not valid with SET_P_TO actions.
- MWBESTFITOPEN**
- : Value will be interpreted as the amount of MW flow change or new MW flow. When using this option the participation factors of interface elements do not impact which elements are opened. The flow on an element, which is determined by its MW flow multiplied by the weighting factor specified with the element, is used to determine which elements should be opened. If the follow on an element is in the appropriate direction to achieve the desired flow change on the interface, that element is eligible fo being opened. To determine if an element will actually be opened, the best fit algorithm attempts to determine the combination of elements that will achieve the desired flow change by opening the least amount of elements and achieving an actual flow change within 5% fo the desired flow change without exceeding the desired amount.
- MWBESTFITOPENEXCEED**
- : Same as MWBESTFITOPEN except that the MW flow change is allowed to exceed the desired amount of change.
- PERCENTBESTFITOPEN or %BESTFITOPEN**
- : Same as MWBESTFITOPEN except that the value will be interpreted as percentage of the contingency reference state MW flow.
- PERCENTBESTFITOPENEXCEED or %BESTFITOPENEXCEED**
- : Same as PERCENTBESTFITOPEN except that the MW flow change is allowed to exceed the desired amount of change.

When using an action that requires participation factors, an optional parameter PPREF can be specified. This indicates that the participation factors should be determined in the contingency reference case.

Interfaces can contain other interfaces. The treatment of interfaces within interfaces is to open the entire contained interface when using the MWMERITORDEROPEN type actions.

Notes: The [name] may be replaced by the label of the interface.

Line Shunt outage or insertion

LINESHUNT	bus1# bus2# bus# ckt	OPEN
	CLOSE	

Takes a line shunt out of service, or puts it in service. bus1# and bus2# identify the line that the line shunt is on and bus# identifies the side of the line that the line shunt is on. bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. bus# values may also be replaced by a string enclosed in single quotes that represents the label of the bus. The sequence [bus1# bus2#] may be replaced by the label of the line to which the line shunt is attached.

Injection Group outage or insertion

INJECTIONGROUP	name	OPEN			
	CLOSE	value	REF	PPREF	
	OPENCBS				
	CLOSECBS				
	OPEN	value	REF	PPREF	

Takes all devices in the injection group out of service, or puts them all in service.

The OPEN action will open all devices in the injection group if no value is specified. If a value is specified, only that number of devices will be opened in the order of highest to lowest participation factor. The CLOSE action will close all devices in the injection group if no value is specified. If a value is specified, only that number of devices will be closed in the order of highest to lowest participation factor. When using an action that requires participation factors, an optional parameter PPREF can be specified. This indicates that the participation factors should be determined in the contingency reference case. This will only be done for participation points using an AutoCalcMethod that indicates the factor should be dynamically determined and the AutoCalc field is set to YES for the participation point.

The [name] may be replaced by the label of the injection group. Bus participation points will be completely ignored in this process.

Injection Group change specific value

INJECTIONGROUP	name	CHANGE_P_BY	value	Option	REF	PPREF
		SET_P_TO				

The following Option settings are allowed to set or change the MW generation/load in an injection group by or to a particular value:

- MW : Value will be interpreted as the amount of MW injection change or new MW injection. Each participation point in the injection group will be changed in proportion to the participation factors of the group.
- PERCENT or % : Same as MW except that the value will be interpreted as percentage of the contingency reference state MW injection.
- MWMERITORDER : Value will be interpreted as the amount of MW injection change or new MW injection. Both generator and load points will be modified in the injection group. Elements will be adjusted in order of highest participation factor to lowest before moving to the next element. This process continues until the desired injection is met. Generators will not be opened in this process, which means all online generators will continue to provide Mvar support. Loads that have both their minimum and maximum MW limits set to zero will not be allowed to increase. They can only decrease towards 0.
- PERCENTMERITORDER or %MERITORDER : Same as MWMERITORDER except that the value will be interpreted as percentage of the contingency reference state MW injection.

MWMERITORDEROPEN

- : Value will be interpreted as the amount of MW injection change or new MW injection. Both generator and load points can be modified in the injection group. If the MW injection change is negative, the generator in the injection group with the highest participation factor will have its status changed to Open, followed by the second generator and so on. This will continue until the amount of MW opened is as close to the desired amount as possible without exceeding the desired amount of drop. If the MW injection change is positive, loads will be opened in the same manner. If an element would cause the desired gen drop amount to be exceeded, that element is skipped and the next element in merit order is processed. If the change requested is positive and there are no loads in the injection group, generators will be increased toward their maximum MW output in the same manner as MWMERITORDER as though the OPEN option was not specified. If the change requested is negative and there are no generators in the injection group, loads will be increased toward their maximum MW output in the same manner as MWMERITORDER as though the OPEN option was not specified.

PERCENTMERITORDEROPEN or %MERITORDEROPEN

- : Same as MWMERITORDEROPEN except that the value will be interpreted as percentage of the contingency reference state MW injection.

MWMERITORDEROPENEXCEED

- : Same as MWMERITORDEROPEN except that the amount of MW opened is allowed to exceed the desired amount of change. Generators or loads will be opened in merit order until the desired amount is met or exceeded.

PERCENTMERITORDEROPENEXCEED or %MERITORDEROPENEXCEED

- : Same as MWMERITORDEROPENEXCEED except that the value will be interpreted as percentage of the contingency reference state MW injection.

MWEFFECTOPEN

- : Value that is specified with the action is the desired MW Effect that the action should have. The participation factors defined with the Injection Group will be interpreted as effectiveness factors akin to transfer distribution factors. These factors are supplied as input by the user when defining the injection group. The effectiveness factors are multiplied by the present output of generators (or loads) in the injection group to determine how much effect they will have if dropped. The action will find the smallest number of generators (or loads) to drop which results in a total MW Effect that is within 5% of the desired MW Effect, but does not exceed the desired MW Effect.

This option is not valid with SET_P_T0 actions.

MWEFFECTOPENEXCEED

- : Same as MWEFFECTOPEN but it will ensure that the total MW Effect is within 5% of the total desired MW Effect and also meets or exceeds the desired MW Effect.

This option is not valid with SET_P_T0 actions.

MWBESTFITOPEN

- : Value will be interpreted as the amount of MW injection change or new MW injection. When using this option the participation factors do not impact which elements are opened. All generators or loads defined with the injection group can participate if they are online. Specifically which generators or loads depends on an algorithm that attempts to get the actual injection change within 5% of the desired injection change by

opening the smallest number of generators or loads without exceeding the desired amount.

MWBESTFITOPENEXCEED

: Same as MWBESTFITOPEN except that the MW injection change is allowed to exceed the desired amount of change.

PERCENTBESTFITOPEN or %BESTFITOPEN

: Same as MWBESTFITOPEN except that the value will be interpreted as percentage of the contingency reference state MW injection.

PERCENTBESTFITOPENEXCEED or %BESTFITOPENEXCEED

: Same as PERCENTBESTFITOPEN except that the MW injection change is allowed to exceed the desired amount of change.

When using an action that requires participation factors, an optional parameter PPREF can be specified. This indicates that the participation factors should be determined in the contingency reference case. This will only be done for participation points using an AutoCalcMethod that indicates the factor should be dynamically determined and the AutoCalc field is set to YES for the participation point.

Injection Groups can contain participation points that reference another injection group. The treatment of injection groups within injection groups will be to drop the entire contained injection group when using the MWMERITOROPEN and MWEFFECTOPEN type actions.

Notes: The [name] may be replaced by the label of the injection group. Bus participation points will be completely ignored in this process.

Series Capacitor Bypass or Inservice

SERIESCAP	bus1# bus2# ckt	BYPASS
		INSERVICE

Bypasses a series capacitor, or puts it in service. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by and underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the branch. Keyword SERIESCAP may also be replaced with BRANCH, which will allow bypassing or not bypassing any branch and is not limited to series capacitors.

Series Capacitor set impedance

SERIESCAP	bus1# bus2# ckt	SET_X_TO	value	PERCENT	REF
				PU	

Changes the impedance a series capacitor either specifying a new per unit value or specifying a percentage of the value in the contingency reference case. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by and underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the branch. Keyword SERIESCAP may also be replaced with BRANCH, which will allow setting the impedance of any branch and is not limited to series capacitors.

DC Transmission or VSC DC Transmission Line outage

DCLINE	bus1# bus2# ckt	OPEN
	OPENCBS	
VSCDCLINE	'Name'	OPEN
	OPENCBS	

Takes DC Line or VSC DC Line out of service. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes

which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the dc transmission line. For the VSC DC Line, the identifiers are replaced simply with the name of the VSCDCLINE instead.

DC Line set a specific value or insertion

DCLINE	bus1# bus2# ckt	SET_P_TO	value	MW	REF
		CHANGE_P_BY		PERCENT	
		SET_I_TO		AMPS	
		CHANGE_I_BY			
		CLOSE			
		CLOSECBS			
VSCDCLINE	'Name'	SET_TO	value	OHMS	REF
		Same options as for the DC Line, except that the AMPS option are not available for VSC			

Use to set the DC Line setpoint to a particular value, or puts it in service. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the dc transmission line. (Note: for the CLOSE and CLOSECBS choice, only the units of MW or AMPS may be used.) For the VSC DC Line, the identifiers are replaced simply with the name of the VSCDCLINE instead.

MTDC Converter outage

DCCONVERTER	rec# bus#	OPEN
		OPENCBS

Takes multi-terminal DC converter out of service. The rec# specifies the multi-terminal DC line record, while bus# specifies the AC bus to which the converter is connected. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in single quotes which represents the label of the bus.

MTDC Converter set a specific value or insertion

DCCONVERTER	rec# bus#	SET_P_TO	value	MW	REF
		CHANGE_P_BY		PERCENT	
		SET_I_TO		AMPS	
		CHANGE_I_BY			
		CLOSE			
		CLOSECBS			

Use to set the multi-terminal DC converter setpoint to a particular value, or puts it in service. The rec# specifies the multi-terminal DC line record, while bus# specifies the AC bus to which the converter is connected. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string enclosed in single quotes which represents the label of the bus. (Note: for the CLOSE and CLOSECBS choice, only the units of MW or AMPS may be used.)

Phase Shifter set a specific value

PHASESHIFTER	bus1# bus2# ckt	SET_P_TO	value	MW	REF
		CHANGE_P_BY		PERCENT	
		SET_TO	value	DEG	REF
		CHANGE_BY			

Use the MW and PERCENT options to change or set the middle of the phase shifter MW regulation range to the specified value. Use the DEG option to change or set the phase shift angle in degrees to a particular value.

Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may

also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# ckt] may be replaced by the label of the branch.

Keyword PHASESHIFTER may also be replaced with BRANCH. If the branch is not a phase shifter, no change will be made.

3-Winding Transformer outage or insertion

3WXFORMER	bus1# bus2# bus3# ckt	OPEN	
		CLOSE	
		OPENCBS	
		CLOSECBS	

Takes all three windings of a 3-winding transformer out of service, or puts them in service. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string enclosed in single quotes which represents the label of the bus. Also, the entire sequence [bus1# bus2# bus#3 ckt] may be replaced by the label of the three winding transformer.

Area Control Type Change

AREA	area#	SET_TO	'OFF'
			'PARTFAC'
			'AREASLACK bus#'
			'IGSLACK injectiongroup name'

Specify to change the make-up power for an area so that it is different during a contingency than the area control settings used in the reference case. The area may be set to toggle the control setting to OFF, PARTFAC, AREASLACK, and IGSLACK. The Area Control topic provides more information about these control types. If selecting Area Slack is chosen, then a bus must be specified which will act as the area slack during the contingency action. If selecting IG Slack, then an injection group must be specified by name.

Note: bus# values may be replaced by a string where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. Note: bus# values may also be replaced by a string which represents the label of the bus.

In order for the Area contingency action to work correctly, there are contingency and power flow solution options that must be set correctly. Simulator does not automatically set these options so the user must make sure they are set.

- Area control must be enabled in the contingency base case, i.e. the Power Flow Solution Option for Island-Based AGC must be set to Disable (Use the Area and Super Area Dispatch settings).
- The contingency Make-Up Power option must be set to Same as Power Flow case.
- The option to Disable Automatic Generation Control (AGC) found with the Power Flow Solution Options must NOT be selected.

Another suggestion, although not a strict requirement, is that the area should be on area control prior to contingency analysis if a control type other than Off AGC is going to be set during a contingency. If a large ACE exists in the base case with area control off, switching the area on control during the contingency will zero out the ACE in addition to compensating for required make-up power.

Substation outage

SUBSTATION	sub#	OPEN			
		OPENCBS			
		SET_P_TO	value	MW	REF
		CHANGE_P_BY		PERCENT	

OPEN and OPENCBS will take a substation out of service. The set and change actions will set the MW output of online generators in the substation to the specified value. sub# is the number that identifies the

substation. sub# can be replaced by a string enclosed in single quotes where the string is the name or label of the substation.

Abort

ABORT

Include this action to cause the solution of the contingency to be aborted.

Execute a Power Flow Solution

SOLVEPOWERFLOW

Include this action to cause the solution of the contingency to be split into pieces. Actions that are listed before each SOLVEPOWERFLOW call will be performed as a group.

Calling of a name ContingencyBlock

CONTINGENCYBLOCK | name

Calls a ContingencyBlock and executes each of the actions in that block.

Make-Up Power Compensation

Only valid immediately following a SET, CHANGE, OPEN or CLOSE action on a Generator, Shunt or Load. This describes how the change in MW or MVAR are picked up by buses throughout the system. The values specify participation factors. Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by and underscore character and then the nominal voltage of the bus.

```
COMPENSATION
bus#1    value1
bus#2    value2
...
END
```

Example:

```
<SUBDATA CTGElement>
// just some comments
// action
"BRANCH 40821 40869 1 OPEN" "" Model Criteria Status TimeDelay comment
"GEN 45041 1 OPEN" "" ALWAYS 0 //Raver - Paul 500 kV
"BRANCH 42702 42727 1 OPEN" "Line X Limited" CHECK 0 //Open Fern Hill
"GEN 40221 1 OPEN" "Interface L1" CHECK 0 //Drop ~600 MW
"GEN 40227 1 OPEN" "Interface L2" CHECK 0 //Drop ~1200 MW
"GEN 40221 1 OPEN" "Interface L3" CHECK 0 //Drop ~600 MW
</SUBDATA>
```

Note: ContingencyElement object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the name of the contingency to which the ContingencyElement belongs. The Action string will remain the same.

LimitViol

A LimitViol is used to describe the results of a contingency analysis run. Each Limit Violation lists nine possible values:

ViolType	: One of six values describing the type of violation. BAMP : branch amp limit violation BMVA : branch MVA limit violation VLOW : bus low voltage limit violation VHIGH : bus high voltage limit violation INTER : interface MW limit violation CUSTOM : Custom Monitor value
ViolElement	: This field depends on the ViolType. for VLOW, VHIGH : "bus1#" or "busname_buskv" or "buslabel" for INTER : "interfacename" or "interfacelabel" for BAMP, BMVA : "bus1# bus2# ckt violationbus# MWFlowDirection" violationbus# is the bus number for the end of the branch which is violated MWFlowDirection is the direction of the MW flow on the line. Potential values are "FROMTO" or "TOFROM". Note: each bus# may be replaced with the name underscore nominal kV string enclosed in single quotations. Or bus# values may be replaced by a string enclosed in single quotes representing the label of the bus. Also the entire sequence [bus1# bus2# ckt] may be replaced by the label of the branch. for CUSTOM : "custommonitorname deviceidentifier" where the deviceidentifier will use the key fields or label as specified by the option selected when saving
Limit	: This is the numerical limit which was violated.
ViolValue	: This is the numerical value of the violation.
PTDF	: This field is optional. It only makes sense for interface or branch violations. It stores a sensitivity of the flow on the violating element during in the base case with respect to a transfer direction This must be calculated using the Contingency Analysis Other Actions related to Sensitivities.
OTDF	: Same as for the PTDF.
InitialValue	: This stores a number. This stores the base case value for the element which is being violated. This is used to compare against when looking at change violations.
Reason	: This will say whether this was a pure violation, or is being reported as a violation because the change from the base case is higher than a specified threshold. LIMIT : means this is a violation of a line/interface/bus limit or simply a Custom Monitor CHANGE : means this is being reported as a limit because the change from the initial value is higher than allowed
CTG Specified Limit	: This specifies if the Limit originated from a contingency action or from the rating specified with the line and Limit Monitoring Settings. NO : the Limit originated from the line and Limit Monitoring Settings YES : the Limit originated from a contingency action

Example:

```
<SUBDATA LimitViol>
  BAMP "1 3 1 1 FROMTO" 271.94031 398.48096 10.0 15.01 //Note OTDF/PTDF
  // values can also be specified with name underscore nominal kV string
  // enclosed inside a single quote as shown next
  BAMP "'One_138' 'Three_138' 1 1 FROMTO" 271.94031 398.48096 10.0 15.01
  INTER "Right-Top" 45.00000 85.84451 None None 56.000 LIMIT NO
</SUBDATA>
```

ViolationCTG object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the name of the contingency to which the ViolationCTG belongs.

Sim_Solution_Options

These describe the power flow solution options which should be used under this particular contingency. The format of the subdata section is two lines of text. The first line is a list of the fieldtypes for Sim_Solution_Options which should be changed. The second line is a list of the values. Note that in general, power flow solution options are stored at three different locations in contingency analysis. When implementing a contingency, Simulator gives precedence to these three locations in the following order:

1. Contingency Record Options (stored with the particular contingency).
2. Contingency Tool Options (stored with CTG_Options).
3. The global solution options.

WhatOccurredDuringContingency

Each line of this subdata section is part of a text description of what actually ended up being implemented for this contingency. This will list which actions were executed and which actions ended up being skipped because of their model criteria. Each line of the subdata section must be enclosed in quotes.

Example:

```
<SUBDATA WhatOccurredDuringContingency>
  "Applied: "
  " OPEN Branch Two (2) TO Five (5) CKT 1 | | CHECK | | ELEMENT"
</SUBDATA>
```

ContingencyMonitoringException

Each line of this subdata section contains a string identifying a specially handled monitored element for this contingency followed by a string indicating how this monitored element should be handled with this contingency. The elements can be identified by their primary or secondary key fields or by label. The element descriptions should be enclosed in quotes because they contain spaces.

Example:

```
<SUBDATA ContingencyMonitoringException>
  "Branch '2' '3' '1'" "Exclude"
  "Branch 'Three_138.00' 'Four_138.00'" "Include"
  "Branch 'Line_2_5'" "Default"
</SUBDATA>
```

CTG_Options

Sim_Solution_Options

These describe the power flow solution options which should be used under this particular contingency. The format of the subdata section is two lines of text. The first line is a list of the fieldtypes for Sim_Solution_Options which should be changed. The second line is a list of the values. Note that in general, power flow solution options are stored at three different locations in contingency analysis. When implementing a contingency, Simulator gives precedence to these three locations in the following order:

1. Contingency Record Options (stored with the particular contingency).
2. Contingency Tool Options (stored with CTG_Options).

3. The global solution options.

CTGElementBlock

CTGElement

This format is the same as for the Contingency objecttype, however, you cannot call a ContingencyBlock from within a contingencyblock.

CTGElementAppend

When a subdata section is defined as CTGElementAppend rather than CTGElement, the actions of this subdata section will be appended to the contingency actions, instead of replacing them. This format is the same as for the Contingency objecttype, however, you cannot call a ContingencyBlock from within a contingencyblock.

Note: CTGElementBlockElement object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the name of the contingency block to which the CTGElementBlockElement belongs.

CustomColors

CustomColors

These describe the customized colors used in Simulator, which are specified by the user. A custom color is an integer describing a color. Each custom color is written on a single line of text and is an integer between 0 and 16,777,216. The value is determined by taking the red, green, and blue components of the color and assigning them a value between 0 and 255. The color is then equal to $\text{red} + 256 * \text{green} + 256 * 256 * \text{blue}$. Each line contains only one integer that corresponds to the color specified.

Example:

```
<SUBDATA CustomColors>
  9823301
  8613240
</SUBDATA>
```

CustomCaseInfo

ColumnInfo

Each line of this SUBDATA section can be used for specifying the column width of particular columns of the respective Custom Case Information Sheet. The line contains two values – the column and then a column width. This is shown in the following example.

Example:

```
<SUBDATA ColumnInfo>
  "SheetCol" 133
  "SheetCol:1" 150
  "SheetCol:2" 50
</SUBDATA>
```

DataGrid

ColumnInfo

Contains a description of the columns which are shown in the respective data grid. Each line of text contains at least four fields: VariableName, ColumnWidth, TotalDigits, DecimalPoints. The remaining fields are used when showing a Data View based on this DataGrid object. See help website for Data View or the OpenDataView script command for more information about this.

VariableName : Contains the variable which is shown in this column.

ColumnWidth	: The column width.
TotalDigits	: The total digits displayed for numerical values.
DecimalPoints	: The decimal points shown for numerical values.
TabBreak	: Optional. Default to NO. Set to YES to indicate that a new tab should be started immediately before this field.
TabCaption	: Optional. Default to blank string. Specifies a caption for the tabbed control for fields occurring after the Tab Break.
RowBreak	: Optional. Default to NO. Set to YES to indicate that a new row should be started immediately before this field.
RowCaption	: Optional. Default to blank string. Specifies a caption for a group box for the fields occurring after the row break.
ColBreak	: Optional. Default to NO. Set to YES to indicate that a new Column should be started immediately before this field. Also, a special feature for column breaks only is you may specify a number after YES to indicate multiple column breaks to skip over a column. For example "YES 2" to skip a column because there are 2 consecutive column breaks.
RowCaption	: Optional. Default to blank string. Specifies a caption for a group box for the fields occurring after the column break.

Example:

```

DataGrid (DataGridName)
{
  BUS
  <SUBDATA COLUMNINFO>
    BusNomVolt 100 8 2
    AreaNum 50 8 2 "YES" "Tab Caption" "NO" "" "NO" ""
    ZoneNum 50 8 2
  </SUBDATA>
  BRANCHRUN
  <subdata COLUMNINFO >
    BusNomVolt:0 100 8 2
    BusNomVolt:1 100 8 2 "NO" "" "NO" "" "YES 2" "Col Caption"
    LineMW:0 100 9 3 "NO" "" "YES" "Row Caption" "NO" ""
  </SUBDATA>
}

```

ColumnContourInfo

Contains a description of the column contour settings

ColumnNumber	: Contains the column index of the contoured column
UseAbsValue	: Contour the absolute value if YES. If NO then contour the signed value.
IgnoreValuesAbove	: If YES values above the maximum percentage are ignored.
IgnoreValuesBelow	: if YES values below the minimum percentage are ignored.
AbsMin	: The minimum value for the colormap
LimMin	: The break low value for the colormap
Nominal	: The nominal value for the colormap
LimMax	: The break high value for the colormap
AbsMax	: The maximum value for the colormap
ColorMapName	: The name of the color map the contour is using. This must reference a color map that exists in the case.
ColorMapBrightness	: The brightness or color saturation. The values range from -0.8 (darker) to 0.8 (brigher).
ColorMapReverseColors	: If YES, the colors in the colormap will be reversed.

Example:

```
DATA (DataGrid,
[DataGridName,BGDisplayFilter,FilterName,NonDefaultFont,CaseInfoRowHeight,FontName,Font
tStyles,S0FontSize,FontColor,VariableName,ConditionType,ViewZoomLevel,FrozenColumns])
{
"Bus" "YES" "" "YES"      13 "Segoe UI" ""      8      0 "" "High To Low" 100.00 -1
  <SUBDATA ColumnInfo>
    "BusNum"      75 8 2
    "BusName"     75 8 2
    "AreaName"    75 8 2
    "BusNomVolt"  75 8 2
    "BusPUVolt"   75 8 5
    "BusKVVolt"   75 8 3
    "BusAngle"    75 8 2
  </SUBDATA>
  <SUBDATA ColumnContourInfo>
    5 NO NO NO 0.996563732624054 1.01118695735931 1.02581024169922
1.03790521621704 1.05000007152557 "Blue=low, Red=High" 0 NO
    7 NO NO NO -1.17415904998779 0.0616339445114136 1.29742693901062
3.84944224357605 6.4014573097229 "Discrete 20 Red/Blue" 0 NO
  </SUBDATA>
}
```

DynamicFormatting

DynamicFormattingContextObject

This subdata section contains a list of the display object types which are chosen to be selected. Each line of the section consists of the following:

DisplayObjectType (WhichFields) (ListOfFields)

- DisplayObjectType : The object type of the display object. These are generally the same as the values seen in the subdata section SelectByCriteriaSetType of SelectByCriteriaSet object types. The only exception is the string CaseInfo, which is used for formatting applying to the case information displays.
- (WhichFields) : For display objects that can reference different fields, this sets which of those fields it should select (e.g. select only Bus Name Fields). The value may be either ALL or SPECIFIED.
- (ListOfFields) : If WhichFields is set to SPECIFIED, then a delimited list of fields follows.

Example:

```
<SUBDATA DynamicFormattingContextObject>
// Note: CaseInfo applies to case information displays
CaseInfo "SPECIFIED" BusName
DisplayAreaField "ALL"
DisplayBus
DisplayBusField "SPECIFIED" BusName BusPUVolt BusNum
DisplayCircuitBreaker
DisplaySubstation
DisplaySubstationField "SPECIFIED" SubName SubNum BusNomVolt BGLoadMVR
DisplayTransmissionLine
DisplayTransmissionLineField "ALL"
</SUBDATA>
```

LineThicknessLookupMap

LineColorLookupMap

FillColorLookupMap

FontColorLookupMap

FontSizeLookupMap

BlinkColorLookupMap

XoutColorLookupMap

FlowColorLookupMap

SecondaryFlowColorLookupMap

The values of the lookup table for the characteristics that can be modified by the dynamic formatting tool.

The first line contains the two following fields:

fieldname	: It is the field that the lookup table is going to look for.
usediscrete	: Set to YES or NO. If set to YES, the characteristic values will be discrete, meaning that the characteristic value will correspond exactly to the one specified in the table. If set to NO, the characteristic values will be continuous, which means the characteristic value will be an interpolation of the high and low closest values specified in the table.

The following lines contain two fields:

fieldvalue	: The value for the field.
characteristicvalue	: The corresponding characteristic value for such field value.

Example:

```
<SUBDATA LineColorLookupMap>
  // FieldName UseDiscrete
  BusPUVlt YES
  // FieldValue Color
  1.02 16711808
  1.05 8454143
  1.1 16744703
</SUBDATA>
```

Filter

Condition

Conditions store the conditions of the filter. Each condition is described by one line of text which can contain up to five fields:

variablename : It is one of the fields for the object_type specified. It may optionally be followed by a colon and a non-negative integer. If not specified, 0 is assumed.

Example: on a LINE, 0 = from bus, 1 = to bus

sgLineMW:0 = the MW flow leaving the from bus

sgLineMW:1 = the MW flow leaving the to bus

Note: this value may also be the string "_UseAnotherfilter" which would then be followed by either meets or notmeets and then the name of another Filter.

Condition	: Possible Values	Alternate1	Alternate2	Requires othervalue
	between	><		yes
	notbetween	~><		yes
	equal	=	==	
	notequal	<>	~=	
	greaterthan	>		
	lessthan	<		
	greaterthanorequal	>=		
	lessthanorequal	<=		
	about			yes
	notabout			yes
	contains			
	notcontains			
	startswith			
	notstartswith			
	inrange			
	notinrange			
	meets			
	notmeets			
	isblank			
	notisblank			
value	: The value used for comparison. For fields associated with strings, this must be a string. For fields associated with real numbers, this must be a number. For fields associated with integers, this is normally an integer, except when the Condition is "inrange" or "notinrange". In this case, value is a comma/dash separated number string.			
(othervalue)	: If required, the other value used for comparison. For conditions "about" and "notabout" this is the tolerance with which the value should be equal or not equal.			
(FieldOpt)	: Optional string with following meanings. Unspecified means that strings are case insensitive, use number fields directly (older files may have had an integer 0 as well). ABS : strings are case sensitive, take absolute value of field values (older files may have had an integer 1 as well)			

Example:

```
FILTER (objecttype, filtername, filtertype, prefilter)
{
BUS "a bus filter" "AND" "no"
  <SUBDATA CONDITION>
    BusNomVolt > 100
    AreaNum   inrange "1 - 5 , 7 , 90-95"
    ZoneNum   between
  </SUBDATA>
BRANCH "a branch filter" "OR" "no"
  <subdata CONDITION>
    BusNomVolt:0 > 100    // Note location 0 means from bus
    BusNomVolt:1 > 100    // Note location 1 means to bus
    LineMW:0     > 100 1  // Note, final field 1 denotes absolute value
    _UseAnotherFilter meets
  </SUBDATA>
}
```

Gen

BidCurve

BidCurve subdata is used to define a piece-wise linear cost curve (or a bid curve). Each bid point consists of two real numbers on a single line of text: a MW output and then the respective bid (or marginal cost).

Example:

```
<SUBDATA BidCurve>
// MW   Price[$/Mwhr]
100.00  10.6
200.00  12.4
400.00  15.7
500.00  16.0
</SUBDATA>
```

ReactiveCapability

Reactive Capability subdata is used to the reactive capability curve of the generator. Each line of text consists of three real numbers: a MW output, and then the respective Minimum MVAR and Maximum MVAR output.

Example:

```
<SUBDATA ReactiveCapability>
// MW   MinMVAR   MaxMVAR
100.00  -60.00    60.00
200.00  -50.00    50.00
400.00  -30.00    20.00
500.00  - 5.00    2.00
</SUBDATA>
```

Note: ReactiveCapability object types can also be directly created inside their own DATA section as well. Two of the key fields of the object are then the bus number and generator ID of the generator to which the ReactiveCapability point belongs.

GeoDataViewStyle

TotalAreaValueMap

This subdata section is used to define the lookup table for determining the total area size of geographic data view objects based on the value of a selected field. Two values are entered for each mapping:

FieldValue	: Value of the field selected for the Total Area attribute.
TotalArea	: The total area size of the object.

Example:

```
<SUBDATA TotalAreaValueMap>
// FieldValue TotalArea
1.000 0
4.000 23
7.000 46
</SUBDATA>
```

RotationRateValueMap

This subdata section is used to define the lookup table for determining the rotation rate of geographic data view objects based on the value of a selected field. Two values are entered for each mapping:

FieldValue	: Value of the field selected for the Rotation Rate attribute.
RotationRate	: The rotation rate of the object. Entered in Hz.

Example:

```
<SUBDATA RotationRateValueMap>
// FieldValue RotationRate
1.000 0.00
4.000 0.10
7.000 0.20
</SUBDATA>
```

RotationAngleValueMap

This subdata section is used to define the lookup table for determining the rotation angle of geographic data view objects based on the value of a selected field. Two values are entered for each mapping:

FieldValue	: Value of the field selected for the Rotation Angle attribute.
RotationAngle	: The rotation angle of the object. Entered in degrees.

Example:

```
<SUBDATA RotationAngleValueMap>
// FieldValue RotationAngle
1.000 -90.0
4.000 0.0
7.000 90.0
</SUBDATA>
```

LineThicknessValueMap

This subdata section is used to define the lookup table for determining the thickness of the border line around geographic data view objects based on the value of a selected field. Two values are entered for each mapping:

FieldValue	: Value of the field selected for the Line Thickness attribute.
LineThickness	: The line thickness of the border line around the object. This should be an integer value.

Example:

```
<SUBDATA LineThicknessValueMap>
// FieldValue LineThickness
1.000 1
4.000 2
7.000 3
</SUBDATA>
```

GlobalContingencyActions

CTGElementAppend

This format is the same as for the Contingency objecttype except that the SolvePowerFlow action is not allowed.

CTGElement

This format is the same as for the Contingency objecttype except that the SolvePowerFlow action is not allowed.

Note: GlobalContingencyActionsElement object types can also be directly created inside their own DATA section as well.

HintDefValues

HintObject

Stores the values for the custom hints. Each line has one value:

FieldDescription	: This is a string enclosed in double quotes. The string itself is delimited by the @ character. The string contains five values: - Name of Field : The name of the field. Special fields that appear on dialog by default have special names. Otherwise these are the same as the fieldnames of the AUX file format (for the "other fields" feature on the dialogs). - Total Digit : Number of total digits for a numeric field. - Decimal Points : Number of decimal points for a numeric field. - Include Suffix : Set to 0 for not including the suffix, and set to 1 to include it. - Field Prefix : The prefix text.
------------------	--

Example:

```
<SUBDATA HintObject>
  "BusPUVlt@4@1@1@PU Vlt ="
  "BusAngle@4@1@1@Angle ="
</SUBDATA>
```

InjectionGroup

PartPoint

A participation point is used to describe the contents of an injection group. Each participation point lists six values:

PointType	: One of five values describing the type of point. GEN : a generator LOAD : a load SHUNT : a switched shunt BUS : a bus INJECTIONGROUP: another injection group
PointBusNum	: The bus number of the partpoint if the type is a GEN, LOAD, SHUNT, or BUS. Value will be blank for an injection group type. Note: bus# values may be replaced by a string enclosed in double quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. These values may also be replaced by a string enclosed in double quotes that represents the label of the bus or a string representing the label of the generator, load, or switched shunt.
PointID	: For GEN, LOAD, or SHUNT type, this is the id for the partpoint. For an INJECTIONGROUP type, this is the name or label of the injection group. This is blank for a BUS type.
PointParFac	: The participation factor for the point.
ParFacCalcType	: How the participation factor is calculated. There are several options depending on the PointType. Generators : SPECIFIED, MAX GEN INC, MAX GEN DEC, or MAX GEN MW Loads : SPECIFIED or LOAD MW Shunts : SPECIFIED, MAX SHUNT INC, MAX SHUNT DEC, or MAX SHUNT MVAR Bus : SPECIFIED Injection Groups : SPECIFIED

All PointTypes can also set their participation factor based on a field associated with the device. To specify this, the tag <Field> should be followed by the variable name of the field: <Field>variablename. All PointTypes can also set their participation factor based on a Model Expression. To specify this, the tag <Expression> should be followed by the name of the Model Expression: <Expression>ModelExpression.

ParFacNotDynamic : Should the participation factor be recalculated dynamically as the system changes.

Example:

```
<SUBDATA PartPoint>
"GEN" 1 "1" 1.00 "SPECIFIED" "NO"
"GEN" 4 "1" 104.96 "MAX GEN INC" "NO"
"GEN" 6 "1" 50.32 "MAX GEN DEC" "YES"
"GEN" 7 "1" 600.00 "MAX GEN MW" "NO"
"LOAD" 2 "1" 5.00 "SPECIFIED" "NO"
"LOAD" 6 "1" 200.00 "LOAD MW" "YES"
</SUBDATA>
```

Note: PartPoint object types can also be directly created inside their own DATA section as well. One of the key fields of the PartPoint object is then the name of the injection group to which the participation point belongs.

Interface

InterfaceElement

A interfaces's subdata contains a list of the elements in the interface. Each line contains a text descriptions of the interface element. Note that this text description must be encompassed by quotation marks. There are eleven kinds of elements allowed in an interface. Please note that the direction specified in the monitoring elements is important.

"BRANCH num1 num2 ckt"	: Monitor the MW flow on the branch starting from bus num1 going to bus num2 with circuit ckt. (order of bus numbers defines the direction)
"AREA num1 num2"	: Monitor the sum of the AC branches that connect area1 and area2.
"ZONE num1 num2"	: Monitor the sum of the AC branches that connect zone1 and zone2.
"BRANCHOPEN num1 num2 ckt"	: When monitoring the elements in this interface, monitor them under the contingency of opening this branch.
"BRANCHCLOSE num1 num2 ckt"	: When monitoring the elements in this interface, monitor them under the contingency of closing this branch.
"DCLINE num1 num2 ckt"	: Monitor the flow on a DC line.
"INJECTIONGROUP 'name'"	: Monitor the net injection from an injection group (generation contributes as a positive injection, loads as negative).
"GEN num1 id"	: Monitor the net injection from a generator (output is positive injection)
"LOAD num1 id"	: Monitor the net injection from a load (output is negative injection).
"MSLINE num1 num2 ckt"	: Monitor the MW flow on the multi-section line starting from bus num1 going to bus num2 with circuit ckt.
"INTERFACE 'name' "	: Monitor the MW flow on the interface given by name.
"GENOPEN num1 id"	: When monitoring the elements in this interface, monitor them under the contingency of opening this generator.
"LOADOPEN num1 id"	: When monitoring the elements in this interface, monitor them under the contingency of opening this load.

Note: bus# values may be replaced by a string enclosed in single quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus. Labels may also be use as follows.

- bus# values for all element types may be replaced by a string enclosed in single quotes where the string is the label of the bus.
- for GEN or LOAD elements, the section num1 id may be replaced by the device's label.
- For MSLINE, DCLINE, or BRANCH elements, the num1 num2 ckt section may be replaced by the device's label.

For the interface element type BRANCH num1 num2 ckt and DCLINE num1 num2 ckt, an optional field can also be written specifying whether the flow should be measured at the far end. This field is either YES or NO.

Example:

```
<SUBDATA InterfaceElement
  "BRANCH 8 9 1" NO // monitor the flow from bus 8 to bus 9 on this branch

  "BRANCH 12 33 1" YES // monitor the flow from bus 12 to bus 33 on branch
                        // measurefarend is set to true, therefore, we are
                        // monitoring the MW flow that arrives at bus 33
  // the following demonstrates the format when bus names and
  // nominal voltages are used.
  "BRANCH 'Twelve_230' 'name33_230' 1" YES

  "AREA 2 1" // monitor tie line flow from area 2 to area 1
  "ZONE 66 53" // monitor tie lines flows from zone 66 to zone 53
  "BRANCHOPEN 5 6 1" // does monitoring after branch opens
  "BRANCHCLOSE 7 10 1" // does monitoring after branch closes
  "GENOPEN" 55 1" // does monitoring after generator opens
</SUBDATA>
```

Note: InterfaceElement object types can also be directly created inside their own DATA section as well. One of the key fields of the InterfaceElement object is then the name of the interface to which the interface element belongs.

KMLExportFormat

DataBlockDescription

This subdata section is used to describe the objects and fields that should be saved to a KML file. Same format as for the AUXFileExportFormatData subdata section.

LimitSet

LimitCost

LimitCost records describe the piece-wise unenforceable constraint cost records for use by unenforceable line/interface limits in the OPF or SCOPF. Each row contains two values

PercentLimit	: Percent of the transmission line limit.
Cost	: Cost used at this line loading percentage value.

Example:

```
<SUBDATA LimitCost>
  //Percent Cost [$/MWhr]
  100.00 50.00
  105.00 100.00
  110.00 500.00
</SUBDATA>
```

Load

BidCurve

BidCurve subdata is used to define a piece-wise linear benefit curve (or a bid curve). Each bid point consists of two real numbers on a single line of text: a MW output and then the respective bid (or marginal cost). These costs must be increasing for loads.

Example:

```
<SUBDATA BidCurve>
  // MW Price[$/MWhr]
  100.00 16.0
  200.00 15.7
  400.00 12.4
  500.00 10.6
</SUBDATA>
```

LPVariable

LPVariableCostSegment

Stores the cost segments for the LP variables. Each line contains four values:

Cost (Up)	:	Cost associated with increasing the LP variable.
Minimum value	:	Minimum limit of the LP variable.
Maximum value	:	Maximum limit of the LP variable.
Artificial	:	Whether the cost segment is artificial or not.

Example:

```
<SUBDATA LPVariableCostSegment>
//Cost(Up)      Minimum      Maximum Artificial
-20000.0000 -10000000000.5801 -0.6000 YES
  16.2343      -0.6000      0.0000 NO
  16.5526      0.0000      0.6000 NO
  16.8708      0.6000      1.2000 NO
  17.1890      1.2000      1.8000 NO
  17.5073      1.8000      2.4000 NO
 20000.0000      2.4000 9999999999.4199 YES
</SUBDATA>
```

ModelCondition

Condition

ModelConditions are the combination of an object and a Filter. They are used to return when the particular object meets the filter specified. As a result, the subdata section here mostly identical to the Condition subdata section of a Filter. See the description there. There is one exception however with the FieldOpt which has additional strings

(FieldOpt)	:	Optional string with following meanings:
ABS	:	strings are case sensitive, take ABS of field values (older files may have had an integer 1 as well)
REF	:	Means that the variablename is evaluated in the contingency reference state
ABSREF	:	means that both ABS and REF are being used

Nothing specified means that strings are case insensitive, use number fields directly and values are evaluated as normal (older files may have had an integer 0 as well)

ModelExpression

LookupTable

LookupTables are used inside Model Expressions sometimes. These lookup table represent either one or two dimensional tables. If the first string in the SUBDATA section is "x1x2", this signals that it is a two dimensional lookup table. From that point on it will read the first row as "x2" lookup points, and the first column in the remainder of the rows as the x1 lookup values.

Example:

```
MODELEXPRESSION (CustomExpression,ObjectType,CustomExpressionStyle,
CustomExpressionString,WhoAmI,VariableName,WhoAmI:1,VariableName:1)
{
// The following demonstrated a one dimensional lookup table
22.0000, "oneD", "Lookup", "", "Gen<KEY1>1</KEY1><KEY2>1</KEY2>",
"Gen<KEY1>1</KEY1><KEY2>1</KEY2><VAR>GenMW</VAR>", "", ""
<SUBDATA LookupTable>
  // because it does not start with the string x1x2 this will
  // represent a one dimensional lookup table
  x1      value
    0.000000  1.000000
    11.000000  22.000000
    111.000000  222.000000
</SUBDATA>
0.0000, "twod", "Lookup", "",
"Gen<KEY1>1</KEY1><KEY2>1</KEY2>",
"Gen<KEY1>1</KEY1><KEY2>1</KEY2><VAR>GenMW</VAR>",
"Gen<KEY1>6</KEY1><KEY2>1</KEY2>",
"Gen<KEY1>6</KEY1><KEY2>1</KEY2><VAR>GenMW</VAR>"
<SUBDATA LookupTable>
  // because this starts with x1x2 this represent a two dimensional
  // lookup table. The first column represents lookup values for x1.
  // The first row represents lookup values for x2
  x1x2      0.100000  0.300000 // these are lookup heading for x2
    0.000000  1.000000  3.000000
    11.000000  22.000000  33.000000
    111.000000  222.000000  333.000000
</SUBDATA>
}
```

ModelFilter

ModelCondition

A Model Filter's subdata contains a list of each ModelCondition in the filter. Because a list of Model Conditions is stored within Simulator, this subdata section only requires the name of each ModelCondition on each line and whether or not the condition is using the NOT operator as part of the Model Filter.

Example:

```
<SUBDATA ModelCondition>
// ModelConditionName NotCondition
  "Name of First Model Condition" "NO"
  "Name of Second Model Condition" "NO"
  "Name of Third Model Condition" "NO"
</SUBDATA>
```

MTDCRecord

An example of the entire multi-terminal DC transmission line record is given at the end of this record description. Each of the SUBDATA sections is discussed first.

MTDCBus

For this SUBDATA section, each DC Bus is described on a single line of text with exactly 8 fields specified.

DCBusNum	: The number of the DC Bus. Note DC bus numbers are independent AC bus numbers.
DCBusName	: The name of the DC bus enclosed in quotes.
ACTerminalBus	: The AC terminal to which this DC bus is connected (via a MTDCConverter). If the DC bus is not connected to any AC buses, then specify as zero. You may also specify this as a string enclosed in double quotes with the bus name followed by an underscore character, following by the nominal voltage of the bus.
DCResistanceToground	: The resistance of the DC bus to ground. Not used by Simulator.
DCBusVoltage	: The DC bus voltage in kV.
DCArea	: The area that this DC bus belongs to.
DCZone	: The zone that this DC bus belongs to.
DCOwner	: The owner that this DC bus belongs to.

MTDCBus object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the number of the MTDCRecord to which the MTDCBus belongs.

MTDCConverter

For this SUBDATA section, each AC/DC Converter is described by exactly 24 field which may be spread across several lines of text. Simulator will keep reading lines of text until it finds 24 fields. All text to the right of the 24th field (on the same line of text) will be ignored. The 24 fields are listed in the following order:

BusNum	: AC terminal bus number.
MTDCNBridges	: Number of bridges for the converter.
MTDCConvEBas	: Converter AC base voltage.
MTDCConvAngMxMn	: Converter firing angle.
MTDCConvAngMxMn:1	: Converter firing angle max.
MTDCConvAngMxMn:2	: Converter firing angle min.
MTDCConvComm	: Converter commutating resistance.
MTDCConvComm:1	: Converter commutating reactance.
MTDCConvXFRat	: Converter transformer ratio.
MTDCFixedACTap	: Fixed AC tap.
MTDCConvTapVals	: Converter tap.
MTDCConvTapVals:1	: Converter tap max.
MTDCConvTapVals:2	: Converter tap min.
MTDCConvTapVals:3	: Converter tap step size.
MTDCConvSetVL	: Converter setpoint value (current or power).
MTDCConvDCPF	: Converter DC participation factor.
MTDCConvMarg	: Converter margin (power or current).
MTDCConvType	: Converter type.
MTDCMaxConvCurrent	: Converter Current Rating.
MTDCConvStatus	: Converter Status.
MTDCConvSchedVolt	: Converter scheduled DC voltage.

MTDCCConvIDC : Converter DC current.
 MTDCCConvPQ : Converter real power.
 MTDCCConvPQ:1 : Converter reactive power.

MTDCCConverter object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the number of the MTDCRecord to which the MTDCCConverter belongs.

MTDCTransmissionLine

For this SUBDATA section, each DC Transmission Line is described on a single line of text with exactly 5 fields specified:

DCFromBusNum : From DC Bus Number.
 DCToBusNum : To DC Bus Number.
 CKTID : The DC Circuit ID.
 Resistance : Resistance of the DC Line in Ohms.
 Inductance : Inductance of the DC Line in mHenries (Not used by Simulator).

Example:

```
MTDCRECORD (Num,Mode,ControlBus)
{
//-----
// The first Multi-Terminal DC Transmission Line Record
//-----
1 "Current" "SYLMAR3 (26098)"
  <SUBDATA Bus>
    //-----
    // DC Bus data must appear on a single line of text
    // The data consists of exactly 8 values
    // DC Bus Num, DC Bus Name, AC Terminal Bus, DC Resistance to ground,
    // DC Bus Voltage, DC Bus Area, DC Bus Zone, DC Bus Owner
    3 "CELIL03P" 0 9999.00 497.92 40 404 1
    4 "SYLMAR3P" 0 9999.00 439.02 26 404 1
    7 "DC7" 41311 9999.00 497.93 40 404 1
    8 "DC8" 41313 9999.00 497.94 40 404 1
    9 "DC9" 26097 9999.00 439.01 26 404 1
    10 "DC10" 26098 9999.00 439.00 26 404 1
  </SUBDATA>
  <SUBDATA Converter>
    //-----
    // convert subdata keeps reading lines of text until it has found
    // values specified for 24 fields. This can span any number of lines
    // any values to the right of the 24th field found will be ignored
    // The next converter will continue on the next line.
    //-----
    41311 2 525.00 20.25 24.00 5.00 0.0000 16.3100
    0.391048 1.050000 1.000000 1.225000 0.950000 0.012500
    1100.0000 1650.0000 0.0000 "Rect" 1650.0000 "Closed"
    497.931 1100.0000 547.7241 295.3274
    41313 4 232.50 15.36 17.50 5.00 0.0000 7.5130
    0.457634 1.008700 1.030000 1.150000 0.990000 0.010000
    2000.0000 2160.0000 0.1550 "Rect" 2160.0000 "Closed"
    497.940 2000.0000 995.8800 561.8186
    26097 2 230.00 20.90 24.00 5.00 0.0000 16.3100
    0.892609 1.000000 1.100000 1.225000 0.950000 0.012500
    -1100.0000 1650.0000 "" "Inv" 1650.0000 "Closed"
    439.009 1100.0000 -482.9099 274.5227
    26098 4 232.00 17.51 20.00 5.00 0.0000 7.5130
    0.458621 1.008700 1.100000 1.120000 0.960000 0.010000
    439.0000 2160.0000 "" "Inv" 2160.0000 "Closed"
    439.000 1999.9999 -878.0000 544.2775
  </SUBDATA>
  <SUBDATA TransmissionLine>
    //-----
    // DC Transmission Segment information appears on a single line of
    // text. It consists of exactly 5 value
```

```

// From DCBus, To DCBus, Circuit ID, Line Resistance, Line Inductance
//-----
3      4      "1"      19.0000      1300.0000
7      3      "1"      0.0100      0.0000
8      3      "1"      0.0100      0.0000
9      4      "1"      0.0100      0.0000
10     4      "1"      0.0100      0.0000
</SUBDATA>
//-----
// A second Multi-Terminal DC Transmission Line Record
//-----
2      "Current"      "SYLMAR4 (26100)"
<SUBDATA Bus>
5      "CELIL04P"      0      9999.00      497.92      40      404      1
6      "SYLMAR4P"      0      9999.00      439.02      26      404      1
11     "DC11"      41312      9999.00      497.93      40      404      1
12     "DC12"      41314      9999.00      497.94      40      404      1
13     "DC13"      26099      9999.00      439.01      26      404      1
14     "DC14"      26100      9999.00      439.00      26      404      1
</SUBDATA>
<SUBDATA Converter>
41312  2      525.00      20.26      24.00      5.00      0.0000      16.3100
0.391048      1.050000      1.000000      1.225000      0.950000      0.012500
1100.0000      1650.0000      0.0000      "Rect"      1650.0000      "Closed"
497.931      1100.0000      547.7241      295.3969
41314  4      232.50      15.45      17.50      5.00      0.0000      7.5130
0.457634      1.008700      1.030000      1.150000      0.990000      0.010000
2000.0000      2160.0000      0.1550      "Rect"      2160.0000      "Closed"
497.940      2000.0000      995.8800      562.9448
26099  2      230.00      20.90      24.00      5.00      0.0000      16.3100
0.892609      1.000000      1.100000      1.225000      0.950000      0.012500
-1100.0000      1650.0000      ""      "Inv"      1650.0000      "Closed"
439.009      1100.0000      -482.9099      274.5227
26100  4      232.00      17.51      20.00      5.00      0.0000      7.5130
0.458621      1.008700      1.100000      1.120000      0.960000      0.010000
439.0000      2160.0000      ""      "Inv"      2160.0000      "Closed"
439.000      1999.9999      -878.0000      544.2775
</SUBDATA>
<SUBDATA TransmissionLine>
5      6      "1"      19.0000      1300.0000
11     5      "1"      0.0100      0.0000
12     5      "1"      0.0100      0.0000
13     6      "1"      0.0100      0.0000
14     6      "1"      0.0100      0.0000
</SUBDATA>
}

```

MTDCTransmissionLine object types can also be directly created inside their own DATA section as well. One of the key fields of the object is then the number of the MTDCRecord to which the MTDCTransmissionLine belongs.

MultiSectionLine

Bus

A multi section line's subdata contains a list of each dummy bus, starting with the one connected to the From Bus of the MultiSectionLine and proceeding in order to the bus connected to the To Bus of the Line. Note: bus# values may be replaced by a string enclosed in double quotes where the string is the name of the bus followed by an underscore character and then the nominal voltage of the bus, or the string may represent the label of the bus.

Example:

```
//-----  
// The following describes a multi-section line that connects bus  
// 2 - 1 - 5 - 6 - 3  
//-----  
MultiSectionLine (BusNum, BusName, BusNum:1, BusName:1,  
                  LineCircuit, MSLineNSections, MSLineStatus)  
{  
2 "Two" 3 "Three" "&1" 2 "Closed"  
  <SUBDATA Bus>  
    1  
    5  
    6  
  </SUBDATA>  
}
```

BusRenumber

This subdata section allows renumbering of the dummy buses. The entries in the subdata section must be the new bus number that should be assigned to each dummy bus followed by the name of the new bus. The entries can be either space or comma delimited. The bus number must be specified, but the name is optional. If the name is not included and a new bus needs to be created, the name will be the same as the number. If an incorrect number of dummy buses is entered for a multi-section line, none of the dummy buses will be updated for that line. If a dummy bus number is specified that matches an existing bus that is another dummy bus, the other dummy bus will be assigned to a new bus number and the current dummy bus will be assigned to the number specified in the data.

Example:

```
MultiSectionLine (BusNum, BusNum:1, LineCircuit)  
{  
1 2 "1"  
  <SUBDATA BusRenumber>  
    3 "Bus 3"  
    4 "Bus 4"  
    5 "Bus 5"  
  </SUBDATA>  
22 33 "1"  
  <SUBDATA BusRenumber>  
    14 "Bus 14"  
    15 "Bus 15"  
  </SUBDATA>  
}
```

Nomogram

InterfaceElementA

InterfaceElementB

InterfaceElementA values represent the interface elements for the first interface of the nomogram. InterfaceElementB values represent the interface elements for the second interface of the nomogram. The format of these SUBDATA sections is identical to the format of the InterfaceElement SUBDATA section of a normal Interface.

NomogramBreakPoint

This subdata section contains a list of the vertex points on the nomogram limit curve.

Example:

```
<SUBDATA NomogramBreakPoint>
// LimA LimB
-100 -20
-100 100
80 50
60 -10
</SUBDATA>
```

NomogramInterface

InterfaceElement

This follows the same convention as the InterfaceElement SUBDATA section described with the Interface objecttype.

Owner

Bus

This subdata section contains a list of the buses which are owned by this owner. Each line of text contains the bus number. As an alternative to specifying the bus number, a string enclosed in double quotes may be used where the string represents the name of the bus followed by an underscore character and then the nominal voltage of the bus, or the string may represent the label of the bus.

Example:

```
<SUBDATA Bus>
1
35
65
</SUBDATA>
```

Load

This subdata section contains a list of the loads which are owned by this owner. Each line of text contains the bus number followed by the load id. As an alternative to specifying the bus number, a string enclosed in double quotes may be used where the string represents the name of the bus followed by an underscore character and then the nominal voltage of the bus, or the string may represent the label of the bus. Also, instead of specifying the bus and load id, the label of the load enclosed in double quotes may be used.

Example:

```
<SUBDATA Load>
5 1 // shows ownership of the load at bus 5 with id of 1
423 1
</SUBDATA>
```

Gen

This subdata section contains a list of the generators which are owned by this owner and the fraction of ownership. Each line of text contains the bus number, followed by the gen id, followed by an integer showing the fraction of ownership. As an alternative to specifying the bus number, a string enclosed in double quotes may be used where the string represents the name of the bus followed by an underscore character and then the nominal voltage of the bus, or the string may represent the label of the bus. Also, instead of specifying the bus and generator id, the label of the generator enclosed in double quotes may be used.

Example:

```
<SUBDATA Gen>
  78 1 50 // shows 50% ownership of generator at bus 78 with id of 1
  23 3 70
</SUBDATA>
```

Branch

This subdata section contains a list of the branches which are owned by this owner and the fraction of ownership. Each line of text contains the from bus number, followed by the to bus number, followed by the circuit id, followed by an integer showing the fraction of ownership. As an alternative to specifying the bus numbers, strings enclosed in double quotes may be used where the string represents the name of the bus followed by an underscore character and then the nominal voltage of the bus, or the string may represent the label of the bus. Also instead of specifying the two numbers and a circuit id, the label of the branch enclosed in double quotes may be used.

Example:

```
<SUBDATA Branch>
  6 10 1 50 // shows 50% ownership of line from bus 6 to 10, circuit 1
</SUBDATA>
```

PostPowerFlowActions

CTGElementAppend

This format is the same as for the Contingency objecttype except that Abort, ContingencyBlock, and SolvePowerFlow actions are not allowed.

CTGElement

This format is the same as for the Contingency objecttype except that Abort, ContingencyBlock, and SolvePowerFlow actions are not allowed. PostPowerFlowActionsElement object types can also be directly created inside their own DATA section as well.

PWCaseInformation

PWCaseHeader

This subdata section contains the Case Description in free-formatted text. Note: as it is read back into Simulator all spaces from the start of each line are removed.

PWFormOptions

PieSizeColorOptions

There can actually be several PieSizeColorOptions subdata sections for each PWFormOptions object. The first line of each subdata section, the first line of text consist of exactly four values

- | | |
|------------------|--|
| ObjectName | : The objectname of the type of object these settings apply to. Will be either be BRANCH or INTERFACE. |
| FieldName | : The fieldname for the pie charts that these settings apply to. |
| UseDiscrete | : Set to YES to use a discrete mapping of colors and size scalars instead of interpolating for intermediate values. |
| UseOtherSettings | : Set to YES to default these settings to the BRANCH MVA values for BRANCH object. This allows you to apply the same settings to all pie charts. |

After this first line of text, if the UseOtherSettings Value is NO, then another line of text will contain exactly three values:

- | | |
|-----------|---|
| ShowValue | : This is the percentage at which the value should be drawn on the pie chart. |
|-----------|---|

NormalSize : This is the scalar size multiplier which should be used for pie charts below the lowest percentage specified in the lookup table.

NormalColor : This is the color which should be used for pie charts below the lowest percentage specified in the lookup table.

Finally the remainder of the subdata section will contain a lookup table by percentage of scalar and color values. This lookup table will consist of consecutive lines of text with exactly three values

Percentage : This is the percentage at which the follow scalar and color should be applied.

Scalar : A scalar (multiplier) on the size of the pie charts.

Color : A color for the pie charts.

Example:

```
<SUBDATA PieSizeColorOptions>
// ObjectName FieldName UseDiscrete UseOtherSettings
Branch MVA YES NO
// ShowValue NormalSize NormalColor
80.0000 1.0000 16776960
// Percentage Scalar Color
80.0000 1.5000 33023
100.0000 2.0000 255
</SUBDATA>
<SUBDATA PieSizeColorOptions>
// ObjectName FieldName UseDiscrete UseOtherSettings
Branch MW YES YES
</SUBDATA>
```

PWLPOPFCTGViol

OPFControlSense

OPFBusSenseP

OPFBusSenseQ

This stores the control sensitivities for each contingency violation during OPF/SCOPF analysis. Each line contains one value:

Sensitivity : The value of the sensitivity with respect to each control in OPFControlSense or with respect to each bus in OPFBusSenseP and OPFBusSenseQ.

Example:

```
<SUBDATA OPFControlSense>
// Value
1.000441679
2.447185E-7
-1.1109307E-6
1.6427327E-7
0
</SUBDATA>
```

PWLPTabRow

LPBasisMatrix

This subdata section stores the basis matrix associated with the final LP OPF solution. Each line contains two values:

Variable : The basic variable.

Value : The sensitivity of the constraint to the basic variable.

Example:

```
<SUBDATA LPBasisMatrix>
// Var Value
1 1.00000
2 1.00000
5 1.00000
6 1.00000
</SUBDATA>
```

PWPVResultListContainer

PWPVResultObject

This subdata section contains the results of a particular PV Curve scenario. The data consists of two general sections: the first three rows of text contain the "independent axis" of the PV Curve. The first row starts with the string INDNOM and is followed by a list of numbers representing the nominal shift, the second row starts with INDEXP and is followed by the export shift, and the third row starts with INDIMP and is followed by the import shift. Following after these rows is a list of all the tracked quantities. Each tracked quantity row consists of three parts which are separated by the strings ?f= and &v= . The first part of the string represents a description of the power system object being tracked, the second part represents the field variable name being tracked, and the third contains a list of all the values at the various shift levels.

Example:

```
<SUBDATA PWPVResultObject>
INDNOM 0.00 500.00 1000.00 1500.00 1750.00 1875.00 1975.00
INDEXP 0.00 500.00 1000.00 1500.00 1750.00 1875.00 1975.00
INDIMP 0.00 -417.23 -701.58 -890.58 -952.60 -975.35 -990.43
Bus '3'?f=BusPUVolt&v= 0.993 0.983 0.964 0.939 0.926 0.919 0.914
Bus '5'?f=BusPUVolt&v= 1.007 1.000 0.982 0.956 0.940 0.932 0.926
Gen '4' '1'?f=GenMVR&v= 19.99 245.27 523.62 831.13 986.84 1060.6 1118.7
Gen '6' '1'?f=GenMVR&v= -6.59 -120.84 -131.37 -39.53 48.35 103.8 154.5
</SUBDATA>
```

LimitViol

This subdata section contains the limit violations of a particular PV Curve scenario. This subdata section would only exist if using the option to monitor limit violations with the PV tool. Each row consists of an identifier, either VLOW or VHIGH, to indicate the type of limit violation followed by the bus identifier based on the key field identifier chosen. The bus can be identified by number, name and nominal kV combination, or label. The bus identifier is followed by the limit in use to identify a voltage violation and this is followed by the voltage at the bus.

Example:

```
<SUBDATA LimitViol>
VLOW 3 1.00000 0.99017
VLOW 5 1.00000 0.98245
</SUBDATA>
```

PVBusInadequateVoltages

This subdata section contains a list of buses that are considered to have inadequate voltages at each transfer level for a particular PV Curve scenario. This subdata section would only exist if using the option to store inadequate voltages. The data consists of two general sections: the first row starts with the string INDNOM and is followed by a list of numbers representing the nominal shift. The second and subsequent rows list the buses and inadequate voltages for any bus that has an inadequate voltage at any transfer level. Each row starts with the bus identifier followed by the voltages at that bus at the corresponding shift levels. If a voltage is not inadequate at a particular transfer level, a blank entry will appear instead of a voltage value. The bus identifier is based on the key field identifier chosen and can be number, name and nominal kV combination, or label.

Example:

```
<SUBDATA PVBUSInadequateVoltages>
// INDNOM ShiftLevel1 ShiftLevel2 ...
// BUS Voltage1 Voltage2 ...
INDNOM      0.000 100.000 200.000 300.000 400.000 500.000
"Bus '3'"    0.99269 0.99278 0.99282 0.99280 0.99273 0.99262
"Bus '4'"    ""    1.00000 "" "" "" ""
</SUBDATA>
```

PWQVResultListContainer

PWPVResultObject

This subdata section contains the results of a particular QV Curve scenario. These results will exist when tracking quantities with the QV curve tool. The data consists of two general sections: the first three rows of text contain the "independent axis" of the QV Curve. The first three rows start with the strings INDNOM, INDEXP, and INDIMP and are followed by a list of numbers representing the setpoint voltage representing the V of the QV curve. Following after these rows is a list of all the tracked quantities. Each tracked quantity row consists of three parts which are separated by the strings ?f= and &v= . The first part of the string represents a description of the power system object being tracked, the second part represents the field variable name being tracked, and the third contains a list of all the values at the various setpoint voltage levels.

Example:

```
<SUBDATA PWPVResultObject>
INDNOM      1.100 1.093 1.083 1.073 1.063
INDEXP      1.100 1.093 1.083 1.073 1.063
INDIMP      1.100 1.093 1.083 1.073 1.063
Bus '1'?f=BusPUVolt&v=1.05000 1.05000 1.05000 1.05000 1.05000
Bus '1'?f=BusKVVolt&v=144.89999 144.89999 144.89999 144.89999 144.89999 </SUBDATA>
```

QVCurve

QVPoints

This subdata section contains a list of the QV Curve points calculated for the respect QVCurve. Each line consists of exactly six values:

PerUnitVoltage	:	The per unit voltage of the bus for a QV point.
FictitiousMvar	:	The amount of Mvar injection from the fictitious generator at this QV point.
ShuntDeviceMvar	:	The Mvar injection from any switched shunts at the bus.
TotalMvar	:	The total Mvar injection from switched shunts and the fictitious generator.
ReservesMvar	:	Total amount of Mvar reserves available at the bus.
ReservesTotalMvar	:	Total Mvar injection from the switched shunts, fictitious generator, and available reserves.

Example:

```
QVCURVE (BusNum, CaseName, qv_VQ0, qv_Q0, qv_Vmax, qv_QVmax, qv_VQmin, qv_Qmin,
         qv_Vmin, qv_QVmin, Qinj_Vmax, Qinj_0, Qinj_min, Qinj_Vmin)
{
5 "BASECASE"  0.880  0.000  1.100 312.490  0.480 -221.072
              0.180 -86.334 191.490 -77.373 -244.075 -89.562
  <SUBDATA QVPoints>
    // NOTE: This bus has a constant impedance
    // switched shunt value of -100 Mvar at it.
    //V(PU), Q(MVR), Q_shunt(MVR), Q_tot(MVR), Q_res(MVR), Q_tot_res(MVR)
    1.1000, 312.4898, -121.0000, 191.4898, 0.0000, 191.4898
    0.9800, 124.6619, -95.9656, 28.6963, 0.0000, 28.6963
    0.7800, -96.6202, -60.7808, -157.4010, 0.0000, -157.4010
    0.5800, -206.9895, -33.5960, -240.5855, 0.0000, -240.5855
    0.3800, -207.4962, -14.4113, -221.9075, 0.0000, -221.9075
  </SUBDATA>
}
```

QVCurve Options

Sim_Solution_Options

This subdata section contains solution options that will be used when running QV Curves. See explanation under the CTG_Options object type for more information.

RemedialAction

CTGElementAppend

This format is the same as for the Contingency objecttype except that the SolvePowerFlow action is not allowed.

CTGElement

This format is the same as for the Contingency objecttype except that the SolvePowerFlow action is not allowed. RemedialActionElement object types can also be directly created inside their own DATA section as well.

SelectByCriteriaSet

SelectByCriteriaSetType

This subdata section contains a list of the display object types which are chosen to be selected. Each line of the section consists of the following:

- | | |
|-------------------|---|
| DisplayObjectType | : The object type of the display object. |
| (FilterName) | : This field is optional, but must be given if either of the following fields is given. See the Using Filters in Script Commands section for more information on specifying the filename. |
| (WhichFields) | : For display objects that can reference different fields, this sets which of those fields it should select (e.g. select only Bus Name Fields). The value may be either ALL or SPECIFIED. |
| (ListOfFields) | : If WhichFields is set to SPECIFIED, then a delimited list of fields follows. |

Example:

```
<SUBDATA SelectByCriteriaSetType>
  DisplayAreaField "" "ALL"
  DisplayBus ""
  DisplayBusField "Name of Bus Filter" "SPECIFIED" BusName BusPUVolt BusNum
  DisplayCircuitBreaker ""
  DisplaySubstation ""
  DisplaySubstationField "" "SPECIFIED" SubName SubNum BusNomVolt BGLoadMVR
  DisplayTransmissionLine ""
  DisplayTransmissionLineField "" "ALL"
</SUBDATA>
```

Area

This subdata section contains a list of areas which were chosen to be selected. Each line of the section consists of either the number or the name. When generated automatically by PowerWorld we also include the other identifier as a comment.

Example:

```
<SUBDATA Area>
  18 // NEVADA
  22 // SANDIEGO
  30 // PG AND E
  52 // AQUILA
</SUBDATA>
```

Zone

This subdata section contains a list of zones which were chosen to be selected. Each line of the section consists of either the number or the name. When generated automatically by PowerWorld we also include the other identifier as a comment.

Example:

```
<SUBDATA Zone>
  680 // ID SOLUT
  682 // WY NE IN
</SUBDATA>
```

ScreenLayer

This subdata section contains a list of screen layers which were chosen to be selected. Each line of the section consists of either the name.

Example:

```
<SUBDATA ScreenLayer>
  "Border"
  "Transmission Line Objects"
</SUBDATA>
```

ShapefileExportDescription

This object uses the same subdata sections as [SelectByCriteriaSet](#). The only distinction is that only buses and lines can be exported.

StudyMWTransactions

ImportExportBidCurve

This subdata section contains the piecewise linear transactions cost curves for areas involved in a MW transaction. Costs are only for areas that are not on OPF control. Curves must be monotonically increasing. Each line corresponds to a point in the cost curve, and it has two values:

MW	: The MW value. Use negative values for imports (purchase) and positive values for exports (sales)
Price	: The price in \$/MWh.

Two different cost curves can be entered for each transaction. One is for the cost curve relative to the Export Area specified in the transaction, and the other is for the cost curve relative to the Import Area specified in the transaction. The first curve that is listed in the SUBDATA section is the curve relative to the Export Area. The curve relative to the Import Area is denoted by the keyword REVERSE. Either or both of the curves can be blank.

Example:

```
<SUBDATA ImportExportBidCurve>
//MW    Price[$/MWh]
-20.00   5.00
-10.00  10.00
  0.00  15.00
 10.00  20.00
 20.00  45.00
 30.00  70.00
REVERSE
-25.00   7.00
-15.00  12.00
  5.00  17.00
 15.00  22.00
 25.00  47.00
 35.00  72.00
</SUBDATA>
```

SuperArea

SuperAreaArea

This subdata section contains a list of areas within each super area. Each line of text contains two values, the area number followed by a participation factor for the area that can be optionally used.

Example:

```
<SUBDATA SuperAreaArea>
 1  48.9
 5  34.2
25  11.2
</SUBDATA>
```

TSSchedule

SchedPoint

This section stores the schedule time points used in Time Step Simulation. Each line contains seven values:

Date	: The date of the point.
Hour	: The hour of the point.
Pointtype	: An integer specifying the point type. 0 : Numeric 1 : Boolean (Yes/No, Closed/Open) 2 : Text
Numeric Value	: The numeric value if point type is Numeric. Otherwise it is just zero.
Boolean Value	: The boolean value if point type is Boolean. Otherwise it is just false.
Text value	: The text value if point type is Text. Otherwise it is just an empty string.
Audiofilename	: The audio filename associated to the point. If none, it is just an empty string.

Example:

```
<SUBDATA SchedPoint>
  //Date      Hour      PointType  NValue  BValue  TValue  AValue
  5/8/2006           0          1.00    NO
  5/8/2006 6:00:00 AM  0          1.10    NO
  5/8/2006 12:00:00 PM 0          1.25    NO
</SUBDATA>
```

UserDefinedDataGrid

ColumnInfo

This follows the same convention as the ColumnInfo SUBDATA section described with the DataGrid objecttype.

SCRIPT Section for Display Auxiliary File

The syntax for script commands in Display Auxiliary Files is the same as for Auxiliary Files. See the [SCRIPT Section](#) and its sub-sections for details on the proper syntax. Any differences for display auxiliary files will be discussed below.

AXD Actions

The following script commands are available for AXD files

AutoInsertBorders;

Use this action to insert borders according to the settings in the AutoInsertBordersOptions object

AutoInsertBuses(LocationSource, MapProjection, AutoInsertBranches, InsertIfNotAlreadyShown, "filename", InsertSelected);

Use this action to insert buses based on specified location data.

LocationSource	: "Bus", "Substation" or "File"
MapProjection	: "Simple Conic", "Mercator", "Alaska" or "xy"
AutoInsertBranches	: YES to insert transmission lines when finished, NO not to
InsertOnlyIfNotAlreadyShown	: YES if only buses that are not already shown should be inserted, NO to insert all buses.
"filename"	: (optional) path to location source file (if LocationSource is "File")

FileCoordinates is no longer used. It should not be included when creating new auxiliary files, but if it is included in existing auxiliary files, it will be read and ignored. If MapProjection is set to "xy" and using a file, the file coordinates are assumed to be in x,y, otherwise, file coordinates are assumed to be lon, lat.

FileCoordinates	: (optional) format of coordinates in file "xy" or "lonlat" (if LocationSource is "File")
InsertSelected	: (optional) Default is NO. YES is only insert buses that are selected (SELECTED = YES).

This command inserts bus display objects using the latitude and longitude stored with each bus, interpreted through the Mercator map projection. It inserts only the buses that have their Selected field set to YES and also adds the connecting branches, regardless of whether the buses are already displayed.

```
AutoInsertBuses("Bus", "Mercator", YES, NO, , YES);
```

AutoInsertGens(MinkV, InsertTextFields);

Use this action to insert generators.

MinkV	: Minimum kV level to insert
InsertTextFields	: (optional) insert text fields (default=YES)

This command inserts generator display objects for all generators connected to buses with a nominal voltage of 140 kV or higher. It places only the generator symbols without any accompanying text fields such as MW or Mvar labels.

```
AutoInsertGens(140, NO);
```

AutoInsertInterfaces(InsertPieCharts, PieChartSize);

Use this action to insert line flow objects.

InsertPieCharts	: (optional) Insert pie charts as well (default=YES)
PieChartSize	: (optional) default size of interface pie charts (default=50.0)

This command inserts interface display objects along with pie charts that visualize flow or loading. The pie charts are included (YES) and are set to a default size of 50.0 screen units.

AutoInsertInterfaces(YES, 50.0);

AutoInsertLineFlowObjects(MinkV, InsertOnlyIfNotAlreadyShown, LineLocation, Size, FieldDigits, FieldDecimals, TextPosition, ShowMW, ShowMvar, ShowMVA, ShowUnits, ShowComplex);

Use this action to insert line flow objects.

MinkV : Minimum kV level to insert (default=0)
InsertOnlyIfNotAlreadyShown: (optional) if existing line flow objects are ignored (default=YES)
LineLocation : (optional) where to insert flow objects (default=0)
 0 : middle
 1 : 10%/90%
 2 : after stubs
Size : (optional) size (default=5.0)
FieldDigits : (optional) total digits in field (default=6)
FieldDecimals : (optional) digits to the right of the decimal (default=2)
TextPosition : (optional) position of fields relative to flow object (default=YES)
 YES : above
 NO : below
ShowMW : (optional) show MW field (default=YES)
ShowMvar : (optional) show Mvar field (default=YES)
ShowMVA : (optional) show MVA field (default=YES)
ShowSuffix : (optional) show field units (default=YES)
ShowComplex : (optional) show complex form (MW+jMvar) (default=NO)

This command inserts line flow arrow objects for transmission lines with a nominal voltage of 100 kV or higher. It inserts arrows in the middle of the line (0), with a size of 5.0, showing MW and Mvar values (but not MVA), with units displayed (e.g., "MW") and text positioned above the arrow. It does not use complex format (MW + jMvar), and skips lines already shown.

AutoInsertLineFlowObjects(100, YES, 0, 5.0, 6, 2, YES, YES, YES, NO, YES, NO);

AutoInsertLineFlowPieCharts(MinkV, InsertOnlyIfNotAlreadyShown, InsertMSLines, Size);

Use this action to insert line flow pie charts.

MinkV : Minimum kV level to insert (default=0)
InsertOnlyIfNotAlreadyShown : (optional) if existing line flow objects are ignored (default=YES)
InsertMSLines : (optional) insert pie charts for Multi-Section Lines (default=YES)
Size : (optional) size (default=5.0)

This command inserts line flow pie charts for all transmission lines with a nominal voltage of 50 kV or higher. It skips lines that already have pie charts (YES), includes Multi-Section Lines (YES), and sets the size of each pie chart to 5.0 screen units.

AutoInsertLineFlowPieCharts(50, YES, YES, 5.0);

AutoInsertLines(MinkV, InsertTextFields, InsertEquivObjects, InsertZBRPieCharts, InsertMSLines, ZBRImpedance, NoStubZBRs, SingleCBZRs);

Use this action to insert lines.

MinkV : (optional) minimum kV level to insert (default=0)
InsertTextFields : (optional) insert text fields (default=YES)
InsertEquivObjects : (optional) insert Equivalenced Objects (default=YES)
InsertZBRPieCharts : (optional) insert pie charts for lines with no limit and bus ties (default=NO)

InsertMSLines	: (optional) insert MultiSection Lines (default=YES)
ZBRImpedance	: (optional) maximum PU impedance for bus ties (default =0.0001)
NoStubsZBRs	: (optional) ignore stubs for bus ties (default=YES)
SingleCBZBRs	: (optional) only insert a single circuit breaker (default=YES)

This command inserts transmission line display objects for lines with a nominal voltage of 50 kV or higher. It includes text fields, inserts equivalenced lines, skips pie charts for zero-impedance bus ties, includes Multi-Section Lines, treats lines with per-unit impedance ≤ 0.0001 as bus ties, ignores stubs when identifying those ties, and inserts only a single circuit breaker for each zero-impedance bus tie.

```
AutoInsertLines(50, YES, YES, NO, YES, 0.0001, YES, YES);
```

AutoInsertLoads(MinkV, InsertTextFields);

Use this action to insert loads.

MinkV	: Minimum kV level to insert (default=0)
InsertTextFields	: (optional) insert text fields (default=YES)

This command inserts load display objects for buses with a nominal voltage of 50 kV or higher and includes text fields showing load details such as MW and Mvar values.

```
AutoInsertLoads(50, YES);
```

AutoInsertSwitchedShunts(MinkV, InsertTextFields);

Use this action to insert switched shunts.

MinkV	: Minimum kV level to insert (default=0)
InsertTextFields	: (optional) insert text fields (default=YES)

This command inserts switched shunt display objects (e.g., capacitor banks or reactors) connected to buses with a nominal voltage of 140 kV or higher and includes text fields showing shunt details like Mvar values.

```
AutoInsertSwitchedShunts(140, YES);
```

AutoInsertSubStations(LocationSource, MapProjection, AutoInsertBranches, InsertIfNotAlreadyShown, "filename", InsertSelected);

Use this action to insert substations based on specified location data.

LocationSource	: "Bus", "Substation" or "File"
MapProjection	: "Simple Conic", "Mercator", "Alaska" or "x,y"
AutoInsertBranches	: YES to insert transmission lines when finished, NO not to
InsertOnlyIfNotAlreadyShown	: YES if only buses that are not already shown should be inserted, NO to insert all buses.
"filename"	: (optional) path to location source file (if LocationSource is "File")

FileCoordinates is no longer used. It should not be included when creating new auxiliary files, but if it is included in existing auxiliary files, it will be read and ignored. If MapProjection is set to "xy" and using a file, the file coordinates are assumed to be in x,y, otherwise, file coordinates are assumed to be lon, lat.

FileCoordinates	: (optional) format of coordinates in file "xy" or "lonlat" (if LocationSource is "File")
InsertSelected	: (optional) Default is NO. YES is only insert buses that are selected (SELECTED = YES).

This command inserts substation display objects using the latitude and longitude coordinates stored in each substation, interpreted with the Simple Conic map projection. It inserts only substations that are not already shown, adds branches between them, and includes all substations regardless of selection status.

```
AutoInsertSubStations("Substation", "Simple Conic", YES, YES, "", NO);
```

AutoInsertAreas(MapProjection, InsertIfNotAlreadyShown, InsertSelected);

Use this action to insert areas based on the latitude and longitude of the area (which is calculated as the average lat/long of buses in the area).

MapProjection : "Simple Conic", "Mercator", "Alaska" or "x,y"

InsertOnlyIfNotAlreadyShown

: (optional) Default is NO. YES if only areas that are not already shown should be inserted, NO to insert all areas.

InsertSelected : (optional) Default is NO. YES is only insert areas that are selected (SELECTED = YES).

This command inserts area display objects based on the average latitude and longitude of the buses within each area, using the Simple Conic map projection. It inserts all areas, regardless of whether they are already displayed or selected.

```
AutoInsertAreas("Simple Conic", NO, NO);
```

AutoInsertInjectionGroups(MapProjection, InsertIfNotAlreadyShown, InsertSelected);

Use this action to insert injection groups based on the latitude and longitude of the injection group (which is calculated as the average lat/long of objects in the injection group).

MapProjection : "Simple Conic", "Mercator", "Alaska" or "x,y"

InsertOnlyIfNotAlreadyShown

: (optional) Default is NO. YES if only injection groups that are not already shown should be inserted, NO to insert all injection groups.

InsertSelected : (optional) Default is NO. YES is only insert injection groups that are selected (SELECTED = YES).

This command inserts injection group display objects based on the average latitude and longitude of the objects within each group, using the Simple Conic map projection. It inserts all injection groups, regardless of whether they are already shown or selected.

```
AutoInsertInjectionGroups("Simple Conic", NO, NO);
```

AutoInsertOwners(MapProjection, InsertIfNotAlreadyShown, InsertSelected);

Use this action to insert owners based on the latitude and longitude of the owner (which is calculated as the average lat/long of objects in the owner).

MapProjection : "Simple Conic", "Mercator", "Alaska" or "x,y"

InsertOnlyIfNotAlreadyShown

: (optional) Default is NO. YES if only owners that are not already shown should be inserted, NO to insert all owners.

InsertSelected : (optional) Default is NO. YES is only insert owners that are selected (SELECTED = YES).

This command inserts owner display objects based on the average latitude and longitude of the elements owned by each owner, using the Mercator map projection. It inserts only owners that are not already displayed and only those with their Selected field set to YES.

```
AutoInsertOwners("Mercator", YES, YES);
```

AutoInsertZones(MapProjection, InsertIfNotAlreadyShown, InsertSelected);

Use this action to insert zones based on the latitude and longitude of the zone (which is calculated as the average lat/long of buses in the zone).

MapProjection : "Simple Conic", "Mercator", "Alaska" or "x,y"
 InsertOnlyIfNotAlreadyShown : (optional) Default is NO. YES if only zones that are not already shown should be inserted, NO to insert all zones.
 InsertSelected : (optional) Default is NO. YES is only insert zones that are selected (SELECTED = YES).

This command inserts zone display objects using the average x,y coordinates of the buses in each zone, interpreted with the x,y coordinate system. It inserts only zones that are not already displayed and only those with their Selected field set to YES.

AutoInsertZones("x,y", YES, YES);

FixFlowArrowLineEnds("OnlineName", "LayerName");

The unmoving line flow arrow indicators that can be displayed on lines may not always be setup correctly. This script corrects the flow arrows so that they look at the end of the line to which they are the closest to determine the direction of flow.

"OnlineName" : Optional – default is blank
 The name of the oneline to which this action should be applied. The oneline must already be open. When not specified this action is applied to the oneline to which the display auxiliary file has been applied.
 "LayerName" : Optional – default is blank
 Name of the screen layer in which this action should be applied. When this is left blank the action is applied to all flow arrow objects regardless of the layer.

This command adjusts the direction of all unmoving flow arrows on the current one-line diagram. It determines the correct flow direction by checking which end of the line the arrow is closest to and orients the arrow accordingly. Since no oneline or layer is specified, it applies to all flow arrows on the active diagram, regardless of screen layer.

FixFlowArrowLineEnds(,);

FixFlowArrowPosition("OnlineName", "LayerName");

The unmoving line flow arrow indicators that can be displayed on lines are difficult to position properly. This script is intended to automate as much of the work as possible for positioning these objects.

"OnlineName" : Optional – default is blank
 The name of the oneline to which this action should be applied. The oneline must already be open. When not specified this action is applied to the oneline to which the display auxiliary file has been applied.
 "LayerName" : Optional – default is blank
 Name of the screen layer in which this action should be applied. When this is left blank the action is applied to all flow arrow objects regardless of the layer.

This command repositions all static flow arrows on the currently active one-line diagram to improve their visual placement along the transmission lines. Since no oneline or layer is specified, it applies to all flow arrows on all layers of the active diagram.

FixFlowArrowPosition(,);

InsertConnectedBuses("BusIdentifier");

Insert oneline display buses that are connected to the identified bus.

- "BusIdentifier" : Identifier for the bus for which connected display buses should be added. Bus can be identified as a data object "BUS Number", "BUS NameNomkV", or "BUS Label". When identified as a data object all display buses that are linked to this data bus will have display buses inserted.
Bus can also be identified as a display object "DISPLAYBUS BusNum SOAuxiliaryID" or "DISPLAYBUS BusName_NomVolt". When identified as a display object only display buses connected to this bus will be inserted.

This command inserts display objects for all buses directly connected to the bus identified as "BUS 1" on the one-line diagram. It places those connected buses visually, allowing you to easily expand the diagram from a single bus outward by showing its immediate electrical connections.

```
InsertConnectedBuses("BUS 1");
```

LoadAXDFromAXD("filename", CreatelfNotFound)

Use this action to apply a display auxiliary file via this script command that is part of another display auxiliary file.

- "filename" : The file name of the display auxiliary file to load. See the [Specifying File Names in Script Commands](#) section for special keywords that can be used when specifying the file name.
CreatelfNotFound : Optional – default is NO when using the Legacy Auxiliary File Header format. Default is YES when using the Concise Auxiliary File Header format.
This parameter is only enforced when the [Create_if_not_found field for a Legacy Auxiliary File Header is set to PROMPT](#). Otherwise, YES is assumed. YES means that objects that cannot be found will be created while reading DATA sections from "filename".

This command loads and applies the display auxiliary file located at the specified path. If any objects referenced in DATA sections are not found on the current oneline, they will be created.

```
LoadAXDFromAXD("G:\Diagrams\SubstationView.axd", YES);
```

PanAndZoomToObject("ObjectID", "DisplayObjectType", "DoZoom");

Use this action to pan to and optionally zoom in on a display object. This action will find the first matching display object linked to the model object identified by ObjectID or the exact display object identified by ObjectID.

- ObjectID : The object identifier that uniquely identifies the power system model object or display object. For example, to identify bus 123354, the ObjectID takes the form "BUS 123354".
DisplayObjectType : (optional) The display object type to find. When a power system model object is passed in for ObjectID, this parameter helps this action narrow down to what kind of display object to pan. For example, to find the first bus display object, use "DISPLAYBUS". (default="")
DoZoom : (optional) Whether or not to zoom after panning. (default=YES)

This command pans and zooms the one-line diagram to center on the display object for bus 1, specifically targeting a DISPLAYBUS object type. The zoom is enabled (YES), so the view will both move and magnify to focus directly on the bus's location in the diagram.

```
PanAndZoomToObject("BUS 1", "DISPLAYBUS", YES);
```

ResetStubLocations(ZBRImpedance, NoStubsZBRs);

Use this action to reset stub locations.

ZBRImpedance : (optional) max P.U. impedance for bus ties (default=0.0001)
NoStubsZBRs : (optional) Ignore stubs for bus ties (default=YES)

This command resets the placement of stub lines for the one-line diagram. It affects lines with a per-unit impedance ≤ 0.0001 , and with YES specified, it preserves the existing layout of the bus objects, adjusting only the stubs for cleaner visual alignment.
--

<code>ResetStubLocations(0.0001, YES);</code>

General Script Commands

The following script commands defined above in the general SCRIPT section are available for display auxiliary files as well:

- ExitProgram
- LoadScript
- LoadData
- SelectAll
- UnSelectAll
- SetData
- SaveData
- SaveDataWithExtra
- CreateData
- DeleteFile
- RenameFile
- CopyFile
- SetCurrentDirectory
- SaveObjectFields

DATA Section for Display Auxiliary File

The syntax for Display Auxiliary Files is the same as for Auxiliary Files. See the [DATA Section](#) and its sub-sections for details on the proper syntax. Any differences for display auxiliary files will be discussed below.

Key Fields

See the [Key Fields](#) topic in the [DATA Section](#) for general details.

Display objects have an additional key field used for identification because multiple objects can be present on the same one-line diagram that represent the same power system element. This extra key field is `SOAuxiliaryID`. This is a field that is unique for each type of display object and other key field combination. If there are two display buses that represent bus one in the power system, the `SOAuxiliaryID` field will be different for both. Simulator will automatically create unique identifiers when these objects are created graphically. They can also be user specified but are forced to be unique. This field does not need to be present when reading in a display auxiliary file, but if it is missing, Simulator assumes that the ID is "1". This field is the only key field identifier for objects that do not link to power system elements such as background lines and pictures, and therefore, should always be included when reading in these objects or the expected results may not be achieved.

By going to the main menu and choosing Help, Export Display Object Fields you will obtain a list of fields available for each display object type. In this output, the key fields will appear with asterisks *.

Special Data Sections

There are several object types that should be noted here because they can impact the reading of an entire display auxiliary file, overall look of the resulting one-line diagram, or require special input to properly import/export the object.

GeographyDisplayOptions

Most objects supported in the display auxiliary file have coordinates that can be specified in the appropriate data sections. What these coordinates specify can be controlled by the `GEOGRAPHYDISPLAYOPTIONS` object. This object has only two fields available: `MapProjection` and `ShowLonLat`. There are four possible settings for `MapProjection`: "x, y", "Simple Conic", "Alaska", and "Mercator". The choice of projection will determine how the x,y values for display objects are interpreted. `ShowLonLat` can be either "YES" or "NO". If `ShowLonLat` is "YES", the setting specified for the `MapProjection` will be the longitude,latitude projection used when reading/writing the object x,y values. If `ShowLonLat` is "NO", the x,y values will always be interpreted as x,y regardless of the `MapProjection` setting. This object should be placed in the display auxiliary file before any other objects containing coordinates are read. If this object is not included in the auxiliary file, the coordinates will be interpreted based on the current settings of map projection and whether or not coordinates are showing longitude,latitude.

Picture

`PICTURE` objects represent background images that cannot be stored in a text file format. To properly include a `PICTURE` object in a display auxiliary file, the file containing the image must be saved and read along with the auxiliary file. The `FileName` field indicates the name and location of the image file. If the image file cannot be found when reading in a display auxiliary file and attempting to create a new object, no `PICTURE` object will be created. If attempting to update an existing object and the image file cannot be found, the object will not be updated with a new image, but the `FileName` field will be updated with the specified file name.

PWFormOptions

One-line display options that affect the current display settings can be changed by using the `PWFORMOPTIONS` object. Usually, this object specifies named sets of options that can be selected and used to change the various one-line display options through the GUI. By including a specially named object, the current options can be changed through a display

auxiliary file. PWFORMOPTIONS are named using the OOName field. Setting this field to "THESE_OPTIONS_ARE_APPLIED_TO_THE_CURRENT_DISPLAY" will apply the specified set of options to the current one-line when the file is read. When saving the entire one-line to a display auxiliary file, a PWFORMOPTIONS object with this name is added to the file by default.

View

Different views can be specified in the display auxiliary file using the VIEW object. Usually, this object is used to specify named sets of options used to select and change the view through the GUI. By including a specially named object, the current view can be changed through a display auxiliary file. VIEW objects are named using the ViewName field. Setting this field to "THIS_VIEW_IS_APPLIED_TO_THE_CURRENT_DISPLAY" will apply the specified set of view options to the current one-line when the file is read. When saving the entire one-line to a display auxiliary file, a VIEW object with this name is added to the file by default.

SubData Sections

The format described thus far works well for most kinds of data in Simulator. It does not work as well however for data that stores a list of objects. For example, a contingency stores some information about itself (such as its name), and then a list of contingency elements, and possibly a list of limit violations as well. For data such as this, Simulator allows <SubData>, </SubData> tags that store lists of information about a particular object. This formatting looks like the following

```
object_type (list_of_fields)
{
value_list_1
  <SUBDATA subobject_type1>
    precise format describing an object_type1
    precise format describing an object_type1
    .
    .
  </SUBDATA>
  <SUBDATA subobject_type2>
    precise format describing an object_type2
    precise format describing an object_type2
    .
    .
  </SUBDATA>
value_list_2
  .
  .
value_list_n
}
```

Note that the information contained inside the <SubData>, </SubData> tags may not be flexibly defined. It must be written in a precisely defined order that will be documented for each SubData type. The description of each of these SubData formats follows.

ColorMap

Same format as in data auxiliary files.

CustomColors

Same format as in data auxiliary files.

DisplayDCTramissionLine

DisplayInterface

DisplayMultiSectionLine

DisplaySeriesCapacitor

DisplayTransformer

DisplayTransmissionLine

Line

Line

This is a list of points defining the graphical line used to represent the object. Each set of coordinates can be enclosed in square brackets, [], or the brackets can be eliminated. The brackets will be included when Simulator generates an auxiliary file. The individual coordinates are separated by the specified delimiter, either a space or a comma, and if the brackets are included, the same delimiter should be used to separate sets of coordinates. The list of points is in a somewhat free form and sets of coordinates can span multiple lines. Each point should either be in x,y coordinates or longitude,latitude coordinates. Which coordinates should be used depends on the current option settings for map projection and whether or not coordinates should be shown in longitude,latitude. If the display auxiliary file is automatically generated by Simulator, a comment will be included in the subdata section indicating the coordinate system in use during file creation.

Example using brackets and a comma delimiter:

```
<SUBDATA Line>
//Coordinates are x,y
  [14.00000000, 63.00000000], [14.00000000, 60.00000000],
  [20.00000000, 45.00000000], [20.00000000, 42.00000000]
</SUBDATA>
```

Example with no brackets and a space delimiter:

```
<SUBDATA Line>
//Coordinates are x,y
  14.00000000 63.00000000 14.00000000 60.00000000
  20.00000000 45.00000000 20.00000000 42.00000000
</SUBDATA>
```

DynamicFormatting

Same format as in data auxiliary files.

Filter

Same format as in data auxiliary files.

GeoDataViewStyle

Same format as in data auxiliary files.

PieChartGaugeStyle

ColorMap

This is a lookup table by percentage of scalar and color values. This lookup table will consist of consecutive lines of text with exactly three values:

Percentage : This is the percentage at which the following scalar and color should be applied.
Scalar : A scalar (multiplier) on the size of the pie chart/gauge.
Color : A color for the pie chart/gauge.

Example:

```
<SUBDATA ColorMap>
//Percentage Scalar Color
85.0000 1.5000 33023
100.0000 2.0000 255
</SUBDATA>
```

PWFormOptions

Same format as in data auxiliary files.

SelectByCriteriaSet

Same format as in data auxiliary files.

UserDefinedDataGrid

Same format as in data auxiliary files.

View

ScreenLayer

This is a list of screen layer names that are hidden in the current view. Each screen layer name is on a separate line of text.

Example:

```
<SUBDATA ScreenLayer>
//These are hidden screen layers
"pie layer"
</SUBDATA>
```